

Algorithmique des graphes

Joël Gay

APP - Polytech Paris Saclay

2026 – 2027

Plan

- 1 Définitions
- 2 Programmation
- 3 Connexité et parcours
- 4 Plus courts chemins
- 5 Arbres couvrants
- 6 Flots
- 7 Coloriage de graphes

Plan

1 Définitions

2 Programmation

3 Connexité et parcours

4 Plus courts chemins

5 Arbres couvrants

6 Flots

7 Coloriage de graphes

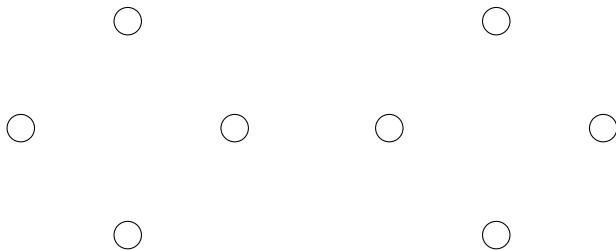
Plan

1 Définitions

- Vulgarisation
- Exemples de problèmes classiques en théorie des graphes
- Vocabulaire
- Modélisation et complexité

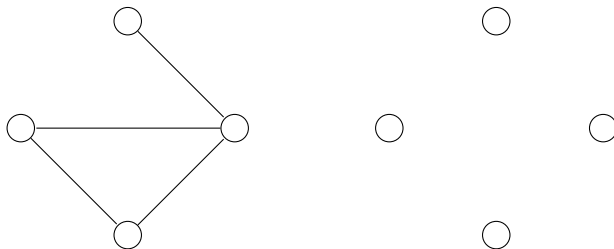
Qu'est-ce que c'est ?

Des *sommets* (ou *nœuds*)



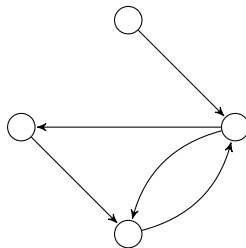
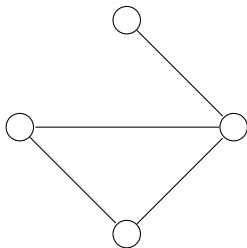
Qu'est-ce que c'est ?

Des *sommets* (ou *nœuds*) reliés par des *arêtes*



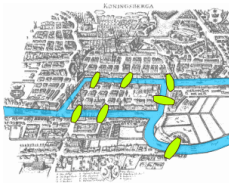
Qu'est-ce que c'est ?

Des *sommets* (ou *nœuds*) reliés par des *arêtes* ou par des *arcs*/*flèches*



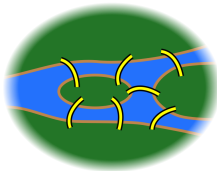
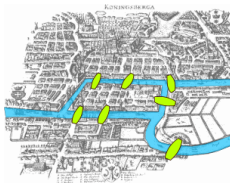
Et alors ?

Problème des sept ponts de Königsberg



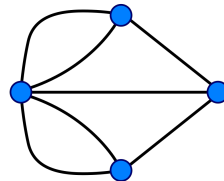
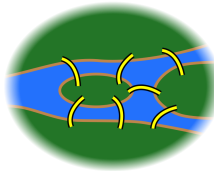
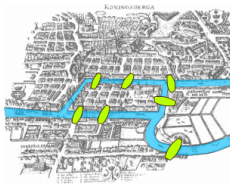
Et alors ?

Problème des sept ponts de Königsberg



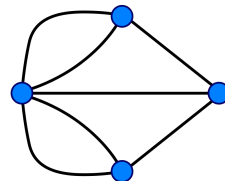
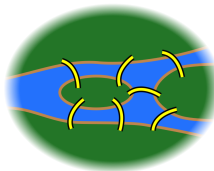
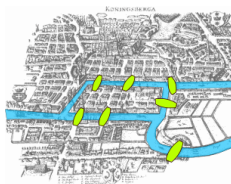
Et alors ?

Problème des sept ponts de Königsberg

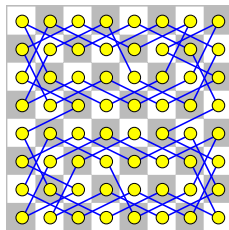


Et alors ?

Problème des sept ponts de Königsberg

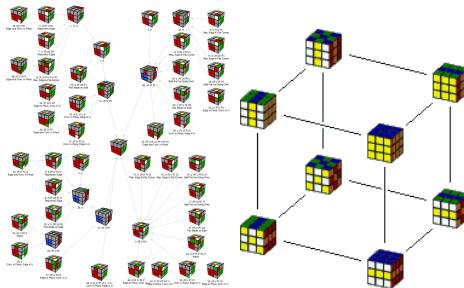
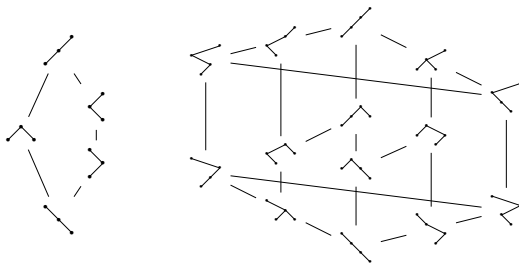


Problème du cavalier

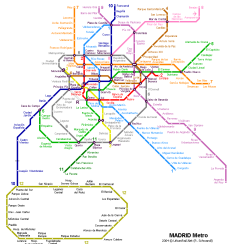


(ex application)

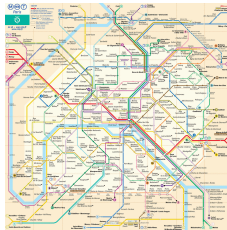
Mais encore ?



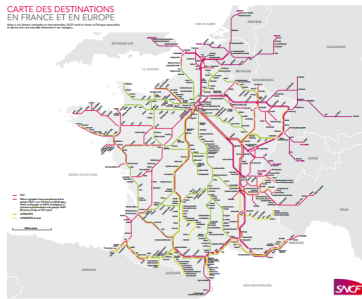
Que des maths ?



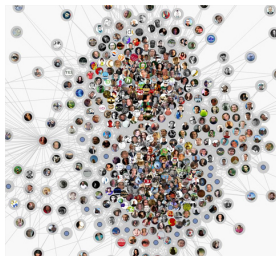
МОСКОВСКИЙ МЕТРОПОЛИТЕН • 2010 • MOSCOW METRO



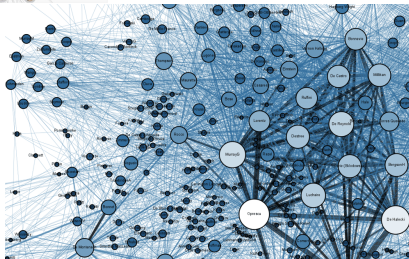
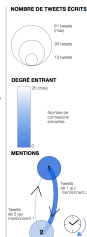
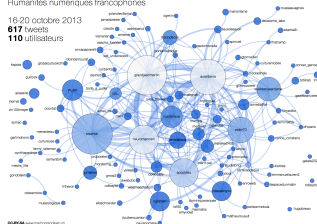
CARTE DES DESTINATIONS
EN FRANCE ET EN EUROPE



Que des maths ? - suite



#THATCampStMalo
Humanités numériques francophones
16-20 octobre 2013
617 tweets
110 utilisateurs



Pour résumer

- relations entre objets mathématiques
- configurations d'un objet
- interconnexions ferroviaires/routières/maritimes
- Plan d'une ville avec rue sens unique
- Liens entre composants électroniques
- Liens entre réseaux de personnes

Dans la vie du programmeur

- Modélisation de réseaux, régulation de flux

Dans la vie du programmeur

- Modélisation de réseaux, régulation de flux
- Data visualisation et machine learning (*random forest, gradient boosting*)

Dans la vie du programmeur

- Modélisation de réseaux, régulation de flux
- Data visualisation et machine learning (*random forest, gradient boosting*)
- API :  GraphQL

Dans la vie du programmeur

- Modélisation de réseaux, régulation de flux
- Data visualisation et machine learning (*random forest, gradient boosting*)
- API :  GraphQL
- No SQL : Graph Databases

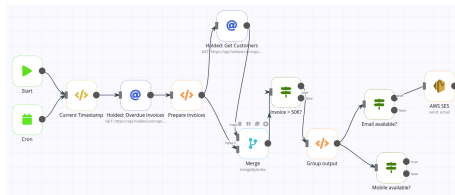


Dans la vie du programmeur

- Modélisation de réseaux, régulation de flux
- Data visualisation et machine learning (*random forest, gradient boosting*)
- API :  GraphQL
- No SQL : Graph Databases



- Plateforme d'automatisation de workflow :  n8n



Plan

1 Définitions

- Vulgarisation
- Exemples de problèmes classiques en théorie des graphes
- Vocabulaire
- Modélisation et complexité

Problèmes classiques

- Parcours et énumération
- Problèmes de distance
- Cycle Eulérien, Cycle Hamiltonien
- Voyageur de commerce, postier chinois
- Problème de flots, coupe minimum
- Couverture par arêtes, couverture par sommets
- Arbre couvrants minimaux
- Ordonnancement, planification
- Recherche de clique maximale
- Nombre chromatique et coloriage de sommets/d'arêtes

Beaucoup de **problèmes NP-complets** ! (cf. cours Recherche Opérationnelle)

Parcours et énumération

Parcours et énumération

Objectif : lister tous les sommets, dans un certain ordre, une et une seule fois

Parcours et énumération

Objectif : lister tous les sommets, dans un certain ordre, une et une seule fois

Algorithmes : parcours en largeur, en profondeur

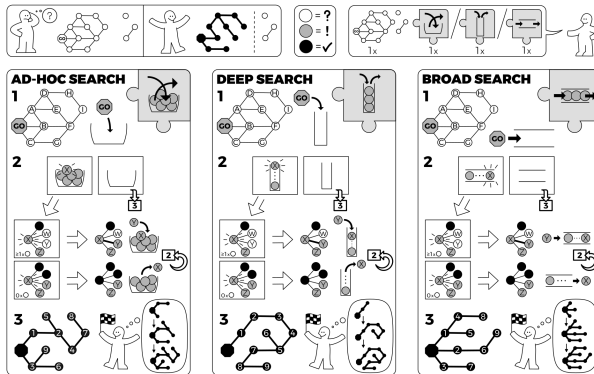
Parcours et énumération

Objectif : lister tous les sommets, dans un certain ordre, une et une seule fois

Algorithmes : parcours en largeur, en profondeur

GRAPH SCÂN

idea-instructions.com/graph-scan/
v1.1, CC by-nc-sa 4.0



Problème de distance



Aller le plus rapidement possible d'un point A à un point B :

Problème de distance



Aller le plus rapidement possible d'un point A à un point B :

- En faisant le moins de correspondance possible

Problème de distance



Aller le plus rapidement possible d'un point A à un point B :

- En faisant le moins de correspondance possible
- En passant le moins de temps possible dans le train (on néglige les correspondances)

Problème de distance



Aller le plus rapidement possible d'un point A à un point B :

- En faisant le moins de correspondance possible : *graphe non valué*
- En passant le moins de temps possible dans le train (on néglige les correspondances)

Problème de distance



Aller le plus rapidement possible d'un point A à un point B :

- En faisant le moins de correspondance possible : *graphe non valué*
- En passant le moins de temps possible dans le train (on néglige les correspondances) : *graphe valué*

Problème de distance



Aller le plus rapidement possible d'un point A à un point B :

- En faisant le moins de correspondance possible : *graphe non valué*
- En passant le moins de temps possible dans le train (on néglige les correspondances) : *graphe valué*

Variantes :

Problème de distance



Aller le plus rapidement possible d'un point A à un point B :

- En faisant le moins de correspondance possible : *graphe non valué*
- En passant le moins de temps possible dans le train (on néglige les correspondances) : *graphe valué*

Variantes :

- Ne pas négliger correspondances

Problème de distance



Aller le plus rapidement possible d'un point A à un point B :

- En faisant le moins de correspondance possible : *graphe non valué*
- En passant le moins de temps possible dans le train (on néglige les correspondances) : *graphe valué*

Variantes :

- Ne pas négliger correspondances
- Distance négative et positive

Problème de distance



Aller le plus rapidement possible d'un point A à un point B :

- En faisant le moins de correspondance possible : *graphe non valué*
- En passant le moins de temps possible dans le train (on néglige les correspondances) : *graphe valué*

Variantes :

- Ne pas négliger correspondances
- Distance négative et positive

Algorithmes

Problème de distance



Aller le plus rapidement possible d'un point A à un point B :

- En faisant le moins de correspondance possible : *graphe non valué*
- En passant le moins de temps possible dans le train (on néglige les correspondances) : *graphe valué*

Variantes :

- Ne pas négliger correspondances
- Distance négative et positive

Algorithmes

- Parcours en largeur (non valué)

Problème de distance



Aller le plus rapidement possible d'un point A à un point B :

- En faisant le moins de correspondance possible : *graphe non valué*
- En passant le moins de temps possible dans le train (on néglige les correspondances) : *graphe valué*

Variantes :

- Ne pas négliger correspondances
- Distance négative et positive

Algorithmes

- Parcours en largeur (non valué)
- Entre deux points : Dijkstra, Bellman-Ford, A^*

Problème de distance



Aller le plus rapidement possible d'un point A à un point B :

- En faisant le moins de correspondance possible : *graphe non valué*
- En passant le moins de temps possible dans le train (on néglige les correspondances) : *graphe valué*

Variantes :

- Ne pas négliger correspondances
- Distance négative et positive

Algorithmes

- Parcours en largeur (non valué)
- Entre deux points : Dijkstra, Bellman-Ford, A^*
- Toutes paires : Floyd-Warshall, Johnson

Cycle Eulérien et Hamiltonien

Cycle Eulérien et Hamiltonien

Cycle Eulérien

Cycle Eulérien et Hamiltonien

Cycle Eulérien

Passer par toutes les **arêtes** une et une seule fois avec un cycle

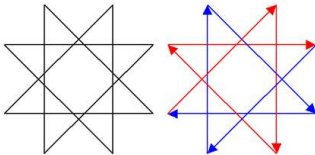
Dessiner sans lever le crayon et revenir point départ

Cycle Eulérien et Hamiltonien

Cycle Eulérien

Passer par toutes les **arêtes** une et une seule fois avec un cycle

Dessiner sans lever le crayon et revenir point départ

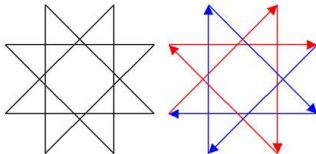


Cycle Eulérien et Hamiltonien

Cycle Eulérien

Passer par toutes les **arêtes** une et une seule fois avec un cycle

Dessiner sans lever le crayon et revenir point départ



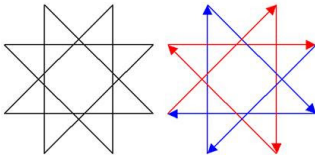
Cycle Hamiltonien

Cycle Eulérien et Hamiltonien

Cycle Eulérien

Passer par toutes les **arêtes** une et une seule fois avec un cycle

Dessiner sans lever le crayon et revenir point départ



Cycle Hamiltonien

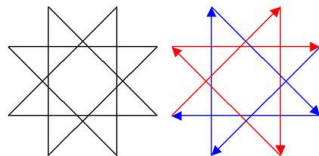
Passer par tous les **sommets** une et une seule fois avec un cycle

Cycle Eulérien et Hamiltonien

Cycle Eulérien

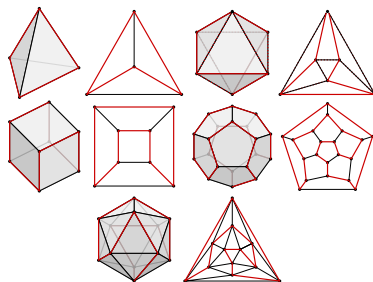
Passer par toutes les **arêtes** une et une seule fois avec un cycle

Dessiner sans lever le crayon et revenir point départ



Cycle Hamiltonien

Passer par tous les **sommets** une et une seule fois avec un cycle

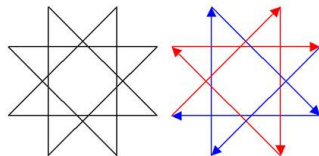


Cycle Eulérien et Hamiltonien

Cycle Eulérien

Passer par toutes les **arêtes** une et une seule fois avec un cycle

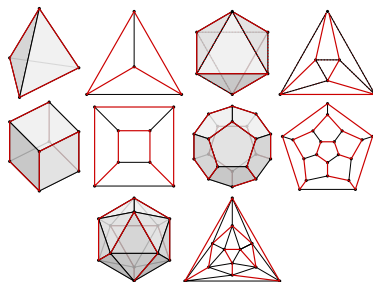
Dessiner sans lever le crayon et revenir point départ



- Problème polynomial

Cycle Hamiltonien

Passer par tous les **sommets** une et une seule fois avec un cycle

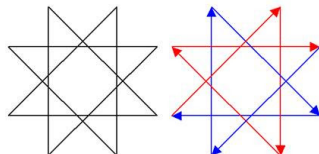


Cycle Eulérien et Hamiltonien

Cycle Eulérien

Passer par toutes les **arêtes** une et une seule fois avec un cycle

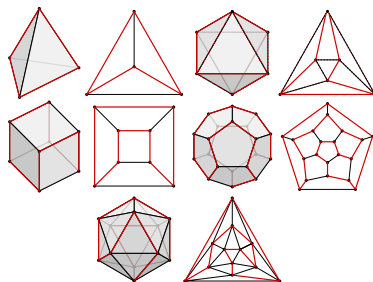
Dessiner sans lever le crayon et revenir point départ



- Problème polynomial
- **Thm** : un graphe admet un cycle eulérien ssi tous ses sommets sont de degré pair

Cycle Hamiltonien

Passer par tous les **sommets** une et une seule fois avec un cycle

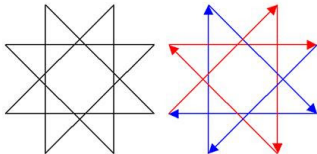


Cycle Eulérien et Hamiltonien

Cycle Eulérien

Passer par toutes les **arêtes** une et une seule fois avec un cycle

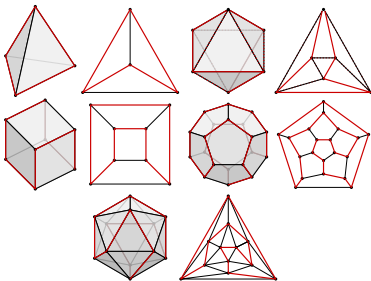
Dessiner sans lever le crayon et revenir point départ



- Problème polynomial
- **Thm** : un graphe admet un cycle eulérien ssi tous ses sommets sont de degré pair
- Algorithme de Fleury, de Hierholzer (linéaire)

Cycle Hamiltonien

Passer par tous les **sommets** une et une seule fois avec un cycle

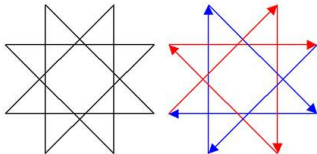


Cycle Eulérien et Hamiltonien

Cycle Eulérien

Passer par toutes les **arêtes** une et une seule fois avec un cycle

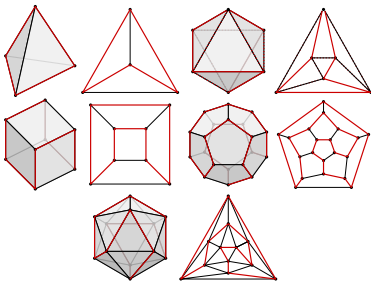
Dessiner sans lever le crayon et revenir point départ



- Problème polynomial
- **Thm** : un graphe admet un cycle eulérien ssi tous ses sommets sont de degré pair
- Algorithme de Fleury, de Hierholzer (linéaire)

Cycle Hamiltonien

Passer par tous les **sommets** une et une seule fois avec un cycle



Problème NP complet !

Voyageurs de commerce, postier chinois

Voyageurs de commerce, postier chinois

Postier chinois

Voyageurs de commerce, postier chinois

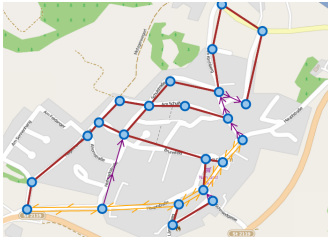
Postier chinois

*Passer par toutes les **arêtes** au moins une fois et minimiser distance.*

Voyageurs de commerce, postier chinois

Postier chinois

*Passer par toutes les **arêtes** au moins une fois et minimiser distance.*

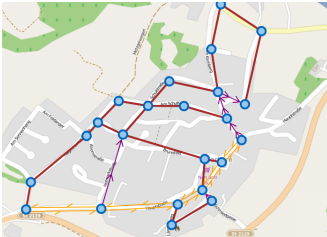


Voyageurs de commerce, postier chinois

Voyageur de commerce

Postier chinois

*Passer par toutes les **arêtes** au moins une fois et minimiser distance.*



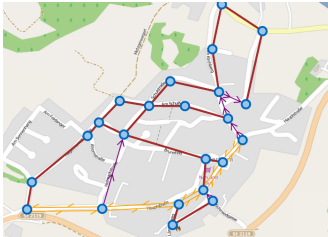
Voyageurs de commerce, postier chinois

Voyageur de commerce

Passer par tous les **sommets** exactement une fois et minimiser distance.

Postier chinois

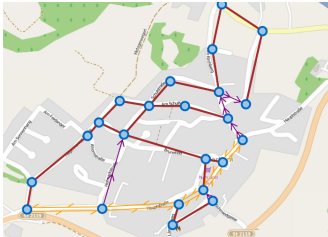
*Passer par toutes les **arêtes** au moins une fois et minimiser distance.*



Voyageurs de commerce, postier chinois

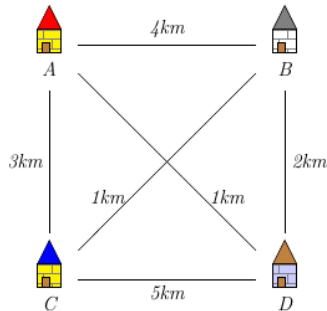
Postier chinois

Passer par toutes les **arêtes** au moins une fois et minimiser distance.



Voyageur de commerce

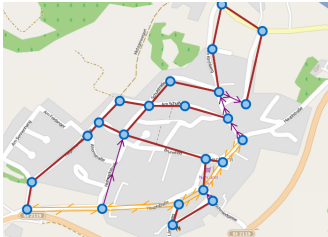
Passer par tous les **sommets** exactement une fois et minimiser distance.



Voyageurs de commerce, postier chinois

Postier chinois

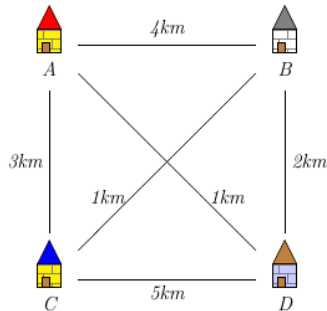
Passer par toutes les **arêtes** au moins une fois et minimiser distance.



- Si graphe eulérien, donne une solution

Voyageur de commerce

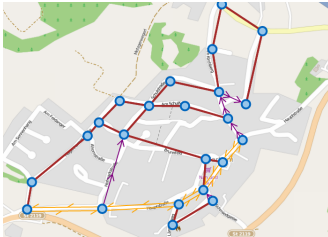
Passer par tous les **sommets** exactement une fois et minimiser distance.



Voyageurs de commerce, postier chinois

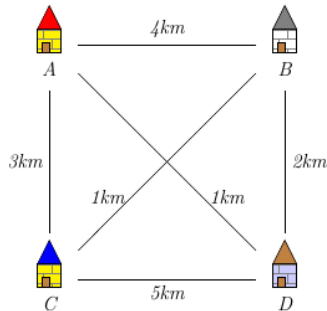
Postier chinois

Passer par toutes les **arêtes** au moins une fois et minimiser distance.



Voyageur de commerce

Passer par tous les **sommets** exactement une fois et minimiser distance.

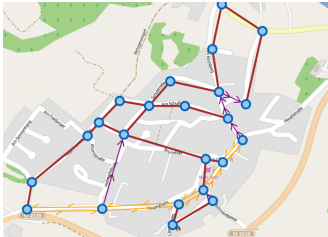


- Si graphe eulérien, donne une solution
- Polynomial en général

Voyageurs de commerce, postier chinois

Postier chinois

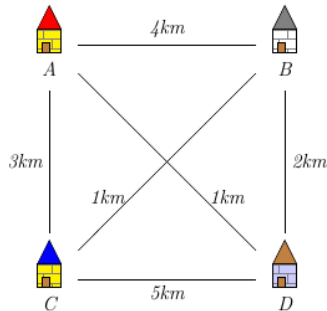
Passer par toutes les **arêtes** au moins une fois et minimiser distance.



- Si graphe eulérien, donne une solution
- Polynomial en général

Voyageur de commerce

Passer par tous les **sommets** exactement une fois et minimiser distance.



Problème NP complet !

Flots, min-cut



Flots, min-cut



- **Contexte** : sur un graphe on fait circuler quelque chose (eau, électricité, embouteillage...)

Flots, min-cut



- **Contexte** : sur un graphe on fait circuler quelque chose (eau, électricité, embouteillage...)
- **Problème de flot** : trouver le flot maximal

Flots, min-cut



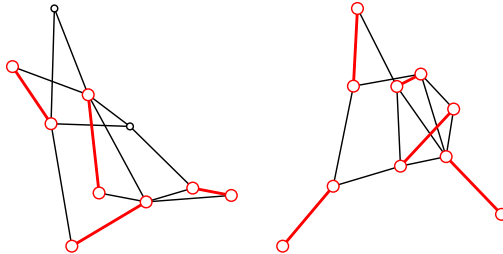
- **Contexte** : sur un graphe on fait circuler quelque chose (eau, électricité, embouteillage...)
- **Problème de flot** : trouver le flot maximal
- **Min-Cut** : où se trouve la saturation

Flots, min-cut

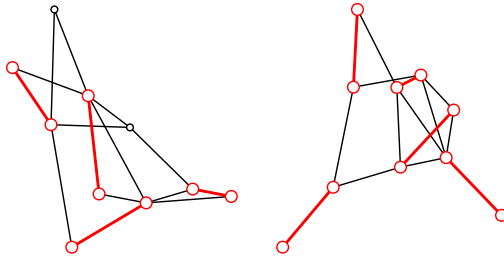


- **Contexte** : sur un graphe on fait circuler quelque chose (eau, électricité, embouteillage...)
- **Problème de flot** : trouver le flot maximal
- **Min-Cut** : où se trouve la saturation
- **Algo** : Ford-Fulkerson

Couplage

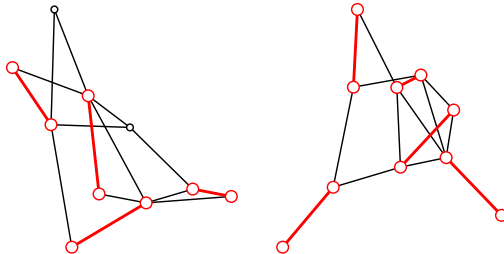


Couplage



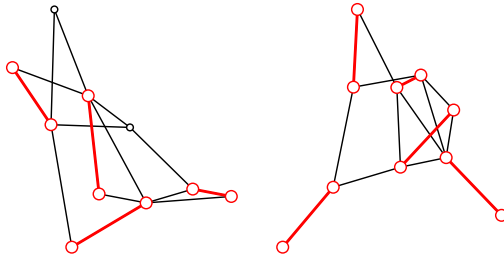
- **Couplage maximal** : ensemble d'arêtes maximal tel que tout sommet incident à au plus une des arêtes

Couplage



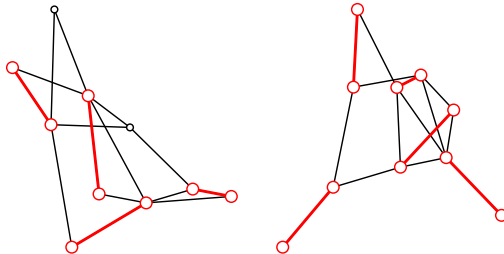
- **Couplage maximal** : ensemble d'arêtes maximal tel que tout sommet incident à au plus une des arêtes
- Problème **polynomial** : algorithme d'Edmonds (algo. des fleurs et des pétales)

Couplage



- **Couplage maximal** : ensemble d'arêtes maximal tel que tout sommet incident à au plus une des arêtes
- Problème **polynomial** : algorithme d'Edmonds (algo. des fleurs et des pétales)
 - ▶ Cas particulier couplage bipartite : on ramène à pb de flots

Couplage



- **Couplage maximal** : ensemble d'arêtes maximal tel que tout sommet incident à au plus une des arêtes
- Problème **polynomial** : algorithme d'Edmonds (algo. des fleurs et des pétales)
 - ▶ Cas particulier couplage bipartite : on ramène à pb de flots
 - ▶ Si notion de préférence, problème des mariages stables, algo de Gale et Shapley (utilisé par Admission Post-Bac)

Couverture par sommets et arêtes

Couverture par sommets et arêtes

Couverture par arêtes

Couverture par sommets et arêtes

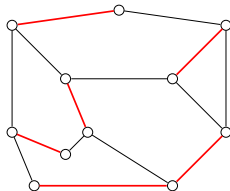
Couverture par arêtes

Colorier les **arêtes** pour que chaque **sommet** ait une arête adjacente colorée

Couverture par sommets et arêtes

Couverture par arêtes

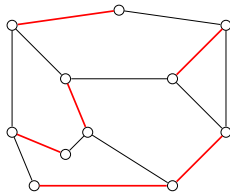
Colorier les **arêtes** pour que chaque **sommet** ait une arête adjacente colorée



Couverture par sommets et arêtes

Couverture par arêtes

Colorier les **arêtes** pour que chaque **sommet** ait une arête adjacente colorée

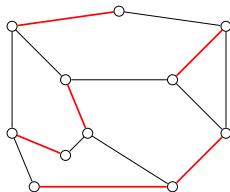


Couverture par sommets

Couverture par sommets et arêtes

Couverture par arêtes

Colorier les **arêtes** pour que chaque **sommet** ait une arête adjacente coloriée

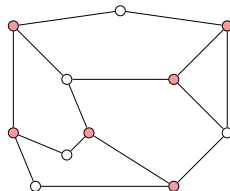


Couverture par sommets

Colorier les **sommets** pour que chaque **arête** ait un sommet adjacent colorié

Couverture par arêtes

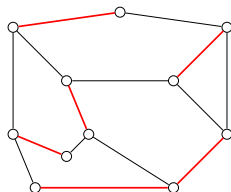
Couverture par sommets



Couverture par sommets et arêtes

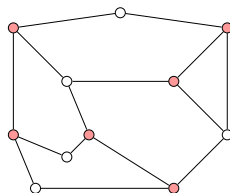
Couverture par arêtes

Colorier les **arêtes** pour que chaque **sommet** ait une arête adjacente coloriée



Couverture par sommets

Colorier les **sommets** pour que chaque **arête** ait un sommet adjacent colorié

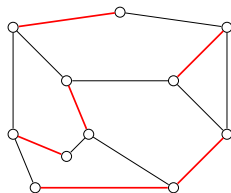


- Problème polynomial

Couverture par sommets et arêtes

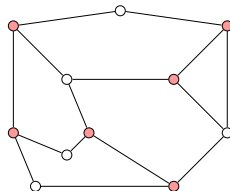
Couverture par arêtes

Colorier les **arêtes** pour que chaque **sommet** ait une arête adjacente colorée



Couverture par sommets

Colorier les **sommets** pour que chaque **arête** ait un sommet adjacent coloré

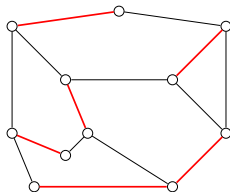


- Problème polynomial
- Algo : commencer par couplage maximal, et compléter en glouton par sommet non atteint

Couverture par sommets et arêtes

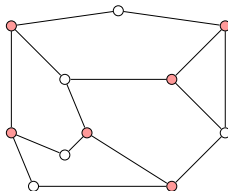
Couverture par arêtes

Colorier les **arêtes** pour que chaque **sommet** ait une arête adjacente colorée



Couverture par sommets

Colorier les **sommets** pour que chaque **arête** ait un sommet adjacent coloré



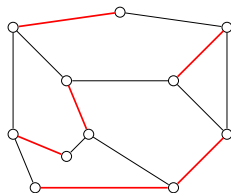
Problème NP complet !

- Problème polynomial
- Algo : commencer par couplage maximal, et compléter en glouton par sommet non atteint

Couverture par sommets et arêtes

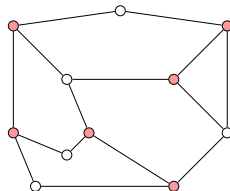
Couverture par arêtes

Colorier les **arêtes** pour que chaque **sommet** ait une arête adjacente colorée



Couverture par sommets

Colorier les **sommets** pour que chaque **arête** ait un sommet adjacent coloré

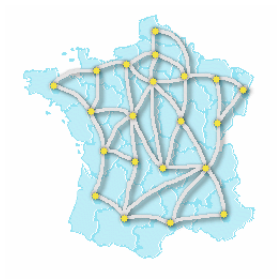


Problème NP complet !

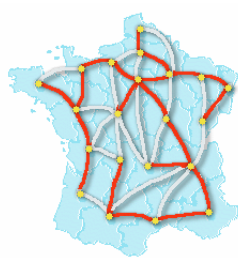
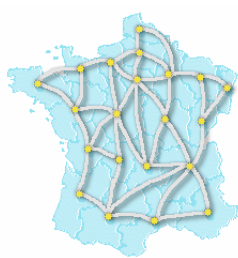
Problème lié à ceux-ci : [problème de la galerie d'art](#)

- Problème polynomial
- Algo : commencer par couplage maximal, et compléter en glouton par sommet non atteint

Arbre couvrant minimal

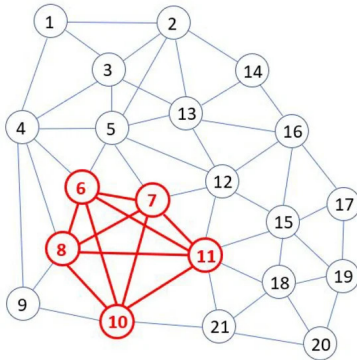


Arbre couvrant minimal



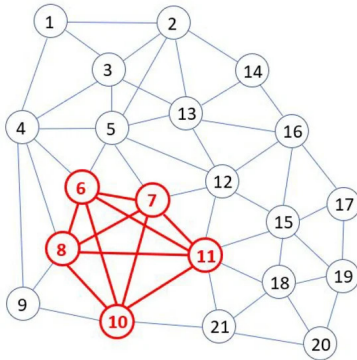
Minimiser le nombre de lignes créées mais qu'on puisse aller de tout sommet à tout sommet

Recherche de clique maximale



Clique : sous-graphe complet

Recherche de clique maximale

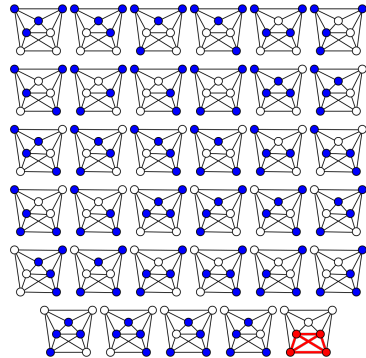
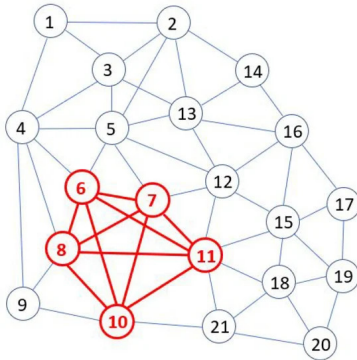


Clique : sous-graphe complet

Plusieurs problèmes :

- Est-ce qu'il existe une clique de taille supérieure à une taille donnée
- Liste de toutes les cliques maximums

Recherche de clique maximale

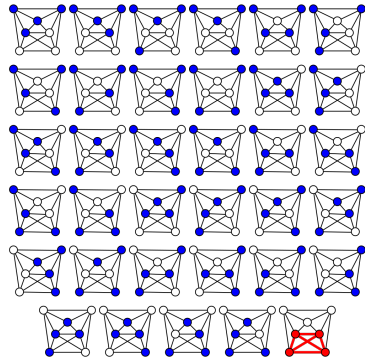
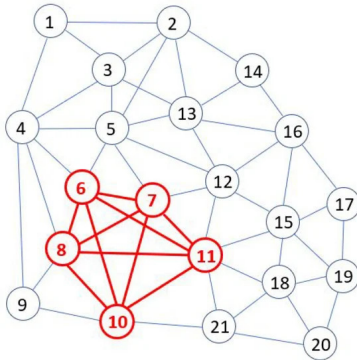


Clique : sous-graphe complet

Plusieurs problèmes :

- Est-ce qu'il existe une clique de taille supérieure à une taille donnée
- Liste de toutes les cliques maximums

Recherche de clique maximale

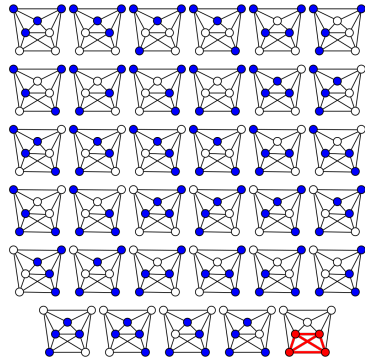
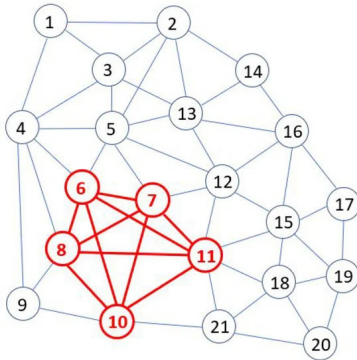


Clique : sous-graphe complet

Plusieurs problèmes :

- Est-ce qu'il existe une clique de taille supérieure à une taille donnée : **NP-complet**
- Liste de toutes les cliques maximums

Recherche de clique maximale

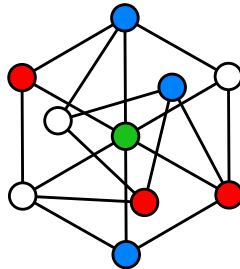
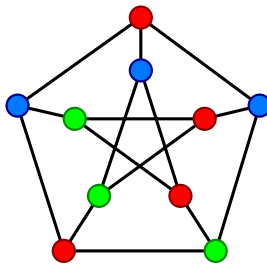


Clique : sous-graphe complet

Plusieurs problèmes :

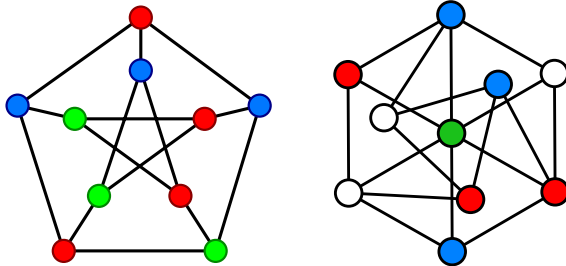
- Est-ce qu'il existe une clique de taille supérieure à une taille donnée : **NP-complet**
- Liste de toutes les cliques maximums : **EXP**

Nombre chromatique et coloriage de sommets/d'arêtes



Trouver un coloriage du graphe où deux sommets voisins n'ont pas la même couleur.

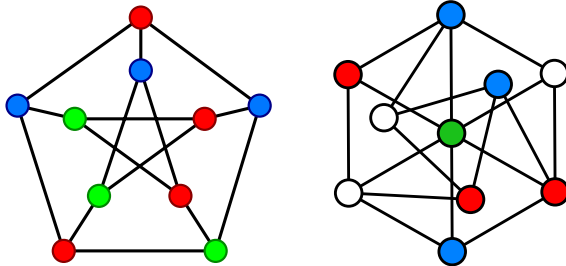
Nombre chromatique et coloriage de sommets/d'arêtes



Trouver un coloriage du graphe où deux sommets voisins n'ont pas la même couleur.

Plus petit nombre : **nombre chromatique**

Nombre chromatique et coloriage de sommets/d'arêtes

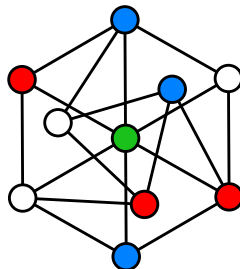
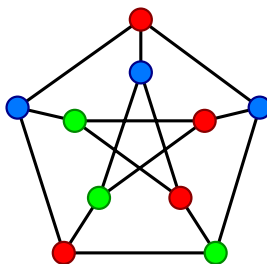


Trouver un coloriage du graphe où deux sommets voisins n'ont pas la même couleur.

Plus petit nombre : **nombre chromatique**

- Problème de décision : peut-on colorier un graphe donné avec k couleurs ?

Nombre chromatique et coloriage de sommets/d'arêtes

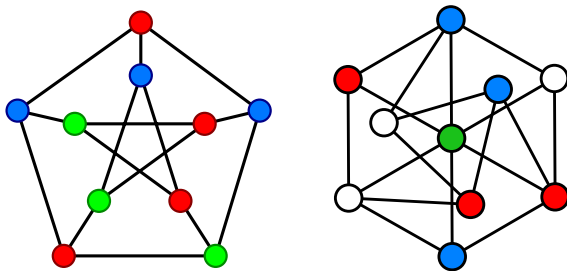


Trouver un coloriage du graphe où deux sommets voisins n'ont pas la même couleur.

Plus petit nombre : **nombre chromatique**

- Problème de décision : peut-on colorier un graphe donné avec k couleurs ?
- Problème d'optimisation : quel est le nombre chromatique ?

Nombre chromatique et coloriage de sommets/d'arêtes

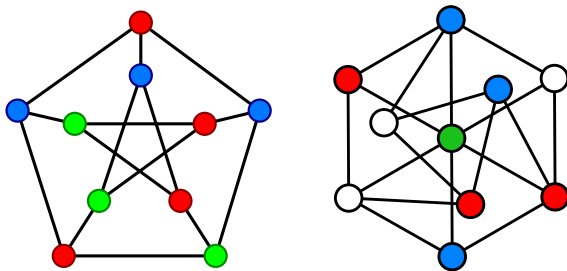


Trouver un coloriage du graphe où deux sommets voisins n'ont pas la même couleur.

Plus petit nombre : **nombre chromatique**

- Problème de décision : peut-on colorier un graphe donné avec k couleurs ? **NP-complet**
- Problème d'optimisation : quel est le nombre chromatique ?

Nombre chromatique et coloriage de sommets/d'arêtes



Trouver un coloriage du graphe où deux sommets voisins n'ont pas la même couleur.

Plus petit nombre : **nombre chromatique**

- Problème de décision : peut-on colorier un graphe donné avec k couleurs ? **NP-complet**
- Problème d'optimisation : quel est le nombre chromatique ? **NP-difficile**

Plan

1 Définitions

- Vulgarisation
- Exemples de problèmes classiques en théorie des graphes
- **Vocabulaire**
- Modélisation et complexité

Graphe

Définition (Graphe)

Un *graphe* G est un couple (V, E) où

- V est un ensemble fini d'objets. Ses éléments sont les *sommets* (ou *nœuds*) du graphe.
- $E \subseteq V \times V$. Ses éléments sont les *arêtes* du graphe.

Graphe

Définition (Graphe)

Un *graphe* G est un couple (V, E) où

- V est un ensemble fini d'objets. Ses éléments sont les *sommets* (ou *nœuds*) du graphe.
- $E \subseteq V \times V$. Ses éléments sont les *arêtes* du graphe.

Une arête e du graphe est une paire $e = (x, y)$ de sommets, avec x et y les *extrémités* de l'arête.

Graphe

Définition (Graphe)

Un *graphe* G est un couple (V, E) où

- V est un ensemble fini d'objets. Ses éléments sont les *sommets* (ou *nœuds*) du graphe.
- $E \subseteq V \times V$. Ses éléments sont les *arêtes* du graphe.

Une arête e du graphe est une paire $e = (x, y)$ de sommets, avec x et y les *extrémités* de l'arête.

Pour un *graphe orienté*, un élément $e = (x, y) \in E$ est un *arc*, x s'appelle la *source* et y la *destination*.

Graphe

Définition (Graphe)

Un **graphe** G est un couple (V, E) où

- V est un ensemble fini d'objets. Ses éléments sont les **sommets** (ou **nœuds**) du graphe.
- $E \subseteq V \times V$. Ses éléments sont les **arêtes** du graphe.

Une arête e du graphe est une paire $e = (x, y)$ de sommets, avec x et y les **extrémités** de l'arête.

Pour un **graphe orienté**, un élément $e = (x, y) \in E$ est un **arc**, x s'appelle la **source** et y la **destination**.

Dans ce cours

Graphes **simples** : pas d'arête **multiple** ni de **boucles**

Représentation - non orienté

$G = (V, E)$ avec

$V = \{a, b, c, d, e, f, g, h\}$

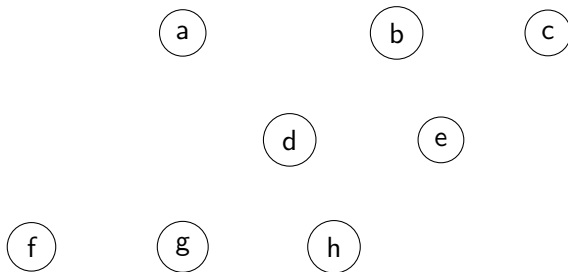
$E = \{(a, d), (b, c), (b, d), (d, e), (e, c), (e, h), (h, d), (f, g), (d, g), (g, h)\}$

Représentation - non orienté

$G = (V, E)$ avec

$V = \{a, b, c, d, e, f, g, h\}$

$E = \{(a, d), (b, c), (b, d), (d, e), (e, c), (e, h), (h, d), (f, g), (d, g), (g, h)\}$

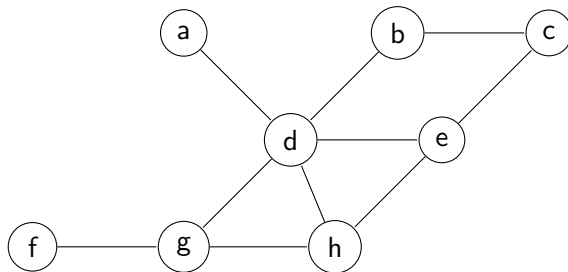


Représentation - non orienté

$G = (V, E)$ avec

$V = \{a, b, c, d, e, f, g, h\}$

$E = \{(a, d), (b, c), (b, d), (d, e), (e, c), (e, h), (h, d), (f, g), (d, g), (g, h)\}$

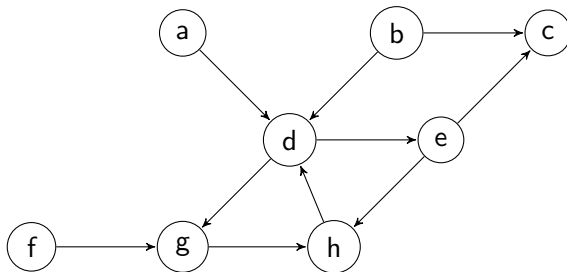


Représentation - orienté

$G = (V, E)$ avec

$V = \{a, b, c, d, e, f, g, h\}$

$E = \{(a, d), (b, c), (b, d), (d, e), (e, c), (e, h), (h, d), (f, g), (d, g), (g, h)\}$

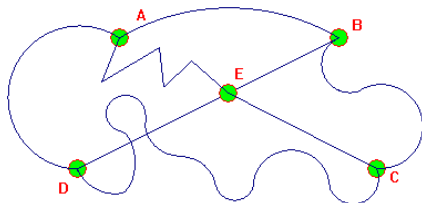
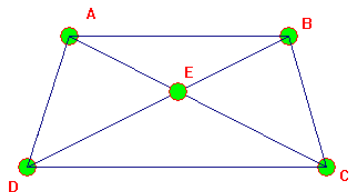


Représentation d'un graphe

On peut tordre les arêtes pour le dessin :

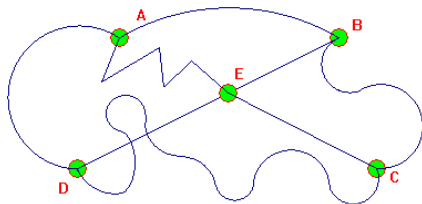
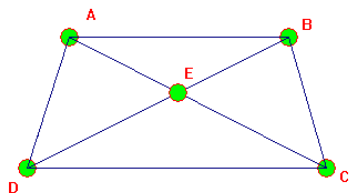
Représentation d'un graphe

On peut tordre les arêtes pour le dessin :

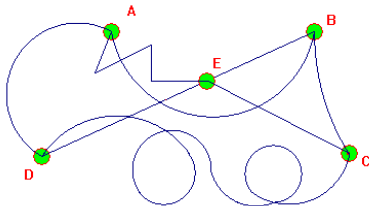


Représentation d'un graphe

On peut tordre les arêtes pour le dessin :



Pas trop non plus *(les arêtes ne se croisent pas elles-mêmes, ici DC)* :



Voisin et degré - non orienté

Soit $G = (V, E)$ un graphe

Définition (Voisin)

Si $e = (x, y) \in E$, x et y sont dit *voisins* ou *adjacents*.

Voisin et degré - non orienté

Soit $G = (V, E)$ un graphe

Définition (Voisin)

Si $e = (x, y) \in E$, x et y sont dit *voisins* ou *adjacents*.

Définition (Degré)

Le *degré* d'un sommet $v \in V$, noté $d(v)$, est son nombre de voisins.

Voisin et degré - non orienté

Soit $G = (V, E)$ un graphe

Définition (Voisin)

Si $e = (x, y) \in E$, x et y sont dit *voisins* ou *adjacents*.

Définition (Degré)

Le *degré* d'un sommet $v \in V$, noté $d(v)$, est son nombre de voisins.

Proposition

La somme des degrés des sommets d'un graphe est égal à 2 fois son nombre d'arêtes.

Voisin et degré - non orienté

Soit $G = (V, E)$ un graphe

Définition (Voisin)

Si $e = (x, y) \in E$, x et y sont dit *voisins* ou *adjacents*.

Définition (Degré)

Le *degré* d'un sommet $v \in V$, noté $d(v)$, est son nombre de voisins.

Proposition

La somme des degrés des sommets d'un graphe est égal à 2 fois son nombre d'arêtes.

Corollaire

Le nombre de sommets de degré impair d'un graphe est pair.

Voisin et degré - orienté

Soit $G = (V, E)$ un graphe **orienté**

Définition (Voisin)

Si $e = (x, y) \in E$, x est dit *prédécesseur* de y (ou *dominant* y) et y est *successeur* de x (ou *dominé* par x).

Voisin et degré - orienté

Soit $G = (V, E)$ un graphe **orienté**

Définition (Voisin)

Si $e = (x, y) \in E$, x est y dit *prédécesseur* de y (ou *dominant* y) et y est *successeur* de x (ou *dominé* par x).

Définition (Degré)

Le *degré sortant* d'un sommet $v \in V$, noté $d^+(v)$, est son nombre de successeurs.

Voisin et degré - orienté

Soit $G = (V, E)$ un graphe **orienté**

Définition (Voisin)

Si $e = (x, y) \in E$, x est y dit *prédécesseur* de y (ou *dominant* y) et y est *successeur* de x (ou *dominé* par x).

Définition (Degré)

Le *degré sortant* d'un sommet $v \in V$, noté $d^+(v)$, est son nombre de successeurs.

Le *degré entrant* d'un sommet $v \in V$, noté $d^-(v)$, est son nombre de prédécesseurs.

Graphe complet

Définition

*Un graphe est **complet** si tous ses sommets sont adjacents les uns aux autres.*

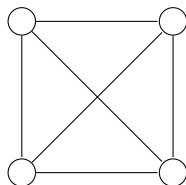
On note K_n le graphe complet à n sommets.

Graphe complet

Définition

Un graphe est **complet** si tous ses sommets sont adjacents les uns aux autres.

On note K_n le graphe complet à n sommets.



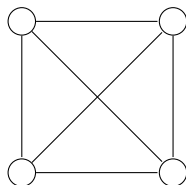
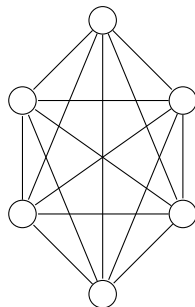
K_4

Graphe complet

Définition

Un graphe est **complet** si tous ses sommets sont adjacents les uns aux autres.

On note K_n le graphe complet à n sommets.

 K_4  K_6

Graphe bipartite

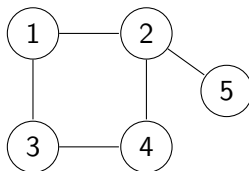
Définition

Un graphe $G = (V, E)$ est *bipartite* si V peut être partitionné en deux sous-ensembles V_1 et V_2 (i.e. $V = V_1 \cup V_2$, $V_1 \cap V_2 = \emptyset$) de sorte que E ne contient aucun arc entre deux sommets de V_1 , ni entre deux sommets de V_2 .

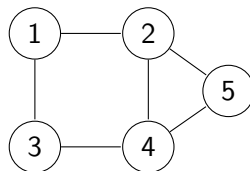
Graphe bipartite

Définition

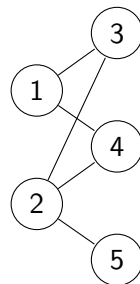
Un graphe $G = (V, E)$ est *bipartite* si V peut être partitionné en deux sous-ensembles V_1 et V_2 (i.e. $V = V_1 \cup V_2$, $V_1 \cap V_2 = \emptyset$) de sorte que E ne contient aucun arc entre deux sommets de V_1 , ni entre deux sommets de V_2 .



Bipartite



Non bipartite

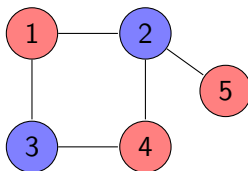


Bipartite

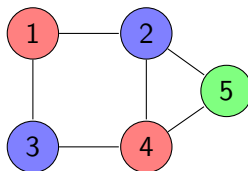
Graphe bipartite

Définition

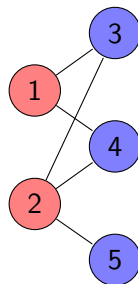
Un graphe $G = (V, E)$ est *bipartite* si V peut être partitionné en deux sous-ensembles V_1 et V_2 (i.e. $V = V_1 \cup V_2$, $V_1 \cap V_2 = \emptyset$) de sorte que E ne contient aucun arc entre deux sommets de V_1 , ni entre deux sommets de V_2 .



Bipartite



Non bipartite



Bipartite

Graphe planaire

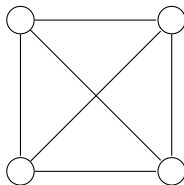
Définition

*Un graphe est dit **planaire** si on peut le représenter sans qu'aucune arête ne se croise.*

Graphe planaire

Définition

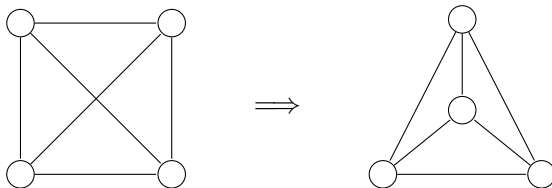
Un graphe est dit *planaire* si on peut le représenter sans qu'aucune arête ne se croise.



Graphe planaire

Définition

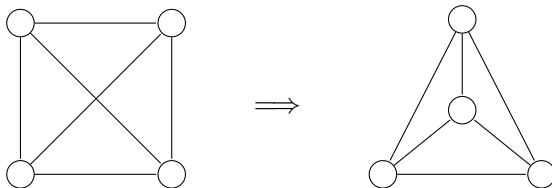
Un graphe est dit *planaire* si on peut le représenter sans qu'aucune arête ne se croise.



Graphe planaire

Définition

Un graphe est dit *planaire* si on peut le représenter sans qu'aucune arête ne se croise.

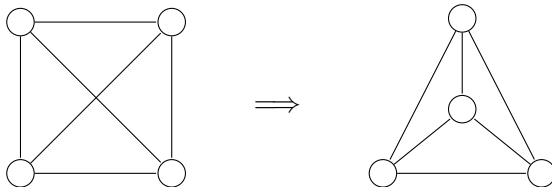


K_4 est planaire

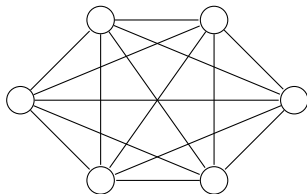
Graphe planaire

Définition

Un graphe est dit *planaire* si on peut le représenter sans qu'aucune arête ne se croise.



K_4 est planaire



K_6 non planaire

Sous-graphe et graphe partiel

Soit $G = (V, E)$ un graphe.

Sous-graphe et graphe partiel

Soit $G = (V, E)$ un graphe.

Définition (Sous-graphe)

Un *sous-graphe* de G est un graphe $H = (W, E(W))$ avec $W \subseteq V$, et

$$E(W) = \{e = (x, y) \in E \mid x, y \in W\}.$$

Sous-graphe et graphe partiel

Soit $G = (V, E)$ un graphe.

Définition (Sous-graphe)

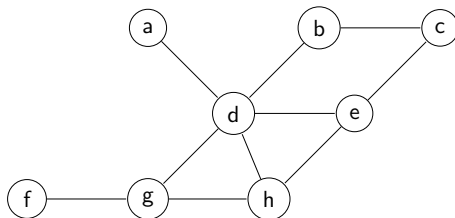
Un *sous-graphe* de G est un graphe $H = (W, E(W))$ avec $W \subseteq V$, et

$$E(W) = \{e = (x, y) \in E \mid x, y \in W\}.$$

Définition (Graphe partiel)

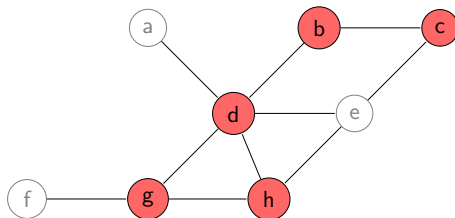
Un *graphe partiel* de G est un graphe $H = (V, F)$ avec $F \subseteq E$.

Sous-graphe et graphe partiel - exemple



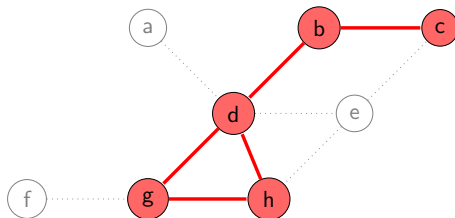
Exemple de sous-graphe

Sous-graphe et graphe partiel - exemple



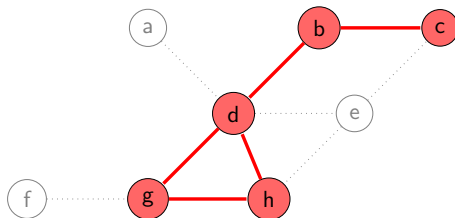
Exemple de sous-graphe

Sous-graphe et graphe partiel - exemple

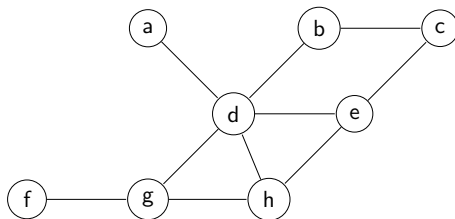


Exemple de sous-graphe

Sous-graphe et graphe partiel - exemple

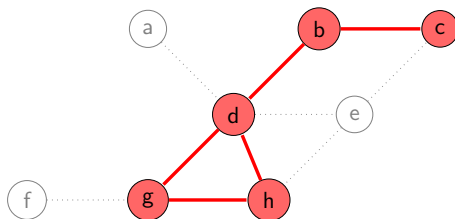


Exemple de sous-graphe

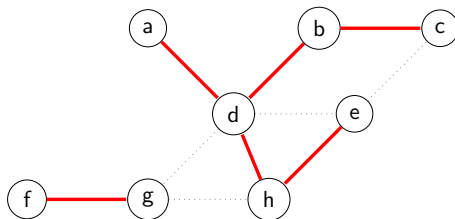


Exemple de graphe partiel

Sous-graphe et graphe partiel - exemple



Exemple de sous-graphe



Exemple de graphe partiel

Chaîne et chemin

Définition (Chaîne)

Une *chaîne* est une liste $p = (x_1, \dots, x_k)$ de sommets telle que deux sommets consécutifs sont voisins :

$$\forall i \leq k - 1, \quad (x_i, x_{i+1}) \in E$$

Chaîne et chemin

Définition (Chaîne)

Une *chaîne* est une liste $p = (x_1, \dots, x_k)$ de sommets telle que deux sommets consécutifs sont voisins :

$$\forall i \leq k - 1, \quad (x_i, x_{i+1}) \in E$$

La *longueur* de la chaîne est le nombre d'arêtes parcourues, soit $k - 1$.

Chaîne et chemin

Définition (Chaîne)

Une *chaîne* est une liste $p = (x_1, \dots, x_k)$ de sommets telle que deux sommets consécutifs sont voisins :

$$\forall i \leq k - 1, \quad (x_i, x_{i+1}) \in E$$

La *longueur* de la chaîne est le nombre d'arêtes parcourues, soit $k - 1$.

Remarque : En orienté, on parle de *chemin*.

Chaîne et chemin

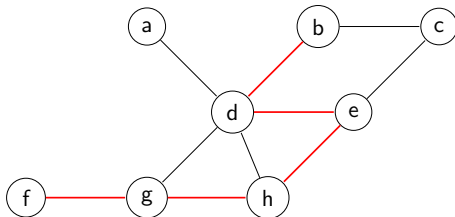
Définition (Chaîne)

Une *chaîne* est une liste $p = (x_1, \dots, x_k)$ de sommets telle que deux sommets consécutifs sont voisins :

$$\forall i \leq k - 1, \quad (x_i, x_{i+1}) \in E$$

La *longueur* de la chaîne est le nombre d'arêtes parcourues, soit $k - 1$.

Remarque : En orienté, on parle de *chemin*.



Chaîne de f à b de longueur 5.

Cycles et circuits

Définition (Simple)

*Un chemin/Une chaîne est dite **simple** si chaque arête est empruntée une seule fois*

Cycles et circuits

Définition (Simple)

*Un chemin/Une chaîne est dite **simple** si chaque arête est empruntée une seule fois*

Définition (Cycle)

*Un **cycle** $c = (x_1, \dots, x_k)$ est une chaîne simple avec $x_1 = x_k$.*

Cycles et circuits

Définition (Simple)

*Un chemin/Une chaîne est dite **simple** si chaque arête est empruntée une seule fois*

Définition (Cycle)

*Un **cycle** $c = (x_1, \dots, x_k)$ est une chaîne simple avec $x_1 = x_k$.*

Remarque : en orienté, on parle de **circuit**

Cycles et circuits

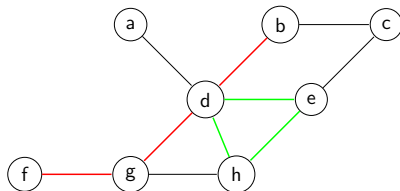
Définition (Simple)

Un chemin/Une chaîne est dite *simple* si chaque arête est empruntée une seule fois

Définition (Cycle)

Un *cycle* $c = (x_1, \dots, x_k)$ est une chaîne simple avec $x_1 = x_k$.

Remarque : en orienté, on parle de *circuit*



(f, g, d, b) et (f, g, d, h, e, d, b) sont simples. Pas $(f, g, d, h, e, d, h, e, d, b)$ car le cycle (d, h, e, d) est emprunté deux fois.

Existence de cycle

Proposition

Si dans un graphe G tout sommet est de degré supérieur ou égal à 2, alors G possède au moins un cycle.

Existence de cycle

Proposition

Si dans un graphe G tout sommet est de degré supérieur ou égal à 2, alors G possède au moins un cycle.

Preuve : algorithme de marquage

Aucun sommet marqué

Choix aléatoire de x_1 , le marquer

$c \leftarrow (x_1)$

tant que le dernier sommet de c a un voisin non marqué **faire**
 choisir un nouveau voisin x_i non marqué

 Le marquer, l'ajouter en fin de c

fin tant que

Existence de cycle

Proposition

Si dans un graphe G tout sommet est de degré supérieur ou égal à 2, alors G possède au moins un cycle.

Preuve : algorithme de marquage

Aucun sommet marqué

Choix aléatoire de x_1 , le marquer

$c \leftarrow (x_1)$

tant que le dernier sommet de c a un voisin non marqué **faire**

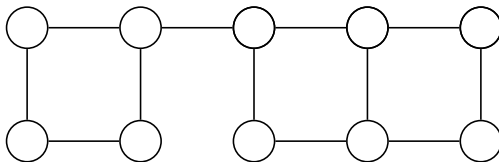
 choisir un nouveau voisin x_i non marqué

 Le marquer, l'ajouter en fin de c

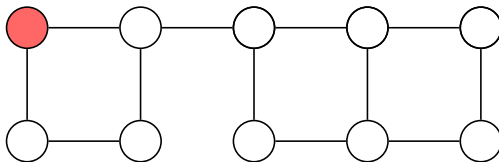
fin tant que

Le dernier sommet de c , x_k , a un voisin autre que x_{k-1} qui est déjà marqué donc dans c .

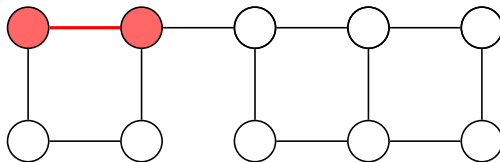
Existence de cycle - visuel



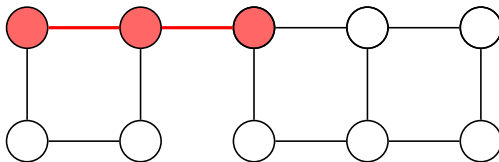
Existence de cycle - visuel



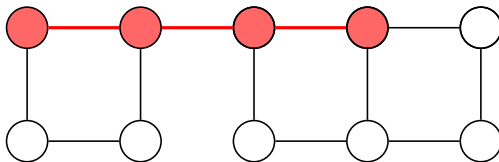
Existence de cycle - visuel



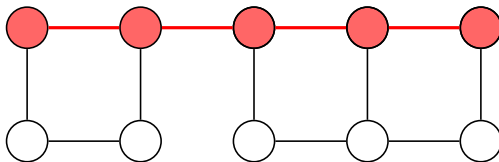
Existence de cycle - visuel



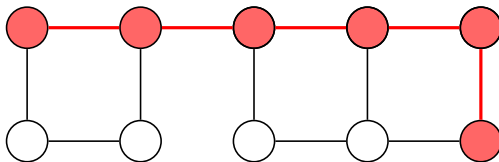
Existence de cycle - visuel



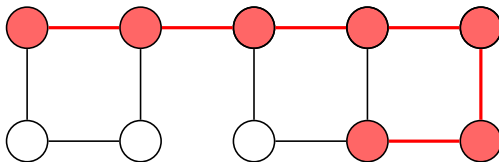
Existence de cycle - visuel



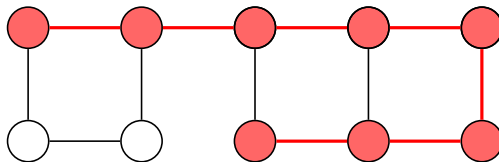
Existence de cycle - visuel



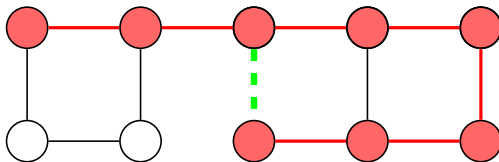
Existence de cycle - visuel



Existence de cycle - visuel



Existence de cycle - visuel



Non existence de cycle

Proposition

Un graphe acyclique G à n sommets possède au plus $n-1$ arêtes.

Non existence de cycle

Proposition

Un graphe acyclique G à n sommets possède au plus $n-1$ arêtes.

Preuve par récurrence

Distance et diamètre

Définition (Distance)

La *distance* entre deux sommets est la longueur de la plus petite chaîne/chemin qui les relie.

Distance et diamètre

Définition (Distance)

La *distance* entre deux sommets est la longueur de la plus petite chaîne/chemin qui les relie.

Définition (Diamètre)

Le *diamètre* d'un graphe est la plus grande distance entre deux sommets.

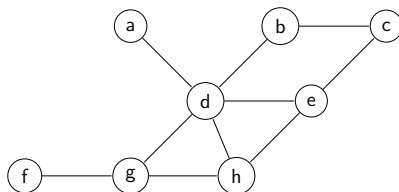
Distance et diamètre

Définition (Distance)

La *distance* entre deux sommets est la longueur de la plus petite chaîne/chemin qui les relie.

Définition (Diamètre)

Le *diamètre* d'un graphe est la plus grande distance entre deux sommets.



Entre f et e la distance est de 3.

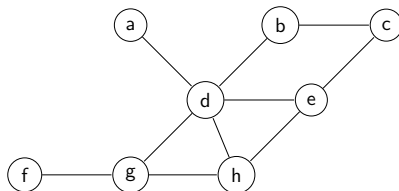
Distance et diamètre

Définition (Distance)

La *distance* entre deux sommets est la longueur de la plus petite chaîne/chemin qui les relie.

Définition (Diamètre)

Le *diamètre* d'un graphe est la plus grande distance entre deux sommets.



Entre f et e la distance est de 3. Le diamètre est de 4.

Plan

1 Définitions

- Vulgarisation
- Exemples de problèmes classiques en théorie des graphes
- Vocabulaire
- Modélisation et complexité

Matrice d'adjacence

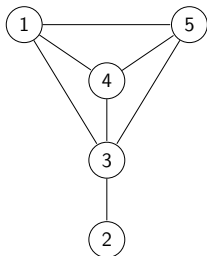
Soit $G = (V, E)$ un graphe

Définition

Si $n = |V|$, on numérote les sommets $V = v_1, \dots, v_n$. La *matrice d'adjacence* de G est la matrice $A = (a_{ij})$ de taille $n \times n$ définie par

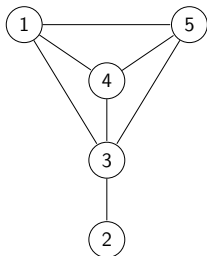
$$a_{ij} = \begin{cases} 1 & \text{si } (v_i, v_j) \in E \\ 0 & \text{sinon.} \end{cases}$$

Matrice d'adjacence - exemple non orienté



$$A = \begin{pmatrix} 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \end{pmatrix}$$

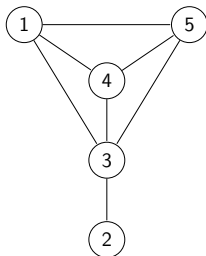
Matrice d'adjacence - exemple non orienté



$$A = \begin{pmatrix} 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \end{pmatrix}$$

Remarques

Matrice d'adjacence - exemple non orienté

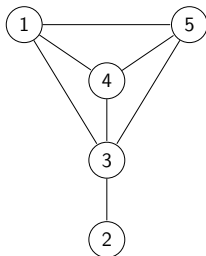


$$A = \begin{pmatrix} 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \end{pmatrix}$$

Remarques

- Diagonale de zéro

Matrice d'adjacence - exemple non orienté

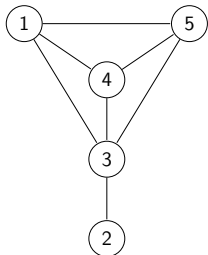


$$A = \begin{pmatrix} 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \end{pmatrix}$$

Remarques

- Diagonale de zéro
- Symétrique

Matrice d'adjacence - exemple non orienté



$$A = \begin{pmatrix} 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \end{pmatrix}$$

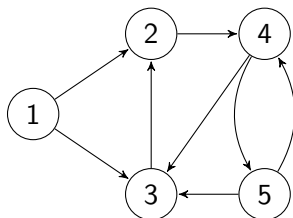
Remarques

- Diagonale de zéro
- Symétrique

- A^k ? $A^2 = \begin{pmatrix} 3 & 1 & 2 & 2 & 2 \\ 1 & 1 & 0 & 1 & 1 \\ 2 & 0 & 4 & 2 & 2 \\ 2 & 1 & 2 & 3 & 2 \\ 2 & 1 & 2 & 2 & 3 \end{pmatrix}$

$$A^3 = \begin{pmatrix} 6 & 2 & 8 & 7 & 7 \\ 2 & 0 & 4 & 2 & 2 \\ 8 & 4 & 6 & 8 & 8 \\ 7 & 2 & 8 & 6 & 7 \\ 7 & 2 & 8 & 7 & 6 \end{pmatrix}$$

Matrice d'adjacence - exemple orienté



$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{pmatrix}$$

Remarques

- Diagonale de zéro
- A^k

Matrice d'incidence

Soit $G = (V, E)$ un graphe **orienté**

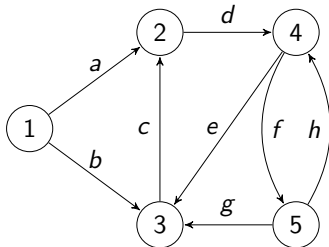
Définition

Si $n = |V|$ et $m = |E|$, on numérote les sommets $V = v_1, \dots, v_n$ et les arcs $E = e_1, \dots, e_m$. La **matrice d'incidence** de G est la matrice $A = (a_{ij})$ de taille $n \times m$ définie par

$$a_{ij} = \begin{cases} -1 & \text{si } e_j \text{ sort de } x_i \\ 1 & \text{si } e_j \text{ entre dans } x_i \\ 0 & \text{sinon.} \end{cases}$$

Remarque : pas même convention que degré entrant et sortant...

Matrice d'incidence - exemple



$$A = \begin{pmatrix} -1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 1 & -1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & -1 & -1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & -1 & -1 \end{pmatrix}$$

Remarques

- Deux valeurs non nulles par colonne, soit $2m$ cases non nulles sur mn

Listes d'adjacences

Soit $G = (V, E)$ un graphe

Listes d'adjacences

Soit $G = (V, E)$ un graphe

Dans le cas d'un graphe **non orienté**

Définition

*Si $n = |V|$ et $V = \{v_1, \dots, v_n\}$, la **liste d'adjacence** du sommet v_i est la liste des sommets qui lui sont adjacents.*

Listes d'adjacences

Soit $G = (V, E)$ un graphe

Dans le cas d'un graphe **non orienté**

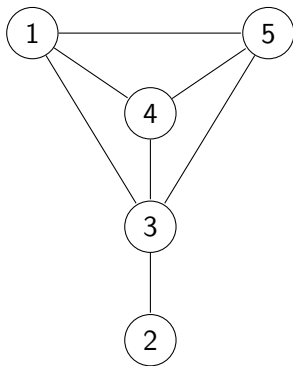
Définition

*Si $n = |V|$ et $V = \{v_1, \dots, v_n\}$, la **liste d'adjacence** du sommet v_i est la liste des sommets qui lui sont adjacents.*

La représentation par liste d'adjacence de G est la donnée de toutes les paires (sommet, liste d'adjacence de ce sommet).

Dans le cas d'un graphe **orienté**, on associe à chaque sommet sa liste de successeurs et/ou sa liste de prédecesseurs en fonction des besoins du problème.

Listes d'adjacence - exemple non orienté



1 : 3, 4, 5

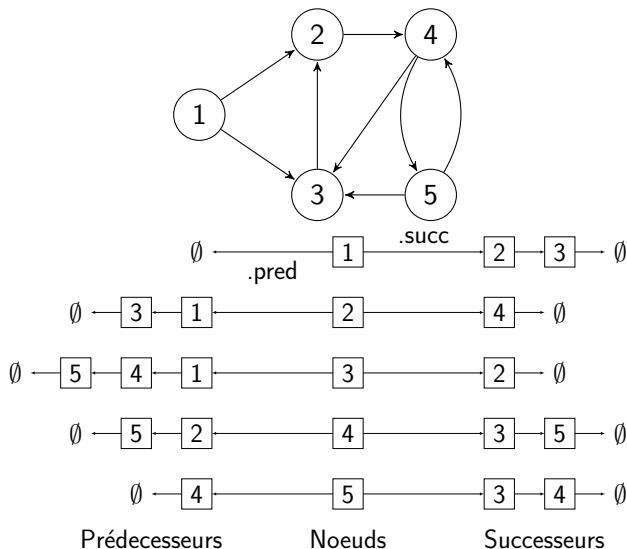
2 : 3

3 : 1, 2, 4, 5

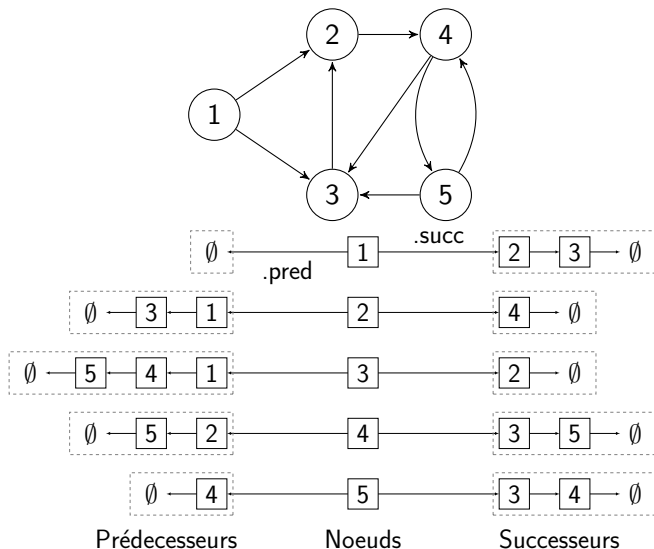
4 : 1, 3, 5

5 : 1, 3, 4

Listes d'adjacence - exemple orienté



Listes d'adjacence - exemple orienté



Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe			

Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe	$O(V + E)$		

Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe	$O(V + E)$	$O(V ^2)$	

Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe	$O(V + E)$	$O(V ^2)$	$O(V \cdot E)$

Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe	$O(V + E)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter sommet			

Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe	$O(V + E)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter sommet	$O(1)$		

Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe	$O(V + E)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter sommet	$O(1)$	$O(V ^2)$	

Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe	$O(V + E)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter sommet	$O(1)$	$O(V ^2)$	$O(V \cdot E)$

Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe	$O(V + E)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter sommet	$O(1)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter arête			

Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe	$O(V + E)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter sommet	$O(1)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter arête	$O(1)$		

Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe	$O(V + E)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter sommet	$O(1)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter arête	$O(1)$	$O(1)$	

Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe	$O(V + E)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter sommet	$O(1)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter arête	$O(1)$	$O(1)$	$O(V \cdot E)$

Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe	$O(V + E)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter sommet	$O(1)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter arête	$O(1)$	$O(1)$	$O(V \cdot E)$
Supprimer sommet			

Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe	$O(V + E)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter sommet	$O(1)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter arête	$O(1)$	$O(1)$	$O(V \cdot E)$
Supprimer sommet	$O(E)$		

Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe	$O(V + E)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter sommet	$O(1)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter arête	$O(1)$	$O(1)$	$O(V \cdot E)$
Supprimer sommet	$O(E)$	$O(V ^2)$	

Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe	$O(V + E)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter sommet	$O(1)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter arête	$O(1)$	$O(1)$	$O(V \cdot E)$
Supprimer sommet	$O(E)$	$O(V ^2)$	$O(V \cdot E)$

Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe	$O(V + E)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter sommet	$O(1)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter arête	$O(1)$	$O(1)$	$O(V \cdot E)$
Supprimer sommet	$O(E)$	$O(V ^2)$	$O(V \cdot E)$
Supprimer arête			

Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe	$O(V + E)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter sommet	$O(1)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter arête	$O(1)$	$O(1)$	$O(V \cdot E)$
Supprimer sommet	$O(E)$	$O(V ^2)$	$O(V \cdot E)$
Supprimer arête	$O(V)$		

Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe	$O(V + E)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter sommet	$O(1)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter arête	$O(1)$	$O(1)$	$O(V \cdot E)$
Supprimer sommet	$O(E)$	$O(V ^2)$	$O(V \cdot E)$
Supprimer arête	$O(V)$	$O(1)$	

Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe	$O(V + E)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter sommet	$O(1)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter arête	$O(1)$	$O(1)$	$O(V \cdot E)$
Supprimer sommet	$O(E)$	$O(V ^2)$	$O(V \cdot E)$
Supprimer arête	$O(V)$	$O(1)$	$O(V \cdot E)$

Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe	$O(V + E)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter sommet	$O(1)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter arête	$O(1)$	$O(1)$	$O(V \cdot E)$
Supprimer sommet	$O(E)$	$O(V ^2)$	$O(V \cdot E)$
Supprimer arête	$O(V)$	$O(1)$	$O(V \cdot E)$
2 sommets adjacents ?			

Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe	$O(V + E)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter sommet	$O(1)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter arête	$O(1)$	$O(1)$	$O(V \cdot E)$
Supprimer sommet	$O(E)$	$O(V ^2)$	$O(V \cdot E)$
Supprimer arête	$O(V)$	$O(1)$	$O(V \cdot E)$
2 sommets adjacents ?	$O(V)$		

Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe	$O(V + E)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter sommet	$O(1)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter arête	$O(1)$	$O(1)$	$O(V \cdot E)$
Supprimer sommet	$O(E)$	$O(V ^2)$	$O(V \cdot E)$
Supprimer arête	$O(V)$	$O(1)$	$O(V \cdot E)$
2 sommets adjacents ?	$O(V)$	$O(1)$	

Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe	$O(V + E)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter sommet	$O(1)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter arête	$O(1)$	$O(1)$	$O(V \cdot E)$
Supprimer sommet	$O(E)$	$O(V ^2)$	$O(V \cdot E)$
Supprimer arête	$O(V)$	$O(1)$	$O(V \cdot E)$
2 sommets adjacents ?	$O(V)$	$O(1)$	$O(E)$

Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe	$O(V + E)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter sommet	$O(1)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter arête	$O(1)$	$O(1)$	$O(V \cdot E)$
Supprimer sommet	$O(E)$	$O(V ^2)$	$O(V \cdot E)$
Supprimer arête	$O(V)$	$O(1)$	$O(V \cdot E)$
2 sommets adjacents ?	$O(V)$	$O(1)$	$O(E)$

Liste d'adjacence adapté à graphe avec peu d'arêtes

Représentation et complexité

	Liste d'adj.	Mat. adj.	Mat. incid.
Créer le graphe	$O(V + E)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter sommet	$O(1)$	$O(V ^2)$	$O(V \cdot E)$
Ajouter arête	$O(1)$	$O(1)$	$O(V \cdot E)$
Supprimer sommet	$O(E)$	$O(V ^2)$	$O(V \cdot E)$
Supprimer arête	$O(V)$	$O(1)$	$O(V \cdot E)$
2 sommets adjacents ?	$O(V)$	$O(1)$	$O(E)$

Liste d'adjacence adapté à graphe avec peu d'arêtes

Matrice d'adjacence adapté à graphe avec beaucoup d'arêtes

Plan

1 Définitions

2 Programmation

3 Connexité et parcours

4 Plus courts chemins

5 Arbres couvrants

6 Flots

7 Coloriage de graphes

Plan

2 Programmation

- Python
- SageMath
- Exercices

Nos amis en Python

Nos amis en Python

- Les **dictionnaires** :

Nos amis en Python

- Les **dictionnaires** :
 - ▶ Associer à une clé **immuable** une valeur (donc tuple plutôt que list par exemple)

Nos amis en Python

- Les **dictionnaires** :

- ▶ Associer à une clé **immuable** une valeur (donc tuple plutôt que list par exemple)
- ▶ Essentiel pour la représentation par liste d'adjacence

Nos amis en Python

- Les **dictionnaires** :

- ▶ Associer à une clé **immuable** une valeur (donc tuple plutôt que list par exemple)
- ▶ Essentiel pour la représentation par liste d'adjacence
- ▶ Pour tout ce qui est marquage, coloriage, informations sur un nœud

Nos amis en Python

- Les **dictionnaires** :
 - ▶ Associer à une clé **immuable** une valeur (donc tuple plutôt que list par exemple)
 - ▶ Essentiel pour la représentation par liste d'adjacence
 - ▶ Pour tout ce qui est marquage, coloriage, informations sur un nœud
- Les **ensembles** (set)

Nos amis en Python

- Les **dictionnaires** :

- ▶ Associer à une clé **immuable** une valeur (donc tuple plutôt que list par exemple)
- ▶ Essentiel pour la représentation par liste d'adjacence
- ▶ Pour tout ce qui est marquage, coloriage, informations sur un nœud

- Les **ensembles** (set)

- ▶ Contiennent sans répétition

Nos amis en Python

- Les **dictionnaires** :

- ▶ Associer à une clé **immuable** une valeur (donc tuple plutôt que list par exemple)
- ▶ Essentiel pour la représentation par liste d'adjacence
- ▶ Pour tout ce qui est marquage, coloriage, informations sur un nœud

- Les **ensembles** (set)

- ▶ Contiennent sans répétition
- ▶ Hashtable : test d'appartenance rapide (temps constant)

Nos amis en Python

- Les **dictionnaires** :
 - ▶ Associer à une clé **immuable** une valeur (donc tuple plutôt que list par exemple)
 - ▶ Essentiel pour la représentation par liste d'adjacence
 - ▶ Pour tout ce qui est marquage, coloriage, informations sur un nœud
- Les **ensembles** (set)
 - ▶ Contiennent sans répétition
 - ▶ Hashtable : test d'appartenance rapide (temps constant)
- Pour les matrices, liste double a priori, avec des librairies on peut faire mieux

Les ensembles

```
# BASIQUES SUR ENSEMBLES
# Création
s = {1, 2, 3}
s2 = set([3, 4, 5]) # Conversion

# Ajout élément
s.add(4)
print(s)           # {1, 2, 3, 4}

# Suppression élément
s.remove(2)        # Contrôle d'erreur
print(s)           # {1, 3, 4}
s.discard(5)       # Pas d'erreur

# Vider set
s.clear()
```


Les ensembles

```
# BASIQUES SUR ENSEMBLES
# Création
s = {1, 2, 3}
s2 = set([3, 4, 5]) # Conversion

# Ajout élément
s.add(4)
print(s)           # {1, 2, 3, 4}

# Suppression élément
s.remove(2)        # Contrôle d'erreur
print(s)           # {1, 3, 4}
s.discard(5)       # Pas d'erreur

# Vider set
s.clear()
```

```
# OPERATIONS ENTRE ENSEMBLES
s1 = {1, 2, 3}
s2 = {3, 4, 5}

# Union
print(s1 | s2)      # {1, 2, 3, 4, 5}
print(s1.union(s2)) # Idem

# Intersection
print(s1 & s2)       # {3}
print(s1.intersection(s2)) # Idem

# Différence
print(s1 - s2)       # {1, 2}
print(s1.difference(s2)) # Idem

# Différence symétrique
print(s1 ^ s2)       # {1, 2, 4, 5}
print(s1.symmetric_difference(s2)) # Idem
# Rq: s1.union_update(s2),... pour modifier
```

Les ensembles

```
# COMPARAISONS ENTRE ENSEMBLES
s1 = {1, 2}
s2 = {1, 2, 3}
s3 = {3, 4}

# Sous-ensemble
print (s1 <= s2)      # True
print(s1.issubset(s2)) # True

# Sur-ensemble
print (s2 >= s1)      # True
print(s2.issuperset(s1)) # True

# Disjoint
print(s1.isdisjoint(s3)) # True
```

Les ensembles

COMPARAISONS ENTRE ENSEMBLES

```
s1 = {1, 2}
s2 = {1, 2, 3}
s3 = {3, 4}
```

Sous-ensemble

```
print (s1 <= s2)      # True
print(s1.issubset(s2)) # True
```

Sur-ensemble

```
print (s2 >= s1)      # True
print(s2.issuperset(s1)) # True
```

Disjoint

```
print(s1.isdisjoint(s3)) # True
```

DIVERS

```
s = {1, 2, 3}
```

Appartenance

```
print(1 in s)      # True
print(4 in s)      # False
print(4 not in s)  # True
for elem in s:
    print(elem)     # ordre arbitraire
```

Conversion

```
lst = list(s)      # [1,2,3] (ordre arbitraire)
```

Les dictionnaires

```
# BASIQUES SUR DICTIONNAIRES
# Création d'un dictionnaire vide
d = {}

# Création avec des paires clé-valeur
d = {"nom": "Alice", "age": 25}

# Utilisation de la fonction 'dict'
d = dict(nom="Alice", age=25)

# Copier dictionnaire
copie = d.copy() # attention si des clés sont mutables

# Valeur par défaut
keys = ["nom", "age"]
d = dict.fromkeys(keys, "Inconnu")
print(d) # {'nom': 'Inconnu', 'age': 'Inconnu'}

# Fusion
d1 = {"nom": "Alice"}
d2 = {"age": 25}
d3 = d1 | d2 # {'nom': 'Alice', 'age': 25}

# Test élément
print("nom" in d3) # True
print("adresse" not in d3) # True
```

Les dictionnaires

```
# BASIQUES SUR DICTIONNAIRES
# Création d'un dictionnaire vide
d = {}

# Création avec des paires clé-valeur
d = {"nom": "Alice", "age": 25}

# Utilisation de la fonction 'dict'
d = dict(nom="Alice", age=25)

# Copier dictionnaire
copie = d.copy() # attention si des clés sont mutables

# Valeur par défaut
keys = ["nom", "age"]
d = dict.fromkeys(keys, "Inconnu")
print(d) # {'nom': 'Inconnu', 'age': 'Inconnu'}

# Fusion
d1 = {"nom": "Alice"}
d2 = {"age": 25}
d3 = d1 | d2 # {'nom': 'Alice', 'age': 25}

# Test élément
print("nom" in d3) # True
print("adresse" not in d3) # True
```

```
# ACCES ET SUPPRESSION
d = {"nom": "Alice", "age": 25}
# Cles, valeurs, paires
print(d.keys()) # dict_keys(['nom', 'age'])
print(d.values()) # dict_values(['Alice', 25])
print(d.items()) # dict_items([('nom', 'Alice'), ('age', 25)])

# Présence élément
print("nom" in d) # True
print("adresse" not in d) # True

# Accès éléments
print(d["nom"]) # Alice
print(d.get("nom")) # Alice
print(d.get("adresse")) # None
print(d.get("adresse", "Inconnue")) # Inconnue

# Ajout éléments
d["age"] = 26 # Modification
d["adresse"] = "Paris" # Ajout clé
print(d) # {'nom': 'Alice', 'age': 26, 'adresse': 'Paris'}

# Suppression avec pop
age = d.pop("age")
print(age) # 26
print(d) # {'nom': 'Alice', 'adresse': 'Paris'}

# Suppression avec del
del d["adresse"]
print(d) # {'nom': 'Alice'}

# Vider
d.clear()
```

Plan

2 Programmation

- Python
- SageMath
- Exercices

Sagemath et usage

Sagemath et usage

- Sagemath 

Sagemath et usage

- Sagemath  :
 - ▶ Logiciel libre de calcul mathématique

Sagemath et usage

- Sagemath  :
 - ▶ Logiciel libre de calcul mathématique
 - ▶ Basé sur du langage Python, orienté objet

Sagemath et usage

- Sagemath  :
 - ▶ Logiciel libre de calcul mathématique
 - ▶ Basé sur du langage Python, orienté objet
 - ▶ Utilisation plus rapide des types algorithmiques complexes

Sagemath et usage

- Sagemath  :
 - ▶ Logiciel libre de calcul mathématique
 - ▶ Basé sur du langage Python, orienté objet
 - ▶ Utilisation plus rapide des types algorithmiques complexes
 - ▶ Tous les algorithmes vus dans ce cours sont déjà codés : il faudra définir le graphe, puis appliquer correctement

Sagemath et usage

- Sagemath  :
 - ▶ Logiciel libre de calcul mathématique
 - ▶ Basé sur du langage Python, orienté objet
 - ▶ Utilisation plus rapide des types algorithmiques complexes
 - ▶ Tous les algorithmes vus dans ce cours sont déjà codés : il faudra définir le graphe, puis appliquer correctement
- Sur [MyDocker](#), environnement JupyterLab 

Définir des graphes

Définir des graphes

- Objets : `Graph()` (non orienté) et `DiGraph()` (Dirigé/orienté)

Définir des graphes

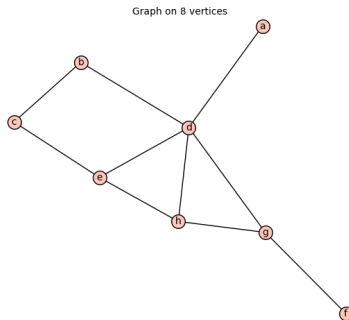
- Objets : `Graph()` (non orienté) et `DiGraph()` (Dlrigé/orienté)

```
G1 = Graph()
G1.add_vertices(['a','b','c','d','e','f','g','h'])
G1.add_edges([(('a','d'), ('b','c')), ('b','d'), ('d','e'), ('e','c'), ('e','h'), ('h','d'), ('f','g'), ('d','g'), ('g','h')])
G1
```


Définir des graphes

- Objets : `Graph()` (non orienté) et `DiGraph()` (Dlrigé/orienté)

```
G1 = Graph()
G1.add_vertices(['a','b','c','d','e','f','g','h'])
G1.add_edges([( 'a','d'), ( 'b','c'), ('b','d'), ('d','e'), ('e','c'), ('e','h'), ('h','d'), ('f','g'), ('d','g'), ('g','h')])
G1
```



Définir des graphes

- Objets : `Graph()` (non orienté) et `DiGraph()` (Dlrigé/orienté)

```
G1 = Graph()
G1.add_vertices(['a','b','c','d','e','f','g','h'])
G1.add_edges([(('a','d'), ('b','c')), ('b','d'), ('d','e'), ('e','c'), ('e','h'), ('h','d'), ('f','g'), ('d','g'), ('g','h')])
G1
```

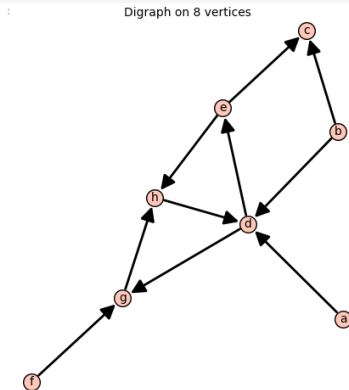
```
G2 = DiGraph()
G2.add_vertices(['a','b','c','d','e','f','g','h'])
G2.add_edges([(('a','d'), ('b','c')), ('b','d'), ('d','e'), ('e','c'), ('e','h'), ('h','d'), ('f','g'), ('d','g'), ('g','h')])
```

Définir des graphes

- Objets : `Graph()` (non orienté) et `DiGraph()` (Dlrigé/orienté)

```
G1 = Graph()
G1.add_vertices(['a','b','c','d','e','f','g','h'])
G1.add_edges([( 'a','d'), ( 'b','c'), ( 'b','d'), ( 'd','e'), ( 'e','c'), ( 'e','h'), ( 'h','d'), ( 'f','g'), ( 'd','g'), ( 'g','h')])
G1
```

```
G2 = DiGraph()
G2.add_vertices(['a','b','c','d','e','f','g','h'])
G2.add_edges([( 'a','d'), ( 'b','c'), ( 'b','d'), ( 'd','e'), ( 'e','c'), ( 'e','h'), ( 'h','d'), ( 'f','g'), ( 'd','g'), ( 'g','h')])
```



Définir des graphes

- Objets : `Graph()` (non orienté) et `DiGraph()` (Dlrigé/orienté)

```
G1 = Graph()
G1.add_vertices(['a','b','c','d','e','f','g','h'])
G1.add_edges([(('a','d'), ('b','c')), ('b','d'), ('d','e'), ('e','c'), ('e','h'), ('h','d'), ('f','g'), ('d','g'), ('g','h')])
G1
```

```
G2 = DiGraph()
G2.add_vertices(['a','b','c','d','e','f','g','h'])
G2.add_edges([(('a','d'), ('b','c')), ('b','d'), ('d','e'), ('e','c'), ('e','h'), ('h','d'), ('f','g'), ('d','g'), ('g','h')])
```

- Mots clés :
 - ▶ `G.add_vertex(nom)`
 - ▶ `G.add_vertices([liste de noms])`

Définir des graphes

- Objets : `Graph()` (non orienté) et `DiGraph()` (Dlrigé/orienté)

```
G1 = Graph()
G1.add_vertices(['a','b','c','d','e','f','g','h'])
G1.add_edges([( 'a','d'), ( 'b','c'), ('b','d'), ('d','e'), ('e','c'), ('e','h'), ('h','d'), ('f','g'), ('d','g'), ('g','h')])
G1
```

```
G2 = DiGraph()
G2.add_vertices(['a','b','c','d','e','f','g','h'])
G2.add_edges([( 'a','d'), ( 'b','c'), ('b','d'), ('d','e'), ('e','c'), ('e','h'), ('h','d'), ('f','g'), ('d','g'), ('g','h')])
```

- Mots clés :
 - ▶ `G.add_vertex(nom)`
 - ▶ `G.add_vertices([liste de noms])`
 - ▶ `G.add_edge(paire de noms)`

Définir des graphes

- Objets : `Graph()` (non orienté) et `DiGraph()` (Dlrigé/orienté)

```
G1 = Graph()
G1.add_vertices(['a','b','c','d','e','f','g','h'])
G1.add_edges([( 'a','d'), ( 'b','c'), ('b','d'), ('d','e'), ('e','c'), ('e','h'), ('h','d'), ('f','g'), ('d','g'), ('g','h')])
G1
```

```
G2 = DiGraph()
G2.add_vertices(['a','b','c','d','e','f','g','h'])
G2.add_edges([( 'a','d'), ( 'b','c'), ('b','d'), ('d','e'), ('e','c'), ('e','h'), ('h','d'), ('f','g'), ('d','g'), ('g','h')])
```

- Mots clés :
 - ▶ `G.add_vertex(nom)`
 - ▶ `G.add_vertices([liste de noms])`
 - ▶ `G.add_edge(paire de noms)`
 - ▶ `G.add_edges([liste de paires de noms])`

Définir des graphes

- Objets : `Graph()` (non orienté) et `DiGraph()` (Dlrigé/orienté)

```
G1 = Graph()
G1.add_vertices(['a','b','c','d','e','f','g','h'])
G1.add_edges([('a','d'), ('b','c'), ('b','d'), ('d','e'), ('e','c'), ('e','h'), ('h','d'), ('f','g'), ('d','g'), ('g','h')])
G1
```

```
G2 = DiGraph()
G2.add_vertices(['a','b','c','d','e','f','g','h'])
G2.add_edges([('a','d'), ('b','c'), ('b','d'), ('d','e'), ('e','c'), ('e','h'), ('h','d'), ('f','g'), ('d','g'), ('g','h')])
```

- Mots clés :
 - ▶ `G.add_vertex(nom)`
 - ▶ `G.add_vertices([liste de noms])`
 - ▶ `G.add_edge(paire de noms)`
 - ▶ `G.add_edges([liste de paires de noms])`

Définir des graphes

- Objets : `Graph()` (non orienté) et `DiGraph()` (Dlrigé/orienté)

```
G1 = Graph()
G1.add_vertices(['a','b','c','d','e','f','g','h'])
G1.add_edges([(('a','d'), ('b','c')), ('b','d'), ('d','e'), ('e','c'), ('e','h'), ('h','d'), ('f','g'), ('d','g'), ('g','h')])
G1
```

```
G2 = DiGraph()
G2.add_vertices(['a','b','c','d','e','f','g','h'])
G2.add_edges([(('a','d'), ('b','c')), ('b','d'), ('d','e'), ('e','c'), ('e','h'), ('h','d'), ('f','g'), ('d','g'), ('g','h')])
```

- Mots clés :
 - ▶ `G.add_vertex(nom)`
 - ▶ `G.add_vertices([liste de noms])`
 - ▶ `G.add_edge(paire de noms)`
 - ▶ `G.add_edges([liste de paires de noms])`
 - ▶ *Rq : les deux derniers ajoutent automatiquement les sommets s'ils n'existent pas*

Définir des graphes par représentation

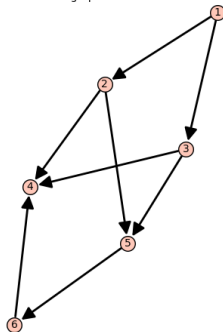
Liste d'adjacence (dict)

Définir des graphes par représentation

Liste d'adjacence (dict)

```
Digraph({1 : [2,3], 2 : [4, 5], 3 : [4, 5], 5: [6], 6 : [4]})
```

Digraph on 6 vertices

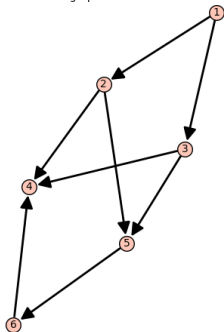


Définir des graphes par représentation

Liste d'adjacence (dict)

```
Digraph({1 : [2,3], 2 : [4, 5], 3 : [4, 5], 5: [6], 6 : [4]})
```

Digraph on 6 vertices



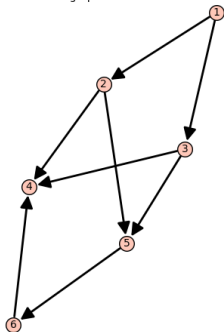
Attention, si non orienté, vérifier conditions

Définir des graphes par représentation

Liste d'adjacence (dict)

```
DiGraph({1 : [2,3], 2 : [4, 5], 3 : [4, 5], 5: [6], 6 : [4]})
```

Digraph on 6 vertices



Matrice d'adjacence (Matrix)

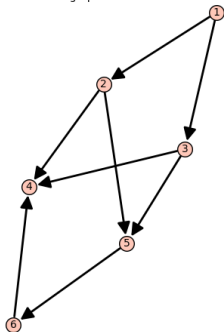
Attention, si non orienté, vérifier conditions

Définir des graphes par représentation

Liste d'adjacence (dict)

```
DiGraph({1 : [2,3], 2 : [4, 5], 3 : [4, 5], 5: [6], 6 : [4]})
```

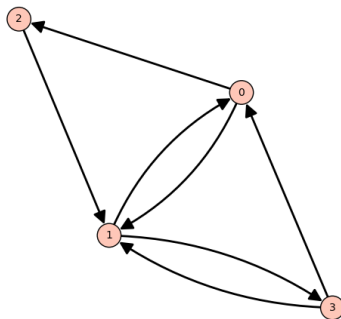
Digraph on 6 vertices



Matrice d'adjacence (Matrix)

```
DiGraph(Matrix([[0,1,1,0],[1, 0,0,1],[0,1,0,0],[1,1,0,0]]))
```

Digraph on 4 vertices



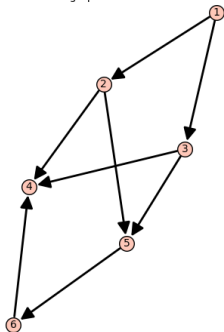
Attention, si non orienté, vérifier conditions

Définir des graphes par représentation

Liste d'adjacence (dict)

```
DiGraph({1 : [2,3], 2 : [4, 5], 3 : [4, 5], 5: [6], 6 : [4]})
```

Digraph on 6 vertices

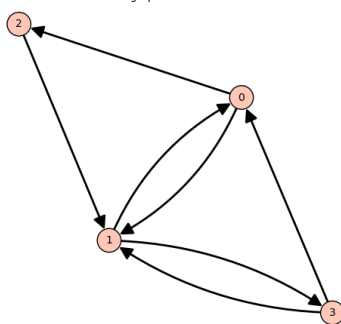


Attention, si non orienté, vérifier conditions

Matrice d'adjacence (Matrix)

```
DiGraph(Matrix([[0,1,1,0],[1, 0,0,1],[0,1,0,0],[1,1,0,0]]))
```

Digraph on 4 vertices



Attention, si non orienté, vérifier conditions

Fonctions sur des graphes

Fonctions sur des graphes

- `for i in G : énumère`
sommets G

Fonctions sur des graphes

- `for i in G : énumère`
sommets G
- `G.show()`

Fonctions sur des graphes

- `for i in G : énumère`
sommets G
- `G.show()`
- `G.to_directed()`

Fonctions sur des graphes

- `for i in G : énumère`
sommets G
- `G.show()`
- `G.to_directed()`
- `G.to_undirected()`

Fonctions sur des graphes

- `for i in G : énumère`
sommets G
- `G.show()`
- `G.to_directed()`
- `G.to_undirected()`
- `G.reverse()` *renverse*
sens flèches

Fonctions sur des graphes

- `for i in G : énumère`
sommets G
- `G.show()`
- `G.to_directed()`
- `G.to_undirected()`
- `G.reverse()` *renverse*
sens flèches
- `G.is_planar()`

Fonctions sur des graphes

- `G.neighbors('d')`
- `for i in G : énumère
sommets G`
- `G.show()`
- `G.to_directed()`
- `G.to_undirected()`
- `G.reverse()` *renverse
sens flèches*
- `G.is_planar()`

Fonctions sur des graphes

- `for i in G : énumère sommets G`
- `G.show()`
- `G.to_directed()`
- `G.to_undirected()`
- `G.reverse()` *renverse sens flèches*
- `G.is_planar()`
- `G.neighbors('d')`
- `G.degree('d')`

Fonctions sur des graphes

- `for i in G : énumère sommets G`
- `G.show()`
- `G.to_directed()`
- `G.to_undirected()`
- `G.reverse()` *renverse sens flèches*
- `G.is_planar()`
- `G.neighbors('d')`
- `G.degree('d')`
- `G.degree_histogram('d')`

Fonctions sur des graphes

- `for i in G : énumère sommets G`
- `G.show()`
- `G.to_directed()`
- `G.to_undirected()`
- `G.reverse()` *renverse sens flèches*
- `G.is_planar()`
- `G.neighbors('d')`
- `G.degree('d')`
- `G.degree_histogram('d')`
- `G.neighbors_in('d')`

Fonctions sur des graphes

- `for i in G : énumère sommets G`
- `G.show()`
- `G.to_directed()`
- `G.to_undirected()`
- `G.reverse()` *renverse sens flèches*
- `G.is_planar()`
- `G.neighbors('d')`
- `G.degree('d')`
- `G.degree_histogram('d')`
- `G.neighbors_in('d')`
- `G.neighbors_out('d')`

Fonctions sur des graphes

- `for i in G : énumère sommets G`
- `G.show()`
- `G.to_directed()`
- `G.to_undirected()`
- `G.reverse()` *renverse sens flèches*
- `G.is_planar()`
- `G.neighbors('d')`
- `G.degree('d')`
- `G.degree_histogram('d')`
- `G.neighbors_in('d')`
- `G.neighbors_out('d')`
- `G.in_degree('d')`

Fonctions sur des graphes

- `for i in G : énumère sommets G`
- `G.show()`
- `G.to_directed()`
- `G.to_undirected()`
- `G.reverse()` *renverse sens flèches*
- `G.is_planar()`
- `G.neighbors('d')`
- `G.degree('d')`
- `G.degree_histogram('d')`
- `G.neighbors_in('d')`
- `G.neighbors_out('d')`
- `G.in_degree('d')`
- `G.out_degree('d')`

Fonctions sur des graphes

- `for i in G : énumère sommets G`
- `G.show()`
- `G.to_directed()`
- `G.to_undirected()`
- `G.reverse()` *renverse sens flèches*
- `G.is_planar()`
- `G.neighbors('d')`
- `G.degree('d')`
- `G.degree_histogram('d')`
- `G.neighbors_in('d')`
- `G.neighbors_out('d')`
- `G.in_degree('d')`
- `G.out_degree('d')`
- `G.adjacency_matrix()`

Fonctions sur des graphes

- `for i in G : énumère sommets G`
- `G.show()`
- `G.to_directed()`
- `G.to_undirected()`
- `G.reverse()` *renverse sens flèches*
- `G.is_planar()`
- `G.neighbors('d')`
- `G.degree('d')`
- `G.degree_histogram('d')`
- `G.neighbors_in('d')`
- `G.neighbors_out('d')`
- `G.in_degree('d')`
- `G.out_degree('d')`
- `G.adjacency_matrix()`
- `G.graphplot(pos = d).show()`
d est un dict de positions absolues

Pour plus d'informations

Documentation SageMath :

- https://doc.sagemath.org/html/en/reference/graphs/sage/graphs/generic_graph.html
- <https://doc.sagemath.org/html/en/reference/graphs/sage/graphs/digraph.html>
- https://doc.sagemath.org/html/en/reference/plotting/sage/graphs/graph_plot.html

Pour plus d'informations

Documentation SageMath :

- https://doc.sagemath.org/html/en/reference/graphs/sage/graphs/generic_graph.html
- <https://doc.sagemath.org/html/en/reference/graphs/sage/graphs/digraph.html>
- https://doc.sagemath.org/html/en/reference/plotting/sage/graphs/graph_plot.html

Ne pas hésiter à aller pour documentation, exemples, etc.

Pour plus d'informations

Documentation SageMath :

- https://doc.sagemath.org/html/en/reference/graphs/sage/graphs/generic_graph.html
- <https://doc.sagemath.org/html/en/reference/graphs/sage/graphs/digraph.html>
- https://doc.sagemath.org/html/en/reference/plotting/sage/graphs/graph_plot.html

Ne pas hésiter à aller pour documentation, exemples, etc.

Rien à connaître par cœur, mais savoir chercher !

Plan

2 Programmation

- Python
- SageMath
- Exercices

Jeu de Nim

Les jeux de Nim sont des jeux d'affrontement en 1v1 de logique avec différentes variantes de règles.



Jeu de Nim



Les jeux de Nim sont des jeux d'affrontement en 1v1 de logique avec différentes variantes de règles. Dans la variante appelée Fan Tan, deux joueurs disposent de 2 ou plusieurs tas d'allumettes. À tour de rôle, chaque joueur peut enlever un certain nombre d'allumettes de l'un des tas (selon la règle choisie). Le joueur qui retire la dernière allumette perd la partie.

Jeu de Nim



Les jeux de Nim sont des jeux d'affrontement en 1v1 de logique avec différentes variantes de règles. Dans la variante appelée Fan Tan, deux joueurs disposent de 2 ou plusieurs tas d'allumettes. À tour de rôle, chaque joueur peut enlever un certain nombre d'allumettes de l'un des tas (selon la règle choisie). Le joueur qui retire la dernière allumette perd la partie.

- Modéliser ce jeu à l'aide d'un graphe dans le cas où l'on dispose au départ de deux tas contenant chacun trois allumettes, et où un joueur peut enlever une ou deux allumettes à chaque fois.

Jeu de Nim



Les jeux de Nim sont des jeux d'affrontement en 1v1 de logique avec différentes variantes de règles. Dans la variante appelée Fan Tan, deux joueurs disposent de 2 ou plusieurs tas d'allumettes. À tour de rôle, chaque joueur peut enlever un certain nombre d'allumettes de l'un des tas (selon la règle choisie). Le joueur qui retire la dernière allumette perd la partie.

- Modéliser ce jeu à l'aide d'un graphe dans le cas où l'on dispose au départ de deux tas contenant chacun trois allumettes, et où un joueur peut enlever une ou deux allumettes à chaque fois.
- Que doit jouer le premier joueur pour gagner la partie à coup sûr ?

Jeu de Nim



Les jeux de Nim sont des jeux d'affrontement en 1v1 de logique avec différentes variantes de règles. Dans la variante appelée Fan Tan, deux joueurs disposent de 2 ou plusieurs tas d'allumettes. À tour de rôle, chaque joueur peut enlever un certain nombre d'allumettes de l'un des tas (selon la règle choisie). Le joueur qui retire la dernière allumette perd la partie.

- Modéliser ce jeu à l'aide d'un graphe dans le cas où l'on dispose au départ de deux tas contenant chacun trois allumettes, et où un joueur peut enlever une ou deux allumettes à chaque fois.
- Que doit jouer le premier joueur pour gagner la partie à coup sûr ?
- **Pour aller plus loin :** *faire un programme générique pour plusieurs tas, nombre d'allumettes différents, nombre à retirer variable.*

Aborder ce genre de problème

Pour générer ce genre de graphe :

- Trouver une façon de représenter les données.

Aborder ce genre de problème

Pour générer ce genre de graphe :

- Trouver une façon de représenter les données.

Ici un tuple (a,b) avec le nombre d'allumettes restantes semble bon.

Aborder ce genre de problème

Pour générer ce genre de graphe :

- Trouver une façon de représenter les données.

Ici un tuple (a, b) avec le nombre d'allumettes restantes semble bon. Mais les tas étant indifférenciés $(3, 2)$ et $(2, 3)$ seraient identiques. On choisit donc que $a \geq b$

Aborder ce genre de problème

Pour générer ce genre de graphe :

- Trouver une façon de représenter les données.
Ici un tuple (a, b) avec le nombre d'allumettes restantes semble bon.
Mais les tas étant indifférenciés $(3, 2)$ et $(2, 3)$ seraient identiques. On choisit donc que $a \geq b$
- On a une **graine** initiale :

Aborder ce genre de problème

Pour générer ce genre de graphe :

- Trouver une façon de représenter les données.
Ici un tuple (a,b) avec le nombre d'allumettes restantes semble bon.
Mais les tas étant indifférenciés $(3,2)$ et $(2,3)$ seraient identiques. On choisit donc que $a \geq b$
- On a une **graine** initiale : $(3,3)$

Aborder ce genre de problème

Pour générer ce genre de graphe :

- Trouver une façon de représenter les données.
Ici un tuple (a, b) avec le nombre d'allumettes restantes semble bon.
Mais les tas étant indifférenciés $(3, 2)$ et $(2, 3)$ seraient identiques. On choisit donc que $a \geq b$
- On a une **graine** initiale : $(3, 3)$
- On liste les opérations possibles :

Aborder ce genre de problème

Pour générer ce genre de graphe :

- Trouver une façon de représenter les données.
Ici un tuple (a, b) avec le nombre d'allumettes restantes semble bon.
Mais les tas étant indifférenciés $(3, 2)$ et $(2, 3)$ seraient identiques. On choisit donc que $a \geq b$
- On a une **graine** initiale : $(3, 3)$
- On liste les opérations possibles :

$$(a, b) \mapsto (a - 1, b) \quad (a, b) \mapsto (a - 2, b)$$

$$(a, b) \mapsto (a, b - 1) \quad (a, b) \mapsto (a, b - 2)$$

(il faut éventuellement remettre dans le bon ordre les tuples de la première ligne, et borner les valeurs à 0)

Aborder ce genre de problème

Pour générer ce genre de graphe :

- Trouver une façon de représenter les données.
Ici un tuple (a, b) avec le nombre d'allumettes restantes semble bon.
Mais les tas étant indifférenciés $(3, 2)$ et $(2, 3)$ seraient identiques. On choisit donc que $a \geq b$
- On a une **graine** initiale : $(3, 3)$
- On liste les opérations possibles :

$$(a, b) \mapsto (a - 1, b) \quad (a, b) \mapsto (a - 2, b)$$

$$(a, b) \mapsto (a, b - 1) \quad (a, b) \mapsto (a, b - 2)$$

(il faut éventuellement remettre dans le bon ordre les tuples de la première ligne, et borner les valeurs à 0)

- On engendre le graphe de proche en proche en s'arrêtant quand on ne crée plus d'arêtes/de nœuds

Exemple de code - Python

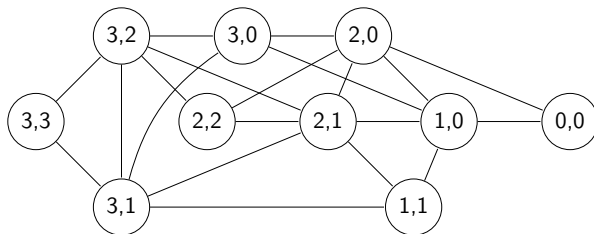
```
def graph() :
    nodes = {(3,3)}                # tous les sommets
    d = {}                        # toutes les arêtes
    d[(3,3)] = set()
    last_nodes = {(3,3)}          # derniers noeuds ajoutés
    while len(last_nodes) > 0 :    # Tant qu'on a des nouveaux noeuds au tour d'avant
        new_nodes = set()
        for node in last_nodes :
            a,b = node
            s = {(max(0, a-1), b), (max(0, a-2), b), (a, max(0, b-1)), (a, max(0, b-2))} # nouveaux noeuds
            s2 = {(max(a,b), min(a,b)) for (a,b) in s} # respecter norme
            for x in s2 :
                if x != node :     # éviter les boucles
                    d[node].add(x)
                if x not in nodes : # Pas encore vu
                    nodes.add(x)
                    new_nodes.add(x)
                    d[x] = set()
            last_nodes = new_nodes # les derniers vus sont les nouveaux de l'itération
    return nodes, d

if __name__ == "__main__":
    nodes, d = graph()
    L = list(nodes)
    L.sort()
    for k in L :
        print(k, " : ", d[k])
```


Exécution et conclusion - Python

Résultat :

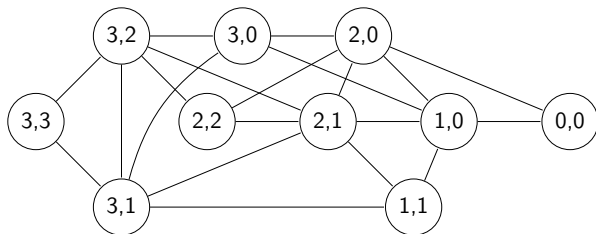
```
(0, 0) : set()
(1, 0) : {(0, 0)}
(1, 1) : {(1, 0)}
(2, 0) : {(1, 0), (0, 0)}
(2, 1) : {(1, 0), (1, 1), (2, 0)}
(2, 2) : {(2, 0), (2, 1)}
(3, 0) : {(1, 0), (2, 0)}
(3, 1) : {(1, 1), (2, 1), (3, 0)}
(3, 2) : {(3, 1), (2, 1), (3, 0), (2, 2)}
```



Exécution et conclusion - Python

Résultat :

```
(0, 0) : set()
(1, 0) : {(0, 0)}
(1, 1) : {(1, 0)}
(2, 0) : {(1, 0), (0, 0)}
(2, 1) : {(1, 0), (1, 1), (2, 0)}
(2, 2) : {(2, 0), (2, 1)}
(3, 0) : {(1, 0), (2, 0)}
(3, 1) : {(1, 1), (2, 1), (3, 0)}
(3, 2) : {(3, 1), (2, 1), (3, 0), (2, 2)}
```

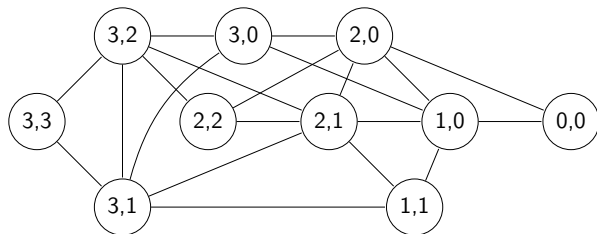


Stratégie : finir à (1,0) pour gagner.

Exécution et conclusion - Python

Résultat :

```
(0, 0) : set()
(1, 0) : {(0, 0)}
(1, 1) : {(1, 0)}
(2, 0) : {(1, 0), (0, 0)}
(2, 1) : {(1, 0), (1, 1), (2, 0)}
(2, 2) : {(2, 0), (2, 1)}
(3, 0) : {(1, 0), (2, 0)}
(3, 1) : {(1, 1), (2, 1), (3, 0)}
(3, 2) : {(3, 1), (2, 1), (3, 0), (2, 2)}
```



Stratégie : finir à (1,0) pour gagner. Si on commence, faire (3,1) et ensuite quoi que l'autre fasse on y arrive.

Exemple de code - SageMath

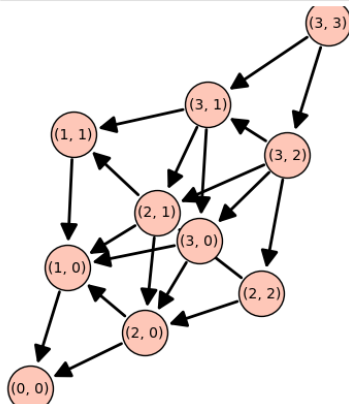
```
G = DiGraph()
G.add_vertex((3,3))
d_vu = {(3,3): False}           # dictionnaire des sommets déjà traités
new = True
while new:
    new = False                 # a-t'on ajouté des sommets ?
    for (a,b) in G.vertices():
        if d_vu[(a,b)] == False:   # si un sommet est non traité
            d_vu[(a,b)] = True      # maintenant il le sera
            L = [(a-1, b), (a-2,b), (a,b-1), (a, b-2)] # tous les possibles
            for (c,d) in L:
                if c>=0 and d>=0:
                    M = max(c,d)
                    m = min(c,d)
                    if (M,m) not in G.vertices():
                        G.add_vertex((M,m))
                        d_vu[(M,m)] = False
                        new = True
                    if ((a,b),(c,d)) not in G.edges() and ((a,b),(d,c)) not in G.edges():
                        G.add_edge(((a,b), (M,m)))

G.show(vertex_size=1000, dist = 50)
```

Exécution et conclusion - SageMath

Résultat :

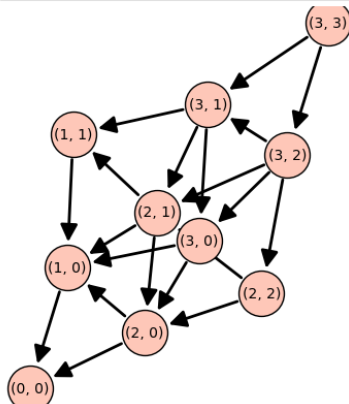
```
G.show(vertex_size=1000, dist = 50)
```



Exécution et conclusion - SageMath

Résultat :

```
G.show(vertex_size=1000, dist = 50)
```

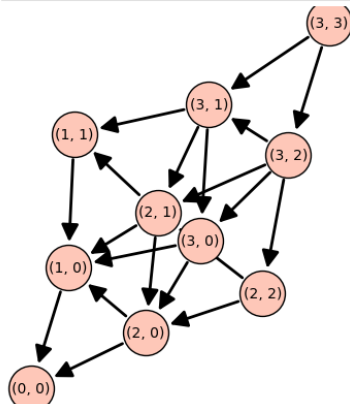


Stratégie : finir à $(1,0)$ pour gagner.

Exécution et conclusion - SageMath

Résultat :

```
G.show(vertex_size=1000, dist = 50)
```



Stratégie : finir à $(1, 0)$ pour gagner. Si on commence, faire $(3, 1)$ et ensuite quoi que l'autre fasse on y arrive.

À vous de jouer



Kristen fait de la cuisine et a besoin d'une quantité d'eau précise. Toutefois elle n'a rien pour peser ni pour doser. Elle ne dispose que de deux cruches de r litres et s litres respectivement ($r, s \in \mathbb{N}^*$). On supposera les deux cruches distinguables. Elle a un robinet pour remplir les cruches et un lavabo pour les vider. Elle fait des mesures précises donc ne remplit jamais "à l'œil" une cruche.

Elle peut donc soit :

- Vider une cruche
- Transvaser d'une cruche à l'autre
- Remplir complètement une cruche

Donner le nombre de configurations possibles de remplissage des deux cruches à un moment donné, puis la liste des quantité d'eau précise atteignable.

Plan

1 Définitions

2 Programmation

3 Connexité et parcours

4 Plus courts chemins

5 Arbres couvrants

6 Flots

7 Coloriage de graphes

Plan

3 Connexité et parcours

- Composantes connexes
- Parcours
- Composantes fortement connexes
- Notion de k -connexité
- Tri topologique

Connexité - définition

Soit $G = (V, E)$ un graphe **non orienté**

Définition (Connexité)

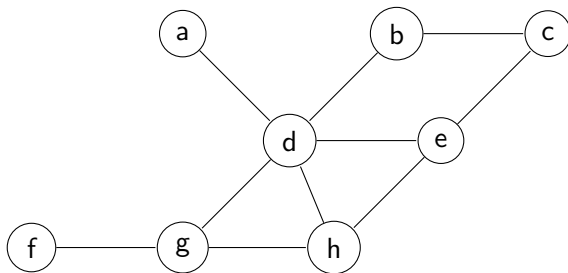
*Un graphe est **connexe** ssi il existe une chaîne entre chaque paire de sommets.*

Connexité - définition

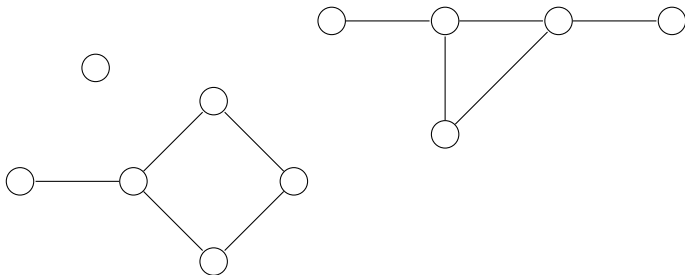
Soit $G = (V, E)$ un graphe **non orienté**

Définition (Connexité)

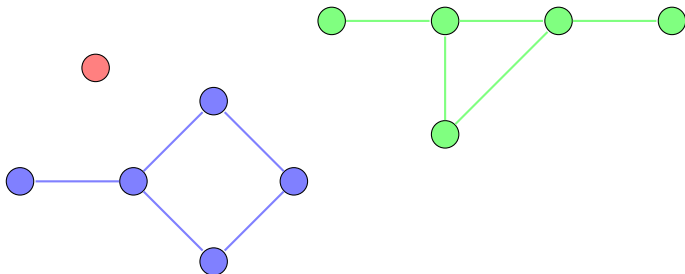
*Un graphe est **connexe** ssi il existe une chaîne entre chaque paire de sommets.*



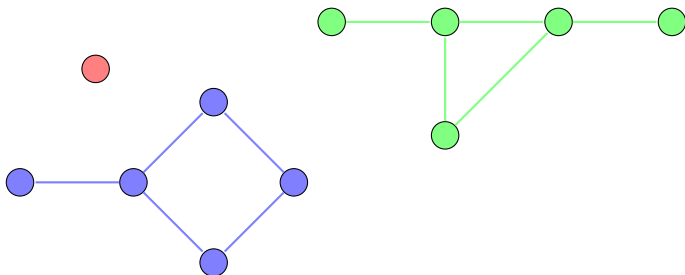
Exemple non connexe



Exemple non connexe



Exemple non connexe



Juxtaposition de graphes connexes : les *composantes connexes* du graphe

Composante connexe

Soit $G = (V, E)$ un graphe **non orienté**

Définition

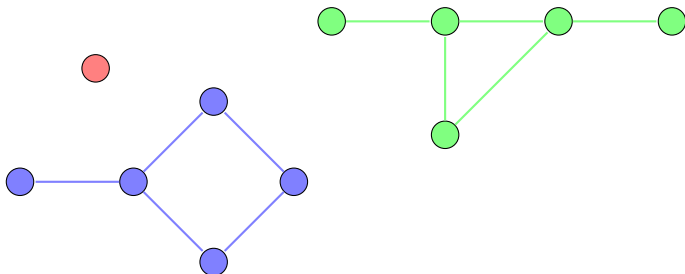
Une *composante connexe* de G est un sous-graphe $G' = (V', E')$ connexe maximal (pour l'inclusion) : il n'est pas possible d'ajouter à V' d'autres sommets en conservant la connexité du sous-graphe.

Composante connexe

Soit $G = (V, E)$ un graphe **non orienté**

Définition

Une **composante connexe** de G est un sous-graphe $G' = (V', E')$ connexe maximal (pour l'inclusion) : il n'est pas possible d'ajouter à V' d'autres sommets en conservant la connexité du sous-graphe.

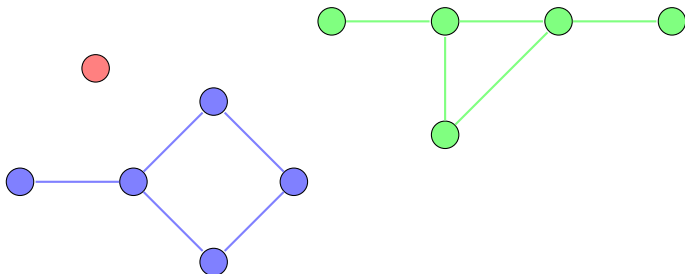


Composante connexe

Soit $G = (V, E)$ un graphe **non orienté**

Définition

Une **composante connexe** de G est un sous-graphe $G' = (V', E')$ connexe maximal (pour l'inclusion) : il n'est pas possible d'ajouter à V' d'autres sommets en conservant la connexité du sous-graphe.



3 Composantes connexes

Propriétés

- Graphe avec une seule composante connexe $\Leftrightarrow$ graphe connexe

Propriétés

- Graphe avec une seule composante connexe $\Leftrightarrow$ graphe connexe
- Un sommet de degré 0 est une composante connexe à lui seul.

Propriétés

- Graphe avec une seule composante connexe $\Leftrightarrow$ graphe connexe
- Un sommet de degré 0 est une composante connexe à lui seul.
- **(Maths)** En **non orienté**, la relation sur les sommets "il existe une chaîne entre ..." est une relation d'équivalence (réflexive, symétrique et transitive). Les composantes connexes d'un graphe correspondent aux classes d'équivalences de cette relation.

Propriétés

- Graphe avec une seule composante connexe $\Leftrightarrow$ graphe connexe
- Un sommet de degré 0 est une composante connexe à lui seul.
- **(Maths)** En **non orienté**, la relation sur les sommets "il existe une chaîne entre ..." est une relation d'équivalence (réflexive, symétrique et transitive). Les composantes connexes d'un graphe correspondent aux classes d'équivalences de cette relation.

Proposition

Un graphe G d'ordre n connexe comporte au moins $n - 1$ arêtes

Plan

3 Connexité et parcours

- Composantes connexes
- **Parcours**
- Composantes fortement connexes
- Notion de k -connexité
- Tri topologique

Structures de données - pile



Structures de données - pile



- Dernier arrivé premier parti

Structures de données - pile



- Dernier arrivé premier parti
- On ajoute en haut

Structures de données - pile



- Dernier arrivé premier parti
- On ajoute en haut
- On retire en haut

Structures de données - pile



- Dernier arrivé premier parti
- On ajoute en haut
- On retire en haut
- Pour accéder à une assiette donné il faut dépiler jusque là

Structures de données - file



Structures de données - file



- Premier arrivé premier parti

Structures de données - file



- Premier arrivé premier parti
- On ajoute en fin

Structures de données - file



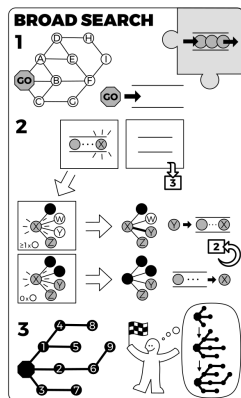
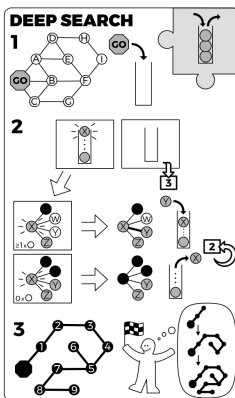
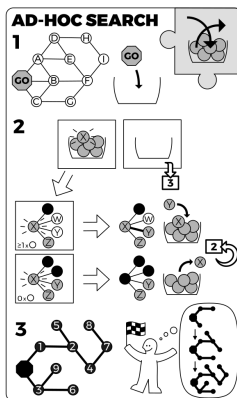
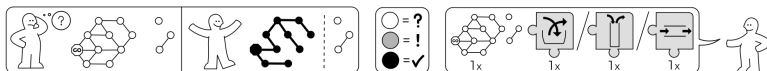
- Premier arrivé premier parti
- On ajoute en fin
- On retire au début

Notion de parcours

GRÂPH SCÂN

idea-instructions.com/graph-scan/
v1.1, CC by-nc-sa 4.0

IDEA

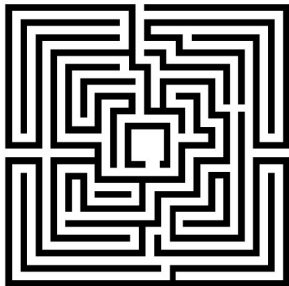


Deux explications intuitives

Parcours en profondeur

Deux explications intuitives

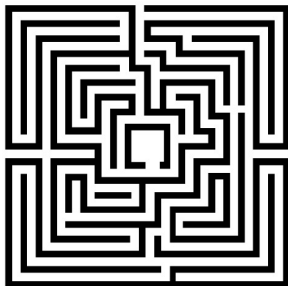
Parcours en profondeur



Deux explications intuitives

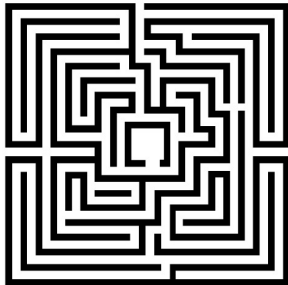
Parcours en profondeur

Parcours en largeur



Deux explications intuitives

Parcours en profondeur



Parcours en largeur



Parcours en profondeur

- Exploration à fond
- Algorithme de marquage
- Parcours une composante connexe
- Complexité en $O(|V| + |E|)$

Parcours en profondeur

- Exploration à fond
- Algorithme de marquage
- Parcours une composante connexe
- Complexité en $O(|V| + |E|)$

Algorithme DFS(graphe G , sommet s) :

Entrée : Graphe G , sommet s

Marquer s

pour tout v voisin de s **faire**

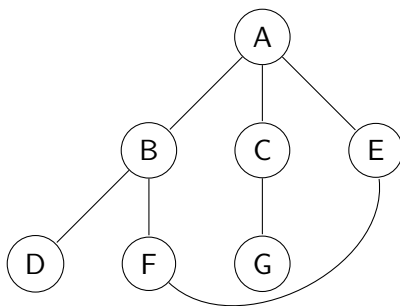
si v non marqué **alors**

 DFS(G , v)

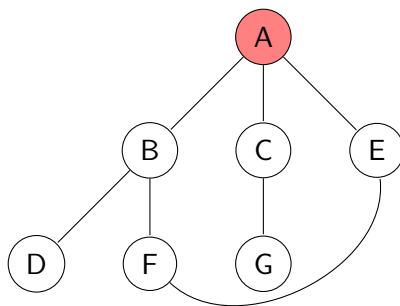
fin si

fin pour

Parcours en profondeur - exemple

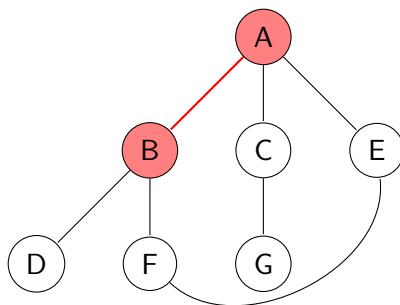


Parcours en profondeur - exemple



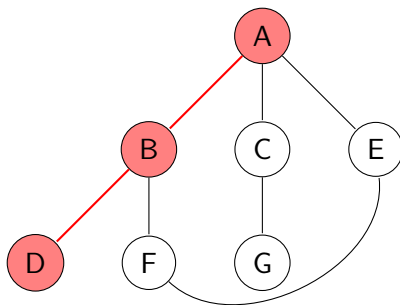
Choix dans l'ordre alphabétique arbitraire

Parcours en profondeur - exemple



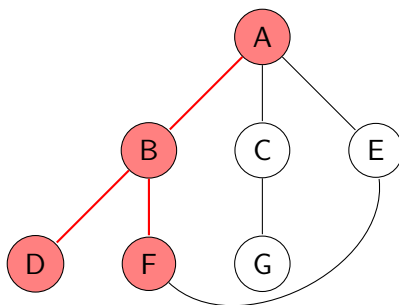
Choix dans l'ordre alphabétique arbitraire

Parcours en profondeur - exemple



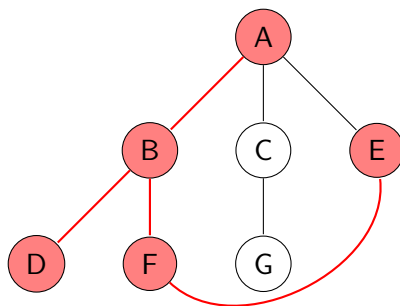
Choix dans l'ordre alphabétique arbitraire

Parcours en profondeur - exemple



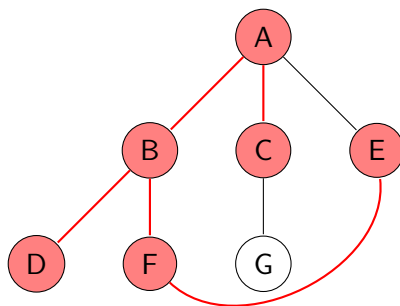
Choix dans l'ordre alphabétique arbitraire

Parcours en profondeur - exemple



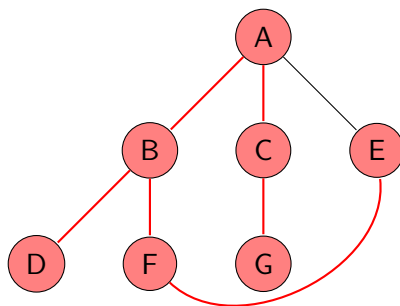
Choix dans l'ordre alphabétique arbitraire

Parcours en profondeur - exemple



Choix dans l'ordre alphabétique arbitraire

Parcours en profondeur - exemple



Choix dans l'ordre alphabétique arbitraire

Parcours en profondeur - algo avec pile

Algorithme DFS(graphe G , sommet s) :

Entrée : Graphe G , sommet s

Empiler s sur pile P

Marquer s

tant que P non vide **faire**

si le sommet de la pile u a un voisin v non marqué **alors**

 Empiler v sur P

 Marquer v

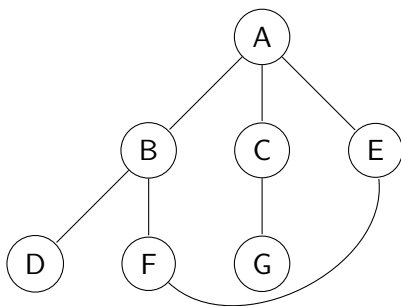
sinon

 Dépiler u de P

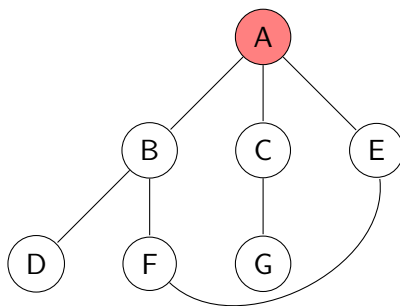
fin si

fin tant que

Parcours en profondeur - exemple avec pile



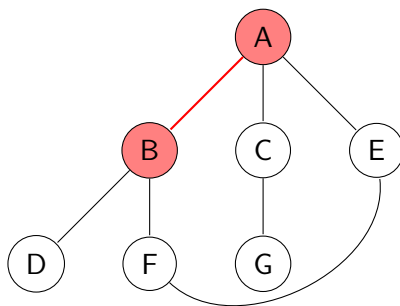
Parcours en profondeur - exemple avec pile



A

Choix dans l'ordre alphabétique arbitraire

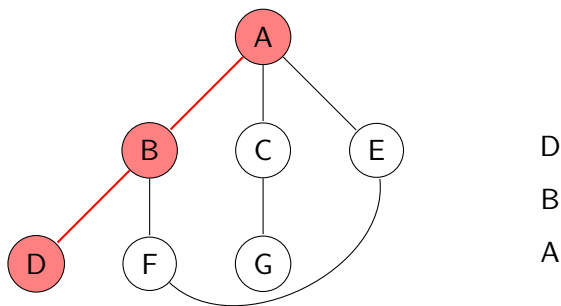
Parcours en profondeur - exemple avec pile



B
A

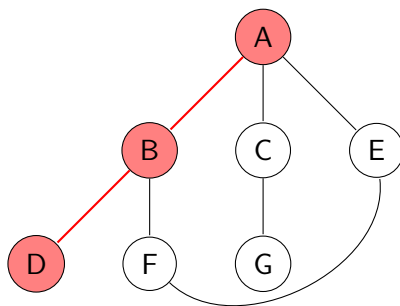
Choix dans l'ordre alphabétique arbitraire

Parcours en profondeur - exemple avec pile



Choix dans l'ordre alphabétique arbitraire

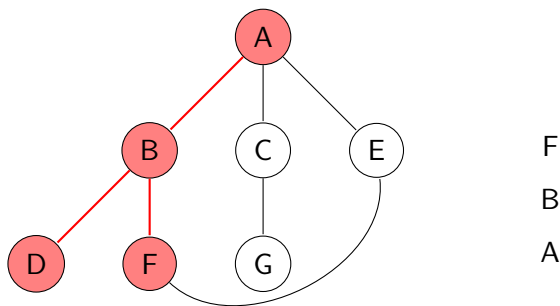
Parcours en profondeur - exemple avec pile



B
A

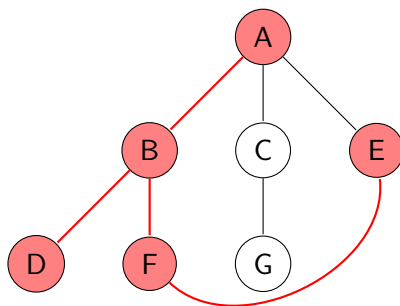
Choix dans l'ordre alphabétique arbitraire

Parcours en profondeur - exemple avec pile



Choix dans l'ordre alphabétique arbitraire

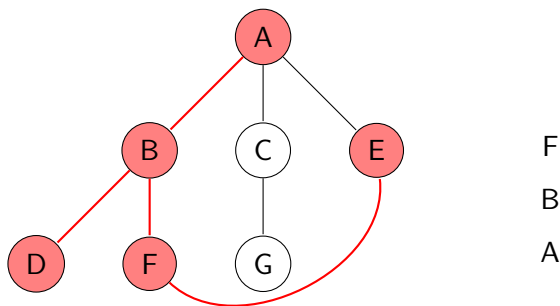
Parcours en profondeur - exemple avec pile



E
F
B
A

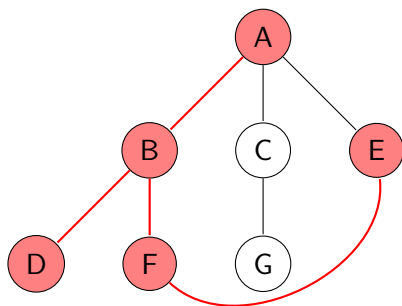
Choix dans l'ordre alphabétique arbitraire

Parcours en profondeur - exemple avec pile



Choix dans l'ordre alphabétique arbitraire

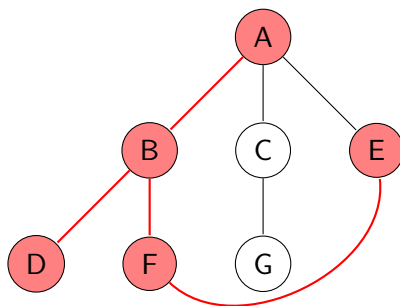
Parcours en profondeur - exemple avec pile



B
A

Choix dans l'ordre alphabétique arbitraire

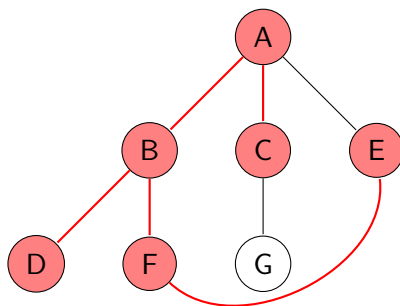
Parcours en profondeur - exemple avec pile



A

Choix dans l'ordre alphabétique arbitraire

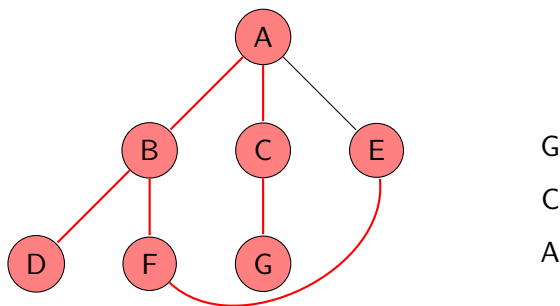
Parcours en profondeur - exemple avec pile



C
A

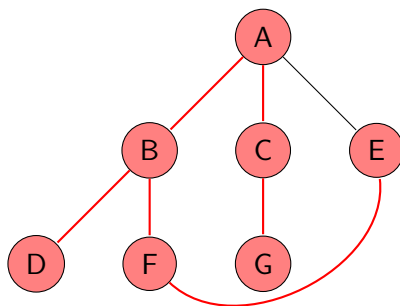
Choix dans l'ordre alphabétique arbitraire

Parcours en profondeur - exemple avec pile



Choix dans l'ordre alphabétique arbitraire

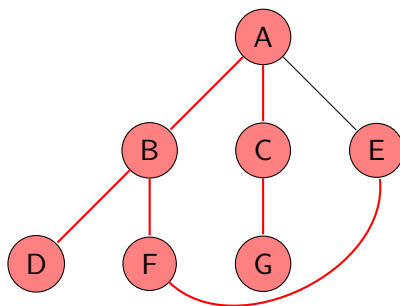
Parcours en profondeur - exemple avec pile



C
A

Choix dans l'ordre alphabétique arbitraire

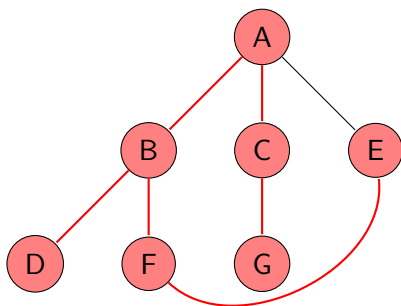
Parcours en profondeur - exemple avec pile



A

Choix dans l'ordre alphabétique arbitraire

Parcours en profondeur - exemple avec pile



Choix dans l'ordre alphabétique arbitraire

Parcours en largeur

- Exploration par **niveaux**
- Algorithme de marquage
- Parcours une composante connexe
- Complexité en $O(|V| + |E|)$
- Utilisation d'une file

Parcours en largeur - algo

Algorithme BFS(graphe G , sommet s) :

Entrée : Graphe G , sommet s

Ajouter s à la file F

Marquer s

tant que F non vide **faire**

 Soit u la tête de F

pour tout v voisin de u non marqué **faire**

 Ajouter v à F

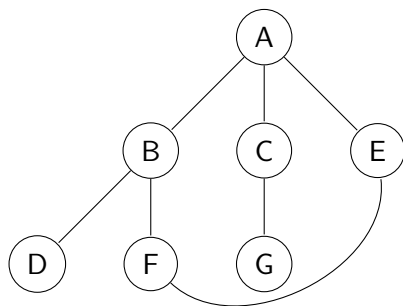
 Marquer v

fin pour

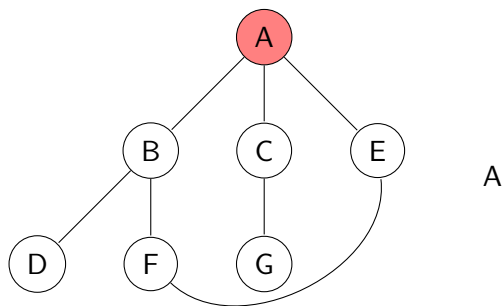
 Décapiter F

fin tant que

Parcours en largeur - exemple avec file

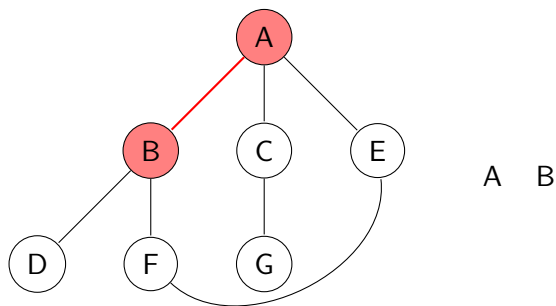


Parcours en largeur - exemple avec file



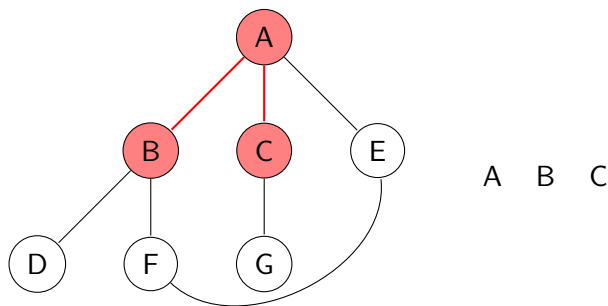
Choix dans l'ordre alphabétique arbitraire

Parcours en largeur - exemple avec file



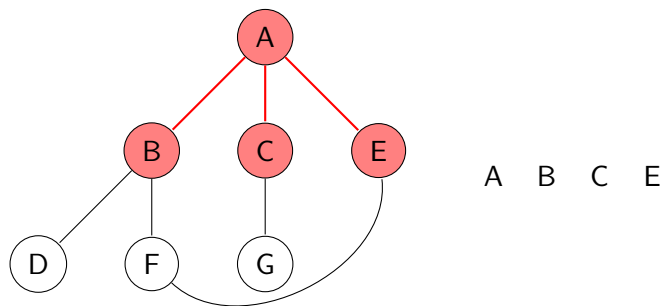
Choix dans l'ordre alphabétique arbitraire

Parcours en largeur - exemple avec file



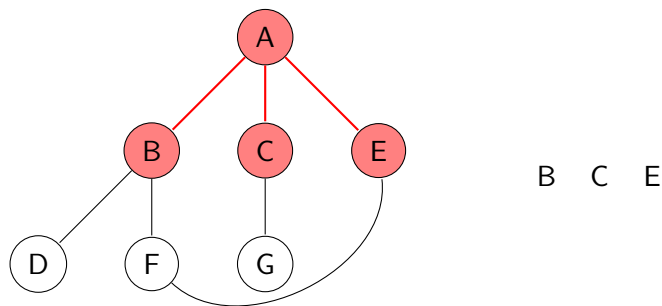
Choix dans l'ordre alphabétique arbitraire

Parcours en largeur - exemple avec file



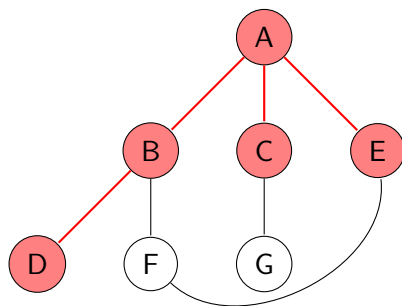
Choix dans l'ordre alphabétique arbitraire

Parcours en largeur - exemple avec file



Choix dans l'ordre alphabétique arbitraire

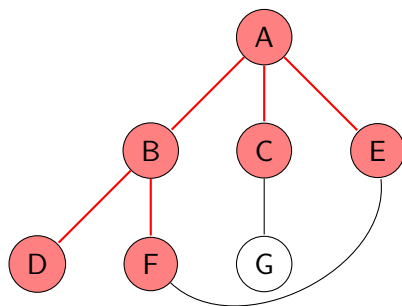
Parcours en largeur - exemple avec file



B C E D

Choix dans l'ordre alphabétique arbitraire

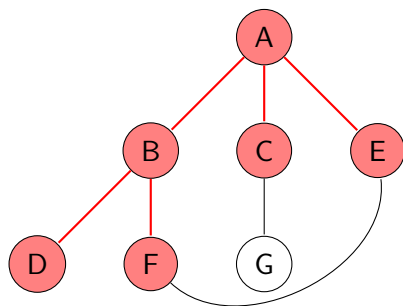
Parcours en largeur - exemple avec file



B C E D F

Choix dans l'ordre alphabétique arbitraire

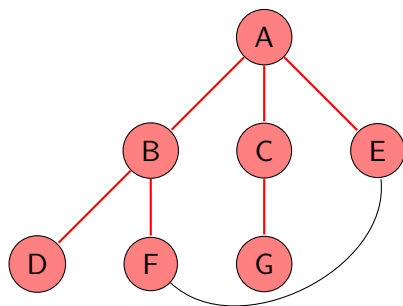
Parcours en largeur - exemple avec file



C E D F

Choix dans l'ordre alphabétique arbitraire

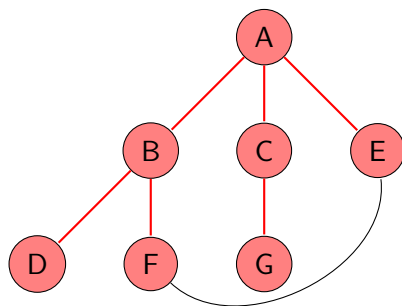
Parcours en largeur - exemple avec file



C E D F G

Choix dans l'ordre alphabétique arbitraire

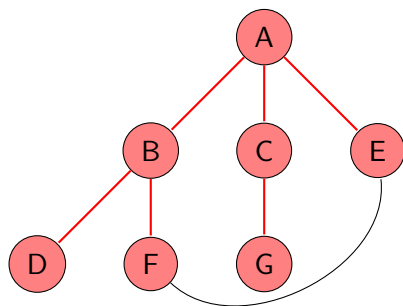
Parcours en largeur - exemple avec file



E D F G

Choix dans l'ordre alphabétique arbitraire

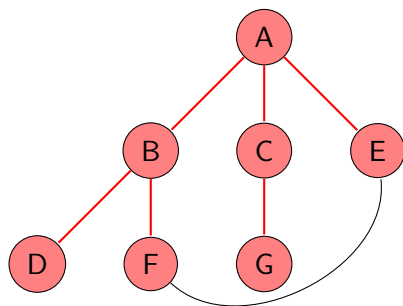
Parcours en largeur - exemple avec file



D F G

Choix dans l'ordre alphabétique arbitraire

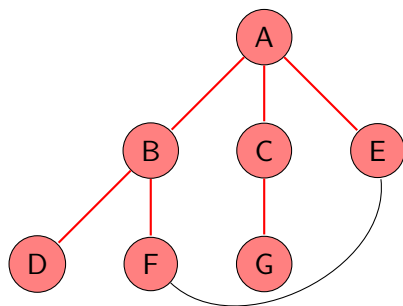
Parcours en largeur - exemple avec file



F G

Choix dans l'ordre alphabétique arbitraire

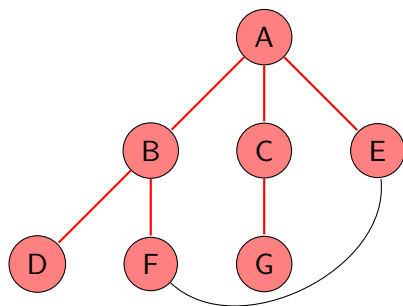
Parcours en largeur - exemple avec file



G

Choix dans l'ordre alphabétique arbitraire

Parcours en largeur - exemple avec file



Choix dans l'ordre alphabétique arbitraire

Deux exemples plus gros

Montrer gif

Plan

3 Connexité et parcours

- Composantes connexes
- Parcours
- Composantes fortement connexes
- Notion de k -connexité
- Tri topologique

Cas orienté - différentes notions de connexité

Définition

Soit $G = (V, E)$ un graphe **orienté**. Il est dit :

Cas orienté - différentes notions de connexité

Définition

Soit $G = (V, E)$ un graphe **orienté**. Il est dit :

- *faiblement connexe* si en oubliant l'orientation des arêtes il est connexe

Cas orienté - différentes notions de connexité

Définition

Soit $G = (V, E)$ un graphe **orienté**. Il est dit :

- *faiblement connexe* si en oubliant l'orientation des arêtes il est connexe
- *unilatéralement connexe* si pour tout $x, y \in V$, il existe un chemin de x à y OU de y à x .

Cas orienté - différentes notions de connexité

Définition

Soit $G = (V, E)$ un graphe **orienté**. Il est dit :

- **faiblement connexe** si en oubliant l'orientation des arêtes il est connexe
- **unilatéralement connexe** si pour tout $x, y \in V$, il existe un chemin de x à y OU de y à x .
- **fortement connexe** si pour tout $x, y \in V$, il existe un chemin de x à y ET de y à x .

Cas orienté - différentes notions de connexité

Définition

Soit $G = (V, E)$ un graphe **orienté**. Il est dit :

- **faiblement connexe** si en oubliant l'orientation des arêtes il est connexe
- **unilatéralement connexe** si pour tout $x, y \in V$, il existe un chemin de x à y OU de y à x .
- **fortement connexe** si pour tout $x, y \in V$, il existe un chemin de x à y ET de y à x .

fortement connexe $\Rightarrow$ unilatéralement connexe $\Rightarrow$ faiblement connexe

Cas orienté - différentes notions de connexité

Définition

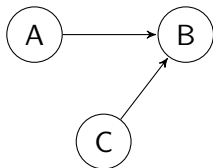
Soit $G = (V, E)$ un graphe **orienté**. Il est dit :

- **faiblement connexe** si en oubliant l'orientation des arêtes il est connexe
- **unilatéralement connexe** si pour tout $x, y \in V$, il existe un chemin de x à y OU de y à x .
- **fortement connexe** si pour tout $x, y \in V$, il existe un chemin de x à y ET de y à x .

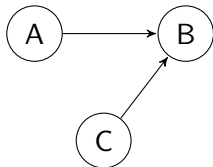
fortement connexe $\Rightarrow$ unilatéralement connexe $\Rightarrow$ faiblement connexe

On s'intéressera surtout à la forte connexité.

Différentes notions de connexité - exemple

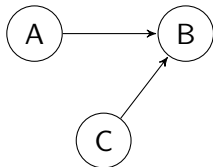


Différentes notions de connexité - exemple

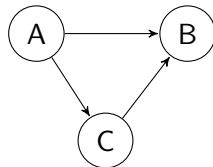


faiblement connexe
non unilatéralement connexe

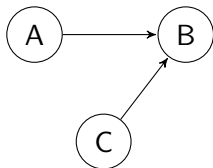
Différentes notions de connexité - exemple



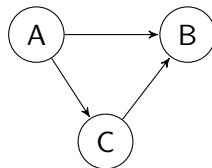
faiblement connexe
non unilatéralement connexe



Différentes notions de connexité - exemple

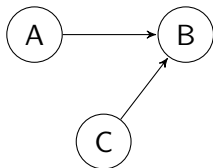


faiblement connexe
non unilatéralement connexe

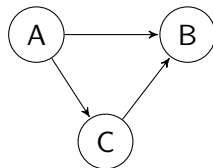


unilatéralement connexe
non fortement connexe

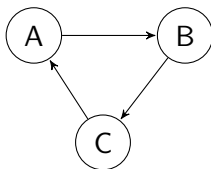
Différentes notions de connexité - exemple



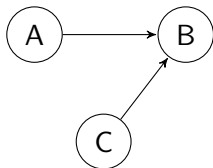
faiblement connexe
non unilatéralement connexe



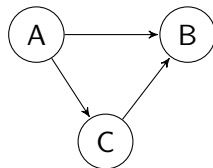
unilatéralement connexe
non fortement connexe



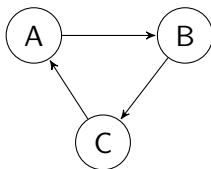
Différentes notions de connexité - exemple



faiblement connexe
non unilatéralement connexe



unilatéralement connexe
non fortement connexe



fortement connexe

Composante fortement connexe

Soit $G = (V, E)$ graphe orienté

Définition

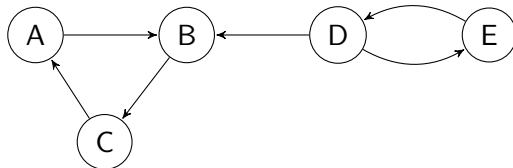
Une *composante fortement connexe* de G est un sous-graphe G' de G fortement connexe maximal pour l'inclusion : on ne peut ajouter de sommet à G' de sorte que G' reste fortement connexe.

Composante fortement connexe

Soit $G = (V, E)$ graphe orienté

Définition

Une *composante fortement connexe* de G est un sous-graphe G' de G fortement connexe maximal pour l'inclusion : on ne peut ajouter de sommet à G' de sorte que G' reste fortement connexe.

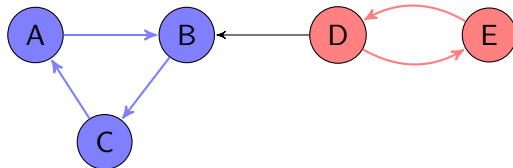


Composante fortement connexe

Soit $G = (V, E)$ graphe orienté

Définition

Une *composante fortement connexe* de G est un sous-graphe G' de G fortement connexe maximal pour l'inclusion : on ne peut ajouter de sommet à G' de sorte que G' reste fortement connexe.



Graphe réduit

Définition

On appelle *graphe réduit* du graphe G le quotient de G par la relation de forte connexité.

Autrement dit c'est le graphe G' où chaque nœud est associé à une composante fortement connexe (cfc) de G . De plus, un arc relie x' à y' dans G' s'il existe un arc qui relie x à y dans G où x appartient à la cfc associée à x' et y à la cfc associée à y' .

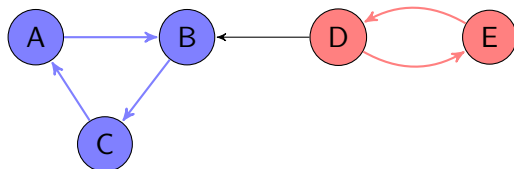
Graphe réduit

Définition

On appelle **graphe réduit** du graphe G le quotient de G par la relation de forte connexité.

Autrement dit c'est le graphe G' où chaque nœud est associé à une composante fortement connexe (cfc) de G . De plus, un arc relie x' à y' dans G' s'il existe un arc qui relie x à y dans G où x appartient à la cfc associée à x' et y à la cfc associée à y' .

Exemple :



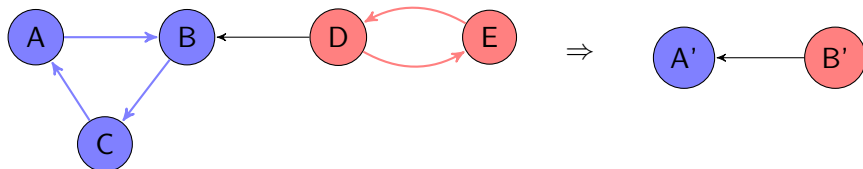
Graphe réduit

Définition

On appelle **graphe réduit** du graphe G le quotient de G par la relation de forte connexité.

Autrement dit c'est le graphe G' où chaque nœud est associé à une composante fortement connexe (cfc) de G . De plus, un arc relie x' à y' dans G' s'il existe un arc qui relie x à y dans G où x appartient à la cfc associée à x' et y à la cfc associée à y' .

Exemple :



Avec $A' = \{A, B, C\}$ et $B' = \{D, E\}$.

Recherche d'une composante fortement connexe

- **Idée** : parcours dans le sens des flèches, puis parcours en sens inverse. Intersection des deux donne la composante fortement connexe.
- Algorithme de marquage

Recherche d'une composante fortement connexe

- **Idée** : parcours dans le sens des flèches, puis parcours en sens inverse. Intersection des deux donne la composante fortement connexe.
- Algorithme de marquage

Algorithme CFC(graphe G , sommet s)

Entrée : Graphe $G = (V, E)$,

sommet s

$V_1 \leftarrow \{s\}$

tant que $\exists u \in V_1$, u non marqué

faire

Marquer u

pour tout $e =$

$(u, v) \in E \mid v \notin V_1$ **faire**

Ajouter v à V_1

fin pour

fin tant que

$V_2 \leftarrow \{s\}$

tant que $\exists u \in V_2$, u non marqué

faire

Marquer u

pour tout $e = (v, u) \in E$

$\mid v \notin V_2$ **faire**

Ajouter v à V_2

fin pour

fin tant que

retourne $V_1 \cap V_2$

Recherche de toutes les composantes fortement connexes

Idée : appliquer algo précédent sur tous les sommets

Recherche de toutes les composantes fortement connexes

Idée : appliquer algo précédent sur tous les sommets

Algorithme CFCs(graphe G)

Entrée : Graphe $G = (V, E)$

Sortie : $C = (C_1, \dots, C_n)$ ensemble des cfc

$V' \leftarrow V$ et $i \leftarrow 1$

tant que $V' \neq \emptyset$ **faire**

 Choisir $x \in V'$

$C_i \leftarrow \text{cfc}(G, x)$

$i \leftarrow i + 1$

$V' \leftarrow V' \setminus C_i$

fin tant que

Plan

3 Connexité et parcours

- Composantes connexes
- Parcours
- Composantes fortement connexes
- Notion de k -connexité
- Tri topologique

Sommet-connexité

Soit $G = (V, E)$ graphe non orienté.

Définition (Sommet-séparateur)

On appelle *sommet-séparateur* un ensemble V' de sommets tel que le graphe $G \setminus V'$ n'est plus connexe.

Sommet-connexité

Soit $G = (V, E)$ graphe non orienté.

Définition (Sommet-séparateur)

On appelle *sommet-séparateur* un ensemble V' de sommets tel que le graphe $G \setminus V'$ n'est plus connexe.

Définition ($k(G)$)

On note $k(G)$ le minimum k tel que G a un sommet séparateur à k sommets. On dit alors que G est *k -sommet-connexe*.

Sommet-connexité

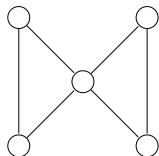
Soit $G = (V, E)$ graphe non orienté.

Définition (Sommet-séparateur)

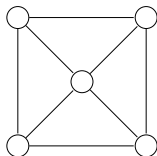
On appelle *sommet-séparateur* un ensemble V' de sommets tel que le graphe $G \setminus V'$ n'est plus connexe.

Définition ($k(G)$)

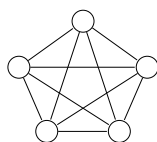
On note $k(G)$ le minimum k tel que G a un sommet séparateur à k sommets. On dit alors que G est *k -sommet-connexe*.



G_1



G_2



G_3

Sommet-connexité

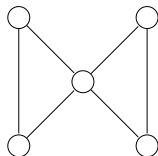
Soit $G = (V, E)$ graphe non orienté.

Définition (Sommet-séparateur)

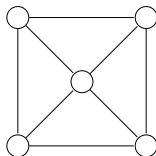
On appelle *sommet-séparateur* un ensemble V' de sommets tel que le graphe $G \setminus V'$ n'est plus connexe.

Définition ($k(G)$)

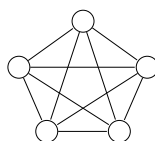
On note $k(G)$ le minimum k tel que G a un sommet séparateur à k sommets. On dit alors que G est *k -sommet-connexe*.



G_1



G_2



G_3

$$k(G_1) =$$

Sommet-connexité

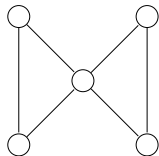
Soit $G = (V, E)$ graphe non orienté.

Définition (Sommet-séparateur)

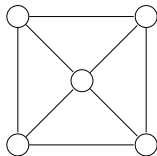
On appelle *sommet-séparateur* un ensemble V' de sommets tel que le graphe $G \setminus V'$ n'est plus connexe.

Définition ($k(G)$)

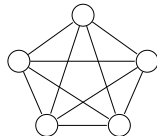
On note $k(G)$ le minimum k tel que G a un sommet séparateur à k sommets. On dit alors que G est *k -sommet-connexe*.



G_1



G_2



G_3

$$k(G_1) = 1,$$

Sommet-connexité

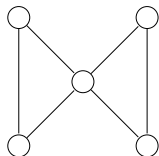
Soit $G = (V, E)$ graphe non orienté.

Définition (Sommet-séparateur)

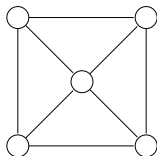
On appelle *sommet-séparateur* un ensemble V' de sommets tel que le graphe $G \setminus V'$ n'est plus connexe.

Définition ($k(G)$)

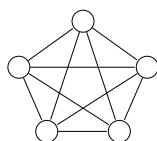
On note $k(G)$ le minimum k tel que G a un sommet séparateur à k sommets. On dit alors que G est *k -sommet-connexe*.



G_1



G_2



G_3

$$k(G_1) = 1, k(G_2) =$$

Sommet-connexité

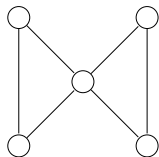
Soit $G = (V, E)$ graphe non orienté.

Définition (Sommet-séparateur)

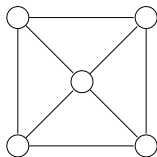
On appelle *sommet-séparateur* un ensemble V' de sommets tel que le graphe $G \setminus V'$ n'est plus connexe.

Définition ($k(G)$)

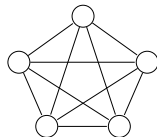
On note $k(G)$ le minimum k tel que G a un sommet séparateur à k sommets. On dit alors que G est *k -sommet-connexe*.



G_1



G_2



G_3

$$k(G_1) = 1, k(G_2) = 3,$$

Sommet-connexité

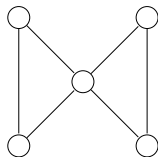
Soit $G = (V, E)$ graphe non orienté.

Définition (Sommet-séparateur)

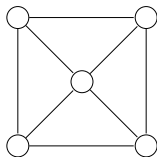
On appelle *sommet-séparateur* un ensemble V' de sommets tel que le graphe $G \setminus V'$ n'est plus connexe.

Définition ($k(G)$)

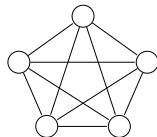
On note $k(G)$ le minimum k tel que G a un sommet séparateur à k sommets. On dit alors que G est *k -sommet-connexe*.



G_1



G_2



G_3

$$k(G_1) = 1, k(G_2) = 3, k(G_3) =$$

Sommet-connexité

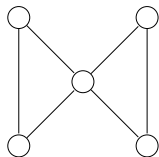
Soit $G = (V, E)$ graphe non orienté.

Définition (Sommet-séparateur)

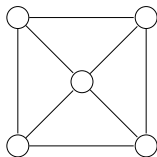
On appelle *sommet-séparateur* un ensemble V' de sommets tel que le graphe $G \setminus V'$ n'est plus connexe.

Définition ($k(G)$)

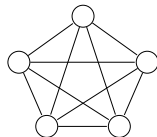
On note $k(G)$ le minimum k tel que G a un sommet séparateur à k sommets. On dit alors que G est *k -sommet-connexe*.



G_1



G_2



G_3

$k(G_1) = 1, k(G_2) = 3, k(G_3) = 4$ (convention)

Arête-connexité

Soit $G = (V, E)$ graphe non orienté.

Définition (Arête-séparateur)

On appelle *arête-séparateur* un ensemble E' d'arêtes tel que le graphe $G \setminus E'$ n'est plus connexe.

Définition ($k'(G)$)

On note $k'(G)$ le minimum k tel que G a un arête-séparateur à k arêtes.
On dit alors que G est *k -arête-connexe*.

Arête-connexité

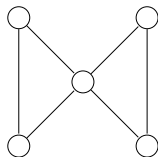
Soit $G = (V, E)$ graphe non orienté.

Définition (Arête-séparateur)

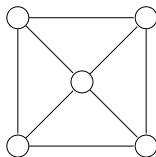
On appelle *arête-séparateur* un ensemble E' d'arêtes tel que le graphe $G \setminus E'$ n'est plus connexe.

Définition ($k'(G)$)

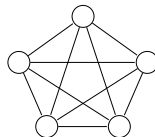
On note $k'(G)$ le minimum k tel que G a un arête-séparateur à k arêtes.
On dit alors que G est *k -arête-connexe*.



G1



G2



G3

Arête-connexité

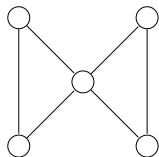
Soit $G = (V, E)$ graphe non orienté.

Définition (Arête-séparateur)

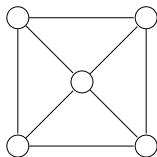
On appelle *arête-séparateur* un ensemble E' d'arêtes tel que le graphe $G \setminus E'$ n'est plus connexe.

Définition ($k'(G)$)

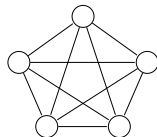
On note $k'(G)$ le minimum k tel que G a un arête-séparateur à k arêtes.
On dit alors que G est *k -arête-connexe*.



G1



G2



G3

$$k'(G_1) =$$

Arête-connexité

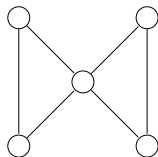
Soit $G = (V, E)$ graphe non orienté.

Définition (Arête-séparateur)

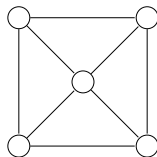
On appelle *arête-séparateur* un ensemble E' d'arêtes tel que le graphe $G \setminus E'$ n'est plus connexe.

Définition ($k'(G)$)

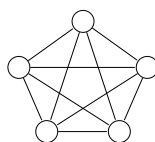
On note $k'(G)$ le minimum k tel que G a un arête-séparateur à k arêtes.
On dit alors que G est *k -arête-connexe*.



G1



G2



G3

$$k'(G_1) = 2,$$

Arête-connexité

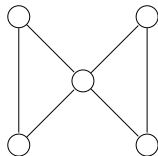
Soit $G = (V, E)$ graphe non orienté.

Définition (Arête-séparateur)

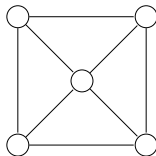
On appelle *arête-séparateur* un ensemble E' d'arêtes tel que le graphe $G \setminus E'$ n'est plus connexe.

Définition ($k'(G)$)

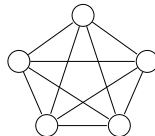
On note $k'(G)$ le minimum k tel que G a un arête-séparateur à k arêtes.
On dit alors que G est *k -arête-connexe*.



G1



G2



G3

$$k'(G_1) = 2, k'(G_2) =$$

Arête-connexité

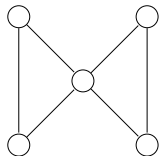
Soit $G = (V, E)$ graphe non orienté.

Définition (Arête-séparateur)

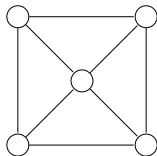
On appelle *arête-séparateur* un ensemble E' d'arêtes tel que le graphe $G \setminus E'$ n'est plus connexe.

Définition ($k'(G)$)

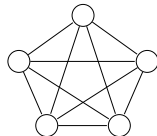
On note $k'(G)$ le minimum k tel que G a un arête-séparateur à k arêtes.
On dit alors que G est *k -arête-connexe*.



G1



G2



G3

$$k'(G_1) = 2, \quad k'(G_2) = 3,$$

Arête-connexité

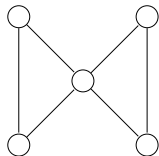
Soit $G = (V, E)$ graphe non orienté.

Définition (Arête-séparateur)

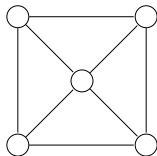
On appelle *arête-séparateur* un ensemble E' d'arêtes tel que le graphe $G \setminus E'$ n'est plus connexe.

Définition ($k'(G)$)

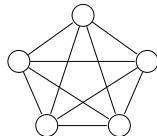
On note $k'(G)$ le minimum k tel que G a un arête-séparateur à k arêtes.
On dit alors que G est *k -arête-connexe*.



G1



G2



G3

$$k'(G_1) = 2, k'(G_2) = 3, k'(G_3) =$$

Arête-connexité

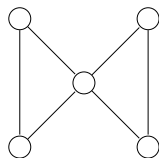
Soit $G = (V, E)$ graphe non orienté.

Définition (Arête-séparateur)

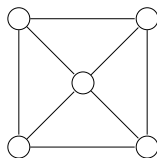
On appelle *arête-séparateur* un ensemble E' d'arêtes tel que le graphe $G \setminus E'$ n'est plus connexe.

Définition ($k'(G)$)

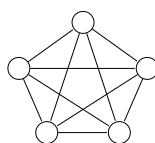
On note $k'(G)$ le minimum k tel que G a un arête-séparateur à k arêtes.
On dit alors que G est *k -arête-connexe*.



G1



G2



G3

$$k'(G_1) = 2, \quad k'(G_2) = 3, \quad k'(G_3) = 4$$

Propriétés

On note $\delta(G)$ le plus petit degré de G .

Théorème

$$k(G) \leq k'(G) \leq \delta(G)$$

Propriétés

On note $\delta(G)$ le plus petit degré de G .

Théorème

$$k(G) \leq k'(G) \leq \delta(G)$$

Théorème (Menger)

Soient G un graphe fini non-orienté, et s et t deux sommets distincts. Le nombre minimum d'arêtes à supprimer pour déconnecter s et t est égal au nombre maximum de chemins arête-disjoints de G reliant s et t .

Ainsi un graphe est k -arête-connexe pour k le plus petit nombre de chemins entre deux sommets.

Propriétés

On note $\delta(G)$ le plus petit degré de G .

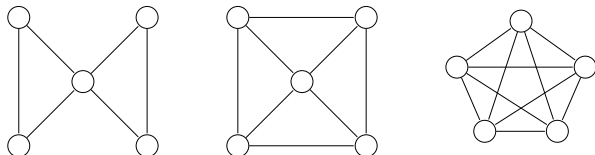
Théorème

$$k(G) \leq k'(G) \leq \delta(G)$$

Théorème (Menger)

Soient G un graphe fini non-orienté, et s et t deux sommets distincts. Le nombre minimum d'arêtes à supprimer pour déconnecter s et t est égal au nombre maximum de chemins arête-disjoints de G reliant s et t .

Ainsi un graphe est k -arête-connexe pour k le plus petit nombre de chemins entre deux sommets.



Plan

3 Connexité et parcours

- Composantes connexes
- Parcours
- Composantes fortement connexes
- Notion de k -connexité
- Tri topologique

Définition

Soit G un graphe **orienté sans circuit**

Définition

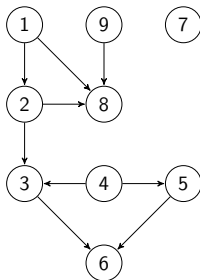
Un *tri topologique* est un ordre total $<$ sur les sommets dans lequel s'il existe un arc de s à t alors $s > t$.

Définition

Soit G un graphe **orienté sans circuit**

Définition

Un *tri topologique* est un ordre total $<$ sur les sommets dans lequel s'il existe un arc de s à t alors $s > t$.

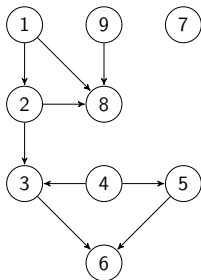


Définition

Soit G un graphe **orienté sans circuit**

Définition

Un *tri topologique* est un ordre total $<$ sur les sommets dans lequel s'il existe un arc de s à t alors $s > t$.



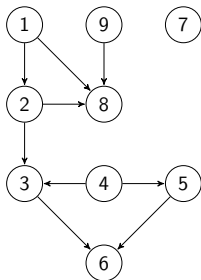
Exemple ordre : $7 > 1 > 2 > 9 > 8 > 4 > 3 > 5 > 6$

Définition

Soit G un graphe **orienté sans circuit**

Définition

Un *tri topologique* est un ordre total $<$ sur les sommets dans lequel s'il existe un arc de s à t alors $s > t$.



Exemple ordre : $7 > 1 > 2 > 9 > 8 > 4 > 3 > 5 > 6$

Application : problème d'ordonnancement de tâches

Algorithme

Parcours en profondeur, l'ordre croissant est donné par l'ordre de dépilement des sommets (numérotation *postfixe*)

Algorithme

Parcours en profondeur, l'ordre croissant est donné par l'ordre de dépilement des sommets (numérotation *postfixe*)

```
tritopoMain(G graphe)
```

```
  pour tout sommet s faire
```

```
    si s non marqué alors
```

```
      tritopoSlave(G, s
```

```
    )
```

```
    fin si
```

```
  fin pour
```

```
tritopoSlave(G graphe, s sommet  
  )
```

```
  Marquer s
```

```
  pour tout v voisin de s faire
```

```
    si v non marqué alors
```

```
      tritopoSlave(G, v  )
```

```
    fin si
```

```
  fin pour
```

Algorithme

Parcours en profondeur, l'ordre croissant est donné par l'ordre de dépilement des sommets (numérotation *postfixe*)

tritopoMain(G graphe)

File $\leftarrow \emptyset$

pour tout sommet s **faire**

si s non marqué **alors**

 tritopoSlave(G , s ,
 File)

fin si

fin pour

retourne File

tritopoSlave(G graphe, s sommet, F File
en réf)

Marquer s

pour tout v voisin de s **faire**

si v non marqué **alors**

 tritopoSlave(G , v , F)

fin si

Ajouter s à F

fin pour

Plan

1 Définitions

2 Programmation

3 Connexité et parcours

4 Plus courts chemins

5 Arbres couvrants

6 Flots

7 Coloriage de graphes

Plan

4 Plus courts chemins

- Présentation du problème
- Graphe non valué
- Graphe valué positif
- Graphe valué quelconque
- Comparaison et bilan

Présentation du problème



Aller le plus rapidement possible d'un point A à un point B :

Présentation du problème



Aller le plus rapidement possible d'un point A à un point B :

- En faisant le moins de correspondance possible

Présentation du problème



Aller le plus rapidement possible d'un point A à un point B :

- En faisant le moins de correspondance possible
- En passant le moins de temps possible dans le train (on néglige les correspondances)

Présentation du problème



Aller le plus rapidement possible d'un point A à un point B :

- En faisant le moins de correspondance possible : *graphe non valué*
- En passant le moins de temps possible dans le train (on néglige les correspondances)

Présentation du problème

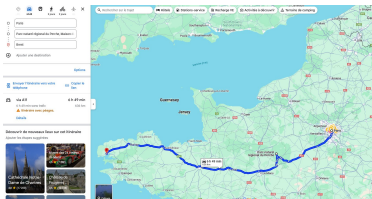


Aller le plus rapidement possible d'un point A à un point B :

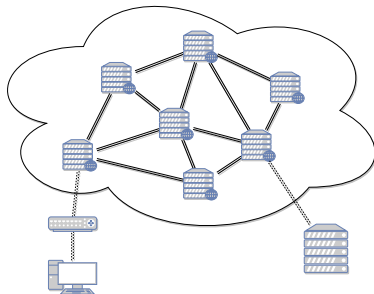
- En faisant le moins de correspondance possible : *graphe non valué*
- En passant le moins de temps possible dans le train (on néglige les correspondances) : *graphe valué*

Et alors ?

Trouver le chemin le plus court



Navigation GPS, itinéraire routier



Routage de paquets réseau : OSPF
(Open Shortest Path First)

Dans la vie du programmeur

Des applications omniprésentes

Dans la vie du programmeur

Des applications omniprésentes

- **Navigation GPS** : graphe valué (temps ou distance), Dijkstra ou A^*

Dans la vie du programmeur

Des applications omniprésentes

- **Navigation GPS** : graphe valué (temps ou distance), Dijkstra ou A*
- **Routage réseau (OSPF)** : graphe valué (latence), Dijkstra

Dans la vie du programmeur

Des applications omniprésentes

- **Navigation GPS** : graphe valué (temps ou distance), Dijkstra ou A^*
- **Routage réseau (OSPF)** : graphe valué (latence), Dijkstra
- **Jeux vidéo** : déplacement des PNJ dans une carte, A^*

Dans la vie du programmeur

Des applications omniprésentes

- **Navigation GPS** : graphe valué (temps ou distance), Dijkstra ou A^*
- **Routage réseau (OSPF)** : graphe valué (latence), Dijkstra
- **Jeux vidéo** : déplacement des PNJ dans une carte, A^*
- **Logistique** : minimiser le coût de livraison entre entrepôts

Dans la vie du programmeur

Des applications omniprésentes

- **Navigation GPS** : graphe valué (temps ou distance), Dijkstra ou A^*
- **Routage réseau (OSPF)** : graphe valué (latence), Dijkstra
- **Jeux vidéo** : déplacement des PNJ dans une carte, A^*
- **Logistique** : minimiser le coût de livraison entre entrepôts
- **Finance** : arbitrage de devises i.e. plusieurs changements de monnaie consécutifs qui au terme font gagner de l'argent (cycles absorbants = opportunité d'arbitrage)

Dans la vie du programmeur

Des applications omniprésentes

- **Navigation GPS** : graphe valué (temps ou distance), Dijkstra ou A^*
- **Routage réseau (OSPF)** : graphe valué (latence), Dijkstra
- **Jeux vidéo** : déplacement des PNJ dans une carte, A^*
- **Logistique** : minimiser le coût de livraison entre entrepôts
- **Finance** : arbitrage de devises i.e. plusieurs changements de monnaie consécutifs qui au terme font gagner de l'argent (cycles absorbants = opportunité d'arbitrage)

Notre programme :

Problème	Graphe	Algorithme
Graphe sans poids	Non valué	BFS
Poids positifs	Valué positif	Dijkstra
Poids quelconques	Valué	Bellman-Ford

Plan

4 Plus courts chemins

- Présentation du problème
- Graphe non valué
- Graphe valué positif
- Graphe valué quelconque
- Comparaison et bilan

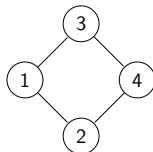
Propriétés des plus courts chemins

On veut minimiser la longueur (nombre d'arêtes) entre deux sommets.

Propriétés des plus courts chemins

On veut minimiser la longueur (nombre d'arêtes) entre deux sommets.

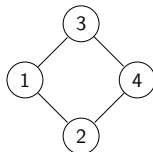
- Pas nécessairement unique



Propriétés des plus courts chemins

On veut minimiser la longueur (nombre d'arêtes) entre deux sommets.

- Pas nécessairement unique



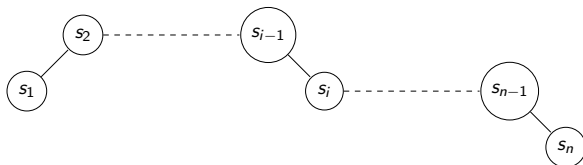
- Un plus court chemin entre deux sommets est *élémentaire* (tous les sommets sont distincts)

Propriétés des plus courts chemins

- *Principe de sous-optimalité* : si $p = (s_1, \dots, s_n)$ est un plus court chemin entre s_1 et s_n , soit $i \leq n$. Alors $(s_1, \dots, s_i)$ est un plus court chemin de s_1 à s_i , et $(s_i, \dots, s_n)$ est un plus court chemin de s_i à s_n .

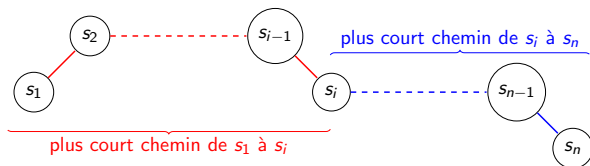
Propriétés des plus courts chemins

- *Principe de sous-optimalité* : si $p = (s_1, \dots, s_n)$ est un plus court chemin entre s_1 et s_n , soit $i \leq n$. Alors $(s_1, \dots, s_i)$ est un plus court chemin de s_1 à s_i , et $(s_i, \dots, s_n)$ est un plus court chemin de s_i à s_n .



Propriétés des plus courts chemins

- *Principe de sous-optimalité* : si $p = (s_1, \dots, s_n)$ est un plus court chemin entre s_1 et s_n , soit $i \leq n$. Alors $(s_1, \dots, s_i)$ est un plus court chemin de s_1 à s_i , et $(s_i, \dots, s_n)$ est un plus court chemin de s_i à s_n .



Parcours en largeur

But : obtenir tous les plus courts chemins à un sommet donné s .

Parcours en largeur

But : obtenir tous les plus courts chemins à un sommet donné s .

Chaque nœud à un label L (longueur à la fin de l'algorithme), et un label $Pred$ (prédécesseur dans le plus court chemin).

Parcours en largeur

But : obtenir tous les plus courts chemins à un sommet donné s .

Chaque nœud à un label L (longueur à la fin de l'algorithme), et un label $Pred$ (prédécesseur dans le plus court chemin).

Algorithme PCM_BFS(graphe G , sommet s) :

Entrée : Graphe G , sommet s

Tous les sommets sont non marqués. Marquer s . Ajouter s à la file F

$L(s) \leftarrow 0$

tant que F non vide **faire**

 Soit u la tête de F

pour tout v voisin de u non marqué **faire**

 Ajouter v à F et marquer v

$L(v) \leftarrow L(u) + 1$

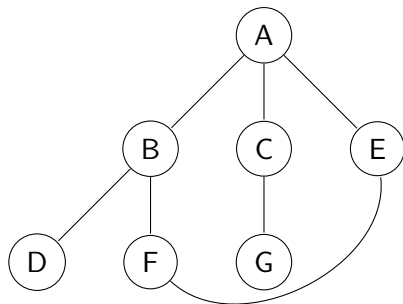
$Pred(v) \leftarrow u$

fin pour

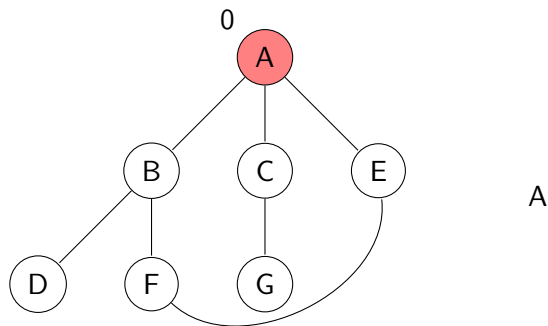
 Défiler F

fin tant que

Parcours en largeur - exemple avec file

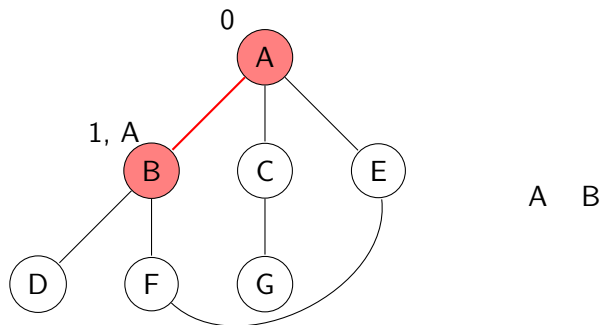


Parcours en largeur - exemple avec file



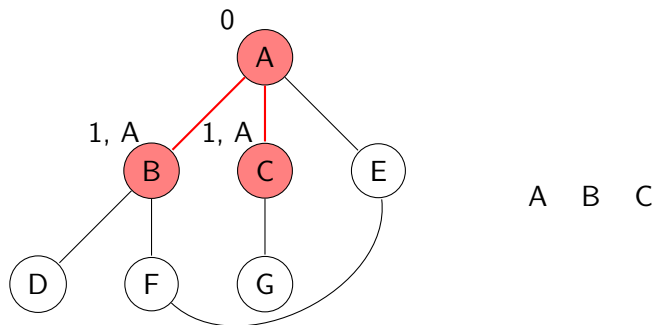
Choix dans l'ordre alphabétique arbitraire

Parcours en largeur - exemple avec file



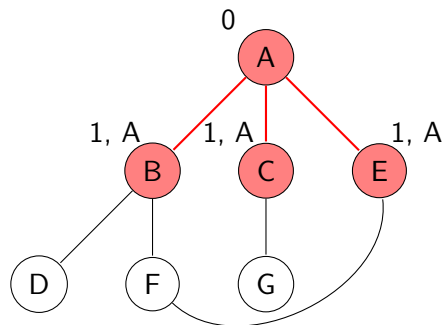
Choix dans l'ordre alphabétique arbitraire

Parcours en largeur - exemple avec file



Choix dans l'ordre alphabétique arbitraire

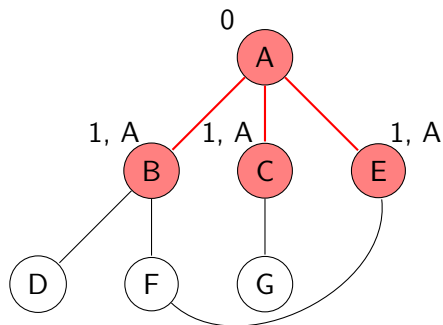
Parcours en largeur - exemple avec file



A B C E

Choix dans l'ordre alphabétique arbitraire

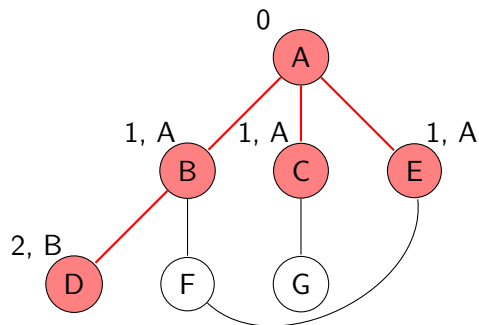
Parcours en largeur - exemple avec file



B C E

Choix dans l'ordre alphabétique arbitraire

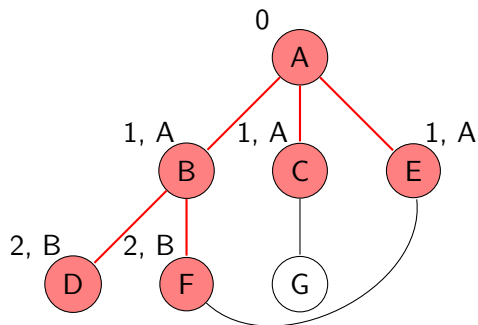
Parcours en largeur - exemple avec file



B C E D

Choix dans l'ordre alphabétique arbitraire

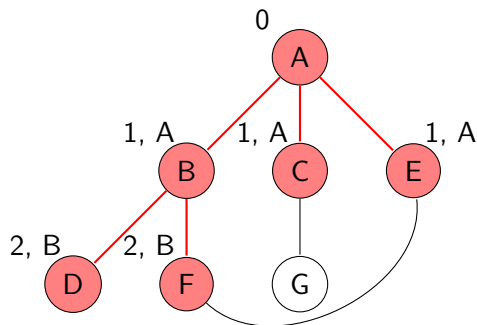
Parcours en largeur - exemple avec file



B C E D F

Choix dans l'ordre alphabétique arbitraire

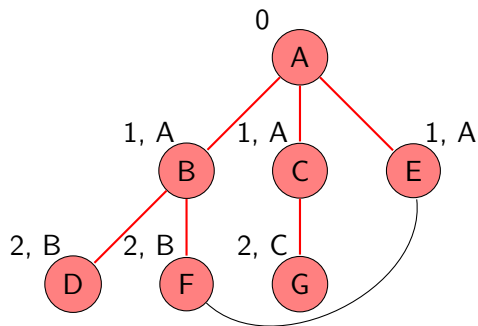
Parcours en largeur - exemple avec file



C E D F

Choix dans l'ordre alphabétique arbitraire

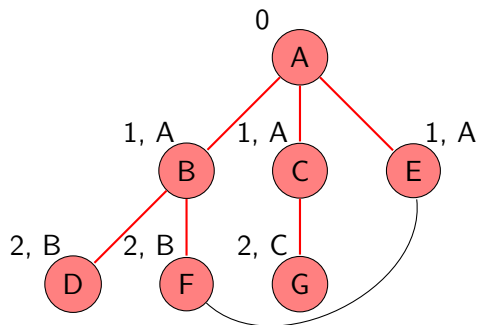
Parcours en largeur - exemple avec file



C E D F G

Choix dans l'ordre alphabétique arbitraire

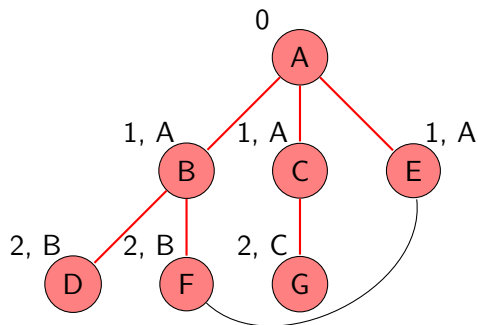
Parcours en largeur - exemple avec file



E D F G

Choix dans l'ordre alphabétique arbitraire

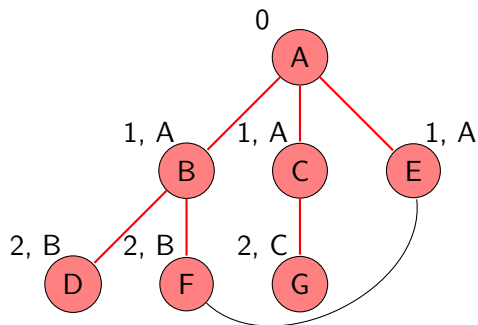
Parcours en largeur - exemple avec file



D F G

Choix dans l'ordre alphabétique arbitraire

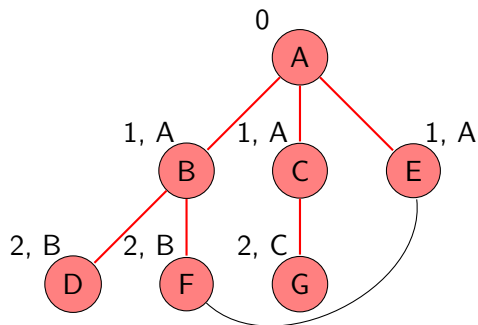
Parcours en largeur - exemple avec file



F G

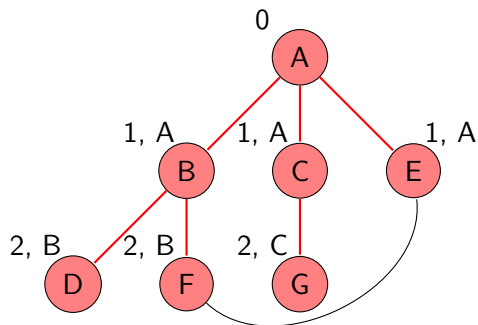
Choix dans l'ordre alphabétique arbitraire

Parcours en largeur - exemple avec file



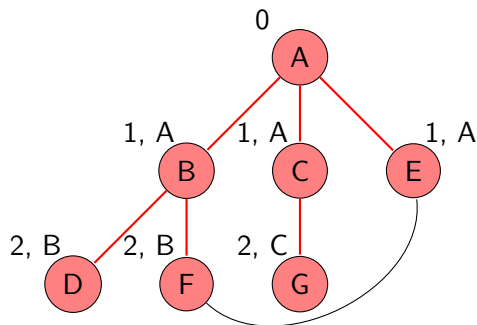
Choix dans l'ordre alphabétique arbitraire

Parcours en largeur - exemple avec file



Choix dans l'ordre alphabétique arbitraire

Parcours en largeur - exemple avec file



Choix dans l'ordre alphabétique arbitraire

Remarque : si on met initialement toutes les longueurs à ∞ , que signifie qu'un sommet est toujours à ∞ à la fin ?

Récupérer le chemin

On veut un plus court chemin de s à t .

Récupérer le chemin

On veut un plus court chemin de s à t .

Algorithme PCM(graphe G , sommet s , sommet t) :

Entrée : Graphe G , sommet s , sommet t

PCM_BFS(G , s)

Initialiser Pile P avec t

tant que Sommet(P) $\neq s$ **faire**

$y \leftarrow \text{Pred}(\text{Sommet}(P))$

 Empiler y sur P

fin tant que

On récupère une pile qui contient le chemin de s à t en dépilant.

Plan

4 Plus courts chemins

- Présentation du problème
- Graphe non valué
- **Graphe valué positif**
- Graphe valué quelconque
- Comparaison et bilan

Graphe valué

Définition

Un graphe $G = (V, E)$ est *valué* si toute arête $e \in E$ possède un poids $p(e) \in \mathbb{Z}$. Si $e = (x, y)$ on note aussi $p(x, y)$.

Le graphe est *valué positif* si tous les poids sont positifs ou nuls.

Le poids $p(c)$ d'un chemin c est la somme du poids des arêtes du chemin.

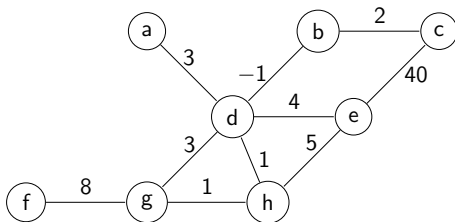
Graphe valué

Définition

Un graphe $G = (V, E)$ est **valué** si toute arête $e \in E$ possède un poids $p(e) \in \mathbb{Z}$. Si $e = (x, y)$ on note aussi $p(x, y)$.

Le graphe est **valué positif** si tous les poids sont positifs ou nuls.

Le poids $p(c)$ d'un chemin c est la somme du poids des arêtes du chemin.



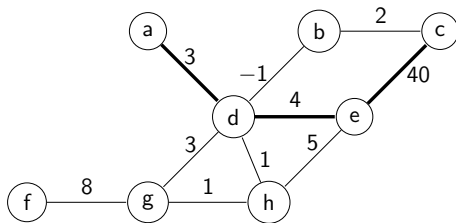
Graphe valué

Définition

Un graphe $G = (V, E)$ est **valué** si toute arête $e \in E$ possède un poids $p(e) \in \mathbb{Z}$. Si $e = (x, y)$ on note aussi $p(x, y)$.

Le graphe est **valué positif** si tous les poids sont positifs ou nuls.

Le poids $p(c)$ d'un chemin c est la somme du poids des arêtes du chemin.



Poids du chemin (a, d, e, c) : $3 + 4 + 40 = 47$

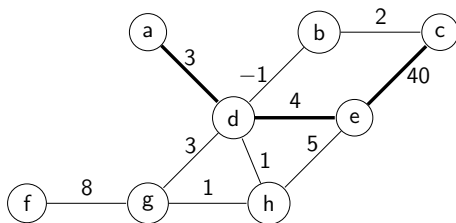
Graphe valué

Définition

Un graphe $G = (V, E)$ est **valué** si toute arête $e \in E$ possède un poids $p(e) \in \mathbb{Z}$. Si $e = (x, y)$ on note aussi $p(x, y)$.

Le graphe est **valué positif** si tous les poids sont positifs ou nuls.

Le poids $p(c)$ d'un chemin c est la somme du poids des arêtes du chemin.



Poids du chemin (a, d, e, c) : $3 + 4 + 40 = 47$

Remarque : le poids est un entier, mais ce n'est pas nécessaire

Sous-optimalité dans un graphe valué

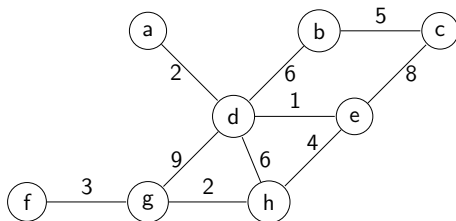
On recherche la distance $D(t)$ d'un sommet t à un sommet donné s :

- Si $t = s$, $D(t) = 0$
- Sinon, $D(t) = \min_{x \text{ voisin de } t} (D(x) + p(x, t))$

Idée : faire grossir progressivement un ensemble de sommets "traités" en propageant avec la formule à tous les non traités. Puis on choisit le non traité de poids minimal, on l'ajoute à notre ensemble et on recommence.

Dijkstra - déroulé naïf

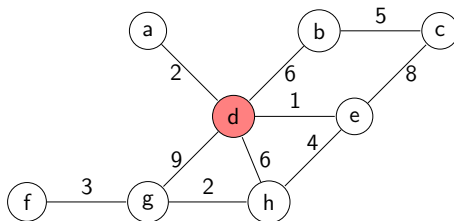
Distance à d



<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>	<i>h</i>
∞	∞	∞	∞	∞	∞	∞	∞

Dijkstra - déroulé naïf

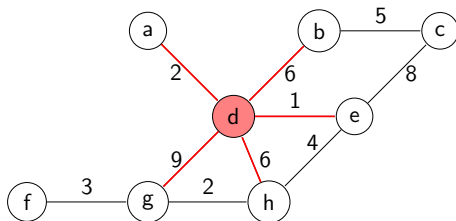
Distance à d



a	b	c	d	e	f	g	h
∞	∞	∞	0	∞	∞	∞	∞

Dijkstra - déroulé naïf

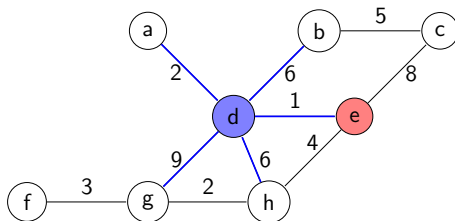
Distance à d



a	b	c	d	e	f	g	h
2	6	∞	0	1	∞	9	6

Dijkstra - déroulé naïf

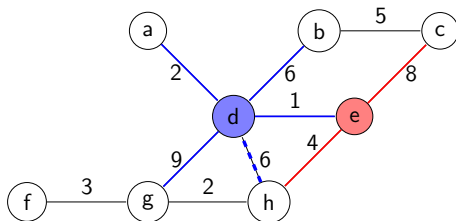
Distance à d



a	b	c	d	e	f	g	h
2	6	∞	0	1	∞	9	6

Dijkstra - déroulé naïf

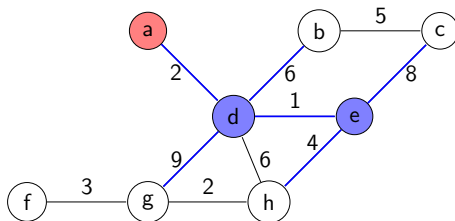
Distance à d



a	b	c	d	e	f	g	h
2	6	9	0	1	∞	9	5

Dijkstra - déroulé naïf

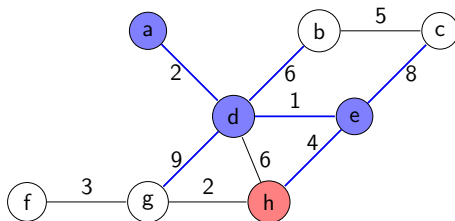
Distance à d



a	b	c	d	e	f	g	h
2	6	9	0	1	∞	9	5

Dijkstra - déroulé naïf

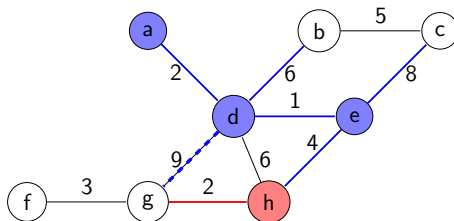
Distance à d



a	b	c	d	e	f	g	h
2	6	9	0	1	∞	9	5

Dijkstra - déroulé naïf

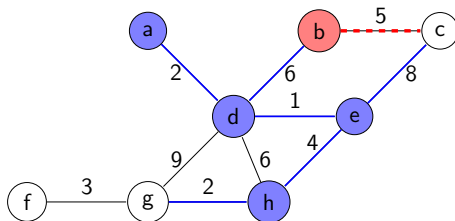
Distance à d



a	b	c	d	e	f	g	h
2	6	9	0	1	∞	7	5

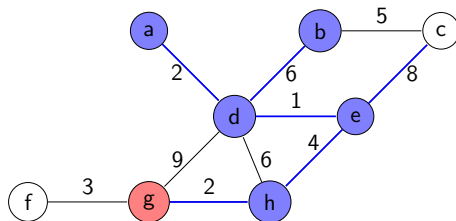
Dijkstra - déroulé naïf

Distance à d



a	b	c	d	e	f	g	h
2	6	9	0	1	∞	7	5

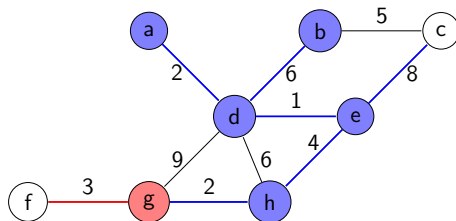
Dijkstra - déroulé naïf

Distance à d 

a	b	c	d	e	f	g	h
2	6	9	0	1	∞	7	5

Dijkstra - déroulé naïf

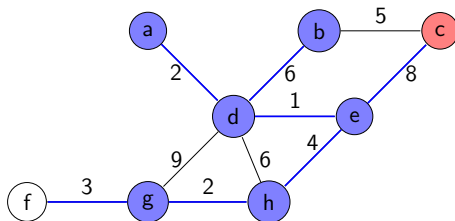
Distance à d



a	b	c	d	e	f	g	h
2	6	9	0	1	11	7	5

Dijkstra - déroulé naïf

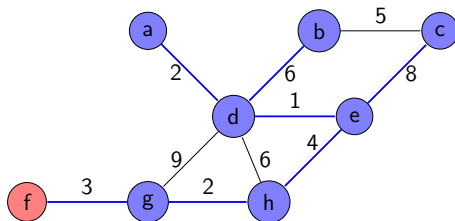
Distance à d



a	b	c	d	e	f	g	h
2	6	9	0	1	11	7	5

Dijkstra - déroulé naïf

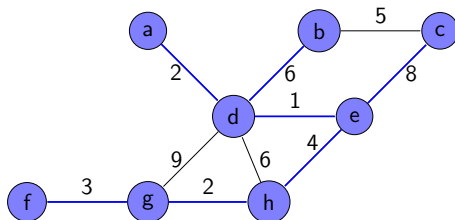
Distance à d



a	b	c	d	e	f	g	h
2	6	9	0	1	10	7	5

Dijkstra - déroulé naïf

Distance à d



a	b	c	d	e	f	g	h
2	6	9	0	1	10	7	5

Remarque : les seuls sommets qui changent de valeur sont les voisins du sommet ajouté

Algorithme de Dijkstra

Deux phases à l'algorithme :

- Sélection du sommet
- Mise à jour des voisins

Algorithme de Dijkstra

Algorithme Dijkstra(graphe G , sommet s) :

Entrée : Graphe G , sommet s

Tous les sommets non marqués. Marquer s . Tous les labels à ∞

$L(s) \leftarrow 0$

F file contient tous les sommets, triée en ordre croissant de poids

tant que il existe sommet non marqué **faire**

Prendre y sommet de poids minimal de F . Marquer y .

pour tout z successeur de y non marqué **faire**

si $L(z) > L(y) + p(y, z)$ **alors**

$L(z) \leftarrow L(y) + p(y, z)$

$Pred(z) \leftarrow y$

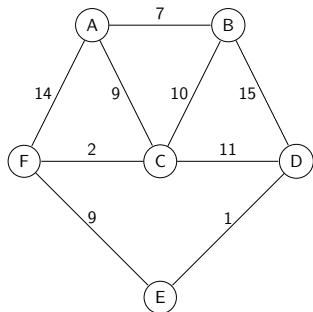
fin si

fin pour

Retirer y de F , et trier F par poids.

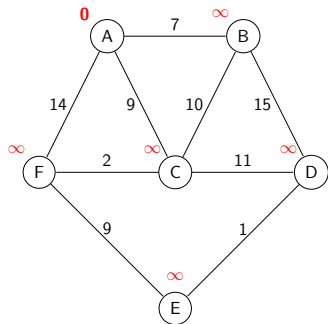
fin tant que

Algorithme de Dijkstra - Exemple 1



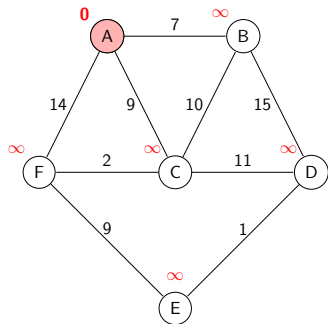
Étape	A	B	C	D	E	F

Algorithme de Dijkstra - Exemple 1



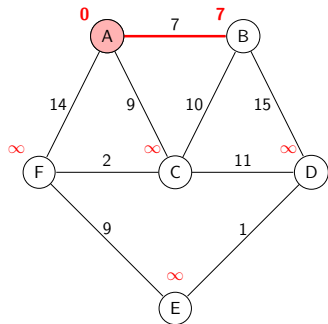
Étape	A	B	C	D	E	F
Initiale	0	∞	∞	∞	∞	∞

Algorithme de Dijkstra - Exemple 1



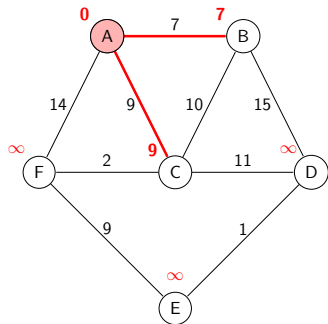
Étape	A	B	C	D	E	F
Initiale A(0)	0	∞	∞	∞	∞	∞

Algorithme de Dijkstra - Exemple 1



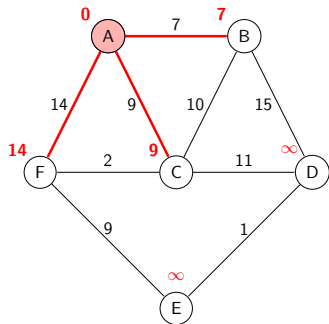
Étape	A	B	C	D	E	F
Initiale	0	∞	∞	∞	∞	∞
A(0)		7_A				

Algorithme de Dijkstra - Exemple 1



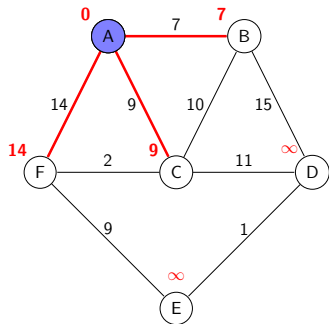
Étape	A	B	C	D	E	F
Initiale	0	∞	∞	∞	∞	∞
A(0)		7_A	9_A			

Algorithme de Dijkstra - Exemple 1



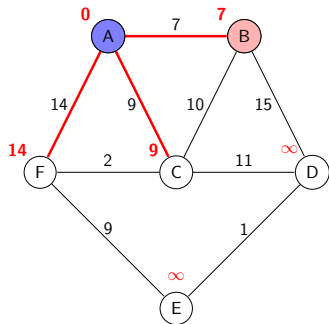
Étape	A	B	C	D	E	F
Initiale	0	∞	∞	∞	∞	∞
A(0)		7_A	9_A			14_A

Algorithme de Dijkstra - Exemple 1



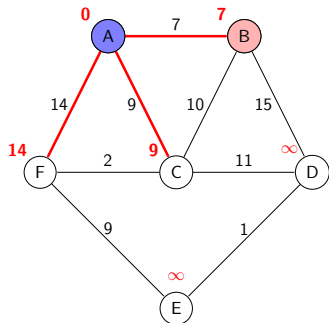
Étape	A	B	C	D	E	F
Initiale	0	∞	∞	∞	∞	∞
A(0)		7_A	9_A	∞	∞	14_A

Algorithme de Dijkstra - Exemple 1



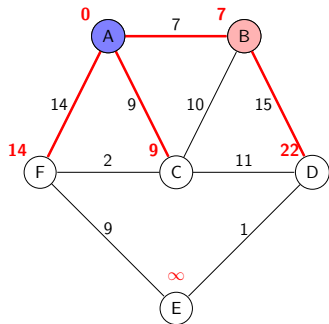
Étape	A	B	C	D	E	F
Initiale	0	∞	∞	∞	∞	∞
A(0)		7_A	9_A	∞	∞	14_A
B(7) _A						

Algorithme de Dijkstra - Exemple 1



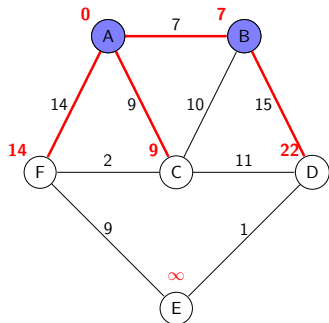
Étape	A	B	C	D	E	F
Initiale	0	∞	∞	∞	∞	∞
A(0)		7_A	9_A	∞	∞	14_A
B(7) _A			9_A			

Algorithme de Dijkstra - Exemple 1



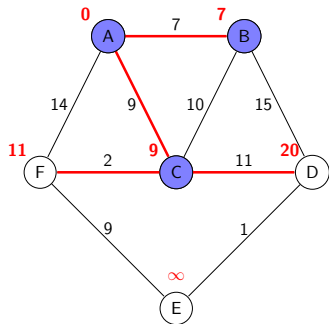
Étape	A	B	C	D	E	F
Initiale	0	∞	∞	∞	∞	∞
A(0)		7_A	9_A	∞	∞	14_A
B(7) _A			9_A	22_B		

Algorithme de Dijkstra - Exemple 1



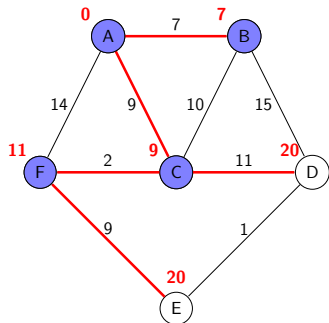
Étape	A	B	C	D	E	F
Initiale	0	∞	∞	∞	∞	∞
A(0)		7_A	9_A	∞	∞	14_A
B(7) _A			9_A	22_B	∞	14_A

Algorithme de Dijkstra - Exemple 1



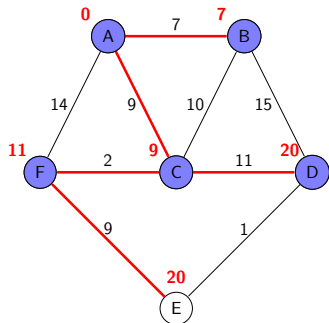
Étape	A	B	C	D	E	F
Initiale	0	∞	∞	∞	∞	∞
A(0)		7_A	9_A	∞	∞	14_A
B(7) _A			9_A	22_B	∞	14_A
C(9) _A				20_C	∞	11_C

Algorithme de Dijkstra - Exemple 1



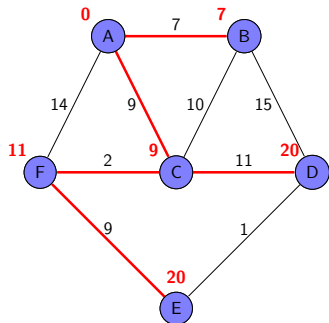
Étape	A	B	C	D	E	F
Initiale	0	∞	∞	∞	∞	∞
A(0)		7_A	9_A	∞	∞	14_A
B(7) _A			9_A	22_B	∞	14_A
C(9) _A				20_C	∞	11_C
F(11) _C				20_C	20_F	

Algorithme de Dijkstra - Exemple 1



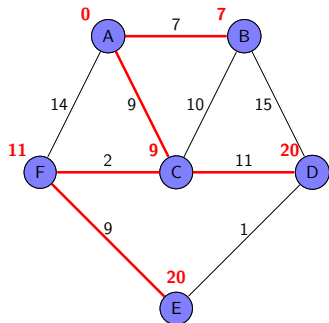
Étape	A	B	C	D	E	F
Initiale	0	∞	∞	∞	∞	∞
A(0)		7_A	9_A	∞	∞	14_A
B(7) _A			9_A	22_B	∞	14_A
C(9) _A				20_C	∞	11_C
F(11) _C				20_C	20_F	
D(20) _C					20_F	

Algorithme de Dijkstra - Exemple 1



Étape	A	B	C	D	E	F
Initiale	0	∞	∞	∞	∞	∞
A(0)		7_A	9_A	∞	∞	14_A
B(7) _A			9_A	22_B	∞	14_A
C(9) _A				20_C	∞	11_C
F(11) _C				20_C	20_F	
D(20) _C					20_F	
E(20) _F						

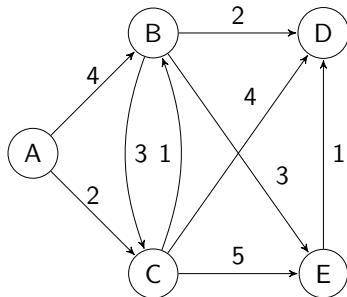
Algorithme de Dijkstra - Exemple 1



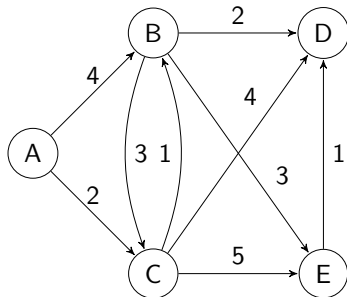
Étape	A	B	C	D	E	F
Initiale	0	∞	∞	∞	∞	∞
A(0)		7_A	9_A	∞	∞	14_A
B(7) _A			9_A	22_B	∞	14_A
C(9) _A				20_C	∞	11_C
F(11) _C				20_C	20_F	
D(20) _C					20_F	
E(20) _F						

Remarque : cela crée un arbre appelé *arbre des plus courts chemins*

Algorithme de Dijkstra - Exemple 2



Algorithme de Dijkstra - Exemple 2



A	B	C	D	E
0	∞	∞	∞	∞
	4_A	2_A	∞	∞
	3_C		6_C	7_C
			5_B	6_B
				6_B

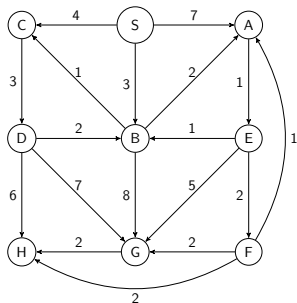
Algorithme de Dijkstra - Exercice

- ① Représenter graphiquement le graphe donné par matrice ci-dessous :

	S	A	B	C	D	E	F	G	H
S		7	3	4					
A						1			
B		2		1				8	
C					3				
D			2					7	6
E			1				2	5	
F		1						2	2
G									2
H									

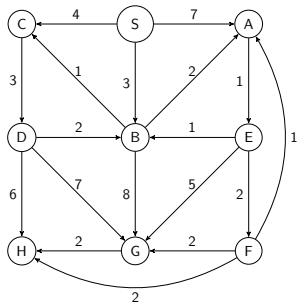
- ② En choisissant A comme sommet initial, faire tourner l'algorithme de Dijkstra des plus courts chemins sur ce graphe.

Algorithme de Dijkstra - exercice - correction



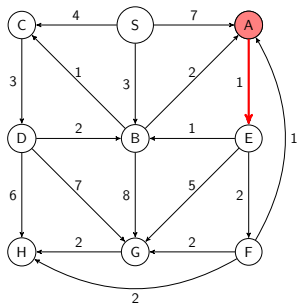
Étape	A	B	C	D	E	F	G	H	S

Algorithme de Dijkstra - exercice - correction



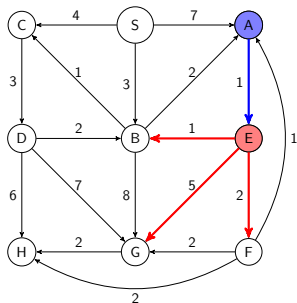
Étape	A	B	C	D	E	F	G	H	S
Initial	0	∞	∞	∞	∞	∞	∞	∞	∞

Algorithme de Dijkstra - exercice - correction



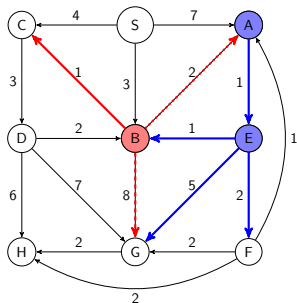
Étape	A	B	C	D	E	F	G	H	S
Initial	0	∞	∞	∞	∞	∞	∞	∞	∞
A(0)		∞	∞	∞	1_A	∞	∞	∞	∞

Algorithme de Dijkstra - exercice - correction



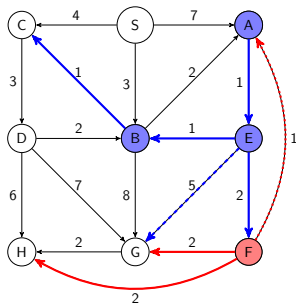
Étape	A	B	C	D	E	F	G	H	S
Initial	0	∞	∞	∞	∞	∞	∞	∞	∞
A(0)		∞	∞	∞	1_A	∞	∞	∞	∞
E(1) _A		2_E	∞	∞		3_E	6_E	∞	∞

Algorithme de Dijkstra - exercice - correction



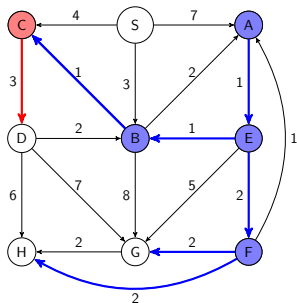
Étape	A	B	C	D	E	F	G	H	S
Initial	0	∞	∞	∞	∞	∞	∞	∞	∞
A(0)		∞	∞	∞	1_A	∞	∞	∞	∞
E(1) _A		2_E	∞	∞		3_E	6_E	∞	∞
B(2) _E			3_B	∞		3_E	6_E	∞	∞

Algorithme de Dijkstra - exercice - correction



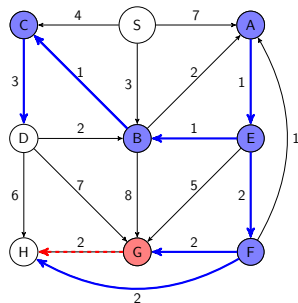
Étape	A	B	C	D	E	F	G	H	S
Initial	0	∞	∞	∞	∞	∞	∞	∞	∞
A(0)		∞	∞	∞	1_A	∞	∞	∞	∞
E(1) _A		2_E	∞	∞		3_E	6_E	∞	∞
B(2) _E			3_B	∞		3_E	6_E	∞	∞
F(3) _E			3_B	∞			5_F	5_F	∞

Algorithme de Dijkstra - exercice - correction



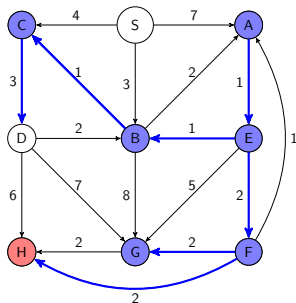
Étape	A	B	C	D	E	F	G	H	S
Initial	0	∞	∞	∞	∞	∞	∞	∞	∞
A(0)		∞	∞	∞	1 _A	∞	∞	∞	∞
E(1) _A		2 _E	∞	∞		3 _E	6 _E	∞	∞
B(2) _E			3 _B	∞		3 _E	6 _E	∞	∞
F(3) _E			3 _B	∞			5 _F	5 _F	∞
C(3) _B				6 _C			5 _F	5 _F	∞

Algorithme de Dijkstra - exercice - correction



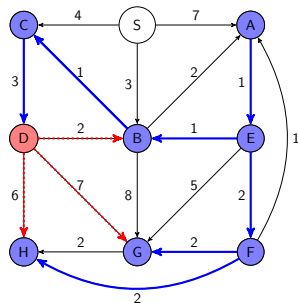
Étape	A	B	C	D	E	F	G	H	S
Initial	0	∞	∞	∞	∞	∞	∞	∞	∞
A(0)		∞	∞	∞	1 _A	∞	∞	∞	∞
E(1) _A		2 _E	∞	∞		3 _E	6 _E	∞	∞
B(2) _E			3 _B	∞		3 _E	6 _E	∞	∞
F(3) _E			3 _B	∞			5 _F	5 _F	∞
C(3) _B				6 _C			5 _F	5 _F	∞
G(5) _F				6 _C				5 _F	∞

Algorithme de Dijkstra - exercice - correction



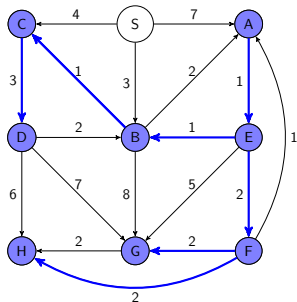
Étape	A	B	C	D	E	F	G	H	S
Initial	0	∞	∞	∞	∞	∞	∞	∞	∞
A(0)		∞	∞	∞	1 _A	∞	∞	∞	∞
E(1) _A		2 _E	∞	∞		3 _E	6 _E	∞	∞
B(2) _E			3 _B	∞		3 _E	6 _E	∞	∞
F(3) _E			3 _B	∞			5 _F	5 _F	∞
C(3) _B				6 _C			5 _F	5 _F	∞
G(5) _F				6 _C				5 _F	∞
H(5) _F				6 _C					∞

Algorithme de Dijkstra - exercice - correction



Étape	A	B	C	D	E	F	G	H	S
Initial	0	∞	∞	∞	∞	∞	∞	∞	∞
A(0)		∞	∞	∞	1 _A	∞	∞	∞	∞
E(1) _A		2 _E	∞	∞		3 _E	6 _E	∞	∞
B(2) _E			3 _B	∞		3 _E	6 _E	∞	∞
F(3) _E			3 _B	∞			5 _F	5 _F	∞
C(3) _B				6 _C			5 _F	5 _F	∞
G(5) _F				6 _C				5 _F	∞
H(5) _F				6 _C					∞
D(6) _C									∞

Algorithme de Dijkstra - exercice - correction



Étape	A	B	C	D	E	F	G	H	S
Initial	0	∞	∞	∞	∞	∞	∞	∞	∞
A(0)		∞	∞	∞	1 _A	∞	∞	∞	∞
E(1) _A		2 _E	∞	∞		3 _E	6 _E	∞	∞
B(2) _E			3 _B	∞		3 _E	6 _E	∞	∞
F(3) _E			3 _B	∞			5 _F	5 _F	∞
C(3) _B				6 _C			5 _F	5 _F	∞
G(5) _F				6 _C				5 _F	∞
H(5) _F				6 _C					∞
D(6) _C									∞
S(∞)									∞

Complexité de Dijkstra

On note $n = |V|$ et $m = |E|$.

Version naïve (tableau non trié)

Complexité de Dijkstra

On note $n = |V|$ et $m = |E|$.

Version naïve (tableau non trié)

- Trouver le minimum à chaque étape : $O(n)$. Répété n fois : $O(n^2)$.

Complexité de Dijkstra

On note $n = |V|$ et $m = |E|$.

Version naïve (tableau non trié)

- Trouver le minimum à chaque étape : $O(n)$. Répété n fois : $O(n^2)$.
- Mise à jour des voisins : $O(m)$ au total.

Complexité de Dijkstra

On note $n = |V|$ et $m = |E|$.

Version naïve (tableau non trié)

- Trouver le minimum à chaque étape : $O(n)$. Répété n fois : $O(n^2)$.
- Mise à jour des voisins : $O(m)$ au total.
- Ainsi on a un $O(n^2 + m)$.

Complexité de Dijkstra

On note $n = |V|$ et $m = |E|$.

Version naïve (tableau non trié)

- Trouver le minimum à chaque étape : $O(n)$. Répété n fois : $O(n^2)$.
- Mise à jour des voisins : $O(m)$ au total.
- Ainsi on a un $O(n^2 + m)$.
- Mais pour un graphe simple $m \leq \binom{n}{2} = \frac{n(n-1)}{2} = O(n^2)$.

Complexité de Dijkstra

On note $n = |V|$ et $m = |E|$.

Version naïve (tableau non trié)

- Trouver le minimum à chaque étape : $O(n)$. Répété n fois : $O(n^2)$.
- Mise à jour des voisins : $O(m)$ au total.
- Ainsi on a un $O(n^2 + m)$.
- Mais pour un graphe simple $m \leq \binom{n}{2} = \frac{n(n-1)}{2} = O(n^2)$.
- **Total** : $O(n^2)$ — adapté aux graphes denses (où $m \approx n^2$).

Complexité de Dijkstra

On note $n = |V|$ et $m = |E|$.

Version naïve (tableau non trié)

- Trouver le minimum à chaque étape : $O(n)$. Répété n fois : $O(n^2)$.
- Mise à jour des voisins : $O(m)$ au total.
- Ainsi on a un $O(n^2 + m)$.
- Mais pour un graphe simple $m \leq \binom{n}{2} = \frac{n(n-1)}{2} = O(n^2)$.
- **Total** : $O(n^2)$ — adapté aux graphes denses (où $m \approx n^2$).

Version avec tas binaire ([wikipédia](#), sorte d'arbre binaire)

Complexité de Dijkstra

On note $n = |V|$ et $m = |E|$.

Version naïve (tableau non trié)

- Trouver le minimum à chaque étape : $O(n)$. Répété n fois : $O(n^2)$.
- Mise à jour des voisins : $O(m)$ au total.
- Ainsi on a un $O(n^2 + m)$.
- Mais pour un graphe simple $m \leq \binom{n}{2} = \frac{n(n-1)}{2} = O(n^2)$.
- **Total** : $O(n^2)$ — adapté aux graphes denses (où $m \approx n^2$).

Version avec tas binaire ([wikipédia](#), sorte d'arbre binaire)

- Extraction du minimum : $O(\log n)$ (tas binaire). Répété n fois : $O(n \log n)$.

Complexité de Dijkstra

On note $n = |V|$ et $m = |E|$.

Version naïve (tableau non trié)

- Trouver le minimum à chaque étape : $O(n)$. Répété n fois : $O(n^2)$.
- Mise à jour des voisins : $O(m)$ au total.
- Ainsi on a un $O(n^2 + m)$.
- Mais pour un graphe simple $m \leq \binom{n}{2} = \frac{n(n-1)}{2} = O(n^2)$.
- **Total** : $O(n^2)$ — adapté aux graphes denses (où $m \approx n^2$).

Version avec tas binaire ([wikipédia](#), sorte d'arbre binaire)

- Extraction du minimum : $O(\log n)$ (tas binaire). Répété n fois : $O(n \log n)$.
- Mise à jour des voisins : $O(\log n)$ (tas binaire). Au plus une fois par arête soit m fois : $O(m \log n)$.

Complexité de Dijkstra

On note $n = |V|$ et $m = |E|$.

Version naïve (tableau non trié)

- Trouver le minimum à chaque étape : $O(n)$. Répété n fois : $O(n^2)$.
- Mise à jour des voisins : $O(m)$ au total.
- Ainsi on a un $O(n^2 + m)$.
- Mais pour un graphe simple $m \leq \binom{n}{2} = \frac{n(n-1)}{2} = O(n^2)$.
- **Total** : $O(n^2)$ — adapté aux graphes denses (où $m \approx n^2$).

Version avec tas binaire ([wikipédia](#), sorte d'arbre binaire)

- Extraction du minimum : $O(\log n)$ (tas binaire). Répété n fois : $O(n \log n)$.
- Mise à jour des voisins : $O(\log n)$ (tas binaire). Au plus une fois par arête soit m fois : $O(m \log n)$.
- **Total** : $O((n + m) \log n)$ — adapté aux graphes creux.

Complexité de Dijkstra

On note $n = |V|$ et $m = |E|$.

Version naïve (tableau non trié)

- Trouver le minimum à chaque étape : $O(n)$. Répété n fois : $O(n^2)$.
- Mise à jour des voisins : $O(m)$ au total.
- Ainsi on a un $O(n^2 + m)$.
- Mais pour un graphe simple $m \leq \binom{n}{2} = \frac{n(n-1)}{2} = O(n^2)$.
- **Total** : $O(n^2)$ — adapté aux graphes denses (où $m \approx n^2$).

Version avec tas binaire ([wikipédia](#), sorte d'arbre binaire)

- Extraction du minimum : $O(\log n)$ (tas binaire). Répété n fois : $O(n \log n)$.
- Mise à jour des voisins : $O(\log n)$ (tas binaire). Au plus une fois par arête soit m fois : $O(m \log n)$.
- **Total** : $O((n + m) \log n)$ — adapté aux graphes creux.

En pratique : tas binaire pour les graphes creux, tableau pour les graphes denses.

Plan

4 Plus courts chemins

- Présentation du problème
- Graphe non valué
- Graphe valué positif
- **Graphe valué quelconque**
- Comparaison et bilan

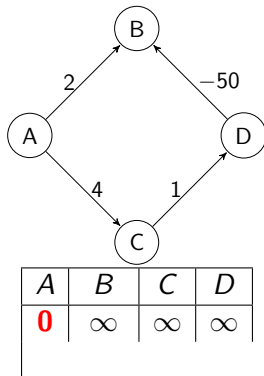
Problèmes

Les arêtes peuvent avoir un poids négatif

Problèmes

Les arêtes peuvent avoir un poids négatif

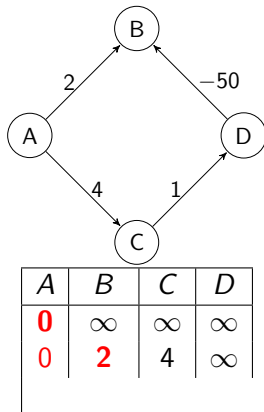
Problème 1 : un sommet "minimal" peut encore changer :



Problèmes

Les arêtes peuvent avoir un poids négatif

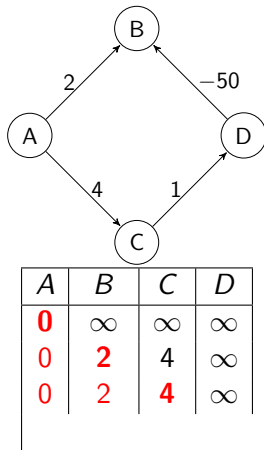
Problème 1 : un sommet "minimal" peut encore changer :



Problèmes

Les arêtes peuvent avoir un poids négatif

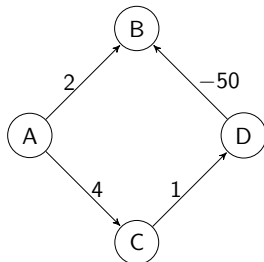
Problème 1 : un sommet "minimal" peut encore changer :



Problèmes

Les arêtes peuvent avoir un poids négatif

Problème 1 : un sommet "minimal" peut encore changer :

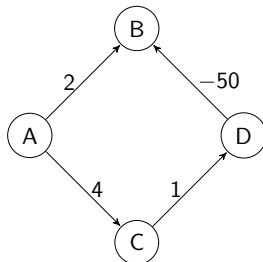


A	B	C	D
0	∞	∞	∞
0	2	4	∞
0	2	4	∞
0	2	4	5

Problèmes

Les arêtes peuvent avoir un poids négatif

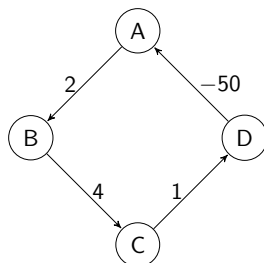
Problème 1 : un sommet "minimal" peut encore changer :



A	B	C	D
0	∞	∞	∞
0	2	4	∞
0	2	4	∞
0	2	4	5
0	-45	4	5

Problèmes

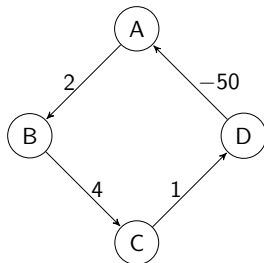
Problème 2 : un *cycle/circuit absorbant*



A	B	C	D
0	∞	∞	∞

Problèmes

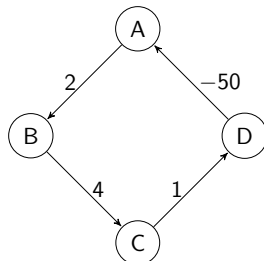
Problème 2 : un *cycle/circuit absorbant*



A	B	C	D
0	∞	∞	∞
0	2	∞	∞

Problèmes

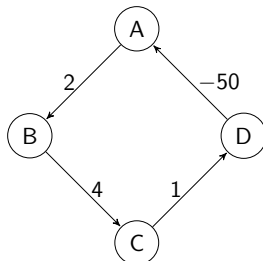
Problème 2 : un *cycle/circuit absorbant*



A	B	C	D
0	∞	∞	∞
0	2	∞	∞
0	2	6	∞

Problèmes

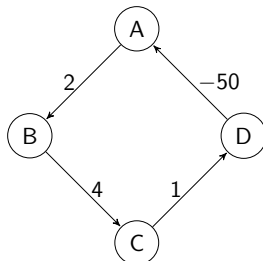
Problème 2 : un *cycle/circuit absorbant*



A	B	C	D
0	∞	∞	∞
0	2	∞	∞
0	2	6	∞
0	2	6	7

Problèmes

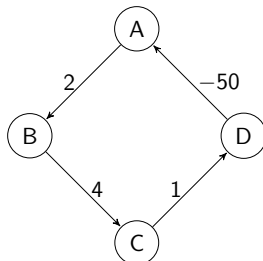
Problème 2 : un *cycle/circuit absorbant*



A	B	C	D
0	∞	∞	∞
0	2	∞	∞
0	2	6	∞
0	2	6	7
-43	2	4	5

Problèmes

Problème 2 : un *cycle/circuit absorbant*



A	B	C	D
0	∞	∞	∞
0	2	∞	∞
0	2	6	∞
0	2	6	7
-43	2	4	5
-43	-41	4	5
...			

Que faire ?

- Pour les cycles absorbants : juste les détecter

Que faire ?

- Pour les cycles absorbants : juste les détecter
- S'il n'y en a pas : avoir une distance

Que faire ?

- Pour les cycles absorbants : juste les détecter
- S'il n'y en a pas : avoir une distance

Algorithme de Bellman-Ford : faire plusieurs tours de boucle de mise à jour de tous les sommets

Remarque : cas particulier de programmation dynamique, cf. [ici](#)

Algorithme de Bellman-Ford

Algorithme Bellman_Ford(graphe G , sommet s) :

Entrée : Graphe G , sommet s

Tous les sommets non marqués. Marquer s . Tous les labels L à ∞

$L(s) \leftarrow 0$

pour $|V| - 1$ fois **faire**

pour tout $u \in V$ **faire**

pour tout u successeur de v **faire**

si $L(v) > L(u) + p(u, v)$ **alors**

$L(v) \leftarrow L(u) + p(u, v)$

$Pred(v) \leftarrow u$

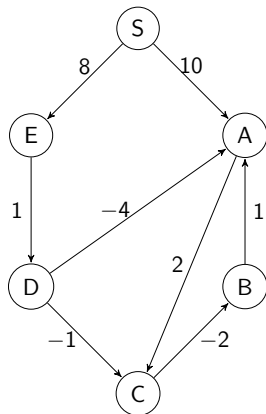
fin si

fin pour

fin pour

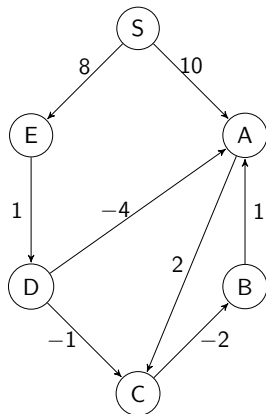
fin pour

Bellman-Ford - exemple



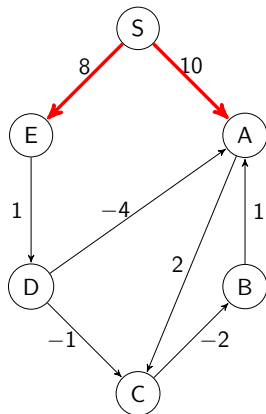
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>

Bellman-Ford - exemple



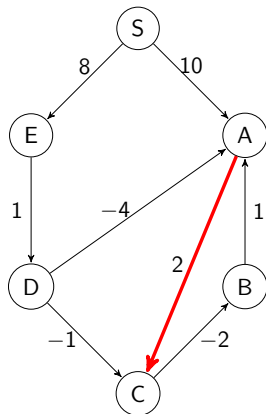
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>
0	∞	∞	∞	∞	∞	<i>k</i> = 0

Bellman-Ford - exemple



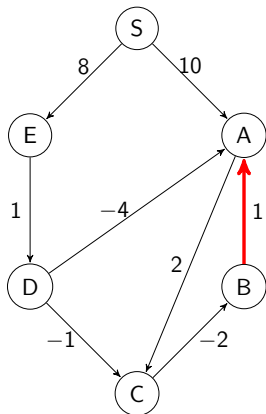
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>
0	∞	∞	∞	∞	∞	$k = 0$
0	10	∞	∞	∞	8	$k = 1$

Bellman-Ford - exemple



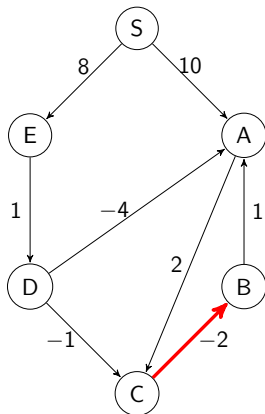
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>
0	∞	∞	∞	∞	∞	$k = 0$
0	10	∞	∞	∞	8	$k = 1$
0	10	∞	12	∞	8	

Bellman-Ford - exemple



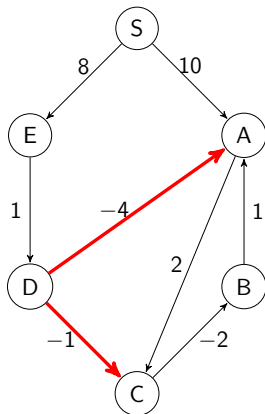
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>
0	∞	∞	∞	∞	∞	$k = 0$
0	10	∞	∞	∞	8	$k = 1$
0	10	∞	12	∞	8	
0	10	∞	12	∞	8	

Bellman-Ford - exemple



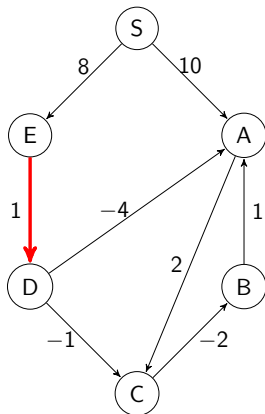
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>
0	∞	∞	∞	∞	∞	$k = 0$
0	10	∞	∞	∞	8	$k = 1$
0	10	∞	12	∞	8	
0	10	10	12	∞	8	
0	10	10	12	∞	8	

Bellman-Ford - exemple



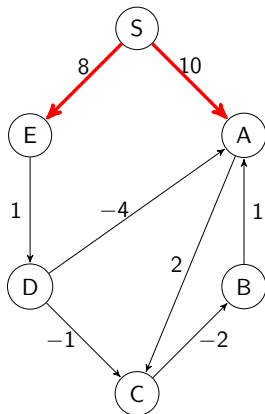
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>
0	∞	∞	∞	∞	∞	$k = 0$
0	10	∞	∞	∞	8	$k = 1$
0	10	∞	12	∞	8	
0	10	10	12	∞	8	
0	10	10	12	∞	8	
0	10	10	12	∞	8	

Bellman-Ford - exemple



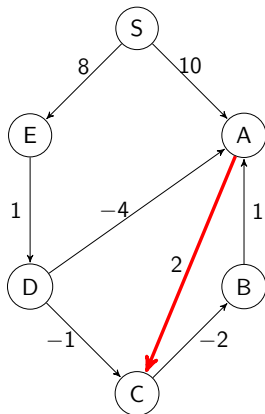
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>
0	∞	∞	∞	∞	∞	$k = 0$
0	10	∞	∞	∞	8	$k = 1$
0	10	∞	12	∞	8	
0	10	10	12	∞	8	
0	10	10	12	∞	8	
0	10	10	12	9	8	
0	10	10	12	9	8	

Bellman-Ford - exemple



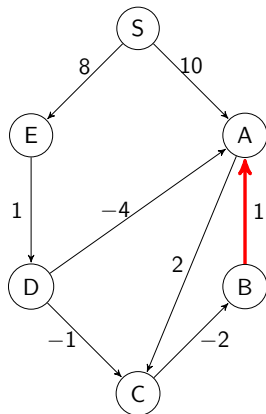
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>
0	∞	∞	∞	∞	∞	$k = 0$
0	10	∞	∞	∞	8	$k = 1$
0	10	∞	12	∞	8	
0	10	10	12	∞	8	
0	10	10	12	∞	8	
0	10	10	12	9	8	
0	10	10	12	9	8	
0	10	10	12	9	8	$k = 2$

Bellman-Ford - exemple



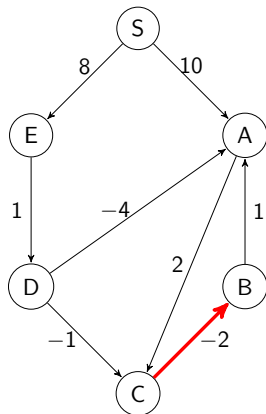
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>
0	∞	∞	∞	∞	∞	$k = 0$
0	10	∞	∞	∞	8	$k = 1$
0	10	∞	12	∞	8	
0	10	10	12	∞	8	
0	10	10	12	∞	8	
0	10	10	12	9	8	
0	10	10	12	9	8	
0	10	10	12	9	8	$k = 2$
0	10	10	12	9	8	

Bellman-Ford - exemple



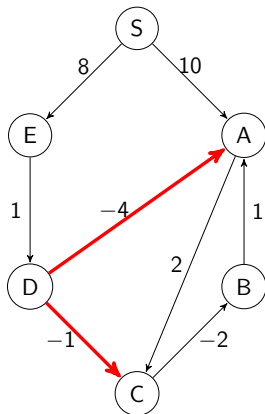
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>
0	∞	∞	∞	∞	∞	<i>k</i> = 0
0	10	∞	∞	∞	8	<i>k</i> = 1
0	10	∞	12	∞	8	
0	10	10	12	∞	8	
0	10	10	12	∞	8	
0	10	10	12	9	8	
0	10	10	12	9	8	
0	10	10	12	9	8	<i>k</i> = 2
0	10	10	12	9	8	
0	10	10	12	9	8	

Bellman-Ford - exemple



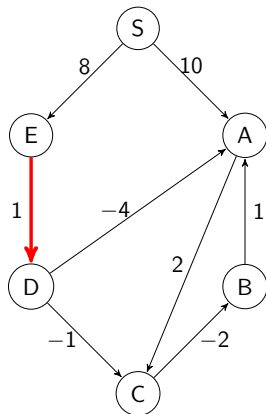
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>
0	∞	∞	∞	∞	∞	$k = 0$
0	10	∞	∞	∞	8	$k = 1$
0	10	∞	12	∞	8	
0	10	10	12	∞	8	
0	10	10	12	∞	8	
0	10	10	12	9	8	
0	10	10	12	9	8	
0	10	10	12	9	8	$k = 2$
0	10	10	12	9	8	
0	10	10	12	9	8	
0	10	10	12	9	8	

Bellman-Ford - exemple



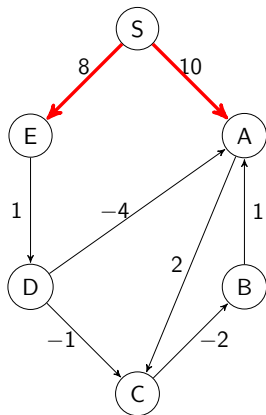
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>
0	∞	∞	∞	∞	∞	<i>k</i> = 0
0	10	∞	∞	∞	8	<i>k</i> = 1
0	10	∞	12	∞	8	
0	10	10	12	∞	8	
0	10	10	12	∞	8	
0	10	10	12	9	8	
0	10	10	12	9	8	
0	10	10	12	9	8	<i>k</i> = 2
0	10	10	12	9	8	
0	10	10	12	9	8	
0	5	10	8	9	8	
0	5	10	8	9	8	

Bellman-Ford - exemple



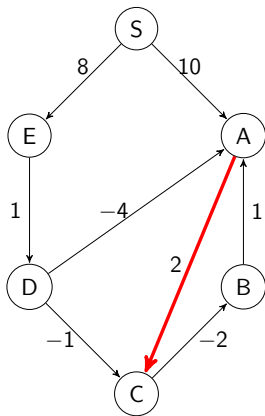
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>
0	∞	∞	∞	∞	∞	$k = 0$
0	10	∞	∞	∞	8	$k = 1$
0	10	∞	12	∞	8	
0	10	10	12	∞	8	
0	10	10	12	∞	8	
0	10	10	12	9	8	
0	10	10	12	9	8	
0	10	10	12	9	8	$k = 2$
0	10	10	12	9	8	
0	10	10	12	9	8	
0	10	10	12	9	8	
0	5	10	8	9	8	
0	5	10	8	9	8	

Bellman-Ford - exemple



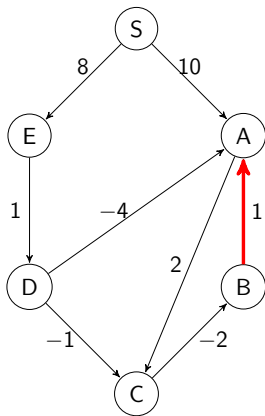
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>
0	5	10	8	9	8	$k = 3$

Bellman-Ford - exemple



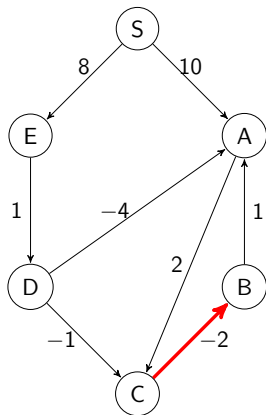
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>
0	5	10	8	9	8	$k = 3$
0	5	10	7	9	8	

Bellman-Ford - exemple



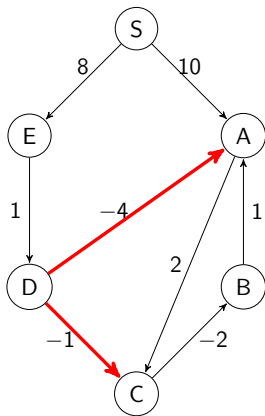
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>
0	5	10	8	9	8	$k = 3$
0	5	10	7	9	8	
0	5	10	7	9	8	

Bellman-Ford - exemple



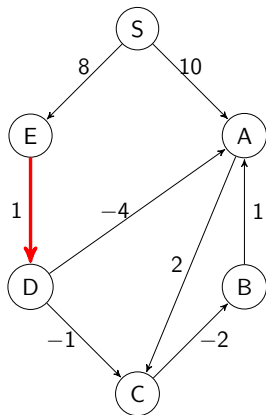
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>
0	5	10	8	9	8	$k = 3$
0	5	10	7	9	8	
0	5	10	7	9	8	
0	5	5	7	9	8	

Bellman-Ford - exemple



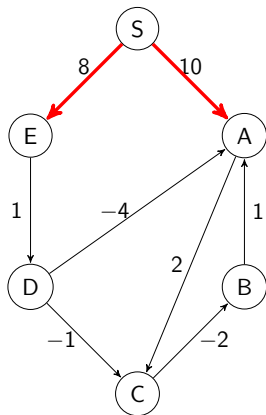
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>
0	5	10	8	9	8	$k = 3$
0	5	10	7	9	8	
0	5	10	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	

Bellman-Ford - exemple



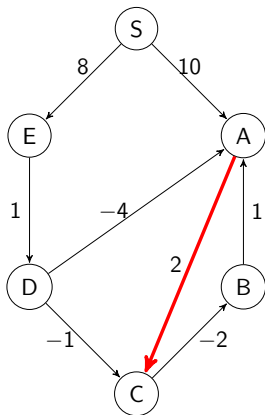
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>
0	5	10	8	9	8	<i>k</i> = 3
0	5	10	7	9	8	
0	5	10	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	

Bellman-Ford - exemple



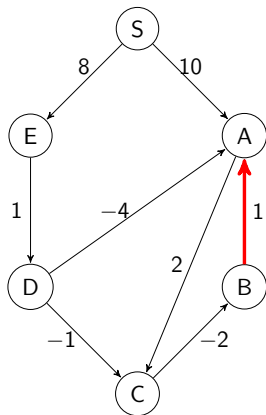
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>
0	5	10	8	9	8	<i>k</i> = 3
0	5	10	7	9	8	
0	5	10	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	<i>k</i> = 4

Bellman-Ford - exemple



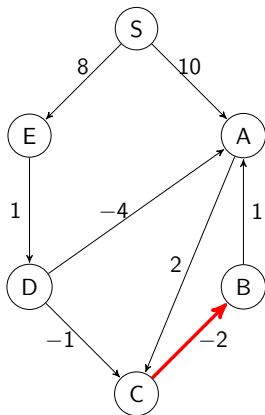
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>
0	5	10	8	9	8	$k = 3$
0	5	10	7	9	8	
0	5	10	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	$k = 4$
0	5	5	7	9	8	

Bellman-Ford - exemple



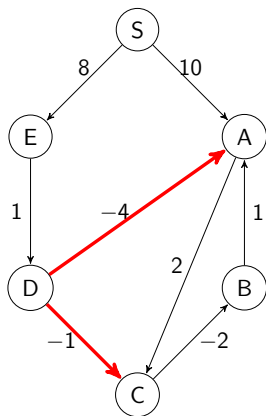
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>
0	5	10	8	9	8	<i>k</i> = 3
0	5	10	7	9	8	
0	5	10	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	<i>k</i> = 4
0	5	5	7	9	8	
0	5	5	7	9	8	

Bellman-Ford - exemple



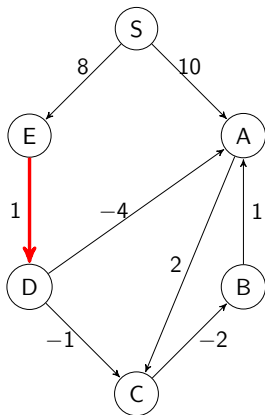
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>
0	5	10	8	9	8	<i>k</i> = 3
0	5	10	7	9	8	
0	5	10	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	<i>k</i> = 4
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	

Bellman-Ford - exemple



<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>
0	5	10	8	9	8	<i>k</i> = 3
0	5	10	7	9	8	
0	5	10	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	<i>k</i> = 4
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	

Bellman-Ford - exemple



<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>Iteration</i>
0	5	10	8	9	8	<i>k</i> = 3
0	5	10	7	9	8	
0	5	10	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	<i>k</i> = 4
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	
0	5	5	7	9	8	

Algorithme de Bellman-Ford - propriétés

- Au passage i , on a vu au moins tous les chemins de longueur inférieure à i . (D'où le nombre d'itérations pour les PCM qui sont élémentaires)

Algorithme de Bellman-Ford - propriétés

- Au passage i , on a vu au moins tous les chemins de longueur inférieure à i . (D'où le nombre d'itérations pour les PCM qui sont élémentaires)
- Si pas de changements entre deux passages, on peut s'arrêter

Algorithme de Bellman-Ford - propriétés

- Au passage i , on a vu au moins tous les chemins de longueur inférieure à i . (D'où le nombre d'itérations pour les PCM qui sont élémentaires)
- Si pas de changements entre deux passages, on peut s'arrêter
- Si on fait $|V|$ itérations et qu'il y a un changement, alors il y a un cycle absorbant. Réciproquement un cycle absorbant se détecte ainsi.

Plan

4 Plus courts chemins

- Présentation du problème
- Graphe non valué
- Graphe valué positif
- Graphe valué quelconque
- Comparaison et bilan

Comparaison des algorithmes

On note $n = |V|$ et $m = |E|$.

	BFS	Dijkstra	Bellman-Ford
Graphe	Non valué	Valué positif	Valué quelconque
Poids négatifs	—	Non	Oui
Cycles absorbants	—	—	Détecte
Complexité	$O(n + m)$	$O((n + m) \log n)$	$O(n \cdot m)$
Chemin reconstruit	Oui	Oui	Oui
Source unique	Oui	Oui	Oui

Comparaison des algorithmes

On note $n = |V|$ et $m = |E|$.

	BFS	Dijkstra	Bellman-Ford
Graphe	Non valué	Valué positif	Valué quelconque
Poids négatifs	—	Non	Oui
Cycles absorbants	—	—	Détecte
Complexité	$O(n + m)$	$O((n + m) \log n)$	$O(n \cdot m)$
Chemin reconstruit	Oui	Oui	Oui
Source unique	Oui	Oui	Oui

- **BFS** : toujours préférer quand les poids sont égaux (ou absents)

Comparaison des algorithmes

On note $n = |V|$ et $m = |E|$.

	BFS	Dijkstra	Bellman-Ford
Graphe	Non valué	Valué positif	Valué quelconque
Poids négatifs	—	Non	Oui
Cycles absorbants	—	—	Détecte
Complexité	$O(n + m)$	$O((n + m) \log n)$	$O(n \cdot m)$
Chemin reconstruit	Oui	Oui	Oui
Source unique	Oui	Oui	Oui

- **BFS** : toujours préférer quand les poids sont égaux (ou absents)
- **Dijkstra** : standard pour les graphes valués positifs (GPS, routage)

Comparaison des algorithmes

On note $n = |V|$ et $m = |E|$.

	BFS	Dijkstra	Bellman-Ford
Graphe	Non valué	Valué positif	Valué quelconque
Poids négatifs	—	Non	Oui
Cycles absorbants	—	—	Détecte
Complexité	$O(n + m)$	$O((n + m) \log n)$	$O(n \cdot m)$
Chemin reconstruit	Oui	Oui	Oui
Source unique	Oui	Oui	Oui

- **BFS** : toujours préférer quand les poids sont égaux (ou absents)
- **Dijkstra** : standard pour les graphes valués positifs (GPS, routage)
- **Bellman-Ford** : indispensable avec des poids négatifs ou pour détecter les cycles absorbants

Récapitulatif

Problème	Plus court chemin source $\rightarrow$ destination
Variantes	Un sommet, tous les sommets, toutes les paires
Propriété clé	Principe de sous-optimalité
Algo 1	BFS — $O(V + E)$ — graphes non valués
Algo 2	Dijkstra — $O((V + E) \log V)$ — poids positifs
Algo 3	Bellman-Ford — $O(V \cdot E)$ — poids quelconques
Pièges	Poids négatifs $\Rightarrow$ Dijkstra incorrect Cycle absorbant $\Rightarrow$ pas de plus court chemin
Applications	GPS, routage réseau, jeux vidéo, finance

Récapitulatif

Problème	Plus court chemin source $\rightarrow$ destination
Variantes	Un sommet, tous les sommets, toutes les paires
Propriété clé	Principe de sous-optimalité
Algo 1	BFS — $O(V + E)$ — graphes non valués
Algo 2	Dijkstra — $O((V + E) \log V)$ — poids positifs
Algo 3	Bellman-Ford — $O(V \cdot E)$ — poids quelconques
Pièges	Poids négatifs $\Rightarrow$ Dijkstra incorrect Cycle absorbant $\Rightarrow$ pas de plus court chemin
Applications	GPS, routage réseau, jeux vidéo, finance

Aller plus loin :

- Toutes les paires : Floyd-Warshall $O(|V|^3)$, Johnson $O(|V||E| + |V|^2 \log |V|)$
- Source unique avec heuristique : A^* (si distance estimable)

Plan

1 Définitions

2 Programmation

3 Connexité et parcours

4 Plus courts chemins

5 Arbres couvrants

6 Flots

7 Coloriage de graphes

Plan

5 Arbres couvrants

- Arbre
- Présentation du problème
- Arbre couvrant minimal
- Algorithme de Kruskal
- Algorithme de Prim
- Propriétés et bilan

Définition

On se met en cadre non orienté.

Définition

On se met en cadre non orienté.

Définition (arbre)

*Un **arbre** est un graphe connexe sans cycles.*

Définition

On se met en cadre non orienté.

Définition (arbre)

Un *arbre* est un graphe connexe sans cycles.

Définition (arbre)

Une *forêt* est un graphe sans cycles.

Définition

On se met en cadre non orienté.

Définition (arbre)

*Un **arbre** est un graphe connexe sans cycles.*

Définition (arbre)

*Une **forêt** est un graphe sans cycles. Chacune de ses composantes connexes est donc un arbre.*

Définition

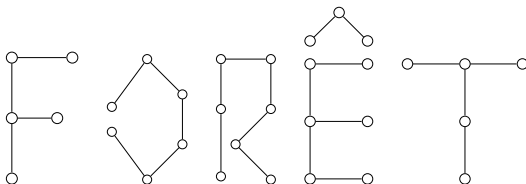
On se met en cadre non orienté.

Définition (arbre)

Un *arbre* est un graphe connexe sans cycles.

Définition (arbre)

Une *forêt* est un graphe sans cycles. Chacune de ses composantes connexes est donc un arbre.



Caractérisations d'un arbre

Pour un graphe T d'ordre n , il y a équivalence entre les propriétés :

Caractérisations d'un arbre

Pour un graphe T d'ordre n , il y a équivalence entre les propriétés :

- 1 T est un arbre

Caractérisations d'un arbre

Pour un graphe T d'ordre n , il y a équivalence entre les propriétés :

- 1 T est un arbre
- 2 Deux sommets quelconques de T sont connectés par une chaîne unique

Caractérisations d'un arbre

Pour un graphe T d'ordre n , il y a équivalence entre les propriétés :

- 1 T est un arbre
- 2 Deux sommets quelconques de T sont connectés par une chaîne unique
- 3 T est un graphe connexe à $n - 1$ arêtes

Caractérisations d'un arbre

Pour un graphe T d'ordre n , il y a équivalence entre les propriétés :

- 1 T est un arbre
- 2 Deux sommets quelconques de T sont connectés par une chaîne unique
- 3 T est un graphe connexe à $n - 1$ arêtes
- 4 T est connexe, et la suppression de toute arête le déconnecte

Caractérisations d'un arbre

Pour un graphe T d'ordre n , il y a équivalence entre les propriétés :

- 1 T est un arbre
- 2 Deux sommets quelconques de T sont connectés par une chaîne unique
- 3 T est un graphe connexe à $n - 1$ arêtes
- 4 T est connexe, et la suppression de toute arête le déconnecte
- 5 T est acyclique à $n - 1$ arêtes

Caractérisations d'un arbre

Pour un graphe T d'ordre n , il y a équivalence entre les propriétés :

- 1 T est un arbre
- 2 Deux sommets quelconques de T sont connectés par une chaîne unique
- 3 T est un graphe connexe à $n - 1$ arêtes
- 4 T est connexe, et la suppression de toute arête le déconnecte
- 5 T est acyclique à $n - 1$ arêtes
- 6 T est acyclique et l'ajout de toute arête le rend cyclique en créant un unique cycle.

Lien avec le cours d'algorithmie

Si T est un arbre, souvent on choisit un sommet r arbitrairement. On l'appelle *racine*.

Lien avec le cours d'algorithmie

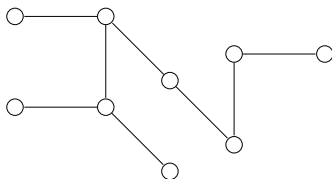
Si T est un arbre, souvent on choisit un sommet r arbitrairement. On l'appelle *racine*.

Cela revient à orienter l'arbre. La racine étant l'ancêtre généalogique de l'arbre généalogique. D'où la notion de *fils*, *père*, *frère*, etc.

Lien avec le cours d'algorithmie

Si T est un arbre, souvent on choisit un sommet r arbitrairement. On l'appelle *racine*.

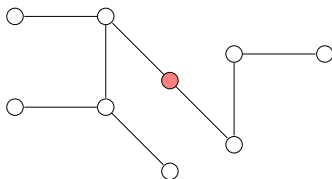
Cela revient à orienter l'arbre. La racine étant l'ancêtre généalogique de l'arbre généalogique. D'où la notion de *fils*, *père*, *frère*, etc.



Lien avec le cours d'algorithmie

Si T est un arbre, souvent on choisit un sommet r arbitrairement. On l'appelle *racine*.

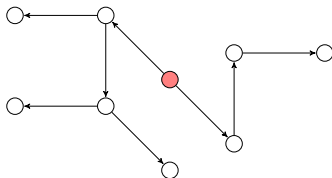
Cela revient à orienter l'arbre. La racine étant l'ancêtre généalogique de l'arbre généalogique. D'où la notion de *fils*, *père*, *frère*, etc.



Lien avec le cours d'algorithmie

Si T est un arbre, souvent on choisit un sommet r arbitrairement. On l'appelle *racine*.

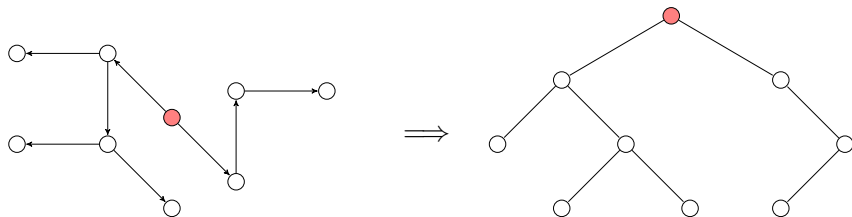
Cela revient à orienter l'arbre. La racine étant l'ancêtre généalogique de l'arbre généalogique. D'où la notion de *fils*, *père*, *frère*, etc.



Lien avec le cours d'algorithmie

Si T est un arbre, souvent on choisit un sommet r arbitrairement. On l'appelle *racine*.

Cela revient à orienter l'arbre. La racine étant l'ancêtre généalogique de l'arbre généalogique. D'où la notion de *fils*, *père*, *frère*, etc.

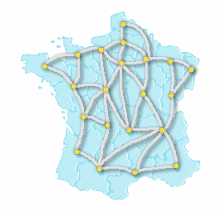


Plan

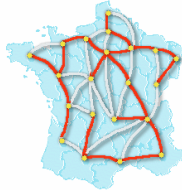
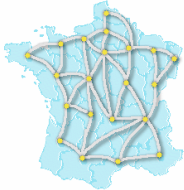
5 Arbres couvrants

- Arbre
- **Présentation du problème**
- Arbre couvrant minimal
- Algorithme de Kruskal
- Algorithme de Prim
- Propriétés et bilan

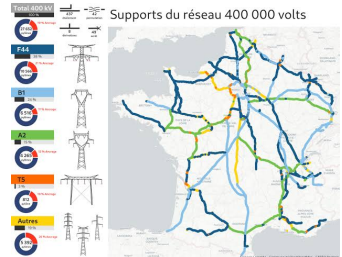
Connecter des objets au moindre coût



Connecter des objets au moindre coût



Réseau ferroviaire : relier toutes les villes avec le moins de voies possibles



Réseau électrique : relier des villes avec le moins de câbles

Dans la vie du programmeur

Des applications très concrètes

Dans la vie du programmeur

Des applications très concrètes

- **Câblage réseau** : relier n machines dans un bâtiment avec le minimum de câble — minimiser le coût d'infrastructure

Dans la vie du programmeur

Des applications très concrètes

- **Câblage réseau** : relier n machines dans un bâtiment avec le minimum de câble — minimiser le coût d'infrastructure
- **Machine Learning** : le single-linkage clustering, algorithme de clustering sur données. On construit graphe complet (sommet = data, arête = distance) avec ACM + suppression des $k - 1$ arêtes de plus gros poids. On a k clusters.

Dans la vie du programmeur

Des applications très concrètes

- **Câblage réseau** : relier n machines dans un bâtiment avec le minimum de câble — minimiser le coût d'infrastructure
- **Machine Learning** : le single-linkage clustering, algorithme de clustering sur données. On construit graphe complet (sommet = data, arête = distance) avec ACM + suppression des $k - 1$ arêtes de plus gros poids. On a k clusters.
- **Approximation du voyageur de commerce** : le problème du voyageur (NP-difficile) a une approximation à un facteur au plus 2 en prenant le cycle hamiltonien obtenu en faisant un parcours en profondeur dans un ACM (si les distances respectent inégalité triangulaire).

Plan

5 Arbres couvrants

- Arbre
- Présentation du problème
- **Arbre couvrant minimal**
- Algorithme de Kruskal
- Algorithme de Prim
- Propriétés et bilan

Arbre couvrant

Définition (Rappel : graphe partiel)

Un *graphe partiel* de G est un graphe $H = (V, F)$ avec $F \subseteq E$.

Arbre couvrant

Définition (Rappel : graphe partiel)

Un *graphe partiel* de G est un graphe $H = (V, F)$ avec $F \subseteq E$.

Définition (Arbre couvrant)

Un *arbre couvrant* de G est un graphe partiel de G qui est un arbre.

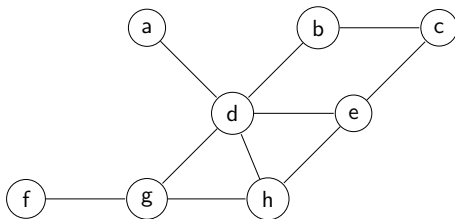
Arbre couvrant

Définition (Rappel : graphe partiel)

Un *graphe partiel* de G est un graphe $H = (V, F)$ avec $F \subseteq E$.

Définition (Arbre couvrant)

Un *arbre couvrant* de G est un graphe partiel de G qui est un arbre.



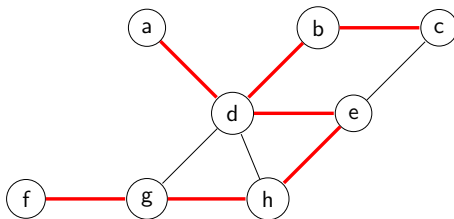
Arbre couvrant

Définition (Rappel : graphe partiel)

Un *graphe partiel* de G est un graphe $H = (V, F)$ avec $F \subseteq E$.

Définition (Arbre couvrant)

Un *arbre couvrant* de G est un graphe partiel de G qui est un arbre.



Propriétés générales

Théorème

Tout graphe connexe G admet un arbre couvrant.

Propriétés générales

Théorème

Tout graphe connexe G admet un arbre couvrant.

Une question naturelle (hors programme) :

Propriétés générales

Théorème

Tout graphe connexe G admet un arbre couvrant.

Une question naturelle (hors programme) : combien d'arbres couvrants ?

Propriétés générales

Théorème

Tout graphe connexe G admet un arbre couvrant.

Une question naturelle (hors programme) : combien d'arbres couvrants ?
Différentes méthodes existent pour cela (Cayley, calcul de mineurs (déterminant) d'une matrice d'adjacence, etc.)

Définition

Soit G un graphe. Le *poids* de G est la somme du poids de ses arêtes.

Définition

Soit G un graphe. Le *poids* de G est la somme du poids de ses arêtes.

Le problème revient donc à chercher un graphe partiel connexe de poids minimal : c'est forcément un arbre : *arbre couvrant minimal* (ACM).

Définition

Soit G un graphe. Le *poids* de G est la somme du poids de ses arêtes.

Le problème revient donc à chercher un graphe partiel connexe de poids minimal : c'est forcément un arbre : *arbre couvrant minimal* (ACM).

Deux algorithmes principaux :

Définition

Soit G un graphe. Le *poids* de G est la somme du poids de ses arêtes.

Le problème revient donc à chercher un graphe partiel connexe de poids minimal : c'est forcément un arbre : *arbre couvrant minimal* (ACM).

Deux algorithmes principaux :

- Kruskal : algorithme glouton sur les arêtes
- Prim : algorithme de marquage sur les sommets

Plan

5 Arbres couvrants

- Arbre
- Présentation du problème
- Arbre couvrant minimal
- **Algorithme de Kruskal**
- Algorithme de Prim
- Propriétés et bilan

Algorithme de Kruskal

Algorithme Kruskal(graphe valué G) :

Entrée : $G = (V, E)$ graphe valué

Soit F une liste d'arêtes vide.

Trier E par ordre croissant de poids

pour $e \in E$ et tant que $|F| < |V| - 1$ **faire**

si $F \cup e$ ne forme pas de cycle **alors**

$F \leftarrow F \cup e$

fin si

fin pour

Algorithme de Kruskal

Algorithme Kruskal(graphe valué G) :

Entrée : $G = (V, E)$ graphe valué

Soit F une liste d'arêtes vide.

Trier E par ordre croissant de poids

pour $e \in E$ et tant que $|F| < |V| - 1$ **faire**

si $F \cup e$ ne forme pas de cycle **alors**

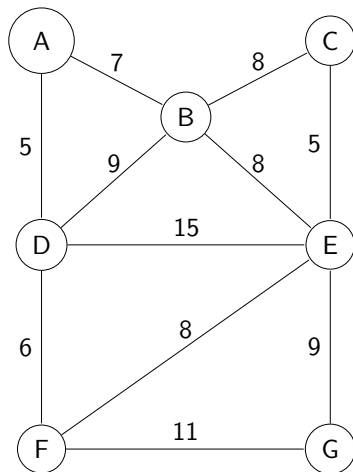
$F \leftarrow F \cup e$

fin si

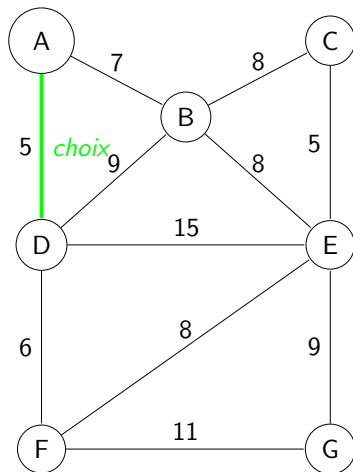
fin pour

- Plusieurs choix possibles
- Calcule une forêt couvrante minimale (donc un ACM si connexe)
- Complexité dépend du tri : $O(|E| \log |E|)$

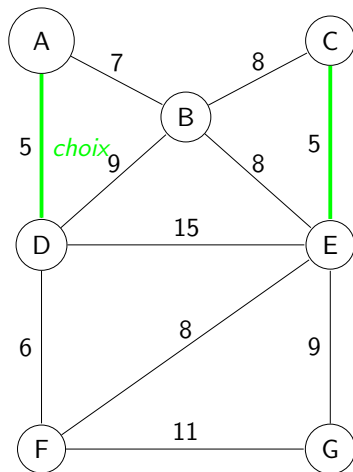
Algorithme de Kruskal - exemple



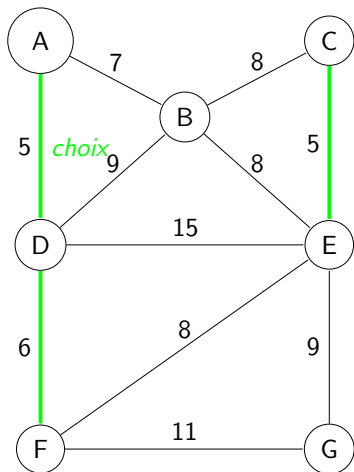
Algorithme de Kruskal - exemple



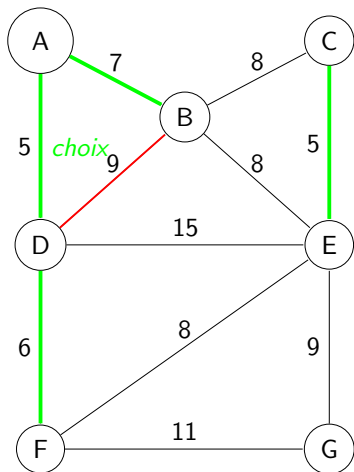
Algorithme de Kruskal - exemple



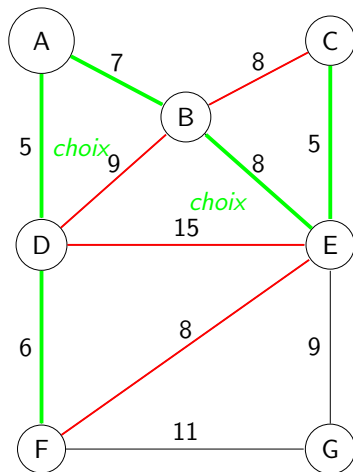
Algorithme de Kruskal - exemple



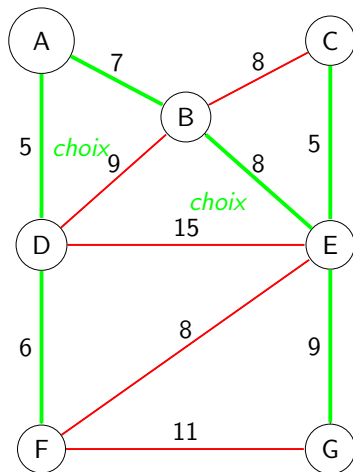
Algorithme de Kruskal - exemple



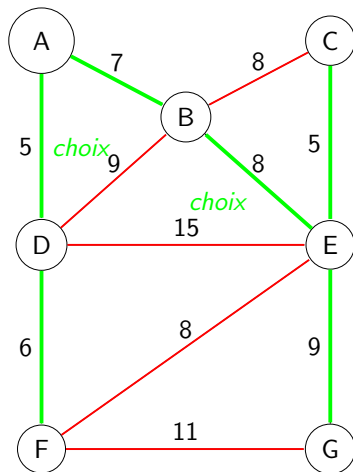
Algorithme de Kruskal - exemple



Algorithme de Kruskal - exemple

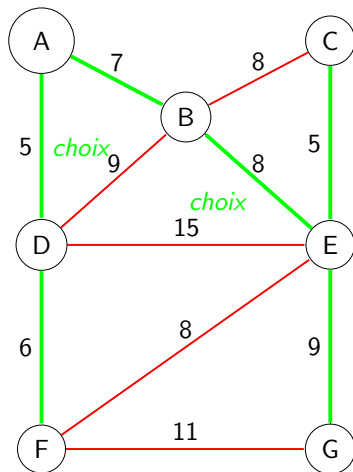


Algorithme de Kruskal - exemple



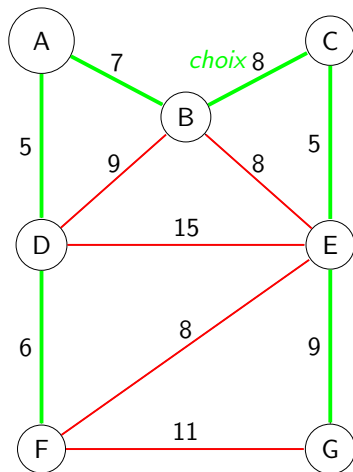
Poids final :

Algorithme de Kruskal - exemple



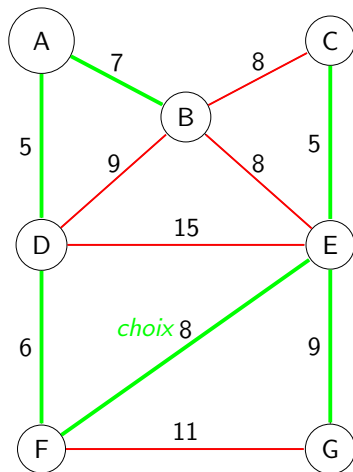
Poids final : $5+5+6+7+8+9 = 40$

Algorithme de Kruskal - autre choix



Poids final : $5+5+6+7+8+9 = 40$

Algorithme de Kruskal - autre choix 2



Poids final : $5+5+6+7+8+9 = 40$

Plan

5 Arbres couvrants

- Arbre
- Présentation du problème
- Arbre couvrant minimal
- Algorithme de Kruskal
- **Algorithme de Prim**
- Propriétés et bilan

Algorithme de Prim

Algorithme Prim(graphe valué G , sommet s) :

Entrée : $G = (V, E)$ graphe valué et $s \in V$

$S \leftarrow \{s\}$

Soit F une liste d'arêtes vide.

tant que $|S| < |V|$ **faire**

 Soit (u, v) arête de poids minimal avec $u \in S$ et $v \in V \setminus S$

$S \leftarrow S \cup \{v\}$

$F \leftarrow F \cup \{(u, v)\}$

fin tant que

Algorithme de Prim

Algorithme Prim(graphe valué G , sommet s) :

Entrée : $G = (V, E)$ graphe valué et $s \in V$

$S \leftarrow \{s\}$

Soit F une liste d'arêtes vide.

tant que $|S| < |V|$ **faire**

 Soit (u, v) arête de poids minimal avec $u \in S$ et $v \in V \setminus S$

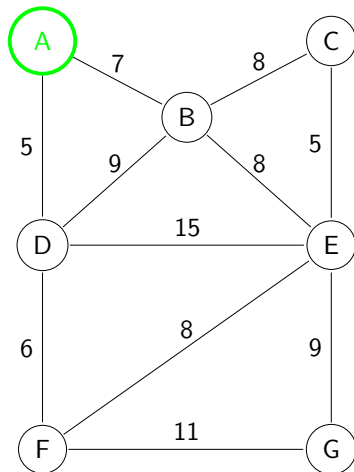
$S \leftarrow S \cup \{v\}$

$F \leftarrow F \cup \{(u, v)\}$

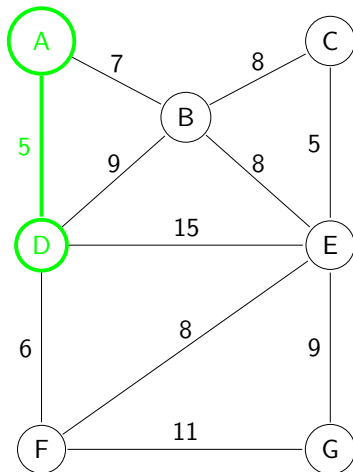
fin tant que

- Plusieurs choix possibles
- Calcul l'arbre couvrant minimal de la composante connexe de s
- Complexité : $O((|E| + |V|) \log |V|)$

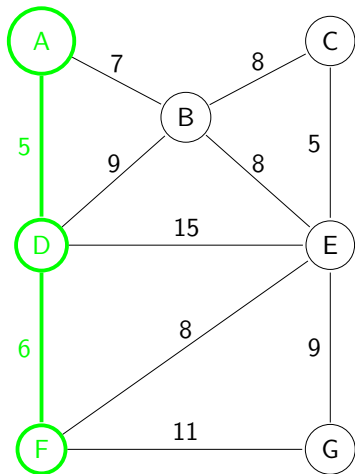
Algorithme de Prim (depuis A)



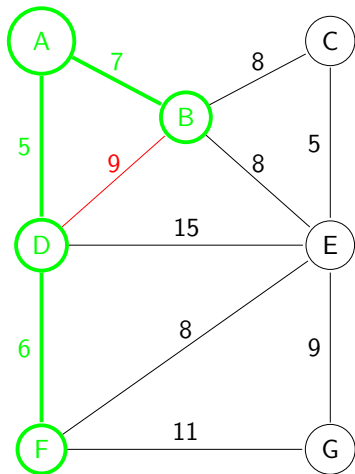
Algorithme de Prim (depuis A)



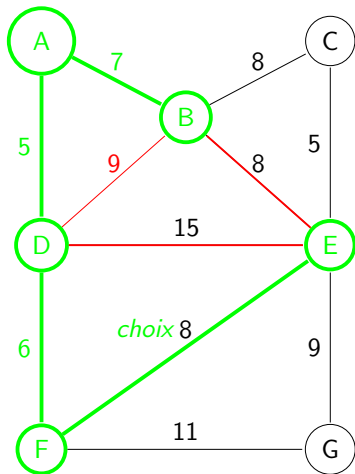
Algorithme de Prim (depuis A)



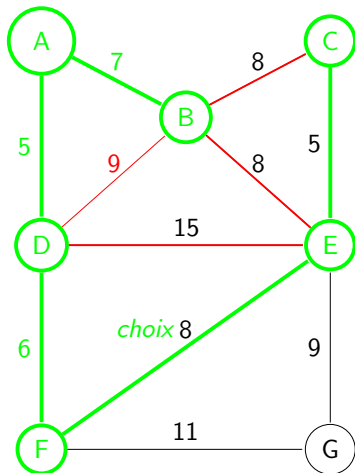
Algorithme de Prim (depuis A)



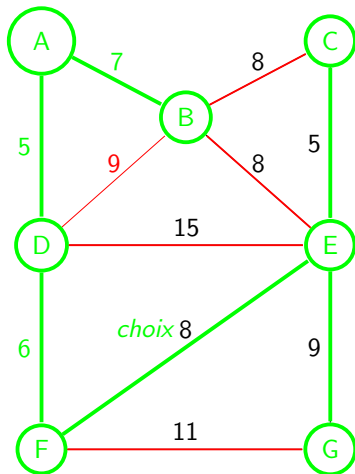
Algorithme de Prim (depuis A)



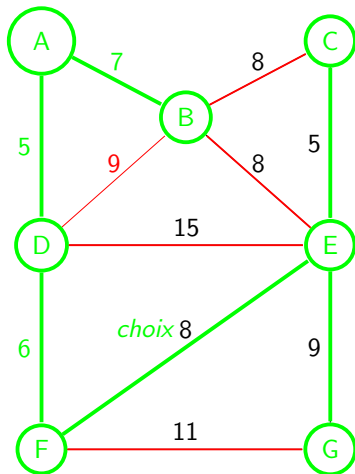
Algorithme de Prim (depuis A)



Algorithme de Prim (depuis A)

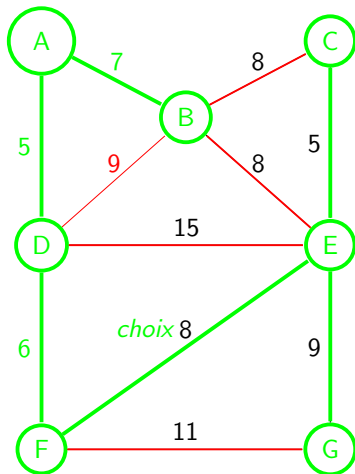


Algorithme de Prim (depuis A)



Poids final :

Algorithme de Prim (depuis A)



Poids final : $5+5+6+7+8+9 = 40$

Plan

5 Arbres couvrants

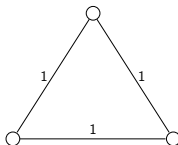
- Arbre
- Présentation du problème
- Arbre couvrant minimal
- Algorithme de Kruskal
- Algorithme de Prim
- Propriétés et bilan

Questions d'unicité

- En général ACM pas unique

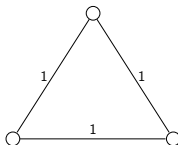
Questions d'unicité

- En général ACM pas unique

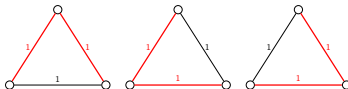


Questions d'unicité

- En général ACM pas unique

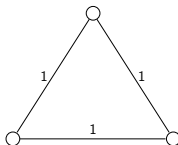


Il y a trois arbres couvrants minimaux :

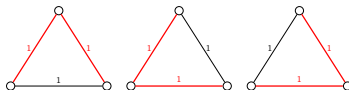


Questions d'unicité

- En général ACM pas unique



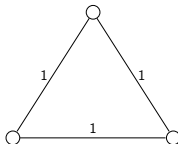
Il y a trois arbres couvrants minimaux :



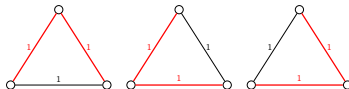
- Si tous les poids sont différents, ACM unique

Questions d'unicité

- En général ACM pas unique



Il y a trois arbres couvrants minimaux :



- Si tous les poids sont différents, ACM unique (mais réciproque fausse)

Pourquoi un glouton marche ici ?

Théorème du cycle (intuition) :

Pourquoi un glouton marche ici ?

Théorème du cycle (intuition) :

Soit e une arête de poids **maximum** dans un cycle C de G . Alors e **n'appartient à aucun ACM**.

Pourquoi un glouton marche ici ?

Théorème du cycle (intuition) :

Soit e une arête de poids **maximum** dans un cycle C de G . Alors e **n'appartient à aucun ACM**.

Pourquoi : si e était dans un ACM T , la supprimer déconnecte T en deux composantes. Or il existe une autre arête e' du cycle C qui reconnecte les deux composantes avec un poids $p(e') \leq p(e)$. On obtient un arbre couvrant de poids $\leq$ celui de T sans e — contradiction avec la minimalité de T .

Pourquoi un glouton marche ici ?

Théorème du cycle (intuition) :

Soit e une arête de poids **maximum** dans un cycle C de G . Alors e **n'appartient à aucun ACM**.

Pourquoi : si e était dans un ACM T , la supprimer déconnecte T en deux composantes. Or il existe une autre arête e' du cycle C qui reconnecte les deux composantes avec un poids $p(e') \leq p(e)$. On obtient un arbre couvrant de poids $\leq$ celui de T sans e — contradiction avec la minimalité de T .

Conséquence pour Kruskal : en triant par poids croissant et en évitant les cycles, on ne rejette jamais une arête qu'on aurait dû garder. ✓

Pourquoi un glouton marche ici ?

Théorème du cycle (intuition) :

Soit e une arête de poids **maximum** dans un cycle C de G . Alors e **n'appartient à aucun ACM**.

Pourquoi : si e était dans un ACM T , la supprimer déconnecte T en deux composantes. Or il existe une autre arête e' du cycle C qui reconnecte les deux composantes avec un poids $p(e') \leq p(e)$. On obtient un arbre couvrant de poids $\leq$ celui de T sans e — contradiction avec la minimalité de T .

Conséquence pour Kruskal : en triant par poids croissant et en évitant les cycles, on ne rejette jamais une arête qu'on aurait dû garder. ✓

Conséquence pour Prim : à chaque étape, l'arête de poids minimal qui sort de l'ensemble S ne peut pas être remplacée avantageusement. ✓

Complexités des algorithmes

Complexité de Kruskal

Complexités des algorithmes

Complexité de Kruskal

- Nœud de Kruskal : " $F \cup e$ ne forme pas de cycle"

Complexités des algorithmes

Complexité de Kruskal

- Nœud de Kruskal : " $F \cup e$ ne forme pas de cycle"
- Structure **Union-find** pour faciliter fusion de composantes connexes.

Complexités des algorithmes

Complexité de Kruskal

- Nœud de Kruskal : " $F \cup e$ ne forme pas de cycle"
- Structure **Union-find** pour faciliter fusion de composantes connexes.
- Grâce à ça, le terme pour éviter un cycle est négligeable devant le tri des arêtes

Complexités des algorithmes

Complexité de Kruskal

- Nœud de Kruskal : " $F \cup e$ ne forme pas de cycle"
- Structure **Union-find** pour faciliter fusion de composantes connexes.
- Grâce à ça, le terme pour éviter un cycle est négligeable devant le tri des arêtes
- Complexité en $O(|E| \log |E|)$ — adapté aux graphes creux.

Complexités des algorithmes

Complexité de Kruskal

- Nœud de Kruskal : " $F \cup e$ ne forme pas de cycle"
- Structure **Union-find** pour faciliter fusion de composantes connexes.
- Grâce à ça, le terme pour éviter un cycle est négligeable devant le tri des arêtes
- Complexité en $O(|E| \log |E|)$ — adapté aux graphes creux.

Complexité de Prim

Complexités des algorithmes

Complexité de Kruskal

- Nœud de Kruskal : " $F \cup e$ ne forme pas de cycle"
- Structure **Union-find** pour faciliter fusion de composantes connexes.
- Grâce à ça, le terme pour éviter un cycle est négligeable devant le tri des arêtes
- Complexité en $O(|E| \log |E|)$ — adapté aux graphes creux.

Complexité de Prim

- Même logique que Dijkstra : on tient à jour les sommets avec la distance minimale à laquelle ils sont de S dans un tas binaire

Complexités des algorithmes

Complexité de Kruskal

- Nœud de Kruskal : " $F \cup e$ ne forme pas de cycle"
- Structure **Union-find** pour faciliter fusion de composantes connexes.
- Grâce à ça, le terme pour éviter un cycle est négligeable devant le tri des arêtes
- Complexité en $O(|E| \log |E|)$ — adapté aux graphes creux.

Complexité de Prim

- Même logique que Dijkstra : on tient à jour les sommets avec la distance minimale à laquelle ils sont de S dans un tas binaire
- Extraction du minimum : $O(\log |V|)$. Répété $|V|$ fois : $O(|V| \log |V|)$.

Complexités des algorithmes

Complexité de Kruskal

- Nœud de Kruskal : " $F \cup e$ ne forme pas de cycle"
- Structure **Union-find** pour faciliter fusion de composantes connexes.
- Grâce à ça, le terme pour éviter un cycle est négligeable devant le tri des arêtes
- Complexité en $O(|E| \log |E|)$ — adapté aux graphes creux.

Complexité de Prim

- Même logique que Dijkstra : on tient à jour les sommets avec la distance minimale à laquelle ils sont de S dans un tas binaire
- Extraction du minimum : $O(\log |V|)$. Répété $|V|$ fois : $O(|V| \log |V|)$.
- Mise à jour des voisins : $O(\log |V|)$ (tas binaire). Au plus une fois par arête soit $|E|$ fois : $O(|E| \log |V|)$.

Complexités des algorithmes

Complexité de Kruskal

- Nœud de Kruskal : " $F \cup e$ ne forme pas de cycle"
- Structure **Union-find** pour faciliter fusion de composantes connexes.
- Grâce à ça, le terme pour éviter un cycle est négligeable devant le tri des arêtes
- Complexité en $O(|E| \log |E|)$ — adapté aux graphes creux.

Complexité de Prim

- Même logique que Dijkstra : on tient à jour les sommets avec la distance minimale à laquelle ils sont de S dans un tas binaire
- Extraction du minimum : $O(\log |V|)$. Répété $|V|$ fois : $O(|V| \log |V|)$.
- Mise à jour des voisins : $O(\log |V|)$ (tas binaire). Au plus une fois par arête soit $|E|$ fois : $O(|E| \log |V|)$.
- **Total** : $O((|V| + |E|) \log |V|)$ — adapté aux graphes denses.

Récapitulatif

Problème	Connexité de poids total minimal
Contrainte	Graphe partiel connexe = arbre couvrant
Objectif	Minimiser la somme des poids des arêtes
Propriété clé	Tout graphe connexe admet un ACM
Garantie	Algorithmes gloutons exacts ici
Algo 1	Kruskal — $O(E \log E)$ — graphes creux
Algo 2	Prim — $O((E + V) \log V)$ — graphes denses
Unicité	Unique si tous les poids sont distincts
Lien Dijkstra	Prim = Dijkstra avec critère local
Applications	Réseaux, câblage, clusters

Plan

- 1 Définitions
- 2 Programmation
- 3 Connexité et parcours
- 4 Plus courts chemins
- 5 Arbres couvrants
- 6 Flots**
- 7 Coloriage de graphes

Plan

6 Flots

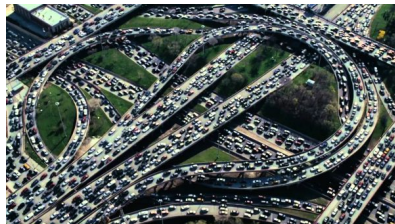
- Présentation du problème
- Définitions
- Ford-Fulkerson
- Coupes
- Application : couplage bipartite
- Bilan

Présentation du problème

Faire passer le maximum de "quelque chose" à travers un réseau



Réseau hydraulique : maximiser le débit d'eau de la source au puits



Trafic routier : maximiser le flux de véhicules entre deux villes

Dans la vie du programmeur

Des applications très variées

Dans la vie du programmeur

Des applications très variées

- **Réseaux hydrauliques et électriques** : maximiser le débit ou la puissance transportée

Dans la vie du programmeur

Des applications très variées

- **Réseaux hydrauliques et électriques** : maximiser le débit ou la puissance transportée
- **Trafic internet** : router le maximum de données entre deux serveurs dans un réseau de bande passante limitée

Dans la vie du programmeur

Des applications très variées

- **Réseaux hydrauliques et électriques** : maximiser le débit ou la puissance transportée
- **Trafic internet** : router le maximum de données entre deux serveurs dans un réseau de bande passante limitée
- **Affectation et couplage** : affecter des tâches à des machines, des candidats à des postes

Dans la vie du programmeur

Des applications très variées

- **Réseaux hydrauliques et électriques** : maximiser le débit ou la puissance transportée
- **Trafic internet** : router le maximum de données entre deux serveurs dans un réseau de bande passante limitée
- **Affectation et couplage** : affecter des tâches à des machines, des candidats à des postes
- **Segmentation d'image** : la coupe minimale sépare un objet de son fond en vision par ordinateur. Vidéo d'explication, résultat de Boykov-Kolmogorov (2004).

Dans la vie du programmeur

Des applications très variées

- **Réseaux hydrauliques et électriques** : maximiser le débit ou la puissance transportée
- **Trafic internet** : router le maximum de données entre deux serveurs dans un réseau de bande passante limitée
- **Affectation et couplage** : affecter des tâches à des machines, des candidats à des postes
- **Segmentation d'image** : la coupe minimale sépare un objet de son fond en vision par ordinateur. Vidéo d'explication, résultat de Boykov-Kolmogorov (2004).

Problème	Flot maximal de s à p
Contrainte	Capacité de chaque arc
Résultat clé	Flot max = Coupe min

Plan

6 Flots

- Présentation du problème
- Définitions
- Ford-Fulkerson
- Coupes
- Application : couplage bipartite
- Bilan

Réseau de transport

Définition (Réseau de transport)

Un *réseau de transport* est un graphe orienté simple (sans boucle) valué. On appelle la valuation d'un arc u , notée $c(u)$, la *capacité* de l'arc u . En outre, un tel réseau vérifie :

- Il existe un seul nœud s qui n'a pas de prédécesseurs. Ce nœud est appelé l'entrée du réseau, ou la *source*.
- Il existe également un seul nœud p qui n'a pas de successeurs. Ce nœud est appelé la sortie du réseau, ou le *puits*.

Réseau de transport

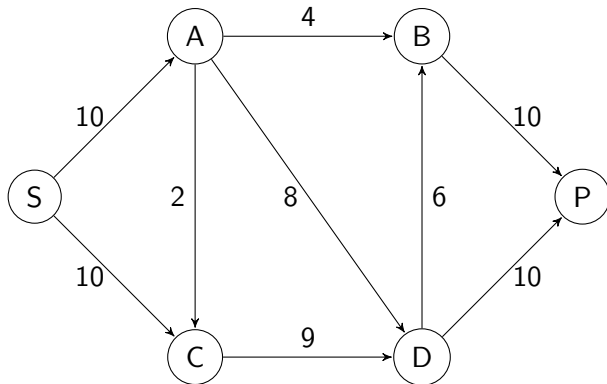
Définition (Réseau de transport)

Un *réseau de transport* est un graphe orienté simple (sans boucle) valué. On appelle la valuation d'un arc u , notée $c(u)$, la *capacité* de l'arc u . En outre, un tel réseau vérifie :

- Il existe un seul nœud s qui n'a pas de prédécesseurs. Ce nœud est appelé l'entrée du réseau, ou la *source*.
- Il existe également un seul nœud p qui n'a pas de successeurs. Ce nœud est appelé la sortie du réseau, ou le *puits*.

Remarque : on ne considérera que des capacités entières positives.

Réseau de transport - exemple



Flot

Définition (Flot)

Un *flot* f dans un réseau de transport associe à chaque arc u un entier $f(u) \in \mathbb{N}$ représentant la *quantité de flot* qui passe par cet arc, en provenance de la source et en destination du puits.

Flot

Définition (Flot)

Un *flot* f dans un réseau de transport associe à chaque arc u un entier $f(u) \in \mathbb{N}$ représentant la *quantité de flot* qui passe par cet arc, en provenance de la source et en destination du puits.

Un flot doit respecter qu'en tout nœud (sauf la source et le puits) la somme des quantités des flots sur les arcs entrants est égal à la somme des quantités des flots sur les arcs sortants.

Flot

Définition (Flot)

Un *flot* f dans un réseau de transport associe à chaque arc u un entier $f(u) \in \mathbb{N}$ représentant la *quantité de flot* qui passe par cet arc, en provenance de la source et en destination du puits.

Un flot doit respecter qu'en tout nœud (sauf la source et le puits) la somme des quantités des flots sur les arcs entrants est égal à la somme des quantités des flots sur les arcs sortants.

La valeur du flot est alors la quantité de flot sortant de la source, ou entrant dans le puits.

Flot

Définition (Flot)

Un **flot** f dans un réseau de transport associe à chaque arc u un entier $f(u) \in \mathbb{N}$ représentant la **quantité de flot** qui passe par cet arc, en provenance de la source et en destination du puits.

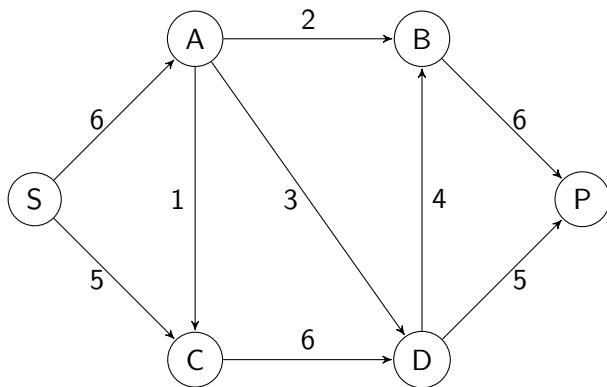
Un flot doit respecter qu'en tout nœud (sauf la source et le puits) la somme des quantités des flots sur les arcs entrants est égal à la somme des quantités des flots sur les arcs sortants.

La valeur du flot est alors la quantité de flot sortant de la source, ou entrant dans le puits.

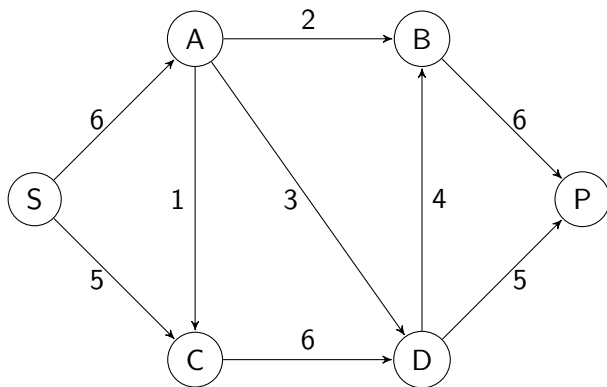
Définition

Un flot f est compatible avec un réseau si pour tout arc u , $0 \leq f(u) \leq c(u)$. Autrement dit, pour chaque arc le flot qui le traverse ne doit pas dépasser la capacité de l'arc.

Flot - exemple

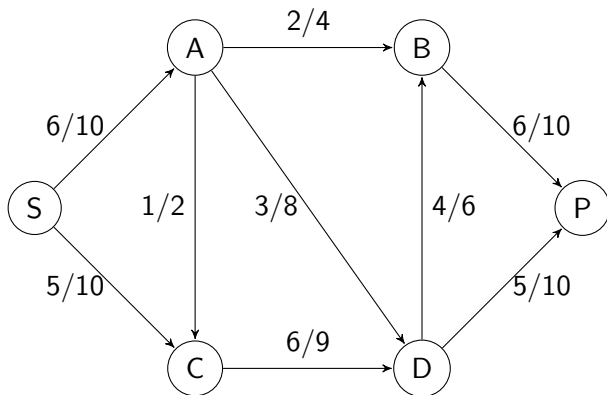


Flot - exemple



Flot de 11

Flot compatible - exemple



En premier la quantité de flot, en second la capacité de l'arc.

Plan

6 Flots

- Présentation du problème
- Définitions
- **Ford-Fulkerson**
- Coupes
- Application : couplage bipartite
- Bilan

Flots maximal

Problématique : trouver le flot maximal d'un graphe

Flots maximal

Problématique : trouver le flot maximal d'un graphe

Algorithme de Ford et Fulkerson :

Flots maximal

Problématique : trouver le flot maximal d'un graphe

Algorithme de Ford et Fulkerson :

- Partir d'un flot compatible : le flot nul classiquement

Flots maximal

Problématique : trouver le flot maximal d'un graphe

Algorithme de Ford et Fulkerson :

- Partir d'un flot compatible : le flot nul classiquement
- Recherche de chaîne augmentante

Flots maximal

Problématique : trouver le flot maximal d'un graphe

Algorithme de Ford et Fulkerson :

- Partir d'un flot compatible : le flot nul classiquement
- Recherche de chaîne augmentante
- Graphe d'écart

Chaîne augmentante

C'est une chaîne C (non orienté !) de s à p telle que :

Chaîne augmentante

C'est une chaîne C (non orienté !) de s à p telle que :

- Si $u \in C$ arc direct, $f(u) < c(u)$

Chaîne augmentante

C'est une chaîne C (non orienté !) de s à p telle que :

- Si $u \in C$ arc direct, $f(u) < c(u)$
- Si $u \in C$ arc indirect, $f(u) > 0$

Chaîne augmentante

C'est une chaîne C (non orienté !) de s à p telle que :

- Si $u \in C$ arc direct, $f(u) < c(u)$
- Si $u \in C$ arc indirect, $f(u) > 0$

On peut alors augmenter le flot de cette chaîne de

$$\min \left[\{c(u) - f(u) \mid u \in C \text{ direct}\} \cup \{f(u) \mid u \in C \text{ indirect}\} \right]$$

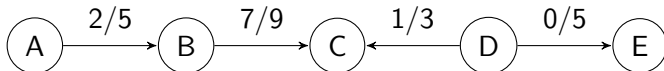
Chaîne augmentante

C'est une chaîne C (non orienté !) de s à p telle que :

- Si $u \in C$ arc direct, $f(u) < c(u)$
- Si $u \in C$ arc indirect, $f(u) > 0$

On peut alors augmenter le flot de cette chaîne de

$$\min \left[\{c(u) - f(u) \mid u \in C \text{ direct}\} \cup \{f(u) \mid u \in C \text{ indirect}\} \right]$$



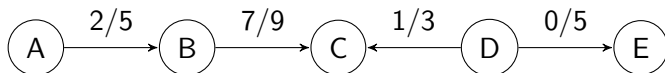
Chaîne augmentante

C'est une chaîne C (non orienté !) de s à p telle que :

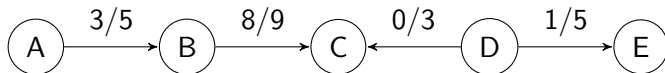
- Si $u \in C$ arc direct, $f(u) < c(u)$
- Si $u \in C$ arc indirect, $f(u) > 0$

On peut alors augmenter le flot de cette chaîne de

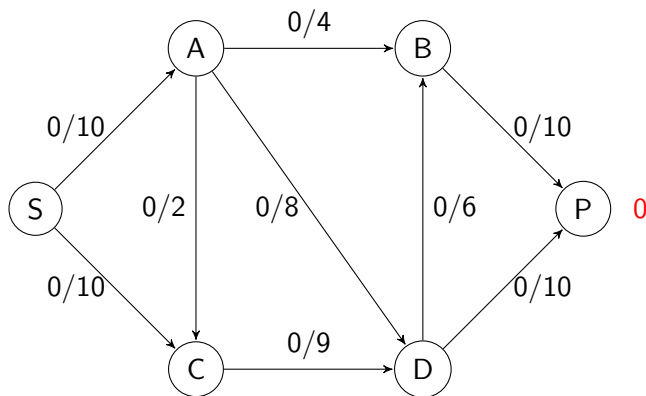
$$\min \left[\{c(u) - f(u) \mid u \in C \text{ direct}\} \cup \{f(u) \mid u \in C \text{ indirect}\} \right]$$



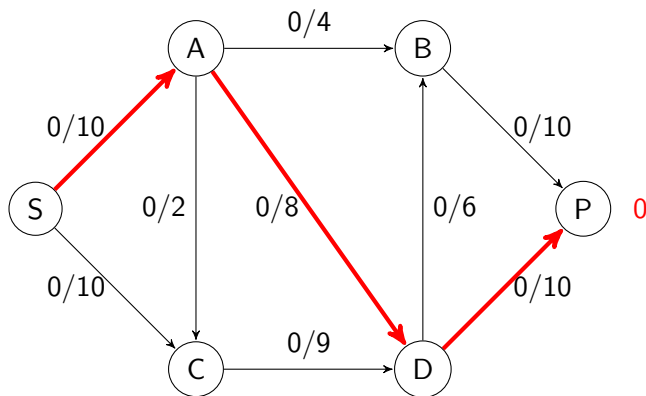
On augmente le flot de 1 :



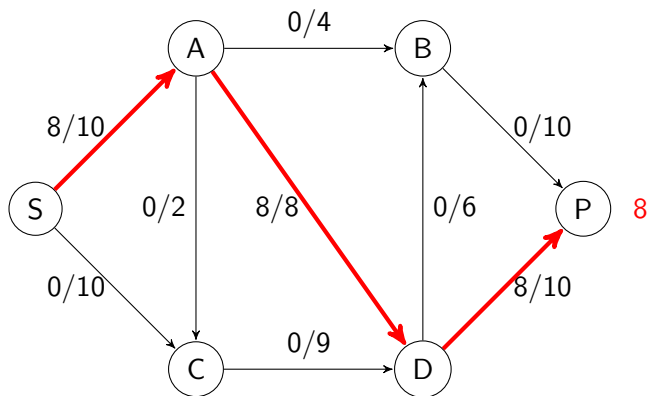
Ford et Fulkerson - Chaîne augmentante - exemple



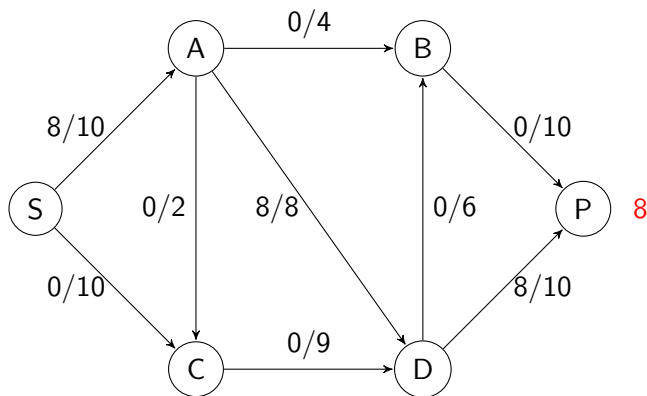
Ford et Fulkerson - Chaîne augmentante - exemple



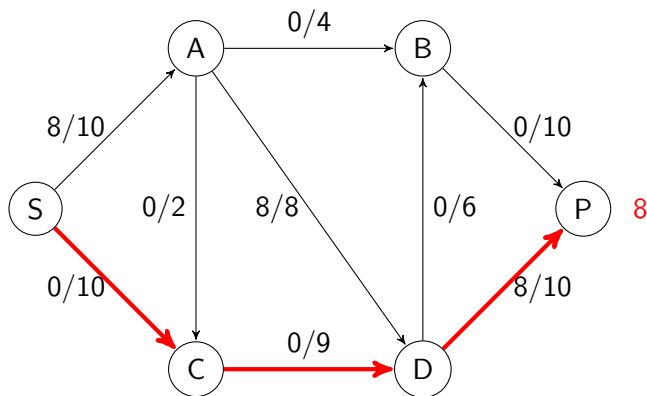
Ford et Fulkerson - Chaîne augmentante - exemple



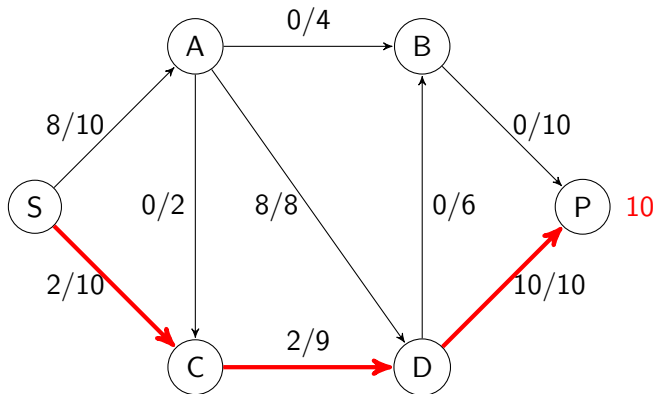
Ford et Fulkerson - Chaîne augmentante - exemple



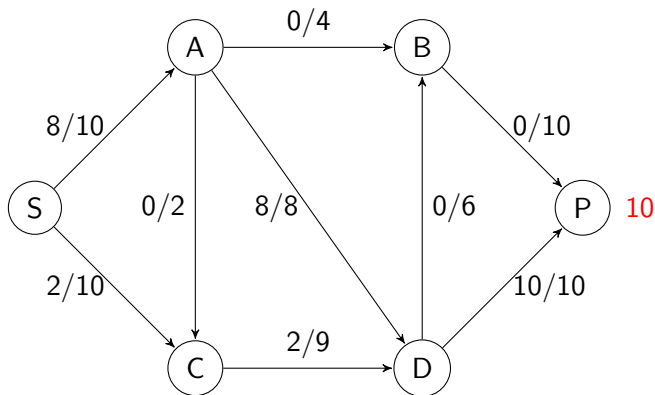
Ford et Fulkerson - Chaîne augmentante - exemple



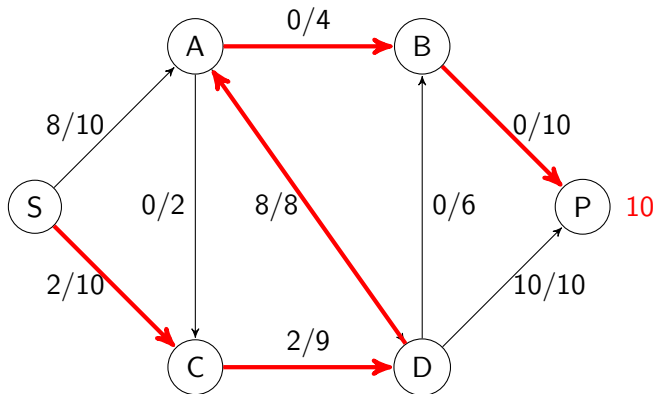
Ford et Fulkerson - Chaîne augmentante - exemple



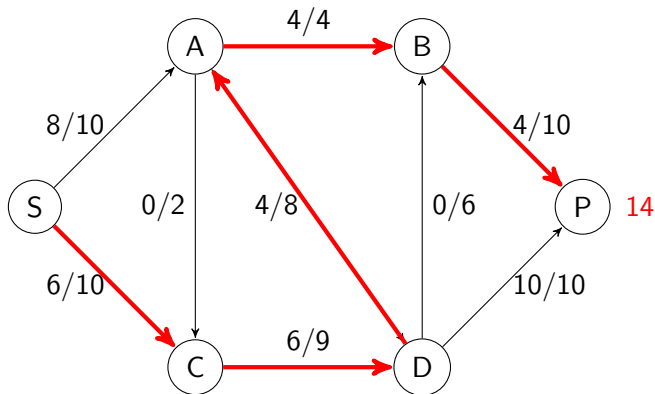
Ford et Fulkerson - Chaîne augmentante - exemple



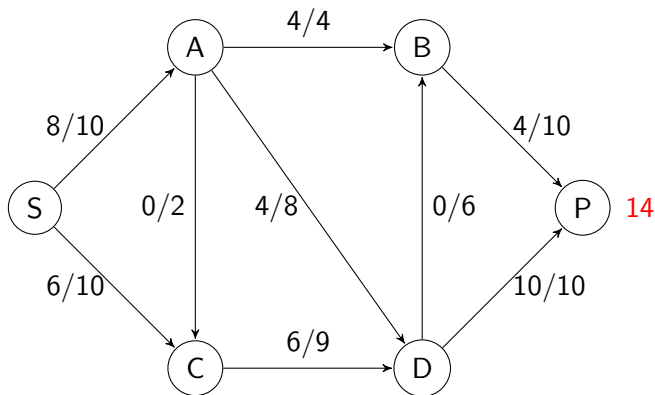
Ford et Fulkerson - Chaîne augmentante - exemple



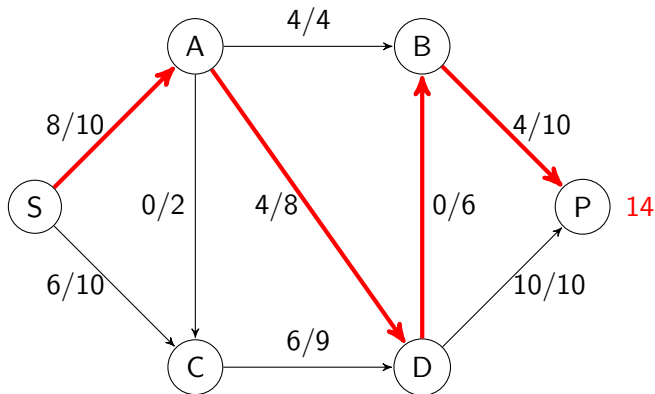
Ford et Fulkerson - Chaîne augmentante - exemple



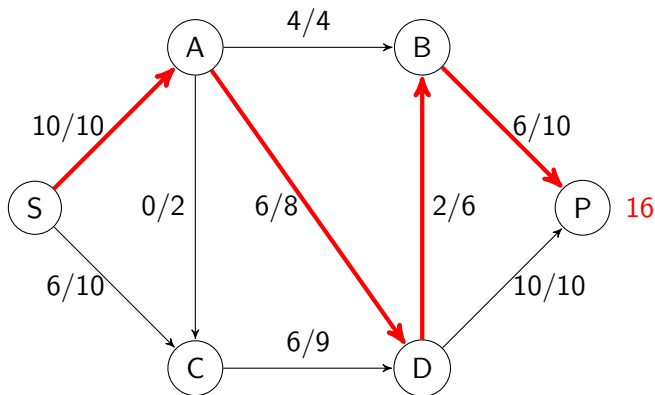
Ford et Fulkerson - Chaîne augmentante - exemple



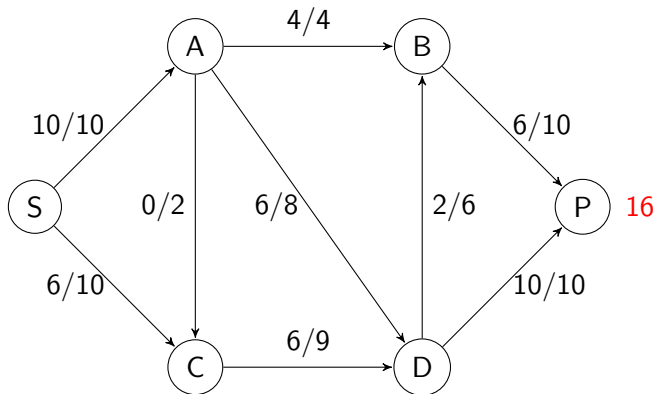
Ford et Fulkerson - Chaîne augmentante - exemple



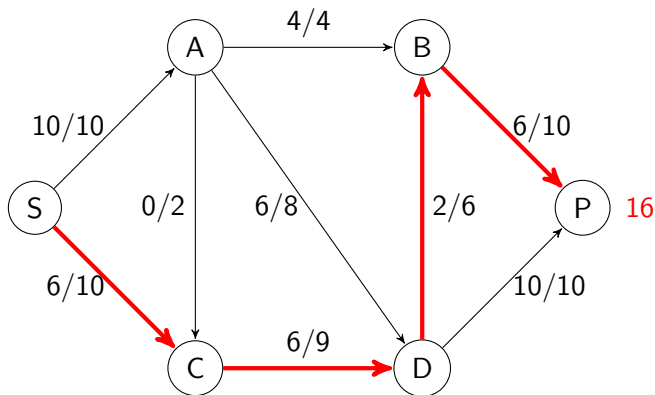
Ford et Fulkerson - Chaîne augmentante - exemple



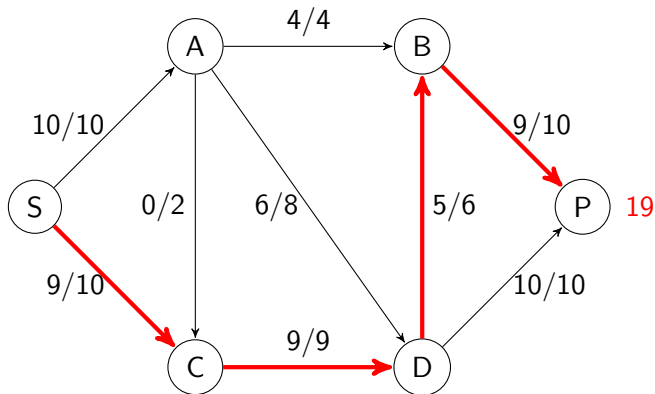
Ford et Fulkerson - Chaîne augmentante - exemple



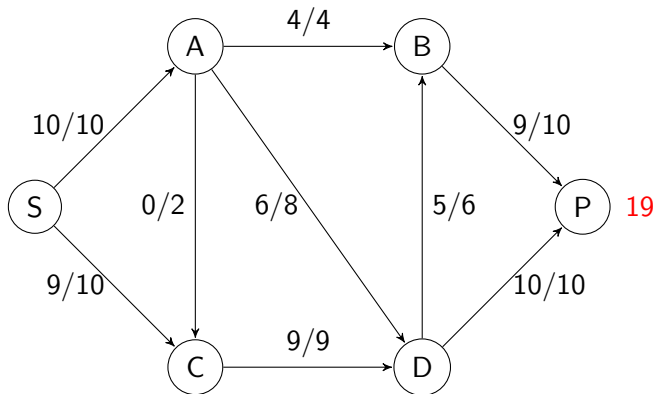
Ford et Fulkerson - Chaîne augmentante - exemple



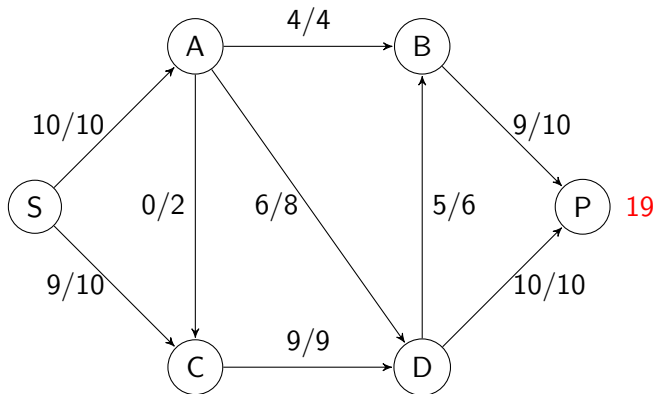
Ford et Fulkerson - Chaîne augmentante - exemple



Ford et Fulkerson - Chaîne augmentante - exemple

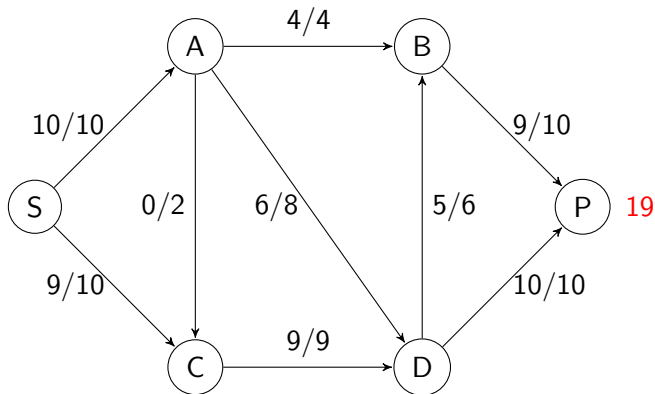


Ford et Fulkerson - Chaîne augmentante - exemple



Plus de chaîne augmentante

Ford et Fulkerson - Chaîne augmentante - exemple



Plus de chaîne augmentante
Flot maximal de 19

Ford et Fulkerson - Chaîne augmentante - algo

Algorithme Ford_Fulkerson_Chaine(réseau de transport G) :

Entrée : $G = (V, E)$ réseau de transport, s sa source, p son puits
pour tout Arc $e \in E$ **faire**
 $f(e) \leftarrow 0$
fin pour
tant que $\exists$ chaîne augmentante C **faire**
 $m \leftarrow \infty$
 pour tout $e \in C$ **faire**
 si e dans le sens direct **alors**
 $a \leftarrow c(e) - f(e)$
 sinon
 $a \leftarrow f(e)$

fin si
 si $a < m$ **alors**
 $m \leftarrow a$
 fin si
 fin pour
 pour tout $e \in C$ **faire**
 si e dans le sens direct **alors**
 $f(e) \leftarrow f(e) + m$
 sinon
 $f(e) \leftarrow f(e) - m$
 fin si
 fin pour
fin tant que

Ford et Fulkerson - algo recherche chaîne augmentante

Algorithme Recherche_Chaine(réseau de transport G) :

Entrée : $G = (V, E)$ réseau de transport, s sa source, p son puits

pour tout $v \in V$ **faire**

Mettre v à non marqué,
 $pred(v) \leftarrow \emptyset$

fin pour

$V' = \{s\}$; Marquer s

tant que $V' \neq \emptyset$ et p non marqué **faire**

Choisir $v \in V'$, retirer v de V'

pour tout $e = (v, u) \in E$ **faire**

si u non marqué et
 $f(e) < c(e)$ **alors**

Marquer u , ajouter u à
 V' , $pred(u) \leftarrow v$

fin si

fin pour

pour tout $e = (u, v) \in E$ **faire**

si u non marqué et $f(e) > 0$
alors

Marquer u , ajouter u à
 V' , $pred(u) \leftarrow v$

fin si

fin pour

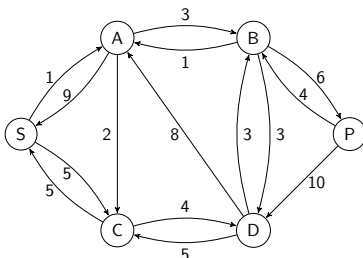
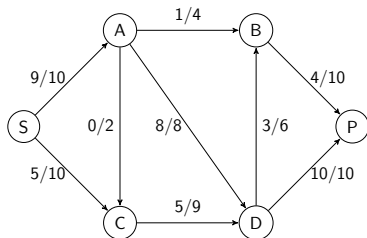
fin tant que

Graphe d'écart

Définition (Graphe d'écart)

Soit $G = (V, E)$ un réseau de transport avec un flot f courant. On définit le **graphe d'écart** comme réseau de transport avec sommets V , et pour tout $e = (u, v) \in E$ on a l'arc :

- (u, v) de capacité $c(e) - f(e)$ si $c(e) \neq f(e)$
- (v, u) de capacité $f(e)$ si $f(e) \neq 0$.



Ford Fulkerson - graphe d'écart

Principe :

Ford Fulkerson - graphe d'écart

Principe :

- Partir du flot vide

Ford Fulkerson - graphe d'écart

Principe :

- Partir du flot vide
- À chaque étape construire le graphe d'écart

Ford Fulkerson - graphe d'écart

Principe :

- Partir du flot vide
- À chaque étape construire le graphe d'écart
- Chercher un chemin de s à p dans le graphe d'écart G' , et soit m le minimum des valuations sur ce chemin

Ford Fulkerson - graphe d'écart

Principe :

- Partir du flot vide
- À chaque étape construire le graphe d'écart
- Chercher un chemin de s à p dans le graphe d'écart G' , et soit m le minimum des valuations sur ce chemin
- Augmenter ou réduire de m sur la chaîne de s à p correspondante dans G

Ford Fulkerson - graphe d'écart

Principe :

- Partir du flot vide
- À chaque étape construire le graphe d'écart
- Chercher un chemin de s à p dans le graphe d'écart G' , et soit m le minimum des valuations sur ce chemin
- Augmenter ou réduire de m sur la chaîne de s à p correspondante dans G

Remarque : ça revient juste à chercher la chaîne augmentante comme chemin dans un graphe orienté

Complexité et terminaison

Version chaîne augmentante (Ford-Fulkerson classique)

Complexité et terminaison

Version chaîne augmentante (Ford-Fulkerson classique)

- Chaque itération augmente le flot d'au moins 1 (si capacités entières)

Complexité et terminaison

Version chaîne augmentante (Ford-Fulkerson classique)

- Chaque itération augmente le flot d'au moins 1 (si capacités entières)
- Il y a au plus $f_{\max}$ itérations

Complexité et terminaison

Version chaîne augmentante (Ford-Fulkerson classique)

- Chaque itération augmente le flot d'au moins 1 (si capacités entières)
- Il y a au plus $f_{\max}$ itérations
- Chercher une chaîne augmentante : $O(|V| + |E|)$ par BFS/DFS

Complexité et terminaison

Version chaîne augmentante (Ford-Fulkerson classique)

- Chaque itération augmente le flot d'au moins 1 (si capacités entières)
- Il y a au plus $f_{\max}$ itérations
- Chercher une chaîne augmentante : $O(|V| + |E|)$ par BFS/DFS
- **Total** : $O(f_{\max} \cdot (|E| + |V|))$

Complexité et terminaison

Version chaîne augmentante (Ford-Fulkerson classique)

- Chaque itération augmente le flot d'au moins 1 (si capacités entières)
- Il y a au plus $f_{\max}$ itérations
- Chercher une chaîne augmentante : $O(|V| + |E|)$ par BFS/DFS
- **Total** : $O(f_{\max} \cdot (|E| + |V|))$

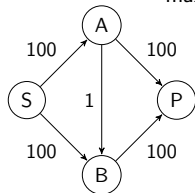
Problème : si $f_{\max}$ est très grand, trop d'itérations ! Exemple :

Complexité et terminaison

Version chaîne augmentante (Ford-Fulkerson classique)

- Chaque itération augmente le flot d'au moins 1 (si capacités entières)
- Il y a au plus $f_{\max}$ itérations
- Chercher une chaîne augmentante : $O(|V| + |E|)$ par BFS/DFS
- **Total** : $O(f_{\max} \cdot (|E| + |V|))$

Problème : si $f_{\max}$ est très grand, trop d'itérations ! Exemple :



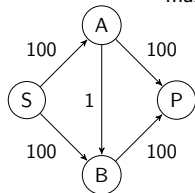
Problème : si pas de chances, on alterne les $S \rightarrow A \rightarrow B \rightarrow P$ et les $S \rightarrow B \rightarrow A \rightarrow P$ pour 200 itérations !

Complexité et terminaison

Version chaîne augmentante (Ford-Fulkerson classique)

- Chaque itération augmente le flot d'au moins 1 (si capacités entières)
- Il y a au plus $f_{\max}$ itérations
- Chercher une chaîne augmentante : $O(|V| + |E|)$ par BFS/DFS
- **Total** : $O(f_{\max} \cdot (|E| + |V|))$

Problème : si $f_{\max}$ est très grand, trop d'itérations ! Exemple :



Problème : si pas de chances, on alterne les $S \rightarrow A \rightarrow B \rightarrow P$ et les $S \rightarrow B \rightarrow A \rightarrow P$ pour 200 itérations !

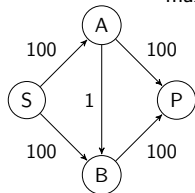
Version Edmonds-Karp (1972) : utiliser BFS pour trouver la *plus courte* chaîne augmentante.

Complexité et terminaison

Version chaîne augmentante (Ford-Fulkerson classique)

- Chaque itération augmente le flot d'au moins 1 (si capacités entières)
- Il y a au plus $f_{\max}$ itérations
- Chercher une chaîne augmentante : $O(|V| + |E|)$ par BFS/DFS
- **Total** : $O(f_{\max} \cdot (|E| + |V|))$

Problème : si $f_{\max}$ est très grand, trop d'itérations ! Exemple :



Problème : si pas de chances, on alterne les $S \rightarrow A \rightarrow B \rightarrow P$ et les $S \rightarrow B \rightarrow A \rightarrow P$ pour 200 itérations !

Version Edmonds-Karp (1972) : utiliser BFS pour trouver la *plus courte* chaîne augmentante.

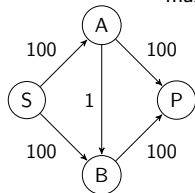
- **Total** : $O(|V| \cdot |E|^2)$ — garanti, indépendant de $f_{\max}$

Complexité et terminaison

Version chaîne augmentante (Ford-Fulkerson classique)

- Chaque itération augmente le flot d'au moins 1 (si capacités entières)
- Il y a au plus $f_{\max}$ itérations
- Chercher une chaîne augmentante : $O(|V| + |E|)$ par BFS/DFS
- **Total** : $O(f_{\max} \cdot (|E| + |V|))$

Problème : si $f_{\max}$ est très grand, trop d'itérations ! Exemple :



Problème : si pas de chances, on alterne les $S \rightarrow A \rightarrow B \rightarrow P$ et les $S \rightarrow B \rightarrow A \rightarrow P$ pour 200 itérations !

Version Edmonds-Karp (1972) : utiliser BFS pour trouver la *plus courte* chaîne augmentante.

- **Total** : $O(|V| \cdot |E|^2)$ — garanti, indépendant de $f_{\max}$
- Même pseudo-code, seul le choix de la chaîne change

Plan

6 Flots

- Présentation du problème
- Définitions
- Ford-Fulkerson
- Coupes
- Application : couplage bipartite
- Bilan

Flots et coupes

Définition

Une coupe de $G = (V, E)$ un réseau de transport est une partition de $V = Y \cup Z$ avec $s \in Y$ et $p \in Z$.

Flots et coupes

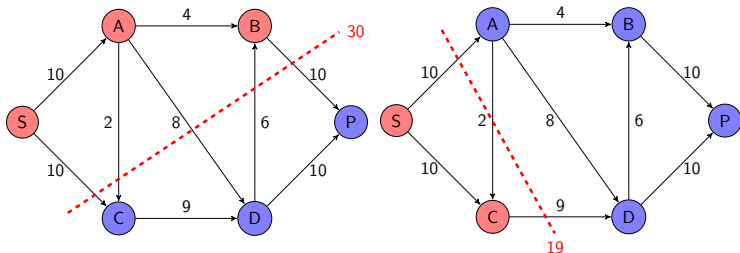
Définition

Une coupe de $G = (V, E)$ un réseau de transport est une partition de $V = Y \cup Z$ avec $s \in Y$ et $p \in Z$. La capacité d'une coupe est la somme des capacités des arcs qui ont une origine dans Y et une extrémité dans Z .

Flots et coupes

Définition

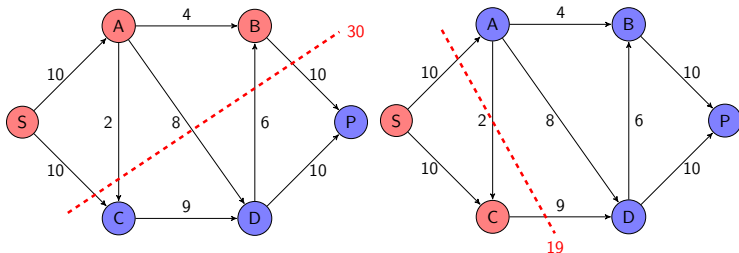
Une coupe de $G = (V, E)$ un réseau de transport est une partition de $V = Y \cup Z$ avec $s \in Y$ et $p \in Z$. La capacité d'une coupe est la somme des capacités des arcs qui ont une origine dans Y et une extrémité dans Z .



Flots et coupes

Définition

Une coupe de $G = (V, E)$ un réseau de transport est une partition de $V = Y \cup Z$ avec $s \in Y$ et $p \in Z$. La capacité d'une coupe est la somme des capacités des arcs qui ont une origine dans Y et une extrémité dans Z .



Théorème (Ford-Fulkerson)

Le flot maximal est égal à la capacité de la coupe minimale.

Intuition : pourquoi flot-max = coupe-min ?

Une coupe borne le flot : tout flot de s à p doit traverser la coupe — il ne peut pas faire mieux que la capacité totale des arcs qui la franchissent.

Intuition : pourquoi flot-max = coupe-min ?

Une coupe borne le flot : tout flot de s à p doit traverser la coupe — il ne peut pas faire mieux que la capacité totale des arcs qui la franchissent.

Donc : flot maximal $\leq$ capacité de toute coupe

Intuition : pourquoi flot-max = coupe-min ?

Une coupe borne le flot : tout flot de s à p doit traverser la coupe — il ne peut pas faire mieux que la capacité totale des arcs qui la franchissent.

Donc : flot maximal $\leq$ capacité de toute coupe

Ford-Fulkerson construit une coupe minimale : quand l'algorithme s'arrête (plus de chaîne augmentante), posons Y = sommets atteignables depuis s dans le graphe d'écart, $Z = V \setminus Y$.

Intuition : pourquoi flot-max = coupe-min ?

Une coupe borne le flot : tout flot de s à p doit traverser la coupe — il ne peut pas faire mieux que la capacité totale des arcs qui la franchissent.

Donc : flot maximal $\leq$ capacité de toute coupe

Ford-Fulkerson construit une coupe minimale : quand l'algorithme s'arrête (plus de chaîne augmentante), posons Y = sommets atteignables depuis s dans le graphe d'écart, $Z = V \setminus Y$.

- Tous les arcs de Y vers Z dans G sont **saturés** (sinon on pourrait avancer dans le graphe d'écart)

Intuition : pourquoi flot-max = coupe-min ?

Une coupe borne le flot : tout flot de s à p doit traverser la coupe — il ne peut pas faire mieux que la capacité totale des arcs qui la franchissent.

Donc : flot maximal $\leq$ capacité de toute coupe

Ford-Fulkerson construit une coupe minimale : quand l'algorithme s'arrête (plus de chaîne augmentante), posons Y = sommets atteignables depuis s dans le graphe d'écart, $Z = V \setminus Y$.

- Tous les arcs de Y vers Z dans G sont **saturés** (sinon on pourrait avancer dans le graphe d'écart)
- Tous les arcs de Z vers Y dans G ont un **flot nul** (sinon on pourrait reculer)

Intuition : pourquoi flot-max = coupe-min ?

Une coupe borne le flot : tout flot de s à p doit traverser la coupe — il ne peut pas faire mieux que la capacité totale des arcs qui la franchissent.

Donc : flot maximal $\leq$ capacité de toute coupe

Ford-Fulkerson construit une coupe minimale : quand l'algorithme s'arrête (plus de chaîne augmentante), posons Y = sommets atteignables depuis s dans le graphe d'écart, $Z = V \setminus Y$.

- Tous les arcs de Y vers Z dans G sont **saturés** (sinon on pourrait avancer dans le graphe d'écart)
- Tous les arcs de Z vers Y dans G ont un **flot nul** (sinon on pourrait reculer)

La valeur du flot courant = capacité de la coupe (Y, Z) $\Rightarrow$ les deux sont simultanément optimaux.

Plan

6 Flots

- Présentation du problème
- Définitions
- Ford-Fulkerson
- Coupes
- Application : couplage bipartite
- Bilan

Rappel : graphe bipartite

On se place dans un graphe bipartite $G = (U \cup W, E)$.

Définition (Couplage)

Un ***couplage*** est un ensemble d'arêtes $M \subseteq E$ tel que chaque sommet est incident à au plus une arête de M .

Rappel : graphe bipartite

On se place dans un graphe bipartite $G = (U \cup W, E)$.

Définition (Couplage)

Un *couplage* est un ensemble d'arêtes $M \subseteq E$ tel que chaque sommet est incident à au plus une arête de M .

Un couplage est dit *maximum* s'il contient le plus grand nombre d'arêtes possible.

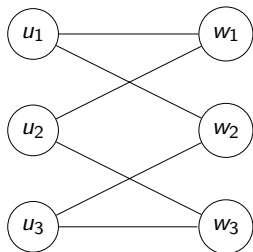
Rappel : graphe bipartite

On se place dans un graphe bipartite $G = (U \cup W, E)$.

Définition (Couplage)

Un **couplage** est un ensemble d'arêtes $M \subseteq E$ tel que chaque sommet est incident à au plus une arête de M .

Un couplage est dit **maximum** s'il contient le plus grand nombre d'arêtes possible.



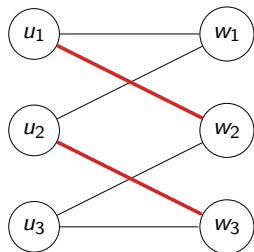
Rappel : graphe bipartite

On se place dans un graphe bipartite $G = (U \cup W, E)$.

Définition (Couplage)

Un **couplage** est un ensemble d'arêtes $M \subseteq E$ tel que chaque sommet est incident à au plus une arête de M .

Un couplage est dit **maximum** s'il contient le plus grand nombre d'arêtes possible.



Couplage de taille 2

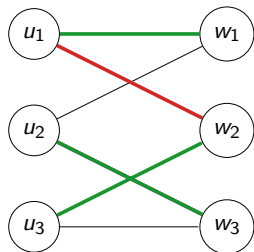
Rappel : graphe bipartite

On se place dans un graphe bipartite $G = (U \cup W, E)$.

Définition (Couplage)

Un **couplage** est un ensemble d'arêtes $M \subseteq E$ tel que chaque sommet est incident à au plus une arête de M .

Un couplage est dit **maximum** s'il contient le plus grand nombre d'arêtes possible.



Couplage de taille 2

3

Couplage maximum de taille

Problèmes de couplage — exemples concrets

- **Affectation de tâches** : U = tâches, W = machines. Une arête (u_i, w_j) signifie que la machine w_j peut exécuter la tâche u_i . Le couplage maximum maximise le nombre de tâches exécutées.

Problèmes de couplage — exemples concrets

- **Affectation de tâches** : U = tâches, W = machines. Une arête (u_i, w_j) signifie que la machine w_j peut exécuter la tâche u_i . Le couplage maximum maximise le nombre de tâches exécutées.
- **Recrutement** : U = candidats, W = postes. Une arête = candidat qualifié pour ce poste.

Problèmes de couplage — exemples concrets

- **Affectation de tâches** : U = tâches, W = machines. Une arête (u_i, w_j) signifie que la machine w_j peut exécuter la tâche u_i . Le couplage maximum maximise le nombre de tâches exécutées.
- **Recrutement** : U = candidats, W = postes. Une arête = candidat qualifié pour ce poste.
- **Affectation de salles** : U = cours, W = salles. Contraintes de capacité et de matériel.

Problèmes de couplage — exemples concrets

- **Affectation de tâches** : U = tâches, W = machines. Une arête (u_i, w_j) signifie que la machine w_j peut exécuter la tâche u_i . Le couplage maximum maximise le nombre de tâches exécutées.
- **Recrutement** : U = candidats, W = postes. Une arête = candidat qualifié pour ce poste.
- **Affectation de salles** : U = cours, W = salles. Contraintes de capacité et de matériel.
- **Mariages stables (Gale-Shapley)** : variante avec préférences .

Problèmes de couplage — exemples concrets

- **Affectation de tâches** : U = tâches, W = machines. Une arête (u_i, w_j) signifie que la machine w_j peut exécuter la tâche u_i . Le couplage maximum maximise le nombre de tâches exécutées.
- **Recrutement** : U = candidats, W = postes. Une arête = candidat qualifié pour ce poste.
- **Affectation de salles** : U = cours, W = salles. Contraintes de capacité et de matériel.
- **Mariages stables (Gale-Shapley)** : variante avec préférences .

Question : comment calculer un couplage maximum efficacement ?

Problèmes de couplage — exemples concrets

- **Affectation de tâches** : U = tâches, W = machines. Une arête (u_i, w_j) signifie que la machine w_j peut exécuter la tâche u_i . Le couplage maximum maximise le nombre de tâches exécutées.
- **Recrutement** : U = candidats, W = postes. Une arête = candidat qualifié pour ce poste.
- **Affectation de salles** : U = cours, W = salles. Contraintes de capacité et de matériel.
- **Mariages stables (Gale-Shapley)** : variante avec préférences .

Question : comment calculer un couplage maximum efficacement ?

Idée : se ramener à un problème de flot maximal.

Réduction au flot maximal

Construction : à partir d'un graphe bipartite $G = (U \cup W, E)$, on construit un réseau de transport G' :

Réduction au flot maximal

Construction : à partir d'un graphe bipartite $G = (U \cup W, E)$, on construit un réseau de transport G' :

- 1 Ajouter une **source** s reliée à chaque sommet de U avec capacité 1

Réduction au flot maximal

Construction : à partir d'un graphe bipartite $G = (U \cup W, E)$, on construit un réseau de transport G' :

- ➊ Ajouter une **source** s reliée à chaque sommet de U avec capacité 1
- ➋ Ajouter un **puits** p relié depuis chaque sommet de W avec capacité 1

Réduction au flot maximal

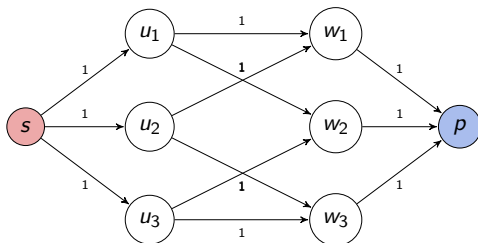
Construction : à partir d'un graphe bipartite $G = (U \cup W, E)$, on construit un réseau de transport G' :

- ➊ Ajouter une **source** s reliée à chaque sommet de U avec capacité 1
- ➋ Ajouter un **puits** p relié depuis chaque sommet de W avec capacité 1
- ➌ Orienter chaque arête de E de U vers W , avec capacité 1

Réduction au flot maximal

Construction : à partir d'un graphe bipartite $G = (U \cup W, E)$, on construit un réseau de transport G' :

- 1 Ajouter une **source** s reliée à chaque sommet de U avec capacité 1
- 2 Ajouter un **puits** p relié depuis chaque sommet de W avec capacité 1
- 3 Orienter chaque arête de E de U vers W , avec capacité 1



Réduction au flot maximal — résultat

Théorème

La valeur du flot maximal dans G' est égale à la taille du couplage maximum dans G .

Réduction au flot maximal — résultat

Théorème

La valeur du flot maximal dans G' est égale à la taille du couplage maximum dans G .

Pourquoi : chaque unité de flot qui passe de s à p emprunte exactement un sommet de U et un sommet de W . Comme toutes les capacités valent 1, chaque sommet n'est traversé qu'une seule fois — c'est exactement la définition d'un couplage.

Réduction au flot maximal — résultat

Théorème

La valeur du flot maximal dans G' est égale à la taille du couplage maximum dans G .

Pourquoi : chaque unité de flot qui passe de s à p emprunte exactement un sommet de U et un sommet de W . Comme toutes les capacités valent 1, chaque sommet n'est traversé qu'une seule fois — c'est exactement la définition d'un couplage.

Complexité : Ford-Fulkerson sur ce graphe s'arrête en au plus $|U|$ itérations (la valeur du flot est bornée par $\min(|U|, |W|)$). Chaque itération prend $O(|V| + |E|)$.

Total : $O(\min(|U|, |W|) \cdot (|V| + |E|))$

Réduction au flot maximal — résultat

Théorème

La valeur du flot maximal dans G' est égale à la taille du couplage maximum dans G .

Pourquoi : chaque unité de flot qui passe de s à p emprunte exactement un sommet de U et un sommet de W . Comme toutes les capacités valent 1, chaque sommet n'est traversé qu'une seule fois — c'est exactement la définition d'un couplage.

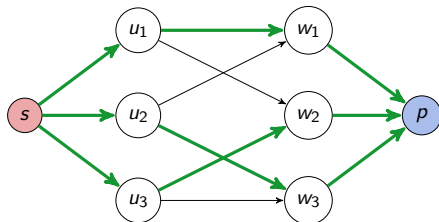
Complexité : Ford-Fulkerson sur ce graphe s'arrête en au plus $|U|$ itérations (la valeur du flot est bornée par $\min(|U|, |W|)$). Chaque itération prend $O(|V| + |E|)$.

Total : $O(\min(|U|, |W|) \cdot (|V| + |E|))$

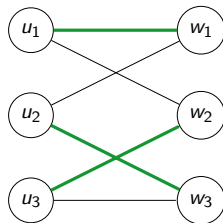
Avec Hopcroft-Karp (1973), on atteint $O(\sqrt{|V|} \cdot |E|)$ — algorithme standard pour le couplage bipartite.

Exemple de couplage par flot

Flot maximal trouvé (valeur 3) :



Couplage maximum correspondant :



Couplage parfait : $u_1 \leftrightarrow w_1$,
 $u_2 \leftrightarrow w_3$, $u_3 \leftrightarrow w_2$

Plan

6 Flots

- Présentation du problème
- Définitions
- Ford-Fulkerson
- Coupes
- Application : couplage bipartite
- Bilan

Récapitulatif

Problème	Flot maximal de s à p
Contrainte	Capacité de chaque arc, conservation aux nœuds
Objectif	Maximiser la valeur du flot
Résultat fondamental	Flot-max = Coupe-min (Ford-Fulkerson 1956)
Algorithme	Ford-Fulkerson (chaîne augmentante)
Complexité	$O(f_{\max} \cdot E)$ — pseudo-polynomial
Amélioration	Edmonds-Karp (BFS) — $O(V \cdot E ^2)$
Application clé	Couplage bipartite maximum
Réduction	Graphe bipartite $\rightarrow$ réseau de transport
Complexité couplage	$O(\min(U , W) \cdot (V + E))$
Applications	Hydraulique, réseau, affectation, vision

Plan

- 1 Définitions
- 2 Programmation
- 3 Connexité et parcours
- 4 Plus courts chemins
- 5 Arbres couvrants
- 6 Flots
- 7 Coloriage de graphes**

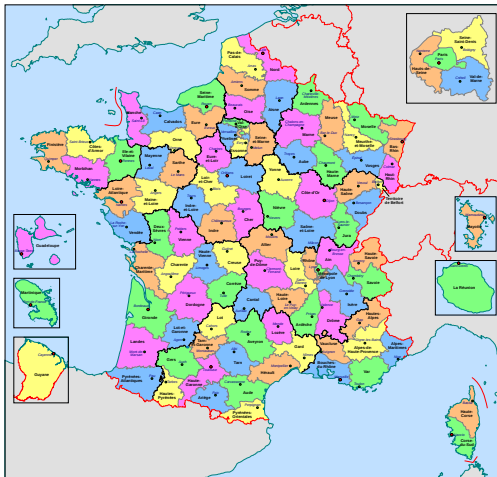
Plan

7 Coloriage de graphes

- Présentation du problème
- Définitions
- NP-complétude et approches
- Welsh-Powell
- DSatur
- Propriétés et comparaison

Présentation du problème

Colorier une carte



Présentation du problème

Planifier des examens sans conflit

			TC					PSO		IIM			EIE			MME	
			EcoG 6	Env Ent EcoE 6	Droit 2	Maths Analyse 10	Algèbre 9	Elec. ElAnal 4	Scien. Lang 1	Algo Algo 7	Model. BDD 2	CS 2	ElectS CptElec 2	Info Algo 2	Robot InfInd 1	StructMtx DP 1	StrucCr 1
Eleve1	EIE	2				X	X										
Eleve2	PSO	1				X	X										
Eleve3	PSO	1						X									
Eleve4	PSO	3				X	X	X									
Eleve5	IIM	1						X									
Eleve6	IIM	2				X				X							
Eleve7	IIM	3				X				X							
Eleve8	IIM	4	X			X	X			X							
Eleve9	IIM	4				X	X				X	X					
Eleve10	EIE	3				X	X			X		X		X			
Eleve11	EIE	2	X				X										
Eleve12	IIM	2	X			X											
Eleve13	EIE	1		X													
Eleve14	EIE	2		X													
Eleve15	IIM	4		X	X	X					X		X				
Eleve16	EIE	4	X	X	X									X			
Eleve17	PSO	2				X		X							X		
Eleve18	PSO	4				X	X	X	X								
Eleve19	EIE	1		X													
Eleve20	IIM	1								X							
Eleve21	IIM	1								X							
Eleve22	MME	2														X	X
Eleve23	IIM	1										X					
Eleve24	EIE	2		X									X				
Eleve25	EIE	3	X	X	X												

Exemple de répartition des EC en APP3

Présentation du problème

Planifier des examens sans conflit

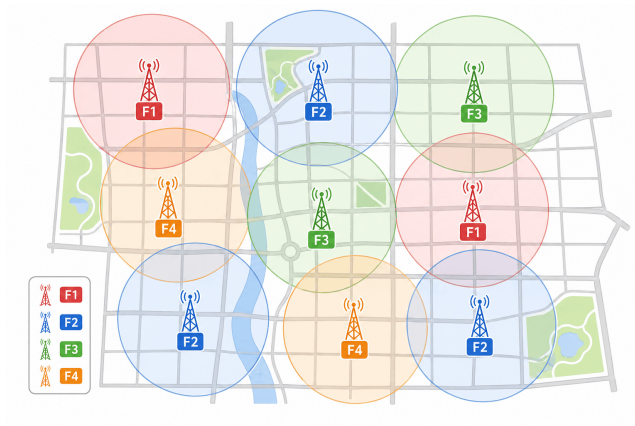
			TC				PSO		IIM			EIE			MME	
			EcoG	Env Ent	Droit	Maths	Elec.	Scien.	Algo	Model.		ElecTS	Info	Robot		
			6	EcoE	2	Analyse	AIAnal	Lang	Algo	BDD	CS	CptElec	Algo	Infind	DP	StrucCr
			6	6	2	10	4	1	7	2	2	2	2	1	1	1
Eleve1	EIE	2				X										
Eleve2	PSO	1				X										
Eleve3	PSO	1					X									
Eleve4	PSO	3				X	X									
Eleve5	IIM	1							X							
Eleve6	IIM	2				X			X							
Eleve7	IIM	3				X			X							
Eleve8	IIM	4	X			X			X							
Eleve9	IIM	4				X			X							
Eleve10	EIE	3				X			X	X						
Eleve11	EIE	2	X			X							X			
Eleve12	IIM	2	X			X										
Eleve13	EIE	1		X												
Eleve14	EIE	2		X												
Eleve15	IIM	4		X	X	X				X		X				
Eleve16	EIE	4	X	X										X		
Eleve17	PSO	2				X	X									
Eleve18	PSO	4				X	X	X								
Eleve19	EIE	1		X												
Eleve20	IIM	1							X							
Eleve21	IIM	1							X							
Eleve22	MME	2													X	X
Eleve23	IIM	1									X					
Eleve24	EIE	2	X	X								X				
Eleve25	EIE	3	X		X											

Exemple de répartition des EC en APP3

Deux examens ne peuvent pas avoir lieu au même créneau si un étudiant est inscrit aux deux.

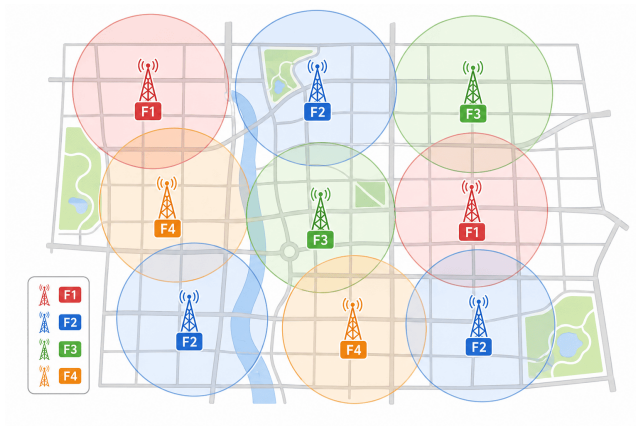
Présentation du problème

Affecter des fréquences radio



Présentation du problème

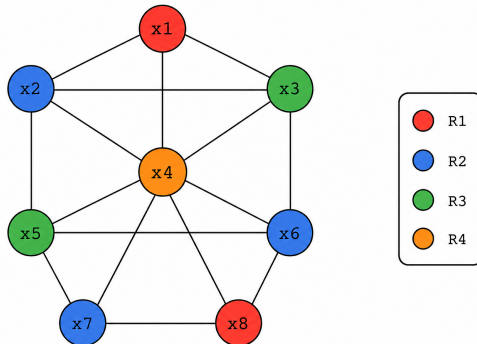
Affecter des fréquences radio



Deux antennes qui se “voient” doivent émettre sur des fréquences différentes.

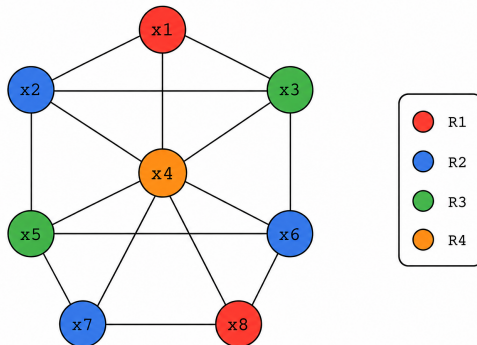
Dans la vie du programmeur

Allocation de registres dans un compilateur



Dans la vie du programmeur

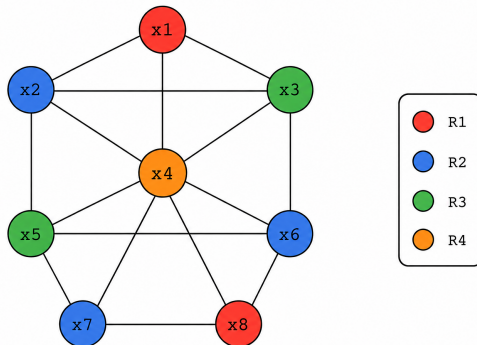
Allocation de registres dans un compilateur



Deux variables *vivantes* simultanément ne peuvent pas partager le même registre CPU.

Dans la vie du programmeur

Allocation de registres dans un compilateur



Deux variables *vivantes* simultanément ne peuvent pas partager le même registre CPU.

Optimiser l'utilisation des registres = colorier le graphe d'interférence.

Pour résumer

Le coloriage de graphes modélise tout problème où :

- On a des **objets** (sommets)

Pour résumer

Le coloriage de graphes modélise tout problème où :

- On a des **objets** (sommets)
- Certains paires d'objets sont **incompatibles** (arêtes)

Pour résumer

Le coloriage de graphes modélise tout problème où :

- On a des **objets** (sommets)
- Certains paires d'objets sont **incompatibles** (arêtes)
- On veut les regrouper en **classes compatibles** (couleurs)

Pour résumer

Le coloriage de graphes modélise tout problème où :

- On a des **objets** (sommets)
- Certains paires d'objets sont **incompatibles** (arêtes)
- On veut les regrouper en **classes compatibles** (couleurs)
- En **minimisant le nombre de classes**

Pour résumer

Le coloriage de graphes modélise tout problème où :

- On a des **objets** (sommets)
- Certains paires d'objets sont **incompatibles** (arêtes)
- On veut les regrouper en **classes compatibles** (couleurs)
- En **minimisant le nombre de classes**

Problème	Sommets	Arêtes
Carte géographique	Régions	Frontières communes
Planning d'examens	Matières	Étudiants en commun
Fréquences radio	Antennes	Zones qui se chevauchent
Allocation registres	Variables	Vivantes simultanément

Plan

7 Coloriage de graphes

- Présentation du problème
- Définitions
- NP-complétude et approches
- Welsh-Powell
- DSatur
- Propriétés et comparaison

Une définition utile : clique

Définition (Clique)

Une *clique* de $G = (V, E)$ est un sous-ensemble $C \subseteq V$ de sommets **tous reliés entre eux** (sous-graphe complet).

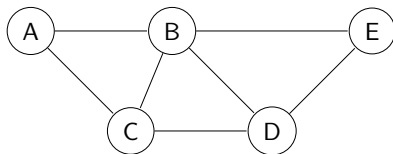
La *taille de la clique maximale* est notée $\omega(G)$.

Une définition utile : clique

Définition (Clique)

Une *clique* de $G = (V, E)$ est un sous-ensemble $C \subseteq V$ de sommets **tous reliés entre eux** (sous-graphe complet).

La *taille de la clique maximale* est notée $\omega(G)$.



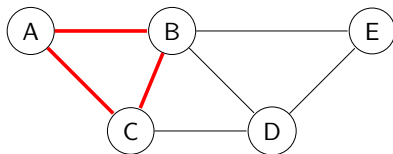
$$\omega(G) =$$

Une définition utile : clique

Définition (Clique)

Une *clique* de $G = (V, E)$ est un sous-ensemble $C \subseteq V$ de sommets **tous reliés entre eux** (sous-graphe complet).

La *taille de la clique maximale* est notée $\omega(G)$.



$\omega(G) = 3$ (triangle A-B-C par exemple)

Coloriage propre

Définition (Coloriage propre)

Un *coloriage propre* de $G = (V, E)$ est une fonction $c : V \rightarrow \mathbb{N}$ telle que

$$\forall (u, v) \in E, \quad c(u) \neq c(v).$$

On dit que G est *k-coloriable* s'il admet un coloriage propre avec au plus k couleurs.

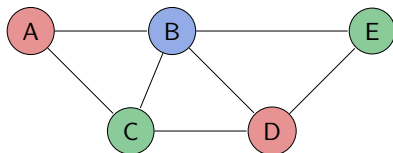
Coloriage propre

Définition (Coloriage propre)

Un *coloriage propre* de $G = (V, E)$ est une fonction $c : V \rightarrow \mathbb{N}$ telle que

$$\forall (u, v) \in E, \quad c(u) \neq c(v).$$

On dit que G est *k-coloriable* s'il admet un coloriage propre avec au plus k couleurs.



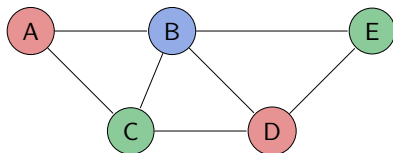
Coloriage propre

Définition (Coloriage propre)

Un *coloriage propre* de $G = (V, E)$ est une fonction $c : V \rightarrow \mathbb{N}$ telle que

$$\forall (u, v) \in E, \quad c(u) \neq c(v).$$

On dit que G est *k-coloriable* s'il admet un coloriage propre avec au plus k couleurs.



3 couleurs utilisées — est-ce optimal ?

Nombre chromatique

Définition (Nombre chromatique)

Le *nombre chromatique* $\chi(G)$ est le plus petit entier k tel que G soit k -coloriable.

Nombre chromatique

Définition (Nombre chromatique)

Le *nombre chromatique* $\chi(G)$ est le plus petit entier k tel que G soit k -coloriable.

Proposition

$$\chi(G) \geq \omega(G)$$

Nombre chromatique

Définition (Nombre chromatique)

Le *nombre chromatique* $\chi(G)$ est le plus petit entier k tel que G soit k -coloriable.

Proposition

$$\chi(G) \geq \omega(G)$$

Proposition

$$\chi(G) \leq \Delta(G) + 1, \text{ où } \Delta(G) \text{ est le degré maximal de } G.$$

Quelques valeurs remarquables

Graphe G	$\chi(G)$
Chemin P_n	2
Cycle pair C_{2k}	2
Cycle impair C_{2k+1}	3
Graphe complet K_n	n
Graphe bipartite	≤ 2
Arbre	2

Quelques valeurs remarquables

Graphe G	$\chi(G)$
Chemin P_n	2
Cycle pair C_{2k}	2
Cycle impair C_{2k+1}	3
Graphe complet K_n	n
Graphe bipartite	≤ 2
Arbre	2

Théorème (Four Color Theorem, Appel & Haken 1976)

Tout graphe planaire vérifie $\chi(G) \leq 4$.

Quelques valeurs remarquables

Graphe G	$\chi(G)$
Chemin P_n	2
Cycle pair C_{2k}	2
Cycle impair C_{2k+1}	3
Graphe complet K_n	n
Graphe bipartite	≤ 2
Arbre	2

Théorème (Four Color Theorem, Appel & Haken 1976)

Tout graphe planaire vérifie $\chi(G) \leq 4$.

(Première preuve assistée par ordinateur — vérification de 1936 cas par un programme informatique)

Plan

7 Coloriage de graphes

- Présentation du problème
- Définitions
- NP-complétude et approches
- Welsh-Powell
- DSatur
- Propriétés et comparaison

Un problème difficile

Calculer $\chi(G)$ est NP-difficile.

Un problème difficile

Calculer $\chi(G)$ est NP-difficile.

Plus précisément :

- Décider si $\chi(G) \leq k$ pour $k \geq 3$ est **NP-complet**

Un problème difficile

Calculer $\chi(G)$ est NP-difficile.

Plus précisément :

- Décider si $\chi(G) \leq k$ pour $k \geq 3$ est **NP-complet**
- Il n'existe **pas** d'algorithme polynomial connu pour trouver un coloriage optimal en général

Un problème difficile

Calculer $\chi(G)$ est NP-difficile.

Plus précisément :

- Décider si $\chi(G) \leq k$ pour $k \geq 3$ est **NP-complet**
- Il n'existe **pas** d'algorithme polynomial connu pour trouver un coloriage optimal en général
- Même approcher $\chi(G)$ à un facteur constant est difficile

Un problème difficile

Calculer $\chi(G)$ est NP-difficile.

Plus précisément :

- Décider si $\chi(G) \leq k$ pour $k \geq 3$ est **NP-complet**
- Il n'existe **pas** d'algorithme polynomial connu pour trouver un coloriage optimal en général
- Même approcher $\chi(G)$ à un facteur constant est difficile

k	Décision
$k = 1$	trivial (G sans arête ?)
$k = 2$	polynomial (BFS/DFS, test bipartite)
$k \geq 3$	NP-complet

Approches possibles

Puisque le problème est NP-complet, on dispose de trois types d'approches :

Approches possibles

Puisque le problème est NP-complet, on dispose de trois types d'approches :

- **Algorithmes exacts** — backtracking, branch & bound. Exponentiels mais optimaux. Utilisables sur des petits graphes. Cf. cours de R.O.

Approches possibles

Puisque le problème est NP-complet, on dispose de trois types d'approches :

- **Algorithmes exacts** — backtracking, branch & bound. Exponentiels mais optimaux. Utilisables sur des petits graphes. Cf. cours de R.O.
- **Algorithmes d'approximation** — garantissent un résultat “pas trop loin” de l'optimal.

Approches possibles

Puisque le problème est NP-complet, on dispose de trois types d'approches :

- **Algorithmes exacts** — backtracking, branch & bound. Exponentiels mais optimaux. Utilisables sur des petits graphes. Cf. cours de R.O.
- **Algorithmes d'approximation** — garantissent un résultat “pas trop loin” de l'optimal.
- **Heuristiques gloutonnes** — rapides, bons résultats en pratique, **aucune garantie d'optimalité**.

Ce sont celles qu'on étudie ici : Welsh-Powell et DSatur.

Approches possibles

Puisque le problème est NP-complet, on dispose de trois types d'approches :

- **Algorithmes exacts** — backtracking, branch & bound. Exponentiels mais optimaux. Utilisables sur des petits graphes. Cf. cours de R.O.
- **Algorithmes d'approximation** — garantissent un résultat “pas trop loin” de l'optimal.
- **Heuristiques gloutonnes** — rapides, bons résultats en pratique, **aucune garantie d'optimalité**.

Ce sont celles qu'on étudie ici : Welsh-Powell et DSatur.

Remarque : les heuristiques sont le standard industriel — GCC, LLVM, les planificateurs d'examens utilisent tous des approches gloutonnes.

Plan

7 Coloriage de graphes

- Présentation du problème
- Définitions
- NP-complétude et approches
- **Welsh-Powell**
- DSatur
- Propriétés et comparaison

Idée de l'algorithme glouton

Glouton naïf : parcourir les sommets dans un ordre quelconque, affecter à chacun la plus petite couleur non utilisée par ses voisins déjà coloriés.

Idée de l'algorithme glouton

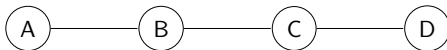
Glouton naïf : parcourir les sommets dans un ordre quelconque, affecter à chacun la plus petite couleur non utilisée par ses voisins déjà coloriés.

Problème : l'ordre de parcours a un impact majeur sur le résultat.

Idée de l'algorithme glouton

Glouton naïf : parcourir les sommets dans un ordre quelconque, affecter à chacun la plus petite couleur non utilisée par ses voisins déjà coloriés.

Problème : l'ordre de parcours a un impact majeur sur le résultat.



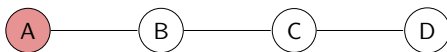
Ordre A, B, C, D :

Ordre A, D, B, C :

Idée de l'algorithme glouton

Glouton naïf : parcourir les sommets dans un ordre quelconque, affecter à chacun la plus petite couleur non utilisée par ses voisins déjà coloriés.

Problème : l'ordre de parcours a un impact majeur sur le résultat.



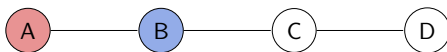
Ordre A, B, C, D : $A=1$

Ordre A, D, B, C :

Idée de l'algorithme glouton

Glouton naïf : parcourir les sommets dans un ordre quelconque, affecter à chacun la plus petite couleur non utilisée par ses voisins déjà coloriés.

Problème : l'ordre de parcours a un impact majeur sur le résultat.



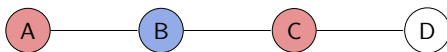
Ordre A, B, C, D : $A=1, B=2$

Ordre A, D, B, C :

Idée de l'algorithme glouton

Glouton naïf : parcourir les sommets dans un ordre quelconque, affecter à chacun la plus petite couleur non utilisée par ses voisins déjà coloriés.

Problème : l'ordre de parcours a un impact majeur sur le résultat.



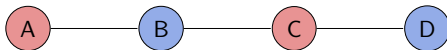
Ordre A, B, C, D : $A=1$, $B=2$, $C=1$

Ordre A, D, B, C :

Idée de l'algorithme glouton

Glouton naïf : parcourir les sommets dans un ordre quelconque, affecter à chacun la plus petite couleur non utilisée par ses voisins déjà coloriés.

Problème : l'ordre de parcours a un impact majeur sur le résultat.



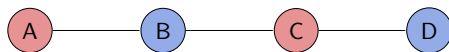
Ordre A, B, C, D : $A=1, B=2, C=1, D=2$.

Ordre A, D, B, C :

Idée de l'algorithme glouton

Glouton naïf : parcourir les sommets dans un ordre quelconque, affecter à chacun la plus petite couleur non utilisée par ses voisins déjà coloriés.

Problème : l'ordre de parcours a un impact majeur sur le résultat.



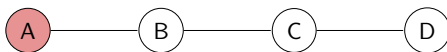
Ordre A, B, C, D : $A=1, B=2, C=1, D=2$. **2 couleurs.**✓

Ordre A, D, B, C :

Idée de l'algorithme glouton

Glouton naïf : parcourir les sommets dans un ordre quelconque, affecter à chacun la plus petite couleur non utilisée par ses voisins déjà coloriés.

Problème : l'ordre de parcours a un impact majeur sur le résultat.



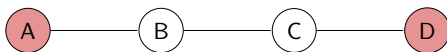
Ordre A, B, C, D : $A=1$, $B=2$, $C=1$, $D=2$. **2 couleurs.**✓

Ordre A, D, B, C : $A=1$

Idée de l'algorithme glouton

Glouton naïf : parcourir les sommets dans un ordre quelconque, affecter à chacun la plus petite couleur non utilisée par ses voisins déjà coloriés.

Problème : l'ordre de parcours a un impact majeur sur le résultat.



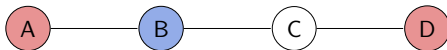
Ordre A, B, C, D : $A=1, B=2, C=1, D=2$. **2 couleurs.**✓

Ordre A, D, B, C : $A=1, D=1$

Idée de l'algorithme glouton

Glouton naïf : parcourir les sommets dans un ordre quelconque, affecter à chacun la plus petite couleur non utilisée par ses voisins déjà coloriés.

Problème : l'ordre de parcours a un impact majeur sur le résultat.



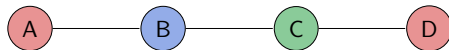
Ordre A, B, C, D : $A=1, B=2, C=1, D=2$. **2 couleurs.**✓

Ordre A, D, B, C : $A=1, D=1, B=2$

Idée de l'algorithme glouton

Glouton naïf : parcourir les sommets dans un ordre quelconque, affecter à chacun la plus petite couleur non utilisée par ses voisins déjà coloriés.

Problème : l'ordre de parcours a un impact majeur sur le résultat.



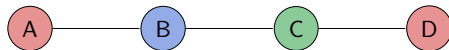
Ordre A, B, C, D : $A=1, B=2, C=1, D=2$. **2 couleurs.**✓

Ordre A, D, B, C : $A=1, D=1, B=2, C=3$.

Idée de l'algorithme glouton

Glouton naïf : parcourir les sommets dans un ordre quelconque, affecter à chacun la plus petite couleur non utilisée par ses voisins déjà coloriés.

Problème : l'ordre de parcours a un impact majeur sur le résultat.



Ordre A, B, C, D : $A=1, B=2, C=1, D=2$. **2 couleurs.**✓

Ordre A, D, B, C : $A=1, D=1, B=2, C=3$. **3 couleurs.**

Algorithme de Welsh-Powell

Idée : traiter en priorité les sommets de **fort degré** (les plus contraints)
— ils réduiront plus le choix de couleur de leurs voisins.

Algorithme de Welsh-Powell

Idée : traiter en priorité les sommets de **fort degré** (les plus contraints)
— ils réduiront plus le choix de couleur de leurs voisins.

Algorithme WelshPowell(graphe G) :

Entrée : $G = (V, E)$

Trier V par degré décroissant : $v_1, v_2, \dots, v_n$

pour $i = 1$ à n **faire**

$c(v_i) \leftarrow \min\{k \in \mathbb{N}^* \mid k \notin \{c(u) : u \text{ voisin de } v_i, u \text{ déjà colorié}\}\}$

fin pour

Sortie : $c : V \rightarrow \mathbb{N}^*$ coloriage propre

Algorithme de Welsh-Powell

Idee : traiter en priorité les sommets de **fort degré** (les plus contraints)
— ils réduiront plus le choix de couleur de leurs voisins.

Algorithme WelshPowell(graphe G) :

Entrée : $G = (V, E)$

Trier V par degré décroissant : $v_1, v_2, \dots, v_n$

pour $i = 1$ à n **faire**

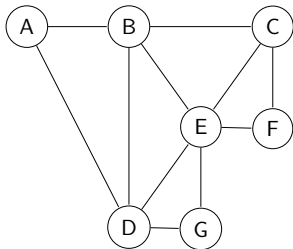
$c(v_i) \leftarrow \min\{k \in \mathbb{N}^* \mid k \notin \{c(u) : u \text{ voisin de } v_i, u \text{ déjà colorié}\}\}$

fin pour

Sortie : $c : V \rightarrow \mathbb{N}^*$ coloriage propre

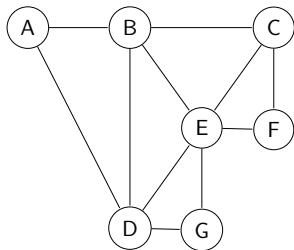
- Complexité : $O(|V|^2)$
- **Pas garanti optimal** — dépend de l'ordre des ex-æquo
- Simple à implémenter

Welsh-Powell — exemple détaillé



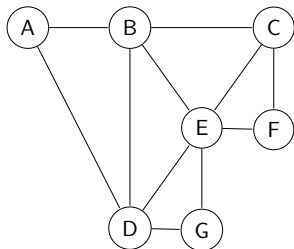
Welsh-Powell — exemple détaillé

Ordre : $E(5)$, $B(4)$, $D(4)$, $C(3)$, $A(2)$, $F(2)$, $G(2)$



Welsh-Powell — exemple détaillé

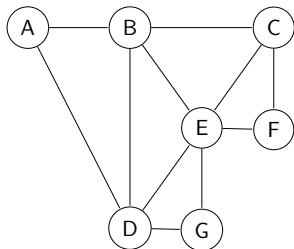
Ordre : $E(5)$, $B(4)$, $D(4)$, $C(3)$, $A(2)$, $F(2)$, $G(2)$



Som.	Deg.	Voisins colorés	Coul.
E	5		
B	4		
D	4		
C	3		
A	2		
F	2		
G	2		

Welsh-Powell — exemple détaillé

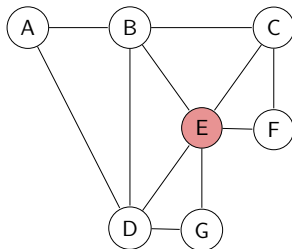
Ordre : $E(5)$, $B(4)$, $D(4)$, $C(3)$, $A(2)$, $F(2)$, $G(2)$



Som.	Deg.	Voisins colorés	Coul.
E	5	—	
B	4		
D	4		
C	3		
A	2		
F	2		
G	2		

Welsh-Powell — exemple détaillé

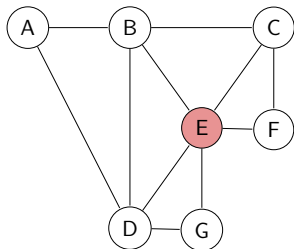
Ordre : $E(5)$, $B(4)$, $D(4)$, $C(3)$, $A(2)$, $F(2)$, $G(2)$



Som.	Deg.	Voisins colorés	Coul.
E	5	—	• 1
B	4		
D	4		
C	3		
A	2		
F	2		
G	2		

Welsh-Powell — exemple détaillé

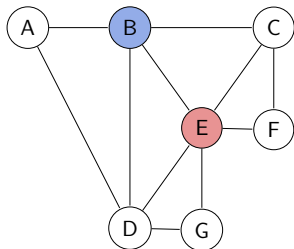
Ordre : $E(5)$, $B(4)$, $D(4)$, $C(3)$, $A(2)$, $F(2)$, $G(2)$



Som.	Deg.	Voisins colorés	Coul.
E	5	—	• 1
B	4	$E:1$	
D	4		
C	3		
A	2		
F	2		
G	2		

Welsh-Powell — exemple détaillé

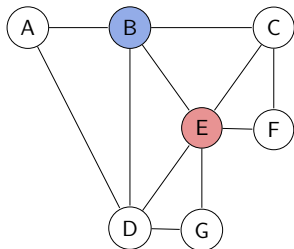
Ordre : $E(5)$, $B(4)$, $D(4)$, $C(3)$, $A(2)$, $F(2)$, $G(2)$



Som.	Deg.	Voisins colorés	Coul.
E	5	—	• 1
B	4	$E:1$	• 2
D	4		
C	3		
A	2		
F	2		
G	2		

Welsh-Powell — exemple détaillé

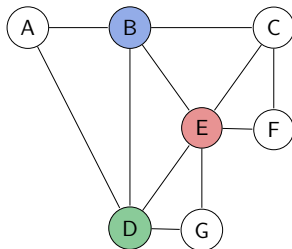
Ordre : $E(5)$, $B(4)$, $D(4)$, $C(3)$, $A(2)$, $F(2)$, $G(2)$



Som.	Deg.	Voisins colorés	Coul.
E	5	—	• 1
B	4	$E:1$	• 2
D	4	$E:1, B:2$	
C	3		
A	2		
F	2		
G	2		

Welsh-Powell — exemple détaillé

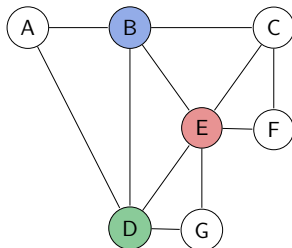
Ordre : $E(5)$, $B(4)$, $D(4)$, $C(3)$, $A(2)$, $F(2)$, $G(2)$



Som.	Deg.	Voisins colorés	Coul.
E	5	—	● 1
B	4	$E:1$	● 2
D	4	$E:1, B:2$	● 3
C	3		
A	2		
F	2		
G	2		

Welsh-Powell — exemple détaillé

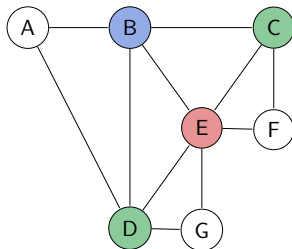
Ordre : $E(5)$, $B(4)$, $D(4)$, $C(3)$, $A(2)$, $F(2)$, $G(2)$



Som.	Deg.	Voisins colorés	Coul.
E	5	—	● 1
B	4	$E:1$	● 2
D	4	$E:1, B:2$	● 3
C	3	$B:2, E:1$	
A	2		
F	2		
G	2		

Welsh-Powell — exemple détaillé

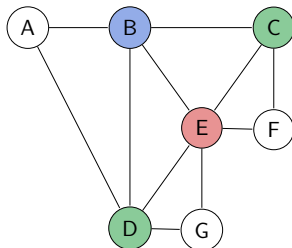
Ordre : $E(5)$, $B(4)$, $D(4)$, $C(3)$, $A(2)$, $F(2)$, $G(2)$



Som.	Deg.	Voisins colorés	Coul.
E	5	—	● 1
B	4	$E:1$	● 2
D	4	$E:1, B:2$	● 3
C	3	$B:2, E:1$	● 3
A	2		
F	2		
G	2		

Welsh-Powell — exemple détaillé

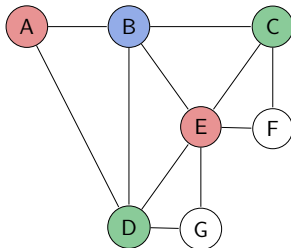
Ordre : $E(5)$, $B(4)$, $D(4)$, $C(3)$, $A(2)$, $F(2)$, $G(2)$



Som.	Deg.	Voisins colorés	Coul.
E	5	—	● 1
B	4	$E:1$	● 2
D	4	$E:1, B:2$	● 3
C	3	$B:2, E:1$	● 3
A	2	$B:2, D:3$	
F	2		
G	2		

Welsh-Powell — exemple détaillé

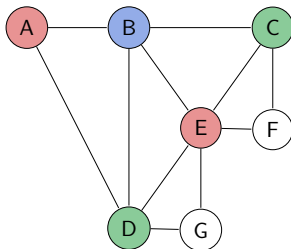
Ordre : $E(5)$, $B(4)$, $D(4)$, $C(3)$, $A(2)$, $F(2)$, $G(2)$



Som.	Deg.	Voisins colorés	Coul.
E	5	—	● 1
B	4	$E:1$	● 2
D	4	$E:1, B:2$	● 3
C	3	$B:2, E:1$	● 3
A	2	$B:2, D:3$	● 1
F	2		
G	2		

Welsh-Powell — exemple détaillé

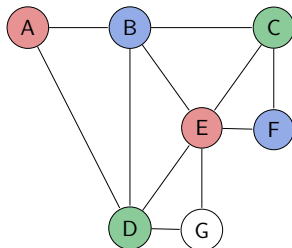
Ordre : $E(5)$, $B(4)$, $D(4)$, $C(3)$, $A(2)$, $F(2)$, $G(2)$



Som.	Deg.	Voisins colorés	Coul.
E	5	—	● 1
B	4	$E:1$	● 2
D	4	$E:1, B:2$	● 3
C	3	$B:2, E:1$	● 3
A	2	$B:2, D:3$	● 1
F	2	$C:3, E:1$	
G	2		

Welsh-Powell — exemple détaillé

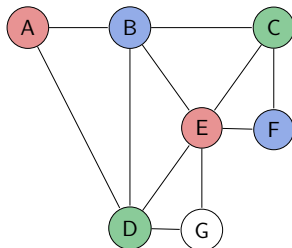
Ordre : $E(5)$, $B(4)$, $D(4)$, $C(3)$, $A(2)$, $F(2)$, $G(2)$



Som.	Deg.	Voisins colorés	Coul.
E	5	—	● 1
B	4	$E:1$	● 2
D	4	$E:1, B:2$	● 3
C	3	$B:2, E:1$	● 3
A	2	$B:2, D:3$	● 1
F	2	$C:3, E:1$	● 2
G	2		

Welsh-Powell — exemple détaillé

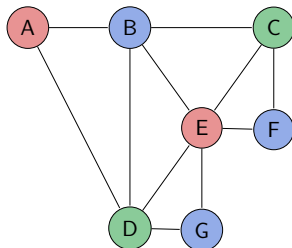
Ordre : $E(5)$, $B(4)$, $D(4)$, $C(3)$, $A(2)$, $F(2)$, $G(2)$



Som.	Deg.	Voisins colorés	Coul.
E	5	—	● 1
B	4	$E:1$	● 2
D	4	$E:1, B:2$	● 3
C	3	$B:2, E:1$	● 3
A	2	$B:2, D:3$	● 1
F	2	$C:3, E:1$	● 2
G	2	$D:3, E:1$	

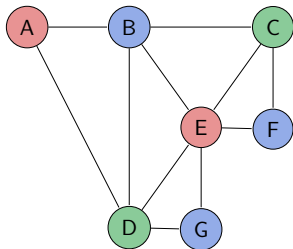
Welsh-Powell — exemple détaillé

Ordre : $E(5)$, $B(4)$, $D(4)$, $C(3)$, $A(2)$, $F(2)$, $G(2)$



Som.	Deg.	Voisins colorés	Coul.
E	5	—	● 1
B	4	$E:1$	● 2
D	4	$E:1, B:2$	● 3
C	3	$B:2, E:1$	● 3
A	2	$B:2, D:3$	● 1
F	2	$C:3, E:1$	● 2
G	2	$D:3, E:1$	● 2

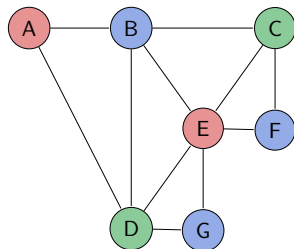
Welsh-Powell — exemple détaillé — résultat



Coloriage final :

- **Couleur 1** : A, E
- **Couleur 2** : B, F, G
- **Couleur 3** : C, D

Welsh-Powell — exemple détaillé — résultat

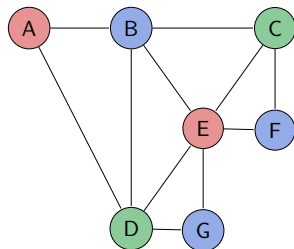


Coloriage final :

- **Couleur 1** : A, E
- **Couleur 2** : B, F, G
- **Couleur 3** : C, D

Vérification : aucune arête ne relie deux sommets de même couleur. ✓

Welsh-Powell — exemple détaillé — résultat



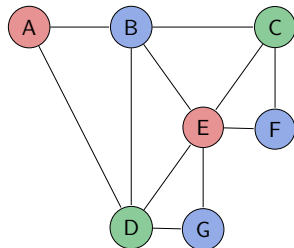
Coloriage final :

- **Couleur 1** : A, E
- **Couleur 2** : B, F, G
- **Couleur 3** : C, D

Vérification : aucune arête ne relie deux sommets de même couleur. ✓

Optimalité ?

Welsh-Powell — exemple détaillé — résultat



Coloriage final :

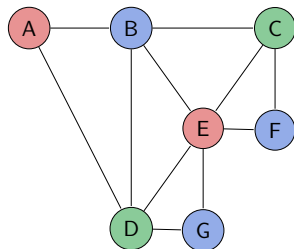
- **Couleur 1** : A, E
- **Couleur 2** : B, F, G
- **Couleur 3** : C, D

Vérification : aucune arête ne relie deux sommets de même couleur. ✓

Optimalité :

B-D-E forment un triangle

Welsh-Powell — exemple détaillé — résultat



Coloriage final :

- **Couleur 1** : A, E
- **Couleur 2** : B, F, G
- **Couleur 3** : C, D

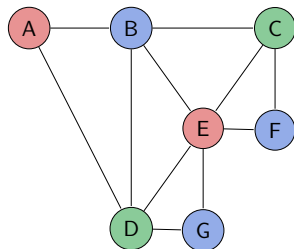
Vérification : aucune arête ne relie deux sommets de même couleur. ✓

Optimalité :

$B-D-E$ forment un triangle

$$\Rightarrow \omega(G) = 3$$

Welsh-Powell — exemple détaillé — résultat



Coloriage final :

- **Couleur 1** : A, E
- **Couleur 2** : B, F, G
- **Couleur 3** : C, D

Vérification : aucune arête ne relie deux sommets de même couleur. ✓

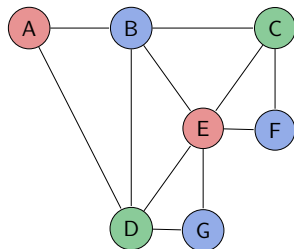
Optimalité :

$B-D-E$ forment un triangle

$$\Rightarrow \omega(G) = 3$$

$$\Rightarrow \chi(G) \geq 3.$$

Welsh-Powell — exemple détaillé — résultat



Coloriage final :

- **Couleur 1** : A, E
- **Couleur 2** : B, F, G
- **Couleur 3** : C, D

Vérification : aucune arête ne relie deux sommets de même couleur. ✓

Optimalité :

$B-D-E$ forment un triangle

$$\Rightarrow \omega(G) = 3$$

$$\Rightarrow \chi(G) \geq 3.$$

Welsh-Powell a trouvé 3 : **optimal**.

Welsh-Powell — contre-exemple

Welsh-Powell **n'est pas optimal en général.**

Welsh-Powell — contre-exemple

Welsh-Powell **n'est pas optimal en général.**

Problème : dans les cas d'ex-aequo, l'ordre de parcours a toujours un impact majeur sur le résultat.

Welsh-Powell — contre-exemple

Welsh-Powell **n'est pas optimal en général.**

Problème : dans les cas d'ex-aequo, l'ordre de parcours a toujours un impact majeur sur le résultat.



Ordre sur les degrés : B,C,D,E de degré 2 ; A et F de degré 1.

Welsh-Powell — contre-exemple

Welsh-Powell **n'est pas optimal en général.**

Problème : dans les cas d'ex-aequo, l'ordre de parcours a toujours un impact majeur sur le résultat.



Ordre sur les degrés : B,C,D,E de degré 2 ; A et F de degré 1.

Ordre *B, E, C, D, A, F* :

Welsh-Powell — contre-exemple

Welsh-Powell **n'est pas optimal en général.**

Problème : dans les cas d'ex-aequo, l'ordre de parcours a toujours un impact majeur sur le résultat.



Ordre sur les degrés : B,C,D,E de degré 2 ; A et F de degré 1.

Ordre B, E, C, D, A, F : $B=1$

Welsh-Powell — contre-exemple

Welsh-Powell **n'est pas optimal en général.**

Problème : dans les cas d'ex-aequo, l'ordre de parcours a toujours un impact majeur sur le résultat.



Ordre sur les degrés : B,C,D,E de degré 2 ; A et F de degré 1.

Ordre B, E, C, D, A, F : **B=1, E=1**

Welsh-Powell — contre-exemple

Welsh-Powell **n'est pas optimal en général.**

Problème : dans les cas d'ex-aequo, l'ordre de parcours a toujours un impact majeur sur le résultat.



Ordre sur les degrés : B,C,D,E de degré 2 ; A et F de degré 1.

Ordre B, E, C, D, A, F : $B=1$, $E=1$, $C=2$

Welsh-Powell — contre-exemple

Welsh-Powell **n'est pas optimal en général.**

Problème : dans les cas d'ex-aequo, l'ordre de parcours a toujours un impact majeur sur le résultat.



Ordre sur les degrés : B,C,D,E de degré 2 ; A et F de degré 1.

Ordre B, E, C, D, A, F : B=1, E=1, C=2, D=3

Welsh-Powell — contre-exemple

Welsh-Powell **n'est pas optimal en général.**

Problème : dans les cas d'ex-aequo, l'ordre de parcours a toujours un impact majeur sur le résultat.



Ordre sur les degrés : B,C,D,E de degré 2 ; A et F de degré 1.

Ordre B, E, C, D, A, F : $B=1$, $E=1$, $C=2$, $D=3$, $A=1$

Welsh-Powell — contre-exemple

Welsh-Powell **n'est pas optimal en général.**

Problème : dans les cas d'ex-aequo, l'ordre de parcours a toujours un impact majeur sur le résultat.



Ordre sur les degrés : B,C,D,E de degré 2 ; A et F de degré 1.

Ordre B, E, C, D, A, F : $B=1$, $E=1$, $C=2$, $D=3$, $A=1$, $F=1$

Welsh-Powell — contre-exemple

Welsh-Powell **n'est pas optimal en général.**

Problème : dans les cas d'ex-aequo, l'ordre de parcours a toujours un impact majeur sur le résultat.



Ordre sur les degrés : B,C,D,E de degré 2 ; A et F de degré 1.

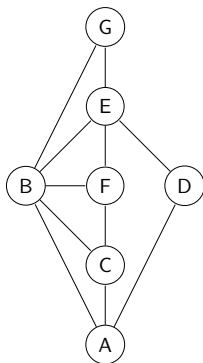
Ordre B, E, C, D, A, F : $B=1$, $E=1$, $C=2$, $D=3$, $A=1$, $F=1$

Trois couleurs, alors que deux optimal !

Welsh-Powell — contre-exemple 2

Welsh-Powell **n'est pas optimal en général.**

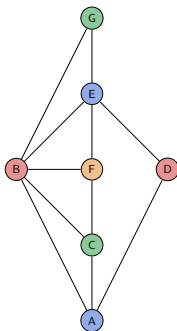
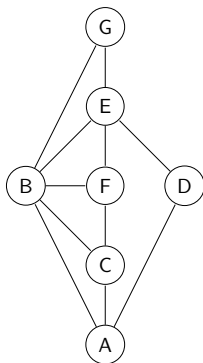
Dérouler l'algorithme et trouver deux nombre de couleurs différents en fonction de l'ordre choisi.



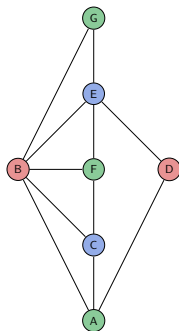
Welsh-Powell — contre-exemple 2

Welsh-Powell **n'est pas optimal en général.**

Dérouler l'algorithme et trouver deux nombre de couleurs différents en fonction de l'ordre choisi.



Ordre : B, E, A, C, F, D, G

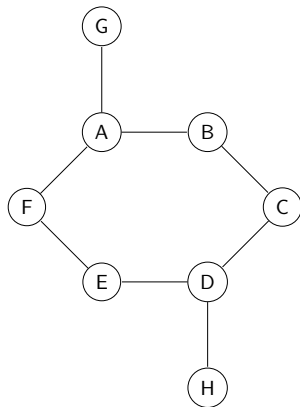


Ordre : B, E, F, C, A, D, G

Welsh-Powell — contre-exemple 3

Welsh-Powell **peut aussi ne jamais être optimal.**

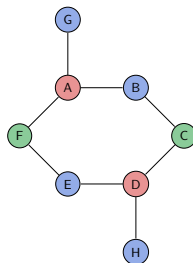
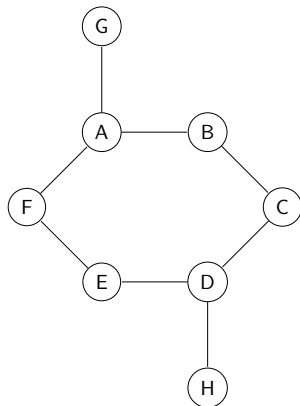
Dérouler l'algorithme, et trouver le problème.



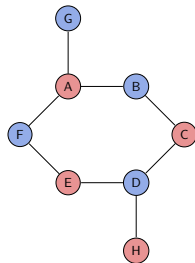
Welsh-Powell — contre-exemple 3

Welsh-Powell **peut** aussi ne jamais être optimal.

Dérouler l'algorithme, et trouver le problème.



Welsh-Powell



Optimal

Remarque : en fonction de l'ordre choisi sur B,C,F,E dans Welsh-Powell, on pourrait ne pas avoir le même coloriage, mais on aurait toujours trois couleurs.

Plan

7 Coloriage de graphes

- Présentation du problème
- Définitions
- NP-complétude et approches
- Welsh-Powell
- **DSatur**
- Propriétés et comparaison

Limite de Welsh-Powell

Trier par degré **une fois pour toutes** au début ignore l'évolution de la situation au fil du coloriage.

Limite de Welsh-Powell

Trier par degré **une fois pour toutes** au début ignore l'évolution de la situation au fil du coloriage.

Intuition : le sommet le plus “contraint” à l'étape k n'est pas forcément celui qui avait le plus grand degré au départ.

Limite de Welsh-Powell

Trier par degré **une fois pour toutes** au début ignore l'évolution de la situation au fil du coloriage.

Intuition : le sommet le plus “contraint” à l'étape k n'est pas forcément celui qui avait le plus grand degré au départ.

Définition (Saturation)

La *saturation* d'un sommet v à l'instant t est le nombre de **couleurs distinctes** présentes dans son voisinage déjà colorié :

$$sat_t(v) = |\{c(u) : u \text{ voisin de } v, u \text{ colorié à } t\}|$$

Limite de Welsh-Powell

Trier par degré **une fois pour toutes** au début ignore l'évolution de la situation au fil du coloriage.

Intuition : le sommet le plus “contraint” à l'étape k n'est pas forcément celui qui avait le plus grand degré au départ.

Définition (Saturation)

La *saturation* d'un sommet v à l'instant t est le nombre de **couleurs distinctes** présentes dans son voisinage déjà colorié :

$$sat_t(v) = |\{c(u) : u \text{ voisin de } v, u \text{ colorié à } t\}|$$

Un sommet de forte saturation a **beaucoup de couleurs déjà interdites**
 $\Rightarrow$ il est prioritaire.

Algorithme DSatur

Degree of **S**aturation — Brélaz, 1979.

Algorithme DSatur(graphe G) :

Entrée : $G = (V, E)$

$S \leftarrow V$ (ensemble des sommets non coloriés)

tant que $S \neq \emptyset$ **faire**

Choisir $v \in S$ de saturation maximale

(ex-æquo : choisir celui de plus grand degré)

$c(v) \leftarrow \min\{k \in \mathbb{N}^* \mid k \notin \{c(u) : u \text{ voisin de } v, u \text{ colorié}\}\}$

$S \leftarrow S \setminus \{v\}$

fin tant que

Sortie : $c : V \rightarrow \mathbb{N}^*$ coloriage propre

Algorithme DSatur

Degree of **Saturation** — Brélaz, 1979.

Algorithme DSatur(graphe G) :

Entrée : $G = (V, E)$

$S \leftarrow V$ (ensemble des sommets non coloriés)

tant que $S \neq \emptyset$ **faire**

Choisir $v \in S$ de saturation maximale

(ex-æquo : choisir celui de plus grand degré)

$c(v) \leftarrow \min\{k \in \mathbb{N}^* \mid k \notin \{c(u) : u \text{ voisin de } v, u \text{ colorié}\}\}$

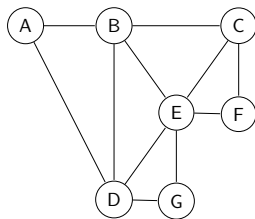
$S \leftarrow S \setminus \{v\}$

fin tant que

Sortie : $c : V \rightarrow \mathbb{N}^*$ coloriage propre

- Complexité : $O(|V|^2)$
- **Pas garanti optimal** non plus — mais meilleur en pratique
- Optimal sur les graphes parfaits et bipartites

DSatur — exemple détaillé

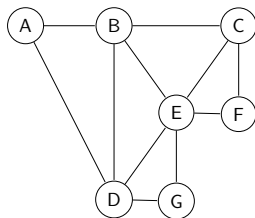


DSatur — exemple détaillé

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$\{ 5 : \{ E \}, 4 : \{ B, D \}, 3 : \{ C \}, 2 : \{ A, F, G \} \}$

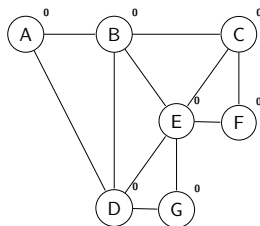


DSatur — exemple détaillé

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$\{ 5 : \{ E \}, 4 : \{ B, D \}, 3 : \{ C \}, 2 : \{ A, F, G \} \}$



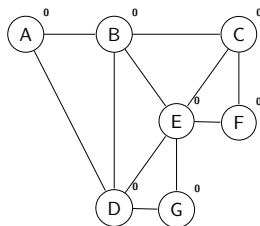
Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	E		

DSatur — exemple détaillé

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$\{ 5 : \{ E \}, 4 : \{ B, D \}, 3 : \{ C \}, 2 : \{ A, F, G \} \}$



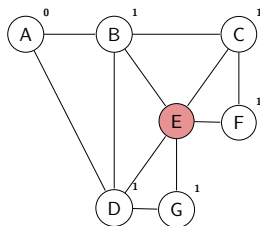
Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	<i>E</i>	—	

DSatur — exemple détaillé

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$\{ 5 : \{ \textcolor{red}{E} \}, 4 : \{ B, D \}, 3 : \{ C \}, 2 : \{ B, F, G \} \}$



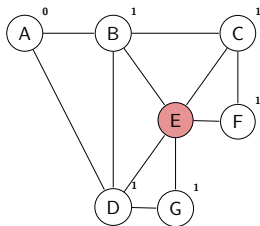
Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	E	—	• 1

DSatur — exemple détaillé

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$\{ 5 : \{ E \}, 4 : \{ B, D \}, 3 : \{ C \}, 2 : \{ B, F, G \} \}$



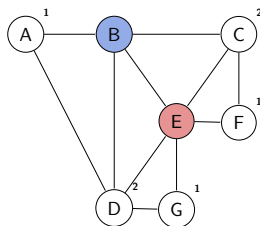
Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	E	—	• 1
1	4	B	E:1	

DSatur — exemple détaillé

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$\{ 5 : \{ \text{E} \}, 4 : \{ \text{B}, \text{D} \}, 3 : \{ \text{C} \}, 2 : \{ \text{B}, \text{F}, \text{G} \} \}$



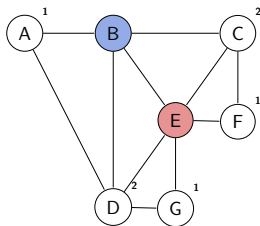
Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	E	—	• 1
1	4	B	E:1	• 2

DSatur — exemple détaillé

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$\{ 5 : \{ E \}, 4 : \{ B, D \}, 3 : \{ C \}, 2 : \{ A, F, G \} \}$



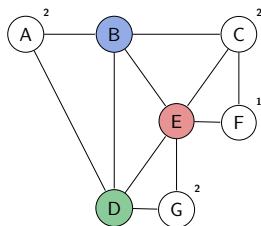
Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	E	—	• 1
1	4	B	E:1	• 2
2	4	D	E:1, B:2	

DSatur — exemple détaillé

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 5 : \{ E \}, 4 : \{ B, D \}, 3 : \{ C \}, 2 : \{ A, F, G \} \}$$



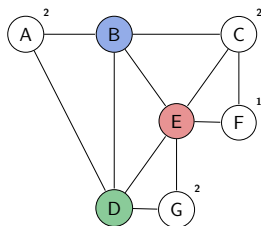
Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	E	—	• 1
1	4	B	E:1	• 2
2	4	D	E:1, B:2	• 3

DSatur — exemple détaillé

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 5 : \{ E \}, 4 : \{ B, D \}, 3 : \{ C \}, 2 : \{ A, F, G \} \}$$



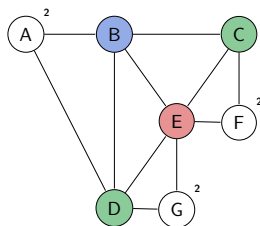
Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	E	—	• 1
1	4	B	E:1	• 2
2	4	D	E:1, B:2	• 3
2	3	C	B:2, E:1	

DSatur — exemple détaillé

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 5 : \{ E \}, 4 : \{ B, D \}, 3 : \{ C \}, 2 : \{ A, F, G \} \}$$



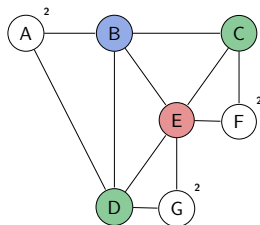
Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	E	—	• 1
1	4	B	E:1	• 2
2	4	D	E:1, B:2	• 3
2	3	C	B:2, E:1	• 3

DSatur — exemple détaillé

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 5 : \{ E \}, 4 : \{ B, D \}, 3 : \{ C \}, 2 : \{ A, F, G \} \}$$



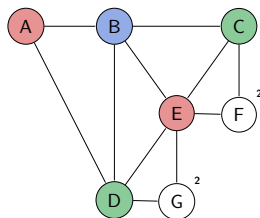
Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	E	—	• 1
1	4	B	E:1	• 2
2	4	D	E:1, B:2	• 3
2	3	C	B:2, E:1	• 3
2	2	A	B:2, D:3	

DSatur — exemple détaillé

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 5 : \{ E \}, 4 : \{ B, D \}, 3 : \{ C \}, 2 : \{ A, F, G \} \}$$



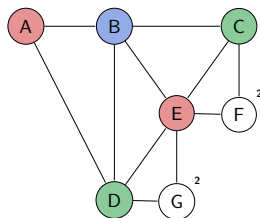
Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	E	—	● 1
1	4	B	E:1	● 2
2	4	D	E:1, B:2	● 3
2	3	C	B:2, E:1	● 3
2	2	A	B:2, D:3	● 1

DSatur — exemple détaillé

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 5 : \{ E \}, 4 : \{ B, D \}, 3 : \{ C \}, 2 : \{ A, F, G \} \}$$



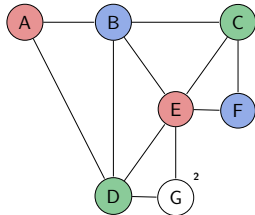
Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	E	—	• 1
1	4	B	E:1	• 2
2	4	D	E:1, B:2	• 3
2	3	C	B:2, E:1	• 3
2	2	A	B:2, D:3	• 1
2	2	F	C:3, E:1	

DSatur — exemple détaillé

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 5 : \{ E \}, 4 : \{ B, D \}, 3 : \{ C \}, 2 : \{ A, F, G \} \}$$



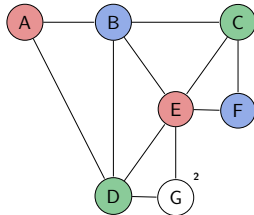
Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	E	—	● 1
1	4	B	E:1	● 2
2	4	D	E:1, B:2	● 3
2	3	C	B:2, E:1	● 3
2	2	A	B:2, D:3	● 1
2	2	F	C:3, E:1	● 2

DSatur — exemple détaillé

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 5 : \{ E \}, 4 : \{ B, D \}, 3 : \{ C \}, 2 : \{ A, F, G \} \}$$



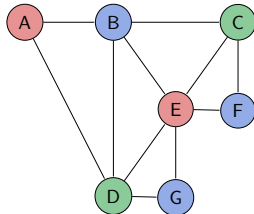
Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	E	—	● 1
1	4	B	E:1	● 2
2	4	D	E:1, B:2	● 3
2	3	C	B:2, E:1	● 3
2	2	A	B:2, D:3	● 1
2	2	F	C:3, E:1	● 2
2	2	G	D:3, E:1	

DSatur — exemple détaillé

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

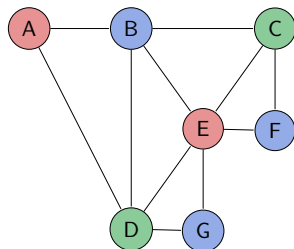
Liste des degrés :

$$\{ 5 : \{ E \}, 4 : \{ B, D \}, 3 : \{ C \}, 2 : \{ A, F, G \} \}$$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	E	—	● 1
1	4	B	E:1	● 2
2	4	D	E:1, B:2	● 3
2	3	C	B:2, E:1	● 3
2	2	A	B:2, D:3	● 1
2	2	F	C:3, E:1	● 2
2	2	G	D:3, E:1	● 2

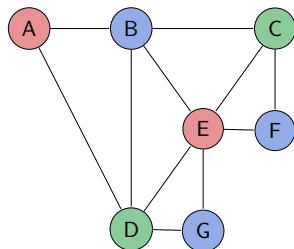
DSatur — résultat et comparaison



Coloriage final :

- **Couleur 1** : A, E
- **Couleur 2** : B, F, G
- **Couleur 3** : C, D

DSatur — résultat et comparaison

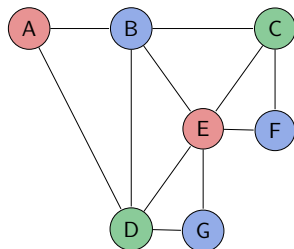


Coloriage final :

- **Couleur 1** : A, E
- **Couleur 2** : B, F, G
- **Couleur 3** : C, D

Même résultat que Welsh-Powell sur cet exemple.

DSatur — résultat et comparaison



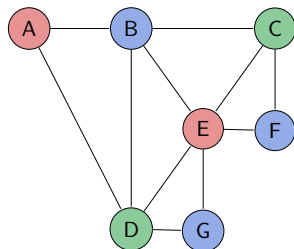
Coloriage final :

- **Couleur 1** : A, E
- **Couleur 2** : B, F, G
- **Couleur 3** : C, D

Même résultat que Welsh-Powell sur cet exemple.

Différence clé : DSatur a choisi l'ordre *E, B, D, C, A, F, G dynamiquement* — la saturation de chaque sommet est réévaluée à chaque étape.

DSatur — résultat et comparaison



Coloriage final :

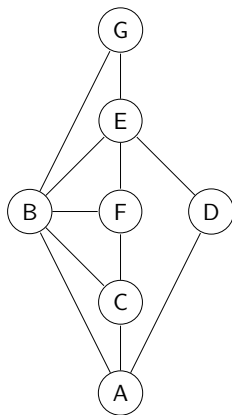
- **Couleur 1** : A, E
- **Couleur 2** : B, F, G
- **Couleur 3** : C, D

Même résultat que Welsh-Powell sur cet exemple.

Différence clé : DSatur a choisi l'ordre *E, B, D, C, A, F, G dynamiquement* — la saturation de chaque sommet est réévaluée à chaque étape.

DSatur — exemple détaillé 2

Retour sur un contre-exemple de Welsh-Powell.



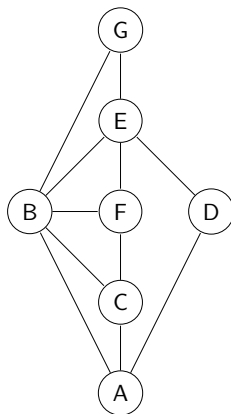
DSatur — exemple détaillé 2

Retour sur un contre-exemple de Welsh-Powell.

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

{ 5 : { B }, 4 : { E }, 3 : { F, C, A }, 2 : { D, G } }



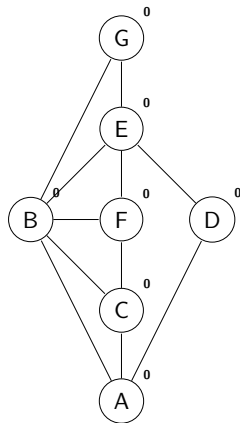
DSatur — exemple détaillé 2

Retour sur un contre-exemple de Welsh-Powell.

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$\{ 5 : \{ B \}, 4 : \{ E \}, 3 : \{ F, C, A \}, 2 : \{ D, G \} \}$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	<i>B</i>		

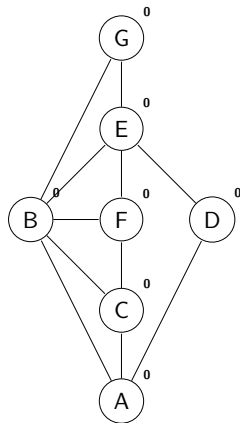
DSatur — exemple détaillé 2

Retour sur un contre-exemple de Welsh-Powell.

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$\{ 5 : \{ B \}, 4 : \{ E \}, 3 : \{ F, C, A \}, 2 : \{ D, G \} \}$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	<i>B</i>	—	

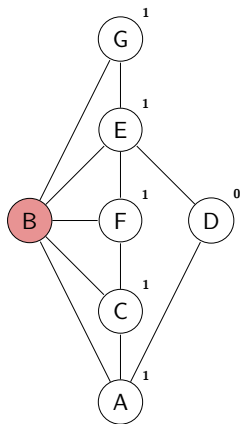
DSatur — exemple détaillé 2

Retour sur un contre-exemple de Welsh-Powell.

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$\{ 5 : \{ B \}, 4 : \{ E \}, 3 : \{ F, C, A \}, 2 : \{ D, G \} \}$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	B	—	• 1

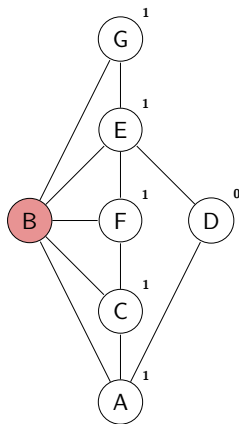
DSatur — exemple détaillé 2

Retour sur un contre-exemple de Welsh-Powell.

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$\{ 5 : \{ B \}, 4 : \{ E \}, 3 : \{ F, C, A \}, 2 : \{ D, G \} \}$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	B	—	• 1
1	4	E	B:1	

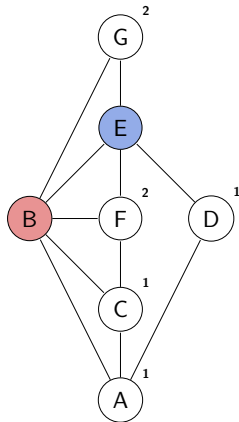
DSatur — exemple détaillé 2

Retour sur un contre-exemple de Welsh-Powell.

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$\{ 5 : \{ B \}, 4 : \{ E \}, 3 : \{ F, C, A \}, 2 : \{ D, G \} \}$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	B	—	• 1
1	4	E	B:1	• 2

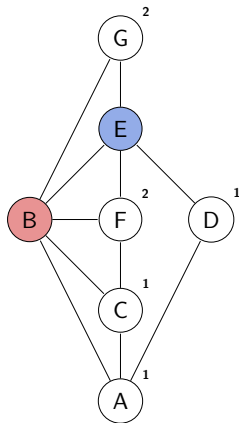
DSatur — exemple détaillé 2

Retour sur un contre-exemple de Welsh-Powell.

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$\{ 5 : \{ B \}, 4 : \{ E \}, 3 : \{ F, C, A \}, 2 : \{ D, G \} \}$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	B	—	• 1
1	4	E	B:1	• 2
2	3	F	B:1, E:2	

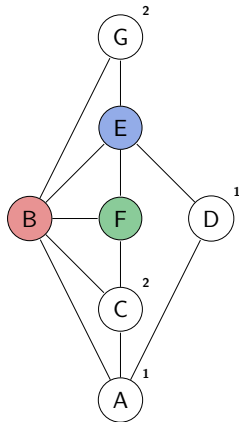
DSatur — exemple détaillé 2

Retour sur un contre-exemple de Welsh-Powell.

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$\{ 5 : \{ B \}, 4 : \{ E \}, 3 : \{ F, C, A \}, 2 : \{ D, G \} \}$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	B	—	• 1
1	4	E	B:1	• 2
2	3	F	B:1, E:2	• 3

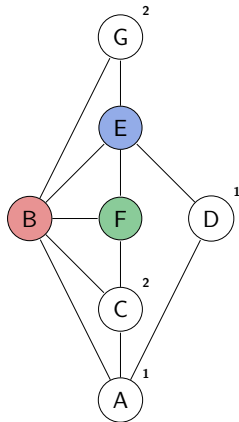
DSatur — exemple détaillé 2

Retour sur un contre-exemple de Welsh-Powell.

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$\{ 5 : \{ B \}, 4 : \{ E \}, 3 : \{ F, C, A \}, 2 : \{ D, G \} \}$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	B	—	• 1
1	4	E	B:1	• 2
2	3	F	B:1, E:2	• 3
2	3	C	B:1, F:3	

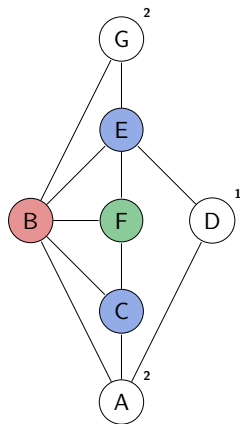
DSatur — exemple détaillé 2

Retour sur un contre-exemple de Welsh-Powell.

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$\{ 5 : \{ B \}, 4 : \{ E \}, 3 : \{ F, C, A \}, 2 : \{ D, G \} \}$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	B	—	● 1
1	4	E	B:1	● 2
2	3	F	B:1, E:2	● 3
2	3	C	B:1, F:3	● 2

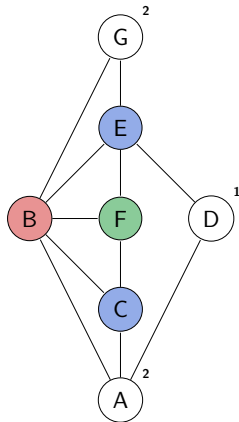
DSatur — exemple détaillé 2

Retour sur un contre-exemple de Welsh-Powell.

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$\{ 5 : \{ B \}, 4 : \{ E \}, 3 : \{ F, C, A \}, 2 : \{ D, G \} \}$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	B	—	● 1
1	4	E	B:1	● 2
2	3	F	B:1, E:2	● 3
2	3	C	B:1, F:3	● 2
2	3	A	B:1, C:2	

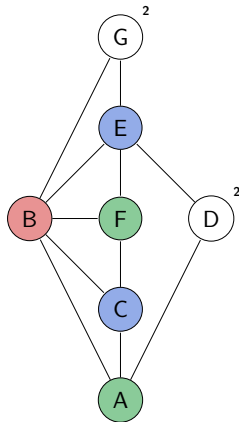
DSatur — exemple détaillé 2

Retour sur un contre-exemple de Welsh-Powell.

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$\{ 5 : \{ B \}, 4 : \{ E \}, 3 : \{ F, C, A \}, 2 : \{ D, G \} \}$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	B	—	● 1
1	4	E	B:1	● 2
2	3	F	B:1, E:2	● 3
2	3	C	B:1, F:3	● 2
2	3	A	B:1, C:2	● 3

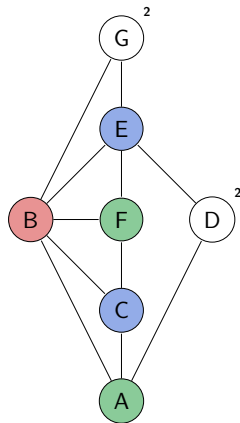
DSatur — exemple détaillé 2

Retour sur un contre-exemple de Welsh-Powell.

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$\{ 5 : \{ B \}, 4 : \{ E \}, 3 : \{ F, C, A \}, 2 : \{ D, G \} \}$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	B	—	● 1
1	4	E	B:1	● 2
2	3	F	B:1, E:2	● 3
2	3	C	B:1, F:3	● 2
2	3	A	B:1, C:2	● 3
2	2	D	E:2, A:3	

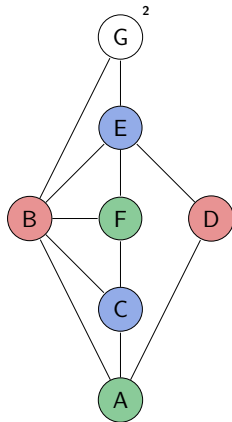
DSatur — exemple détaillé 2

Retour sur un contre-exemple de Welsh-Powell.

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$\{ 5 : \{ B \}, 4 : \{ E \}, 3 : \{ F, C, A \}, 2 : \{ D, G \} \}$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	B	—	• 1
1	4	E	B:1	• 2
2	3	F	B:1, E:2	• 3
2	3	C	B:1, F:3	• 2
2	3	A	B:1, C:2	• 3
2	2	D	E:2, A:3	• 1

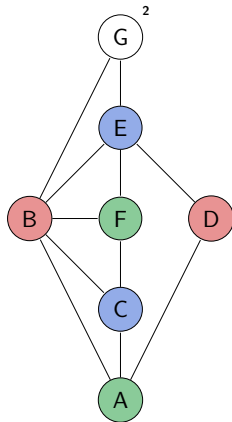
DSatur — exemple détaillé 2

Retour sur un contre-exemple de Welsh-Powell.

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$\{ 5 : \{ B \}, 4 : \{ E \}, 3 : \{ F, C, A \}, 2 : \{ D, G \} \}$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	B	—	• 1
1	4	E	B:1	• 2
2	3	F	B:1, E:2	• 3
2	3	C	B:1, F:3	• 2
2	3	A	B:1, C:2	• 3
2	2	D	E:2, A:3	• 1
2	2	G	B:1, E:2	

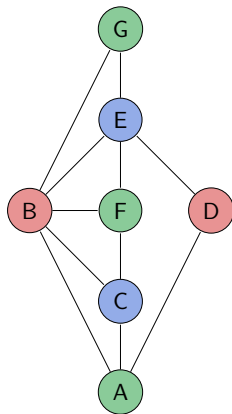
DSatur — exemple détaillé 2

Retour sur un contre-exemple de Welsh-Powell.

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$\{ 5 : \{ B \}, 4 : \{ E \}, 3 : \{ F, C, A \}, 2 : \{ D, G \} \}$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	B	—	● 1
1	4	E	B:1	● 2
2	3	F	B:1, E:2	● 3
2	3	C	B:1, F:3	● 2
2	3	A	B:1, C:2	● 3
2	2	D	E:2, A:3	● 1
2	2	G	B:1, E:2	● 3

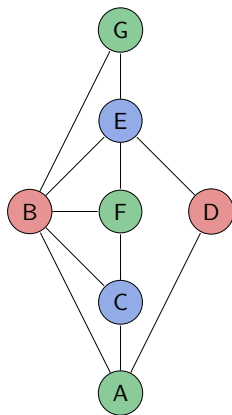
DSatur — exemple détaillé 2

Retour sur un contre-exemple de Welsh-Powell.

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 5 : \{ B \}, 4 : \{ E \}, 3 : \{ F, C, A \}, 2 : \{ D, G \} \}$$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	B	—	● 1
1	4	E	B:1	● 2
2	3	F	B:1, E:2	● 3
2	3	C	B:1, F:3	● 2
2	3	A	B:1, C:2	● 3
2	2	D	E:2, A:3	● 1
2	2	G	B:1, E:2	● 3

3 couleurs — optimal ✓ (triangle A-B-C)

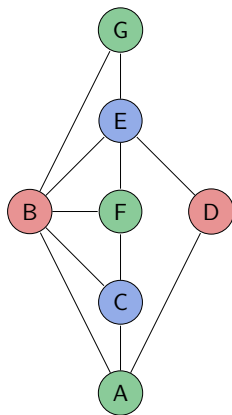
DSatur — exemple détaillé 2

Retour sur un contre-exemple de Welsh-Powell.

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 5 : \{ B \}, 4 : \{ E \}, 3 : \{ F, C, A \}, 2 : \{ D, G \} \}$$



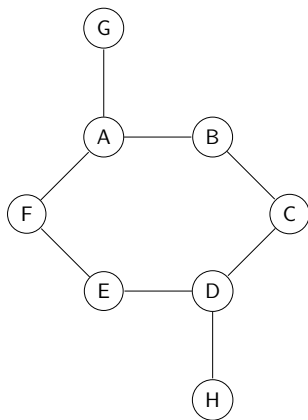
Sat.	Deg.	Som.	Voisins colorés	Coul.
0	5	B	—	● 1
1	4	E	B:1	● 2
2	3	F	B:1, E:2	● 3
2	3	C	B:1, F:3	● 2
2	3	A	B:1, C:2	● 3
2	2	D	E:2, A:3	● 1
2	2	G	B:1, E:2	● 3

3 couleurs — optimal ✓ (triangle A-B-C

DSatur a donc trouvé tout seul le bon parcours des sommets.

DSatur — exemple détaillé 3

Retour sur un contre-exemple de Welsh-Powell qui échouait toujours !



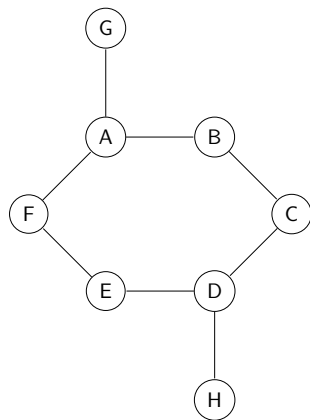
DSatur — exemple détaillé 3

Retour sur un contre-exemple de Welsh-Powell qui échouait toujours !

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$\{ 3 : \{ A, D \}, 2 : \{ B, C, E, F \}, 1 : \{ G, H \} \}$



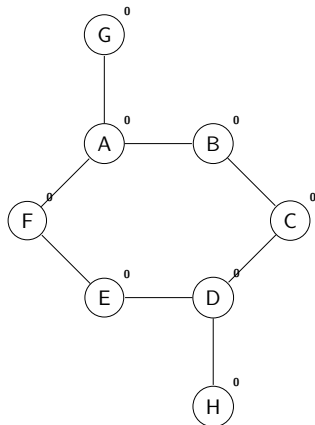
DSatur — exemple détaillé 3

Retour sur un contre-exemple de Welsh-Powell qui échouait toujours !

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 3 : \{ A, D \}, 2 : \{ B, C, E, F \}, 1 : \{ G, H \} \}$$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	3	A		

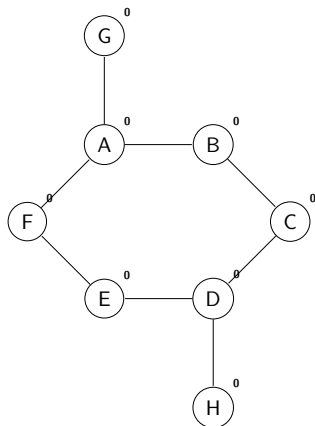
DSatur — exemple détaillé 3

Retour sur un contre-exemple de Welsh-Powell qui échouait toujours !

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 3 : \{ A, D \}, 2 : \{ B, C, E, F \}, 1 : \{ G, H \} \}$$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	3	A	—	

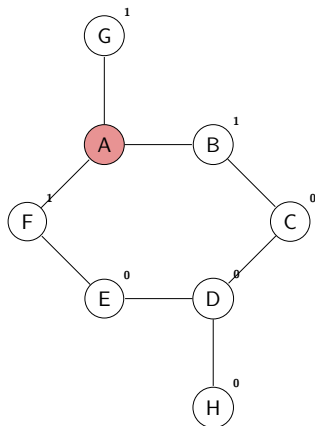
DSatur — exemple détaillé 3

Retour sur un contre-exemple de Welsh-Powell qui échouait toujours !

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 3 : \{ A, D \}, 2 : \{ B, C, E, F \}, 1 : \{ G, H \} \}$$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	3	A	—	• 1

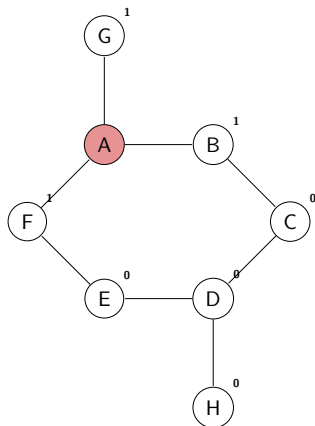
DSatur — exemple détaillé 3

Retour sur un contre-exemple de Welsh-Powell qui échouait toujours !

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 3 : \{ A, D \}, 2 : \{ B, C, E, F \}, 1 : \{ G, H \} \}$$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	3	A	—	• 1
1	2	B	A:1	

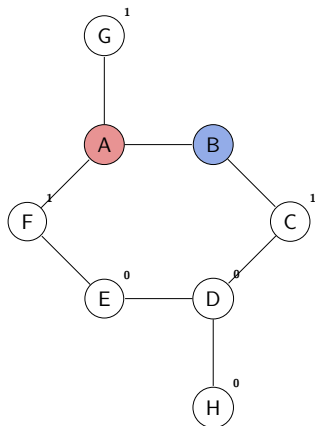
DSatur — exemple détaillé 3

Retour sur un contre-exemple de Welsh-Powell qui échouait toujours !

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 3 : \{ A, D \}, 2 : \{ B, C, E, F \}, 1 : \{ G, H \} \}$$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	3	A	—	• 1
1	2	B	A:1	• 2

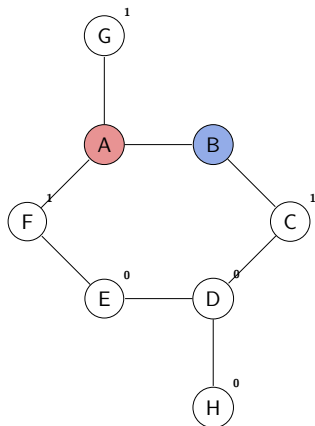
DSatur — exemple détaillé 3

Retour sur un contre-exemple de Welsh-Powell qui échouait toujours !

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 3 : \{ A, D \}, 2 : \{ B, C, E, F \}, 1 : \{ G, H \} \}$$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	3	A	—	• 1
1	2	B	A:1	• 2
1	2	C	B:2	

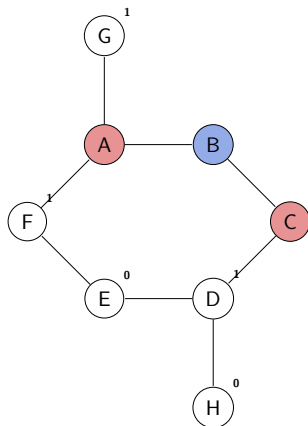
DSatur — exemple détaillé 3

Retour sur un contre-exemple de Welsh-Powell qui échouait toujours !

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 3 : \{ A, D \}, 2 : \{ B, C, E, F \}, 1 : \{ G, H \} \}$$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	3	A	—	• 1
1	2	B	A:1	• 2
1	2	C	B:2	• 1

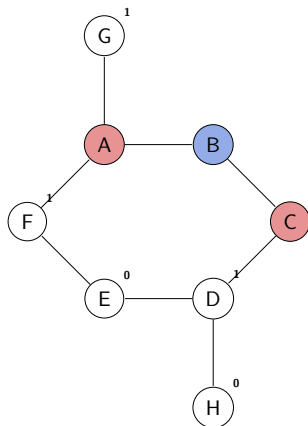
DSatur — exemple détaillé 3

Retour sur un contre-exemple de Welsh-Powell qui échouait toujours !

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 3 : \{ A, D \}, 2 : \{ B, C, E, F \}, 1 : \{ G, H \} \}$$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	3	A	—	• 1
1	2	B	A:1	• 2
1	2	C	B:2	• 1
1	3	D	C:1	

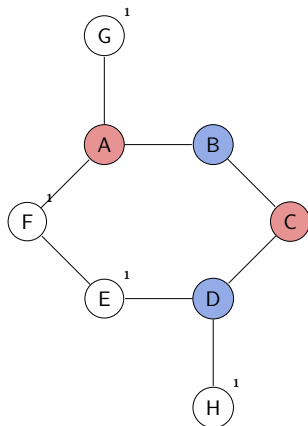
DSatur — exemple détaillé 3

Retour sur un contre-exemple de Welsh-Powell qui échouait toujours !

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 3 : \{ A, D \}, 2 : \{ B, C, E, F \}, 1 : \{ G, H \} \}$$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	3	A	—	• 1
1	2	B	A:1	• 2
1	2	C	B:2	• 1
1	3	D	C:1	• 2

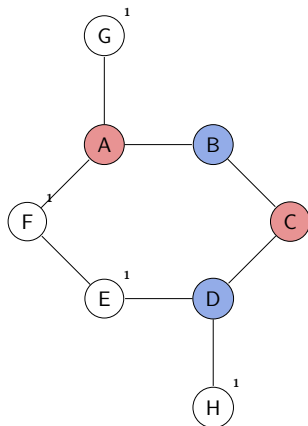
DSatur — exemple détaillé 3

Retour sur un contre-exemple de Welsh-Powell qui échouait toujours !

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 3 : \{ A, D \}, 2 : \{ B, C, E, F \}, 1 : \{ G, H \} \}$$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	3	A	—	• 1
1	2	B	A:1	• 2
1	2	C	B:2	• 1
1	3	D	C:1	• 2
1	2	E	D:2	

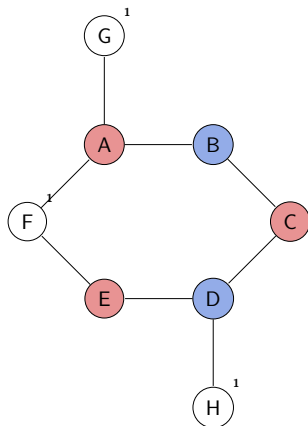
DSatur — exemple détaillé 3

Retour sur un contre-exemple de Welsh-Powell qui échouait toujours !

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 3 : \{ A, D \}, 2 : \{ B, C, E, F \}, 1 : \{ G, H \} \}$$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	3	A	—	• 1
1	2	B	A:1	• 2
1	2	C	B:2	• 1
1	3	D	C:1	• 2
1	2	E	D:2	• 1

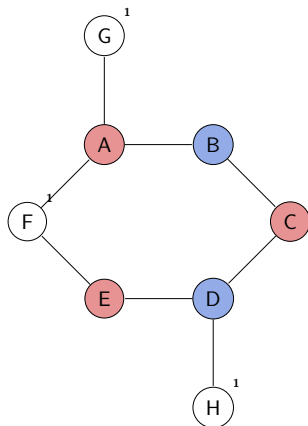
DSatur — exemple détaillé 3

Retour sur un contre-exemple de Welsh-Powell qui échouait toujours !

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 3 : \{ A, D \}, 2 : \{ B, C, E, F \}, 1 : \{ G, H \} \}$$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	3	A	—	• 1
1	2	B	A:1	• 2
1	2	C	B:2	• 1
1	3	D	C:1	• 2
1	2	E	D:2	• 1
1	2	F	E:1, A:1	

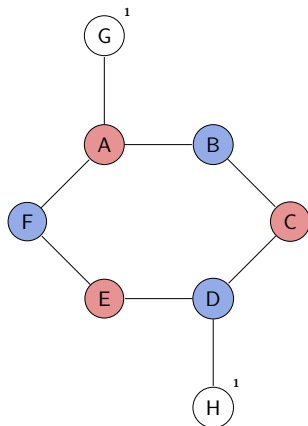
DSatur — exemple détaillé 3

Retour sur un contre-exemple de Welsh-Powell qui échouait toujours !

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 3 : \{ A, D \}, 2 : \{ B, C, E, F \}, 1 : \{ G, H \} \}$$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	3	A	—	• 1
1	2	B	A:1	• 2
1	2	C	B:2	• 1
1	3	D	C:1	• 2
1	2	E	D:2	• 1
1	2	F	E:1, A:1	• 2

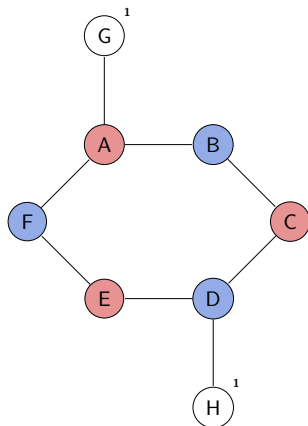
DSatur — exemple détaillé 3

Retour sur un contre-exemple de Welsh-Powell qui échouait toujours !

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 3 : \{ A, D \}, 2 : \{ B, C, E, F \}, 1 : \{ G, H \} \}$$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	3	A	—	• 1
1	2	B	A:1	• 2
1	2	C	B:2	• 1
1	3	D	C:1	• 2
1	2	E	D:2	• 1
1	2	F	E:1, A:1	• 2
1	1	G	A:1	

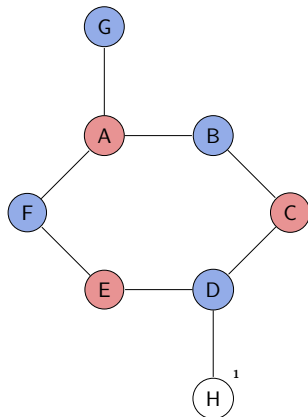
DSatur — exemple détaillé 3

Retour sur un contre-exemple de Welsh-Powell qui échouait toujours !

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 3 : \{ A, D \}, 2 : \{ B, C, E, F \}, 1 : \{ G, H \} \}$$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	3	A	—	• 1
1	2	B	A:1	• 2
1	2	C	B:2	• 1
1	3	D	C:1	• 2
1	2	E	D:2	• 1
1	2	F	E:1, A:1	• 2
1	1	G	A:1	• 2

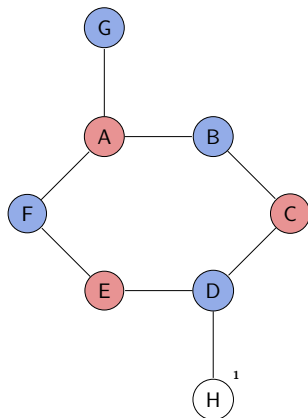
DSatur — exemple détaillé 3

Retour sur un contre-exemple de Welsh-Powell qui échouait toujours !

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 3 : \{ A, D \}, 2 : \{ B, C, E, F \}, 1 : \{ G, H \} \}$$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	3	A	—	• 1
1	2	B	A:1	• 2
1	2	C	B:2	• 1
1	3	D	C:1	• 2
1	2	E	D:2	• 1
1	2	F	E:1, A:1	• 2
1	1	G	A:1	• 2
1	1	H	D:2	

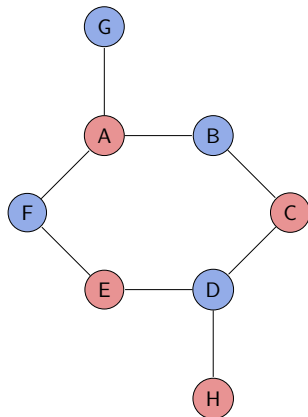
DSatur — exemple détaillé 3

Retour sur un contre-exemple de Welsh-Powell qui échouait toujours !

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 3 : \{ A, D \}, 2 : \{ B, C, E, F \}, 1 : \{ G, H \} \}$$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	3	A	—	• 1
1	2	B	A:1	• 2
1	2	C	B:2	• 1
1	3	D	C:1	• 2
1	2	E	D:2	• 1
1	2	F	E:1, A:1	• 2
1	1	G	A:1	• 2
1	1	H	D:2	• 1

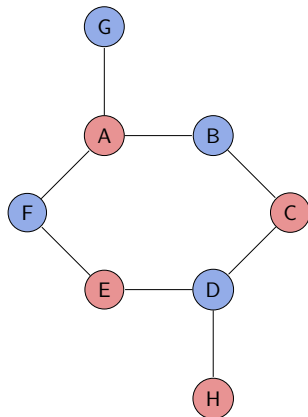
DSatur — exemple détaillé 3

Retour sur un contre-exemple de Welsh-Powell qui échouait toujours !

Critère : saturation max (affichée sur le graphe), puis degré max pour ex-æquo

Liste des degrés :

$$\{ 3 : \{ A, D \}, 2 : \{ B, C, E, F \}, 1 : \{ G, H \} \}$$



Sat.	Deg.	Som.	Voisins colorés	Coul.
0	3	A	—	• 1
1	2	B	A:1	• 2
1	2	C	B:2	• 1
1	3	D	C:1	• 2
1	2	E	D:2	• 1
1	2	F	E:1, A:1	• 2
1	1	G	A:1	• 2
1	1	H	D:2	• 1

2 couleurs — optimal ✓ (graphe bipartite, $\chi = 2$)

DSatur — contre-exemple

DSatur **n'est pas non plus optimal** en général.

Plan

7 Coloriage de graphes

- Présentation du problème
- Définitions
- NP-complétude et approches
- Welsh-Powell
- DSatur
- Propriétés et comparaison

Zoom sur la complexité

On note $n = |V|$ et $m = |E|$.

Welsh-Powell

Zoom sur la complexité

On note $n = |V|$ et $m = |E|$.

Welsh-Powell

- Tri des degrés en $O(n \log(n))$.

Zoom sur la complexité

On note $n = |V|$ et $m = |E|$.

Welsh-Powell

- Tri des degrés en $O(n \log(n))$.
- Il y a n étapes. À l'étape k on visite au plus les $k - 1$ sommets déjà coloriés soit $0 + 1 + 2 + \dots + (n - 1) = \frac{n(n-1)}{2} = O(n^2)$.

Zoom sur la complexité

On note $n = |V|$ et $m = |E|$.

Welsh-Powell

- Tri des degrés en $O(n \log(n))$.
- Il y a n étapes. À l'étape k on visite au plus les $k - 1$ sommets déjà coloriés soit $0 + 1 + 2 + \dots + (n - 1) = \frac{n(n-1)}{2} = O(n^2)$.
- Donc $O(n^2)$

DSatur

Zoom sur la complexité

On note $n = |V|$ et $m = |E|$.

Welsh-Powell

- Tri des degrés en $O(n \log(n))$.
- Il y a n étapes. À l'étape k on visite au plus les $k - 1$ sommets déjà coloriés soit $0 + 1 + 2 + \dots + (n - 1) = \frac{n(n-1)}{2} = O(n^2)$.
- Donc $O(n^2)$

DSatur

- Il y a n étapes. On tient à jour le tableau des saturations.

Zoom sur la complexité

On note $n = |V|$ et $m = |E|$.

Welsh-Powell

- Tri des degrés en $O(n \log(n))$.
- Il y a n étapes. À l'étape k on visite au plus les $k - 1$ sommets déjà coloriés soit $0 + 1 + 2 + \dots + (n - 1) = \frac{n(n-1)}{2} = O(n^2)$.
- Donc $O(n^2)$

DSatur

- Il y a n étapes. On tient à jour le tableau des saturations.
- À chaque étape recherche de la saturation maximale : $O(n)$.

Zoom sur la complexité

On note $n = |V|$ et $m = |E|$.

Welsh-Powell

- Tri des degrés en $O(n \log(n))$.
- Il y a n étapes. À l'étape k on visite au plus les $k - 1$ sommets déjà coloriés soit $0 + 1 + 2 + \dots + (n - 1) = \frac{n(n-1)}{2} = O(n^2)$.
- Donc $O(n^2)$

DSatur

- Il y a n étapes. On tient à jour le tableau des saturations.
- À chaque étape recherche de la saturation maximale : $O(n)$.
- Mise à jour de la saturation : sur l'ensemble de l'algorithme toutes les arêtes seront vues une seule fois soit un $O(m)$.

Zoom sur la complexité

On note $n = |V|$ et $m = |E|$.

Welsh-Powell

- Tri des degrés en $O(n \log(n))$.
- Il y a n étapes. À l'étape k on visite au plus les $k - 1$ sommets déjà coloriés soit $0 + 1 + 2 + \dots + (n - 1) = \frac{n(n-1)}{2} = O(n^2)$.
- Donc $O(n^2)$

DSatur

- Il y a n étapes. On tient à jour le tableau des saturations.
- À chaque étape recherche de la saturation maximale : $O(n)$.
- Mise à jour de la saturation : sur l'ensemble de l'algorithme toutes les arêtes seront vues une seule fois soit un $O(m)$.
- Ainsi on a un $O(n^2 + m)$.

Zoom sur la complexité

On note $n = |V|$ et $m = |E|$.

Welsh-Powell

- Tri des degrés en $O(n \log(n))$.
- Il y a n étapes. À l'étape k on visite au plus les $k - 1$ sommets déjà coloriés soit $0 + 1 + 2 + \dots + (n - 1) = \frac{n(n-1)}{2} = O(n^2)$.
- Donc $O(n^2)$

DSatur

- Il y a n étapes. On tient à jour le tableau des saturations.
- À chaque étape recherche de la saturation maximale : $O(n)$.
- Mise à jour de la saturation : sur l'ensemble de l'algorithme toutes les arêtes seront vues une seule fois soit un $O(m)$.
- Ainsi on a un $O(n^2 + m)$.
- Mais pour un graphe simple $m \leq \binom{n}{2} = \frac{n(n-1)}{2} = O(n^2)$.

Zoom sur la complexité

On note $n = |V|$ et $m = |E|$.

Welsh-Powell

- Tri des degrés en $O(n \log(n))$.
- Il y a n étapes. À l'étape k on visite au plus les $k - 1$ sommets déjà coloriés soit $0 + 1 + 2 + \dots + (n - 1) = \frac{n(n-1)}{2} = O(n^2)$.
- Donc $O(n^2)$

DSatur

- Il y a n étapes. On tient à jour le tableau des saturations.
- À chaque étape recherche de la saturation maximale : $O(n)$.
- Mise à jour de la saturation : sur l'ensemble de l'algorithme toutes les arêtes seront vues une seule fois soit un $O(m)$.
- Ainsi on a un $O(n^2 + m)$.
- Mais pour un graphe simple $m \leq \binom{n}{2} = \frac{n(n-1)}{2} = O(n^2)$.
- Donc $O(n^2)$.

Comparaison des deux algorithmes

	Welsh-Powell	DSatur
Tri	Statique (une fois)	Dynamique (à chaque étape)
Critère	Degré $d(v)$	Saturation $\text{sat}(v)$
Complexité	$O(V ^2)$	$O(V ^2)$ (naïf)
Optimal ?	Non	Non
Résultat	$\leq \Delta(G) + 1$	$\leq \Delta(G) + 1$
En pratique	Correct	Meilleur

Comparaison des deux algorithmes

	Welsh-Powell	DSatur
Tri	Statique (une fois)	Dynamique (à chaque étape)
Critère	Degré $d(v)$	Saturation $\text{sat}(v)$
Complexité	$O(V ^2)$	$O(V ^2)$ (naïf)
Optimal ?	Non	Non
Résultat	$\leq \Delta(G) + 1$	$\leq \Delta(G) + 1$
En pratique	Correct	Meilleur

(Toujours $\leq \Delta(G) + 1$, souvent bien meilleur.)

Propriétés remarquables

- **Graphes bipartites** : $\chi(G) = 2$ exactement si G est connexe non vide.

Propriétés remarquables

- **Graphes bipartites** : $\chi(G) = 2$ exactement si G est connexe non vide. DSatur le trouve toujours (il suit les niveaux BFS).

Propriétés remarquables

- **Graphes bipartites** : $\chi(G) = 2$ exactement si G est connexe non vide. DSatur le trouve toujours (il suit les niveaux BFS).
- **Graphes parfaits** : classe de graphes où $\chi(H) = \omega(H)$ pour tout sous-graphe induit H .

Propriétés remarquables

- **Graphes bipartites** : $\chi(G) = 2$ exactement si G est connexe non vide. DSatur le trouve toujours (il suit les niveaux BFS).
- **Graphes parfaits** : classe de graphes où $\chi(H) = \omega(H)$ pour tout sous-graphe induit H . DSatur est optimal sur ces graphes.

Propriétés remarquables

- **Graphes bipartites** : $\chi(G) = 2$ exactement si G est connexe non vide. DSatur le trouve toujours (il suit les niveaux BFS).
- **Graphes parfaits** : classe de graphes où $\chi(H) = \omega(H)$ pour tout sous-graphe induit H . DSatur est optimal sur ces graphes.
- **Théorème de Brooks (1941)** : si G est connexe et n'est ni un cycle impair ni un graphe complet, alors $\chi(G) \leq \Delta(G)$ (mieux que la borne triviale donc) !

Propriétés remarquables

- **Graphes bipartites** : $\chi(G) = 2$ exactement si G est connexe non vide. DSatur le trouve toujours (il suit les niveaux BFS).
- **Graphes parfaits** : classe de graphes où $\chi(H) = \omega(H)$ pour tout sous-graphe induit H . DSatur est optimal sur ces graphes.
- **Théorème de Brooks (1941)** : si G est connexe et n'est ni un cycle impair ni un graphe complet, alors $\chi(G) \leq \Delta(G)$ (mieux que la borne triviale donc) !
- **Remarque** : $\chi(G) \gg \omega(G)$ est possible. Le graphe de Mycielski prouve que l'écart peut être arbitrairement grand — **la clique ne suffit pas à prédire χ** .

Récapitulatif

Problème	Affecter des couleurs à V
Contrainte	Voisins de couleurs différentes
Objectif	Minimiser $\chi(G)$
Complexité	NP-difficile ($k \geq 3$)
Borne inf.	$\chi(G) \geq \omega(G)$
Borne sup.	$\chi(G) \leq \Delta(G) + 1$
Algo 1	Welsh-Powell — $O(V ^2)$
Algo 2	DSatur — $O(V ^2)$
Applications	Cartes, examens, radio, compilateurs

Récapitulatif

Problème	Affecter des couleurs à V
Contrainte	Voisins de couleurs différentes
Objectif	Minimiser $\chi(G)$
Complexité	NP-difficile ($k \geq 3$)
Borne inf.	$\chi(G) \geq \omega(G)$
Borne sup.	$\chi(G) \leq \Delta(G) + 1$
Algo 1	Welsh-Powell — $O(V ^2)$
Algo 2	DSatur — $O(V ^2)$
Applications	Cartes, examens, radio, compilateurs

Aller plus loin : algorithmes exacts (Bron-Kerbosch + backtracking), heuristiques méta (recuit simulé, algorithmes génétiques), DIMACS benchmarks.